

机器学习方法

bjhh2005, xhkzdepartedream, kiwiizzz, zoomy

2026.6

目录

1	机器学习概论	1	5.4	C4.5 算法	60
1.1	问题定义与数据表征 (The Problem Space)	1	5.4.1	使用信息增益比作为判准	60
1.2	学习范式分类	1	5.4.2	最佳属性划分: 连续值与缺失值处理	60
1.2.1	机器学习技术分类(I): 样本是否带有分类标记	1	5.5	决策树的剪枝	60
1.2.2	按模型分类	2	5.6	决策树的剪枝: 从数学符号到核心逻辑	60
1.2.3	按算法分类	2	5.6.1	底层依赖概念拆解	60
1.3	方法论三要素: “方法=模型+策略+算法”	2	5.6.2	核心机制: 损失函数 $C_{\alpha}(T)$ 的天平博弈	61
1.3.1	模型与假设空间 (Model)	2	5.6.3	局部的“Before 与 After”对决机制	61
1.3.2	策略与学习准则 (Strategy)	3	5.7	CART 算法	61
1.3.3	算法	4	5.8	CART 生成	61
1.4	计算学习理论	4	5.9	CART 剪枝	65
1.4.1	打散、中断点、VC 维度	4	6	集成学习	66
1.4.2	Hoeffding 不等式	5	6.1	定义与概念	66
1.4.3	多重检验问题	5	6.1.1	同质集成/异质集成	66
1.5	误差构成与工程评估 (Operational Excellence)	6	6.1.2	多专家组合(multiexpert combination)	66
1.5.1	泛化能力	6	6.1.3	多级组合(multistage combination)	66
1.5.2	泛化误差分解	8	6.2	集成学习理论基础	67
1.5.3	数据集管理与评估方法	9	6.2.1	Why Ensemble?	67
1.5.4	性能度量	10	6.2.2	How to Ensemble?	67
1.6	模型比较检验	12	6.3	Boosting	67
2	线性回归	13	6.3.1	AdaBoost 算法	67
2.1	线性判别函数与感知机模型	13	6.3.2	AdaBoost 算法的数学本质	70
2.1.1	基本概念	13	6.3.3	提升树	71
2.1.2	回归与分类	15	6.3.4	梯度提升树 (GBDT)	73
2.1.3	线性判别分析 (LDA)	15	6.4	Bagging 与 随机森林	75
2.2	线性回归	17	6.4.1	自主采样(Bootstrap Aggregating)与 Bagging	75
2.2.1	基本概念	17	6.4.2	随机森林 (Random Forest)	76
2.2.2	岭回归与拉索回归	18	6.4.3	OOB 误差估计示例	77
2.2.3	相关量化指标	19	6.4.4	随机森林的优缺点分析	77
2.2.4	广义线性回归	20	6.4.5	代码实现: 特征重要性评估	78
2.3	逻辑斯蒂回归与最大熵模型	23	6.4.6	堆叠	80
2.3.1	逻辑斯蒂回归模型/对数几率回归分类器	23	6.5	多样性	80
2.3.2	最大熵模型	26	6.5.1	多样性度量 (五大指标)	81
3	贝叶斯分类器	29	7	降维与度量学习	81
3.1	贝叶斯决策论 (Bayesian Decision Theory)	29	7.1	低维嵌入	81
3.2	极大似然估计 (Maximum Likelihood Estimation, MLE)	29	7.2	样本主成分分析: 基于 (样本) 协方差矩阵	82
3.3	贝叶斯法	29	7.2.1	算法直觉与流程	82
3.3.1	朴素贝叶斯法的学习与分类	29	7.2.2	深入理解 PCA: 数学推导、性质与应用考量	86
3.3.2	半朴素贝叶斯分类器	31	7.3	样本主成分分析	88
3.3.3	贝叶斯网络	33	7.4	核化线性降维	92
3.3.4	吉布斯采样	35	7.5	流形学习	92
4	支持向量机	35	7.5.1	局部线性嵌入, LLE	93
4.1	引言	35	7.6	度量学习	93
4.2	线性可分支持向量机与硬间隔最大化	36	7.6.1	研究动机	94
4.2.1	函数间隔和几何间隔	37	7.6.2	度量学习基础	94
4.2.2	间隔最大化	37	7.6.3	近邻成分分析 (NCA)	95
4.2.3	求解: 线性可分支持向量机的最优化问题	39	7.6.4	度量学习的扩展	96
4.3	线性支持向量机与软间隔最大化	43	7.6.5	度量学习与降维的关系	96
4.3.1	线性支持向量机与软间隔最大化	43	7.7	k 近邻法	96
4.3.2	学习的对偶算法	43	7.7.1	k 近邻算法	96
4.3.3	支持向量	46	7.7.2	k 近邻模型	97
4.3.4	合页损失函数	47	7.7.3	k 近邻法的实现: kd 树	98
4.4	非线性支持向量机与核函数	48	8	聚类	99
4.5	序列最小最优化算法	51	8.1	聚类方法	99
5	决策树	56	8.1.1	聚类的基本概念	99
5.1	决策树模型与学习	56	8.2	性能指标	100
5.2	特征选择	56	8.2.1	外部指标	100
5.3	ID3 算法	58	8.2.2	内部指标	100
			8.3	距离计算	101
			8.3.1	有序属性	101
			8.3.2	无序属性	102

8.3.3	MinkovDM	103	10.4.1	基本思想与图映射关系	127
8.4	原型聚类	103	10.4.2	亲和矩阵与图拉普拉斯矩阵	127
8.4.1	k 均值聚类	103	10.4.3	二分类标记传播闭式解	127
8.4.2	学习向量量化 (LVQ)	104	10.4.4	多分类标记传播与规范拉普拉斯矩阵	128
8.4.3	高斯混合聚类 (GMM)	105	10.4.5	图半监督学习的局限性	128
8.4.4	两种方法横向对比	107	10.5	基于分歧的方法 (Disagreement-Based Methods)	128
8.4.5	关键概念速查	108	10.5.1	协同训练与多视图数据	128
8.5	密度聚类	108	10.5.2	协同训练算法流程	128
8.5.1	DBSCAN 聚类密度	108	10.5.3	单视图协同训练与分歧特征	129
8.5.2	DBSCAN 聚类簇的定义	109	10.6	半监督聚类 (Semi-Supervised Clustering)	129
8.5.3	DBSCAN 聚类步骤	109	10.6.1	监督信息的形式	129
8.5.4	DBSCAN 聚类分析	111	10.6.2	约束 k 均值 (Constrained k-means) 算法	130
8.5.5	算法复杂度分析	111	10.6.3	约束种子 k 均值 (Constrained Seed k-means) 算法	130
8.5.6	应用场景与局限性	111	11	强化学习	131
8.6	层次聚类	111	11.1	强化学习基本设置	131
8.6.1	层次聚类思想	111	11.1.1	强化学习的直观理解与应用场景	131
8.6.2	凝聚式层次聚类算法	112	11.1.2	马尔可夫决策过程 (Markov Decision Process, MDP)	131
8.6.3	类间距离计算方法	112	11.1.3	优化目标: 累积奖赏 (Cumulative Reward)	131
8.6.4	AGNES 聚类方法	113	11.1.4	强化学习与监督学习的本质对比	132
8.6.5	层次聚类方法的特点	114	11.2	K-摇臂赌博机 (K-armed Bandit)	132
8.6.6	延伸阅读: 复杂网络的社区发现	114	11.2.1	问题定义与背景	132
9	特征选择	114	11.2.2	ϵ -贪心算法	132
9.1	子集搜索与评价	114	11.2.3	Softmax 策略 (Boltzmann 探索)	132
9.1.1	特征的基本概念	114	11.3	有模型学习 (Model-based Learning)	133
9.1.2	特征选择的定义与动机	114	11.3.1	值函数的定义 (Value Functions)	133
9.1.3	特征选择的一般框架	115	11.3.2	贝尔曼等式	133
9.1.4	子集搜索策略	115	11.3.3	最优策略与最优值函数	134
9.1.5	子集评价方法	115	11.3.4	策略迭代算法 (Policy Iteration)	134
9.1.6	子集搜索与评价的结合	116	11.4	免模型学习 (Model-free Learning)	134
9.2	过滤式选择	116	11.4.1	蒙特卡洛方法 (Monte Carlo Methods)	134
9.2.1	Relief 方法	116	11.4.2	时序差分学习 (Temporal Difference, TD)	136
9.2.2	Relief-F 扩展	117	11.4.3	Sarsa 算法与 Q-learning 算法	136
9.3	包裹式选择	117	11.5	值函数近似 (Value Function Approximation)	136
9.3.1	LVW 方法	117	11.5.1	表格型值函数的局限性	136
9.4	嵌入式选择与 L1 正则化	118	11.5.2	值函数近似的概念	137
9.4.1	岭回归 (L2 正则化)	118	11.5.3	值函数近似	137
9.4.2	LASSO (L1 正则化)	118	11.5.4	状态-动作值函数近似	137
9.4.3	L1 正则化的几何解释	118	11.6	模仿学习 (Imitation Learning)	137
9.4.4	L1 正则化问题的求解: 近端梯度下降	119	11.6.1	专家轨迹数据集 (Expert Demonstrations)	137
9.5	稀疏表示与字典学习	120	11.6.2	行为克隆 (Behavioral Cloning)	137
9.5.1	字典学习的基本形式	120	11.6.3	逆强化学习 (Inverse Reinforcement Learning, IRL)	138
9.5.2	变量交替优化	120	11.7	基于人类反馈的强化学习 (RLHF)	138
9.6	压缩感知	121	11.7.1	智能体分类	138
9.6.1	压缩感知的基本模型	121	11.7.2	大模型对齐挑战与 3H 原则	138
9.6.2	限定等距性 (RIP)	121	11.7.3	RLHF 核心流水线 (Pipeline)	138
9.6.3	矩阵补全	122	11.8	强化学习与有监督学习的区别	139
10	半监督学习	122	11.8.1	相较于有监督学习更有可能考虑整体影响	139
10.1	未标记样本与半监督学习	122	11.8.2	强化学习更容易解决幻觉问题	139
10.1.1	背景与现实需求	122	11.8.3	强化学习可以更好地解决多轮对话奖励累积问题	139
10.1.2	未标记样本的效用与直观理解	123	12	名词索引	140
10.1.3	未标记样本利用的核心假设	123			
10.2	生成式方法 (Generative Methods)	123			
10.2.1	基本假设	123			
10.2.2	高斯混合模型 (GMM) 的引入	124			
10.2.3	联合对数似然估计	124			
10.2.4	EM 算法求解步骤	124			
10.2.5	生成式方法的特点	125			
10.3	半监督支持向量机 (S3VM)	125			
10.3.1	TSVM 基本原理	125			
10.3.2	TSVM 形式化优化目标	126			
10.3.3	类别不平衡问题及其改进	126			
10.3.4	TSVM 算法流程与标记置换机制	126			
10.4	图半监督学习 (Graph-Based Semi-Supervised Learning)	127			

1 机器学习概论

1.1 问题定义与数据表征 (The Problem Space)

“机器学习是一门学科,致力于研究如何通过计算的手段,利用**经验**来改善系统自身的性能.在计算机系统中,“经验”通常以“**数据**”形式存在,机器学习研究的主要内容是在计算机上从数据中产生“模型 (model) ”。——周志华” [1] Preliminaries.

Definition 1.1.1

特征向量/样本空间/标记空间

一组描述客体性质的变量称为 **特征变量 (feature)** .

- 变量个数 d 称为 **特征维度**;
- 特征变量组成的向量称为 **特征向量**;
- 所有特征变量张成的空间称为 **特征空间**;
- 变量的具体取值称为 **属性值**.

每个对象实例 (样本) 可由一个特征向量表征,记作 $\mathbf{x}_i$. 样本空间 $\mathcal{X}$ 是有可能存在的、合理的样本集合. 每个 $\mathbf{x}_i$ 定义在 d 维特征空间中,其第 j 个分量对应第 j 个特征:

$$\mathbf{x}_i = \begin{pmatrix} x_{i1} \\ \vdots \\ x_{id} \end{pmatrix} = (x_{i1}, \dots, x_{id})^T$$

与样本空间 $\mathcal{X}$ 对应,样本的分类标签或预测取值的集合称为 **标记空间**,记作 $\mathcal{Y}$. 在多分类问题 ($kg = 2$ 类) 中,常采用 **独热编码 (one-hot encoding)** 表示标签.

Definition 1.1.2

数据集

训练数据由输入 (或特征向量) 与标签或目标值组成,训练集通常表示为:

$$T = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_N, y_N)\}$$

输入输出对又称为样本或样本点.

多变量回归是指由多个输入特征,多元输出回归是指有多个相应变量.

1.2 学习范式分类

1.2.1 机器学习技术分类(I): 样本是否带有分类标记

1.2.1.1 监督学习

监督学习是指从**标注数据**中学习预测模型的机器学习问题,本质是学习输入到输出的**映射的统计规律**.

在监督学习中,将输入与输出所有可能取值的集合分别称为输入空间和输出空间,可以是有限元素的集合,也可以是整个欧氏空间,通常输出空间远小于输入空间.

[a]

特征空间与输入空间是不一定一样的,有时需要将实例从输入空间映射到特征空间. 模型实际上都是定义在特征空间上的.

严格的来说,其实样本和样例不太一样,前者有标签,后者无标签;anyway 不区分也可以.

Definition 1.2.1

回归问题、预测问题、分类问题

输入变量与输出变量均为连续变量的预测问题,称为**回归问题**;

输出变量为有限个离散变量的预测问题,称为**分类问题**;

输入变量与输出变量均为变量序列的预测问题,称为**标注问题**.

监督学习的模型可以是概率模型或者非概率模型,由**条件概率分布 $P(Y|X)$** 或**决策函数 $Y = f(X)$** 表示. 在监督学习中,假设训练数据与测试数据是依联合概率分布 $P(X, Y)$ 独立同分布产生的.

1.2.1.2 无监督学习

从**无标注数据**中学习预测模型的机器学习问题,本质是学习数据中的**统计规律或潜在结构**.

假设 $\mathcal{X}$ 是输入空间, $\mathcal{Z}$ 是隐式结构空间. 要学习的模型可以表示为函数 $z = g(x)$ 、条件概率分布 $P(z|x)$ 或者条件概率分布 $P(x|z)$ 的形式,其中 $x \in \mathcal{X}$ 是输入, $z \in \mathcal{Z}$ 是输出. 包括所有可能的模型的集合称为假设空间,无监督学习旨在从假设空间中选出在给定评价标准下的最优模型.

无监督学习通常使用大量的无标注数据学习或训练,每个样本都是一个实例. 训练数据表示为 $U = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\}$, 其中 $\mathbf{x}_i, i = 1, 2, \dots, N$ 是样本.

1.2.1.3 强化学习

智能系统在**与环境的连续互动**中学习**最优行为策略**的机器学习问题.假设智能系统与环境的互动基于马尔科夫决策过程,智能系统能观测到的是与环境互动得到的数据序列,强化学习本质是学习最优的序贯决策.

强化学习的马尔科夫决策过程是状态、奖励、动作序列上的随机过程,由四元组 $\langle S, A, P, r \rangle$ 组成,目标就是在所有可能的策略中选出价值函数最大的策略 π^* ,而在实际学习中往往从具体的策略出发,不断优化已有策略. γ 表示未来的奖励会有衰减.

1.2.1.4 半监督学习与主动学习

Definition 1.2.2

监督学习与主动学习

半监督学习是指利用标注数据和未标注数据学习预测模型的机器学习问题.

主动学习是指机器不断主动给出实例让教师进行标注,然后利用标注数据学习预测模型的机器学习问题.通常的监督学习使用给定的标注数据,往往是随机得到的,可以看作是被动学习,主动学习可以以较小的标注代价达到较好的学习效果.

Remark

半监督学习和主动学习更接近监督学习.

1.2.2 按模型分类

1.2.2.1 概率模型与非概率模型

- **概率模型**: 决策树、朴素贝叶斯、隐马尔科夫链、条件随机场、概率潜在语义分析、潜在狄利克雷分配、高斯混合模型.
- **非概率模型**: 感知机、支持向量机、 k 近邻、AdaBoost、 k 均值、潜在语义分析以及神经网络都是非概率模型.

Remark

逻辑斯蒂回归(Logistic Regression)既可以看作是概率模型,也可以看作是非概率模型.

条件概率分布和函数是可以相互转化的,条件概率分布最大化后得到函数,函数归一化后得到条件概率分布.因而,这二者的区别不在于输入与输出之间的映射关系,而在于模型的内在结构.

概率模型的代表是概率图模型,概率图模型是联合概率分布由有向图或者无向图表示的概率模型,而联合概率分布可以根据图的结构分解为因子乘积的形式.贝叶斯网络、马尔科夫随机场、条件随机场都是概率图模型.

1.2.2.2 线性模型与非线性模型

机器学习模型,特别是非概率模型,可以分为线性模型和非线性模型.如果函数 $y = f(x)$ 或 $z = g(x)$ 是线性函数,则称该模型是线性模型,否则称该模型是非线性模型.

感知机、线性支持向量机、 k 近邻、 k 均值、潜在语义分析是线性模型,核函数支持向量机、AdaBoost、神经网络是非线性模型.

深度学习实际是复杂神经网络的学习,也就是复杂的非线性模型的学习.

1.2.2.3 参数化模型与非参数化模型

参数化模型假设模型参数的维度固定,模型可以由有限维参数完全刻画;非参数化模型假设模型参数的维度不固定或者无穷大,随着训练数据量的增大而不断增大.

感知机、朴素贝叶斯、逻辑斯蒂回归、 k 均值、高斯混合模型、潜在语义分析、概率潜在语义分析、潜在狄利克雷分配是参数化模型;决策树、支持向量机、AdaBoost、 k 近邻是非参数化模型.

1.2.2.4 生成模型与判别模型

判别式(Discriminative)模型:寻找决策边界,通过该边界将样本划分到对应类别.反映异类数据之间的差异,直接面对预测而设计,学习准确度高.

生成式(Generative)模型:学习每个类别的边界,基于贝叶斯公式计算样本属于每个类的后验概率,一般由最大后验概率确定样本分类.模型能够反映同类数据本身的相似度,包含信息更为丰富,可以用来生成样本.

1.2.3 按算法分类

按照算法分为**在线学习**与**批量学习**.在线学习是指每次接收一个样本进行预测,之后学习模型,并不断重复该操作的机器学习.批量学习一次接受所有数据,学习模型,之后进行预测.

在线学习可以是监督学习,也可以是无监督学习,强化学习本身就具有在线学习的特点,所以以下只考虑在线的监督学习.

学习和预测在一个系统,每次接受一个输入 x_t ,用已有模型给出预测 $\hat{f}(x_t)$,之后得到相应的反馈,即该输入对应的输出 y_t ;系统用损失函数计算两者的差异,更新模型并不断重复.

利用随机梯度下降的感知机学习算法就是在线学习算法,在线学习通常比批量学习更难,很难学到预测精确率更高的模型,因为每次模型更新中,可利用的数据有限.

1.3 方法论三要素:“方法=模型+策略+算法”

1.3.1 模型与假设空间 (Model)

在监督学习过程中,模型就是所有学习的条件概率分布或决策函数.模型的假设空间包含所有可能的条件概率分布或决策函数.假设空间中的模型一般有无穷多个.

Definition 1.3.1

假设空间/参数空间

- 假设空间用 $\mathcal{F}$ 表示, 可以定义为决策函数的集合:

$$\mathcal{F} = \{f|Y = f(X)\}$$

其中, X 和 Y 是定义在输入空间 $\mathcal{X}$ 和输出空间 $\mathcal{Y}$ 上的变量. 这时 $\mathcal{F}$ 通常是由一个参数向量决定的函数族:

$$\mathcal{F} = \{f|Y = f_{\theta}(X), \theta \in \mathbb{R}^n\}$$

- 假设空间也可以定义为条件概率的集合:

$$\mathcal{F} = \{P|P(Y|X)\}$$

其中, X 和 Y 是定义在输入空间 $\mathcal{X}$ 和输出空间 $\mathcal{Y}$ 上的随机变量. 这时 $\mathcal{F}$ 通常是由一个参数向量决定的条件概率分布族:

$$\mathcal{F} = \{P|P_{\theta}(Y|X), \theta \in \mathbb{R}^n\}$$

参数向量 θ 取值于 n 维欧式空间 $\mathbb{R}^n$, 也称为参数空间.

1.3.2 策略与学习准则 (Strategy)

损失函数度量模型一次预测的好坏, 风险函数度量平均意义下模型预测的好坏.

1.3.2.1 损失函数和风险函数

监督学习问题是在假设空间 $\mathcal{F}$ 中选择模型 f 作为决策函数, 对于给定的输入 X , 由 $f(X)$ 给出相应的输出 Y , 这个输出的预测值 $f(X)$ 与真实值 Y 可能一致也可能不一致, 用一个损失函数或代价函数来度量预测错误的程度. 损失函数是 $f(X)$ 和 Y 的非负实值函数, 记作 $L(Y, f(X))$.

常见的损失函数有:

- 0-1 损失函数 $L(Y, f(X)) = \begin{cases} 1 & \text{if } Y \neq f(X) \\ 0 & \text{if } Y = f(X) \end{cases}$
- 平方损失函数(L2) $L(Y, f(X)) = (Y - f(X))^2$
- 绝对损失函数(L1) $L(Y, f(X)) = |Y - f(X)|$
- 对数损失函数 (对数似然损失函数) $L(Y, f(X)) = -\log P(Y|X)$

Definition 1.3.2

期望风险

对于模型的输入、输出 (X, Y) 是随机变量, 遵循联合分布 $P(X, Y)$, 损失函数的期望是

$$R_{\text{exp}}(f) = E_p[L(Y, f(X))] = \int_{\mathcal{X} \times \mathcal{Y}} L(y, f(x)) P(x, y) dx dy$$

这是理论上模型 $f(X)$ 关于联合分布 $P(X, Y)$ 的平均意义下的损失, 称为风险函数或期望损失.

由于 $P(x, y)$ 是不可知的, 所以 $R_{\text{exp}}(f)$ 不可以直接计算, 如果联合分布已知, 那么就可以直接求出条件概率分布 $P(Y|X)$, 也就不需要学习了. 这样一来, 一方面根据期望风险最小学习模型要用到联合分布, 另一方面联合分布是未知的, 所以监督学习就成为了一个病态问题.

Definition 1.3.3

经验风险/经验损失

给定一个训练数据集:

$$T = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$$

模型 $f(X)$ 关于训练数据集的平均损失称为经验风险或经验损失, 记作 R_{emp} :

$$R_{\text{emp}}(f) = \frac{1}{N} \sum_{i=1}^N L(y_i, f(x_i))$$

期望风险 $R_{\text{exp}}(f)$ 是模型关于联合分布的期望损失, 经验风险 $R_{\text{emp}}(f)$ 是模型关于训练样本集的平均损失. 当样本容量 N 趋于无穷时, 经验风险 $R_{\text{emp}}(f)$ 趋于期望风险 $R_{\text{exp}}(f)$. 但由于现实中训练样本数目有限, 甚至很小, 所以用经验风险估计期望风险常常不理想, 要对经验风险进行一定的校正.

Remark

这就是监督学习中的两个基本策略: 经验风险最小化和结构风险最小化. (最小化 Loss 函数以及正则化)

1.3.2.2 经验风险最小化与结构风险最小化

经验风险最小化的策略认为, 经验风险最小的模型是最优的模型, 根据这一策略, 按照经验风险最小化来求最优模型就是求解最优化问题 (其中 $\mathcal{F}$ 是假设空间):

$$\min_{f \in \mathcal{F}} \frac{1}{N} \sum_{i=1}^N L(y_i, f(x_i))$$

当样本容量很小的时候,可能会产生过拟合的现象.结构风险最小化是为了防止过拟合而提出来的策略,在经验风险上加上了表示模型复杂度的正则化项或罚项.

Definition 1.3.4

结构风险、正则化

在假设空间、损失函数以及训练数据集确定的情况下,结构风险的定义是:

$$R_{\text{strm}}(f) = \frac{1}{N} \sum_{i=1}^N L(y_i, f(x_i)) + \lambda J(f)$$

其中, $J(f)$ 为模型的复杂度,是定义在假设空间 $\mathcal{F}$ 上的泛函. $\lambda \geq 0$ 是系数,用以权衡经验风险和模型复杂度.结构风险小需要经验风险与模型复杂度同时小.

Remark

这样,监督学习问题就变成了经验风险或结构风险函数的最优化问题.

1.3.3 算法

特指最优优化代价函数(损失函数)训练模型参数的方法,即求解最优优化问题的算法.

凸优化 ((Quasi) convex optimization): 也称为凸最优化或凸最小化,主要研究定义于凸集中的凸函数的最小化问题.在机器学习中,常把优化目标函数或损失函数设计为凸函数,则很多优化问题都需要通过凸优化算法来求解.

常用的优化算法包括:

- 对偶问题与 Lagrange 乘子法
- 牛顿法 (Newton Method)
- 梯度下降法 (Gradient descent)
- 内点法 (Interior Point Method)

1.4 计算学习理论

本节只讨论二分类的监督学习问题.

1.4.1 打散、中断点、VC 维度

假设空间 H (例如所有可能的线性划分类) 通常包含无数个函数,但在实际训练中,我们仅拥有 m 个有限样本.我们的目标是寻找一个指标,能够不依赖于具体数据分布,仅描述 H 解决问题的“能力上限”.

1. 增长函数 $\Pi_H(m)$: 能力的上限

Definition 1.4.1

增长函数

增长函数描述了模型在 m 个点上能产生的最大分类组合数.

- **定义:** $\Pi_H(m) = \max_{\{x_1, \dots, x_m\} \subseteq \mathcal{X}} |\{(h(x_1), \dots, h(x_m)) | h \in H\}|$
- **物理含义:** 给定 m 个点,模型 H 最多能“解释”出多少种标签排列.
- **取值范围:** $1 \leq \Pi_H(m) \leq 2^m$

2. 打散 (Shattering) 与 中断点 (Break Point)

Definition 1.4.2

打散/中断点

1. 打散 (Shattering): 若对于某组包含 m 个点的集合,模型 H 能实现所有 2^m 种分类方式,即:

$$\Pi_H(m) = 2^m$$

此时称这 m 个点被 H 打散了.这意味着模型在这些点面前是完全“自由”的,没有任何标签组合能难倒它.

2. 中断点 (Break Point) k : 当样本数增加到某个值 k 时,模型 H 开始“力不从心”,没有任何一种包含 k 个点的集合能被 H 完全打散.

- **定义:** 最小的 k ,使得 $\Pi_H(k) < 2^k$.
- **直观理解:** 这是模型复杂度的“天花板”.

3. VC 维度 (Vapnik-Chervonenkis Dimension)

Definition 1.4.3

VC 维是用一个单一数字对上述逻辑的最终封装.

- **定义:** H 能打散的最大样本集的大小.
 - **公式:** $VC(H) = k - 1$
1. 如果 $m \leq VC(H)$, 可能存在某组 m 个点被打散.
 2. 如果 $m > VC(H)$, 任何一组 m 个点都一定不能被打散.

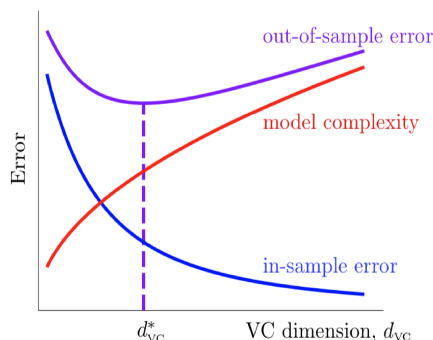


图 1 VC 维用来度量假设 H 的复杂度和学习能力. 当固定样本数 m 时, 随着 VC 维的上升, $E_{in}(g)$ 会不断降低, 同时模型复杂度 Ω 会不断上升. 因此机器学习的模型选择要寻找一个合适的 VC 维使 $E_{out}(g)$ 最小.



图 2 这四个点就没法用线性分割器分割. Beloved CeTz!

1.4.2 Hoeffding 不等式

$$\mathbb{P}[|E_{in}(h) - E_{out}(h)| > \epsilon] \leq 2e^{-2\epsilon^2 m}$$

- 固定假设 (h): 一个具体的模型. 在此语境下, 我们暂不考虑模型选择过程, 仅关注该单一模型的表现.
- 样本量 (m): 训练样本的具体数量.
- 误差容忍度 (ϵ): 一个极小的正数 (如 0.1), 代表我们允许 E_{in} 与 E_{out} 之间存在的最大差距.
- 经验风险 ($E_{in}(h)$): 模型在训练集上的错误率.

$$E_{in}(h) = \frac{1}{m} \sum_{i=1}^m \mathbb{I}(h(x_i) \neq y_i)$$

这是一个**随机变量**, 其数值取决于抽样得到的具体训练数据.

- 期望风险 ($E_{out}(h)$): 又称泛化误差, 指模型在整个数据分布上的真实错误率.

$$E_{out}(h) = P_{(x,y) \sim \mathcal{D}}[h(x) \neq y]$$

对于固定的分布和模型, 这是一个**常数** (真理值), 通常无法直接观测.

- 坏事件 (Bad Event): 指训练误差与真实误差的差距超过了预设的容忍度:

$$|E_{in}(h) - E_{out}(h)| > \epsilon$$

对固定的单个假设 h , 每个样本的预测结果是独立的 0/1 随机变量.

Remark

1. 公式成立的前提是每个样本的预测结果是独立的 (即样本是独立同分布 i.i.d. 采样的). 就像抛硬币, 每一次抛掷互不影响.
2. 0/1 随机变量: 这里的预测结果被看作伯努利试验 (对或错, 1 或 0). Hoeffding 不等式专门处理这种有界随机变量的和.
3. 指数衰减: 对于任何一个固定的模型, 只要数据量 足够大, 训练误差就会以极高的概率紧贴真实误差.

1.4.3 多重检验问题

但学习算法是从 H 中选出一个 h , 这带来了**多重检验问题**: 想象 H 中有 M 个假设, 每个假设都有小概率“碰巧” E_{in} 很低, 算法选中了这个“幸运”的假设, 但它的 E_{out} 其实很高.

对所有假设取并:

$$\mathbb{P}[\exists h \in H : |E_{in}(h) - E_{out}(h)| > \epsilon] \leq 2Me^{-2\epsilon^2 m}$$

- $|H| = M$ 有限时, 右侧仍然可以任意小 (只需 m 足够大)
- $|H| = \infty$ 时, $M = \infty$, 界失效 —— 这就是为什么需要 VC 维.

对于无限假设空间,真正“本质不同”的假设行为只有 $\Pi_H(m)$ 种 (散列数), 于是将 M 替换为增长函数:

$$\mathbb{P}[\exists h \in H : |E_{\text{in}}(h) - E_{\text{out}}(h)| > \varepsilon] \leq 2\Pi_H(2m)e^{-2\varepsilon^2 m/8}$$

再由 **Sauer 引理**, 当 $\Pi_H(m) \leq O(m^{d_{\text{vc}}})$, 多项式增长远远慢于指数衰减, 于是右侧仍然趋于 0.

[b]

假设空间 $\mathcal{H}$, $\text{VC}(\mathcal{H}) = d, \forall m \in \mathbb{N}$,

$$\Pi_{\mathcal{H}}(m) \leq \sum_{i=0}^d \binom{m}{i}$$

Proof: 对 m 和 d 利用数学归纳法. 当 $(m-1, d-1), (m-1, d)$ 都成立的时候, 考虑 m 个样本点 $X = \{x_1, \dots, x_m\}$. 定义两个缩减后的假设空间:

1. $\mathcal{H}_1$: 在 $\{x_1, \dots, x_{m-1}\}$ 上的投影, 显然 $|\mathcal{H}_1| = \Pi_{\mathcal{H}}(m-1) \leq \Phi(m-1, d)$.
2. $\mathcal{H}_2$: 包含那些在前 $m-1$ 个点上表现相同, 但在 x_m 上表现不同的对.

通过计数原理, 可以得到:

$$\Pi_{\mathcal{H}}(m) = |\mathcal{H}_1| + |\mathcal{H}_2|$$

由于 $\mathcal{H}_2$ 在前 $m-1$ 个点中能够“打散”的集合大小减小了 1 (因为它必须在第 m 个点上产生差异), 其有效 VC 维不超过 $d-1$. 因此:

$$|\mathcal{H}_2| \leq \Pi_{\mathcal{H}_2}(m-1) \leq \Phi(m-1, d-1)$$

又 $\binom{n}{k} = \binom{n-1}{k} + \binom{n-1}{k-1}$:

$$\begin{aligned} \Pi_{\mathcal{H}}(m) &\leq \Phi(m-1, d) + \Phi(m-1, d-1) \\ &= \sum_{i=0}^d \binom{m-1}{i} + \sum_{i=0}^{d-1} \binom{m-1}{i} \\ &= \binom{m-1}{0} + \sum_{i=1}^d \left[\binom{m-1}{i} + \binom{m-1}{i-1} \right] \\ &= \binom{m}{0} + \sum_{i=1}^d \binom{m}{i} = \Phi(m, d) \end{aligned}$$

□

[c] 为什么我们要花这么多精力探讨 VC 维度? 如下是 gemini 的一个答案.

VC 维提供了一个分水岭:

有限 VC 维: 模型是 PAC 可学习的 (Probably Approximately Correct). 只要给够数据, 模型最终一定能学到真谛.

无限 VC 维: 模型可以“打散 (Shatter)”任意数量的样本 (比如高斯核 SVM 或深层神经网络在某些条件下). 这意味着模型可以记住任何噪声, 如果不加正则化, 它本质上只是一个极其复杂的“查表程序”, 没有真正的泛化能力.

VC 维的重要性在于它量化了**奥卡姆剃刀 (Occam's Razor)** 原则: 在训练误差相同的情况下, 我们永远应该选择 VC 维更小的模型.

现在很多深度学习的研究 (比如为什么参数量巨大的 Transformer 泛化性依然很好) 本质上都是在挑战或优化传统的 VC 理论边界.

笔者与读者应当相信计算理论是有用的⁻¹

1.5 误差构成与工程评估 (Operational Excellence)

1.5.1 泛化能力

泛化能力和机器学习理论一节有很强的联系.

Definition 1.5.1

泛化误差的数学表达

如果学到的模型是 $\hat{f}$, 那么用这个模型对未知数据预测的误差即为泛化误差:

$$R_{\text{exp}}(\hat{f}) = E_P[L(Y, \hat{f}(X))] = \int_{\mathcal{X} \times \mathcal{Y}} L(y, \hat{f}(x)) P(x, y) dx dy$$

1.5.1.1 泛化误差上界

泛化误差上界通常具有以下性质:

- 是样本容量的函数, 当样本容量增加时, 泛化误差上界趋于 0;

¹可以参考 <https://tangshusen.me/2018/12/09/vc-dimension/>.

- 是假设空间容量的函数,假设空间容量越大,模型就越难学,泛化误差上界就越大.

$R(f)$: 真实误差 (泛化误差). 模型 f 在整个人群 (真实分布) 上的错误率. 这是我们要估计的“真理”, 但看不见.

$\hat{R}(f)$: 经验误差 (训练误差). 模型 f 在训练集 (N 个样本) 上的错误率. 这是我们可以算出来的. 经验误差最小化函数是

$$f_N = \arg \min_{f \in \mathcal{F}} \hat{R}(f)$$

f_N 依赖训练数据集的样本容量 N , 人们更关心的是 f_N 的泛化能力:

$$R(f_N) = E[L(Y, f_N(X))]$$

Example

例如二类分类问题的泛化误差上界. 已知训练数据集 $T = \{(x_1, y_1), \dots, (x_N, y_N)\}$, N 是样本容量, T 是从联合概率分布 $P(X, Y)$ 独立同分布产生的, $X \in \mathcal{R}^n$, $Y \in \{-1, +1\}$. 假设空间是函数的有限集合 $\mathcal{F} = \{f_1, f_2, \dots, f_d\}$, d 是函数个数. 设 f 是从 $\mathcal{F}$ 中选取的函数, 损失函数是 0-1 损失, 关于 f 的期望风险和经验风险分别是

$$R(f) = E[L(Y, f(X))]$$

$$\hat{R}(f) = \frac{1}{N} \sum_{i=1}^N L(y_i, f(x_i))$$

对任意一个函数 $f \in \mathcal{F}$, 至少以概率 $1 - \delta, 0 < \delta < 1$, 以下不等式成立:

$$R(f) \leq \hat{R}(f) + \varepsilon(d, N, \delta)$$

$$\varepsilon(d, N, \delta) = \sqrt{\frac{1}{2N} \left(\log d + \log \frac{1}{\delta} \right)}$$

Remark

不等式右边是泛化误差上界, 第 1 项是训练误差, 训练误差越小, 泛化误差越小; 第 2 项是 N 的单调递减函数, 当 N 趋于无穷时趋于 0; 同时它也是 $\sqrt{\log d}$ 阶的函数, 假设空间 $\mathcal{F}$ 包含的函数越多, 其值越大.

1.5.1.2 证明: 真实误差不会比训练误差大太多

Theorem 1.5.2

Hoeffding 不等式

设 $X_1, X_2, \dots, X_N$ 是独立随机变量, 且 $X_i \in [a_i, b_i], i = 1, 2, \dots, N$;

记 $\bar{X}$ 是 $X_1, X_2, \dots, X_N$ 的经验均值, 即 $\bar{X} = \frac{1}{N} \sum_{i=1}^N X_i$, 则对任意 $t > 0$, 有:

$$P(\bar{X} - E(\bar{X}) \geq t) \leq \exp \left\{ -\frac{2N^2 t^2}{\sum_{i=1}^N (b_i - a_i)^2} \right\}$$

$$P(E(\bar{X}) - \bar{X} \geq t) \leq \exp \left\{ -\frac{2N^2 t^2}{\sum_{i=1}^N (b_i - a_i)^2} \right\}$$

对任意函数 $f \in \mathcal{F}$, $\hat{R}(f)$ 是 N 个独立的随机变量 $L(Y, f(X))$ 的样本均值 $\bar{X}$, $R(f)$ 是随机变量 $L(Y, f(X))$ 的期望值 $E(\bar{X})$.

如果损失函数取值于区间 $[0, 1]$, 即对所有 $i, [a_i, b_i] = [0, 1]$, 那么由 Hoeffding 不等式不难得知, 对于 $\varepsilon > 0$, 有:

$$P(R(f) - \hat{R}(f) \geq \varepsilon) \leq \exp(-2N\varepsilon^2)$$

由于 $\mathcal{F} = \{f_1, f_2, \dots, f_d\}$ 是一个有限集合, 故:

$$\begin{aligned} P(\exists f \in \mathcal{F} : R(f) - \hat{R}(f) \geq \varepsilon) &= P\left(\bigcup_{f \in \mathcal{F}} \{R(f) - \hat{R}(f) \geq \varepsilon\}\right) \\ &\leq \sum_{f \in \mathcal{F}} P(R(f) - \hat{R}(f) \geq \varepsilon) \\ &\leq d \exp(-2N\varepsilon^2) \end{aligned}$$

所以有

$$P(R(f) - \hat{R}(f) < \varepsilon) \geq 1 - d \exp(-2N\varepsilon^2)$$

令

$$\delta = d \exp(-2N\varepsilon^2)$$

则

$$P(R(f) - \hat{R}(f) < \varepsilon) \geq 1 - \delta$$

即至少以概率 $1 - \delta$ 有 $R(f) < \hat{R}(f) + \varepsilon$.

Remark

以上讨论的知识假设空间包含有限个函数情况下的泛化误差上界,对一般的假设空间要找到泛化误差上界并不简单,这里不再介绍.

$$\begin{aligned} \hat{\varepsilon}_1, \dots, \hat{\varepsilon}_k & \text{ i.i.d. } \sim \mathcal{N}(\varepsilon, \theta^2) \\ \mu &= \frac{1}{k} \sum \hat{\varepsilon}_i, \quad z^2 = \frac{1}{k-1} \sum (\hat{\varepsilon}_i - \mu)^2 \\ & \text{则} \\ 1. & \frac{\hat{\varepsilon}_i - \varepsilon}{\theta} \text{ i.i.d. } \sim \mathcal{N}(0, 1) \\ 2. & \frac{(k-1)z^2}{\theta^2} \sim \chi^2(k-1) \\ 3. & \frac{\mu - \varepsilon}{\theta} \sim \mathcal{N}\left(0, \frac{1}{k}\right), \quad \frac{\sqrt{k}(\mu - \varepsilon)}{\theta} \sim \mathcal{N}(0, 1) \\ 4. & \mu \text{ 与 } \theta^2 \text{ 独立} \\ 5. & \text{ 令 } T = \frac{\sqrt{k}(\mu - \varepsilon)/\theta}{\sqrt{((k-1)z^2/\theta^2)/(k-1)}} = \frac{\sqrt{k}(\mu - \varepsilon)}{\theta} \sim t(k-1) \end{aligned}$$

1.5.2 泛化误差分解

1.5.2.1 偏差-方差分解

在机器学习中,算法的泛化误差可分解为三个互不相关的部分: **偏差 (Bias)**、**方差 (Variance)** 和 **噪声 (Noise)**.

- 偏差 (Bias): 期望预测与真实结果的偏离程度,即模型本身的拟合能力.高偏差意味着模型过于简单,是欠拟合的根源.
- 方差 (Variance): 训练集变动引起的学习性能变化,即模型的稳定性.高方差意味着模型对训练集波动敏感,学习了过多噪声,是过拟合的根源.
- 噪声 (Noise): 当前任务能达到的最小泛化误差,即学习任务的上限.由数据集中无法解释的人为疏忽或随机变异产生,不可避免.

直观理解: 方差是“跟自己比”(不同数据集预测结果的散布),偏差是“跟目标比”(预测均值与真值的距离).

以回归任务为例,使用样本数量相同的不同训练集训练出一个模型,然后使用同一个测试用例.设测试用例为 x ,其真实标记为 y ; y_D 是“如果我们在测试点 x 处进行一次实际观测,我们会拿到的那个带有噪声的标签”(而不是训练集里的样本);在 D 上学得的模型预测值为 $\hat{y}_D$.

预测输出的期望: $f(x) = \mathbb{E}_D[\hat{y}_D]$

偏差的平方: $\text{bias}^2 = (f(x) - y)^2$

方差: $\text{var} = \mathbb{E}_D[(\hat{y}_D - f(x))^2]$

训练数据存在噪声,即标记 y_D 与真值 y 有差异,则噪声有:

$$\varepsilon^2 = \mathbb{E}_D[(y_D - y)^2]$$

对期望损失进行展开.首先,引入恒等式:

$$\hat{y}_D - y_D = (\hat{y}_D - f(x)) + (f(x) - y) + (y - y_D)$$

将平方展开:

$$\begin{aligned} \mathbb{E}_D[(\hat{y}_D - y_D)^2] &= \mathbb{E}_D[(\hat{y}_D - f(x))^2] + (f(x) - y)^2 + \mathbb{E}_D[(y_D - y)^2] \\ &\quad + 2\mathbb{E}_D[(\hat{y}_D - f(x))(f(x) - y)] \\ &\quad + 2\mathbb{E}_D[(\hat{y}_D - f(x))(y_D - y)] \\ &\quad + 2\mathbb{E}_D[(f(x) - y)(y_D - y)] \end{aligned}$$

分析交叉项:

1. 第一项 $2\mathbb{E}_D[(\hat{y}_D - f(x))(f(x) - y)] = 2(f(x) - y)\mathbb{E}_D[\hat{y}_D - f(x)] = 0$ (由 $f(x)$ 的定义)
2. 第二项 $2\mathbb{E}_D[(\hat{y}_D - f(x))(y_D - y)] = 2\mathbb{E}_D[\hat{y}_D - f(x)]\mathbb{E}_D[y_D - y] = 0$ (噪声与模型独立)
3. 第三项 $2\mathbb{E}_D[(f(x) - y)(y_D - y)] = 2(f(x) - y)\mathbb{E}_D[y_D - y] = 0$ (噪声期望为零)

因此,仅保留三项:

$$\begin{aligned}\mathbb{E}_D[(\hat{y}_D - y_D)^2] &= \mathbb{E}_D[(\hat{y}_D - f(x))^2] + (f(x) - y)^2 + \mathbb{E}_D[(y_D - y)^2] \\ &= \text{var}(x) + \text{bias}^2(x) + \varepsilon^2\end{aligned}$$

即泛化误差分解:

$$E(f; D) = \text{bias}^2(x) + \text{var}(x) + \varepsilon^2$$

结论: 泛化误差 = 偏差² + 方差 + 噪声.

1.5.2.2 偏差-方差权衡

1. 训练初期 (或模型简单时): 学习器拟合能力弱, 偏差主导泛化误差.
2. 训练充足 (或模型复杂时): 学习器拟合能力强, 数据微小干扰被记录, 方差主导泛化误差.
3. 最优平衡点: 存在合适模型复杂度 (或 VC 维), 使偏差与方差之和最小.

实际工程中, 需在以下三个维度间寻求平衡: 模型复杂度 (VC 维)、数据规模与质量、泛化误差.

- 缓解过拟合 (降方差): 增加训练数据、正则化约束、剪枝、Dropout、集成学习.
- 缓解欠拟合 (降偏差): 增加模型参数、特征工程、减少正则化惩罚.

1.5.3 数据集管理与评估方法

1.5.3.1 留出法

从 D 中选择一部分作为训练集 S, 留出 (hold out) 一部分作为测试集 T.

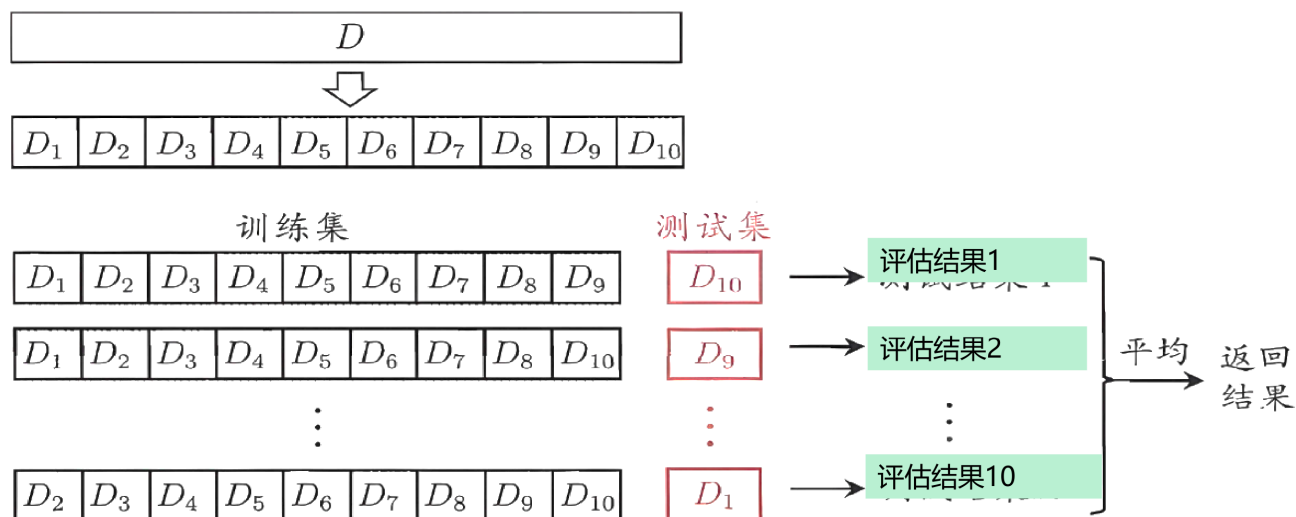
1.5.3.2 分层采样

根据数据集的某些特征或类别将数据划分为多个子集 (即“层”), 然后在每个层中进行采样, 以确保每个子集中都有足够的样本被选中, 保持样本的多样性和代表性 (一致性)

1.5.3.3 交叉验证

将数据集 D 划分为 k 个大小相同的互斥子集. 采用分层采样的方法使得这些子集的数据分布一致. 每次用 k-1 个子集的并集构成训练集, 余下的那个子集作为测试集.

共有 k 种训练集/测试集划分的情况, 从而可进行 k 次交叉验证 (k-fold Cross-Validation), 最终返回 k 次评估结果的均值、标准差. 如果模型性能在不同折之间存在较大差异, 可能意味着数据集存在某种不平衡或模型对特定的样本过于敏感, 则需要进一步的数据清洗或模型调整.



10折交叉验证示意图

图 3

10折交叉验证示意图. 提问: k折交叉验证会得到 k个参数不同的模型, 那么最终使用哪一个呢?

[a] 都不用. 基于平均结果, 确定哪种模型配置 (如不同的超参数、特征选择方法、假设类型等) 的性能最好; 找到最优参数后, 在全量数据上会重新训练一次然后再使用.

1.5.3.4 留一法 (Leave-One-Out, LOO)

假定数据集 D 中包含 m 个样本, 令 k=m.

留一法不受随机样本划分的方式影响,因为 m 个样本只有唯一的方式划分为 m 折; 留一法使用的训练集与初始数据集相比只少了一个样本,使得在绝大多数情况下,留一法中被实际评估的模型与最终模型很相似.(这太蠢了.)

1.5.3.5 自助采样法

又称“有放回采样”：给定包含 m 个样本的数据集 D , 每次随机从 D 中挑选一个样本, 将其副本放入 D' (该样本自身仍在数据集 D 中, 使得下次采样时仍有可能被选中). 重复执行 m 次, 就得到了包含 m 个样本的数据集 D' .

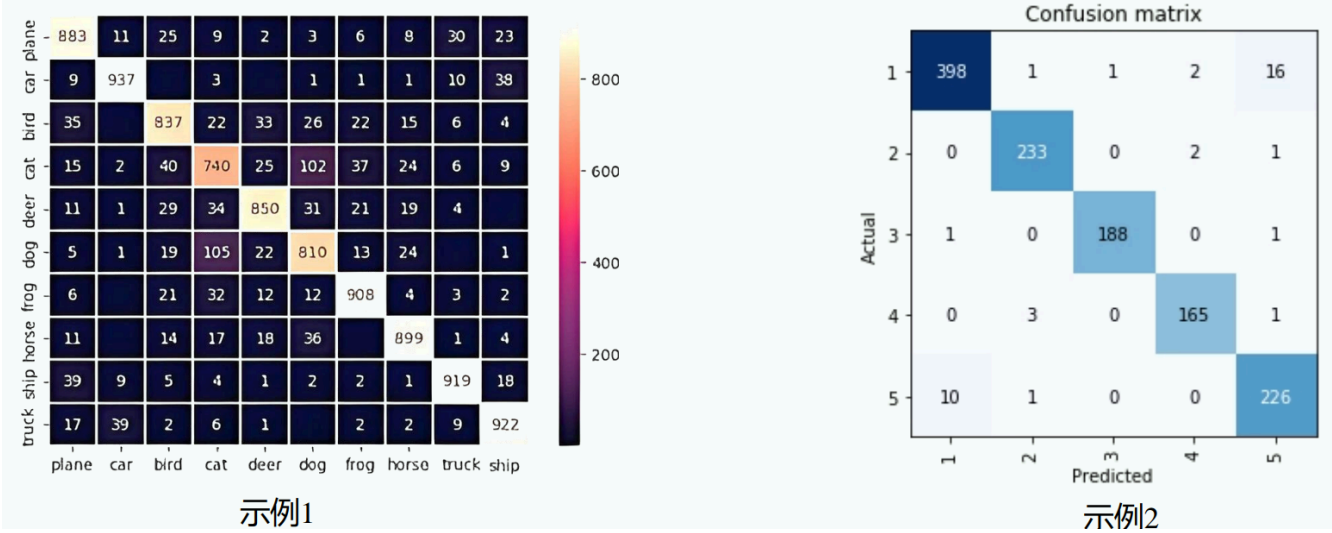
以 D' 作为训练集, $D \setminus D'$ 用作测试集. 这样的测试结果, 被称为“包外估计” (out-of-bag estimate) .

优点: 对于数据集较小、难以有效划分训练/测试集时很有效. 此外, 自助法能从初始数据集中产生多个不同的训练集, 有利于集成学习方法.

缺点: 自助法改变了初始数据集的分布, 会引入估计偏差

1.5.4 性能度量

1. 混淆矩阵 (Confusion Matrix)



下表展示了预测值与真实值之间的四种关系：

	真正正类 (Positive)	真实负类 (Negative)
预测正类 (Positive)	TP (True Positive) 真正：预测为正,实际也为正	FP (False Positive) 假正：预测为正,实际为负
预测负类 (Negative)	FN (False Negative) 假负：预测为负,实际为正	TN (True Negative) 真负：预测为负,实际也为负

表 1 分类结果四种情形定义

2. 核心评价指标定义

精确率 (Precision, P)： 预测为正类的样本中,真正为正类的比例.衡量的是“预测得准不准”。

$$P = \frac{TP}{TP + FP}$$

召回率 (Recall, R)： 实际为正类的样本中,被预测为正类的比例.衡量的是“找得全不全”。

$$R = \frac{TP}{TP + FN}$$

F1 分数 (F1-Score)： 精确率和召回率的调和平均数,用于综合评价模型的性能,尤其在类别不平衡时非常有用。

$$F_1 = 2 \cdot \frac{P \cdot R}{P + R} = \frac{2 TP}{2 TP + FP + FN}$$

[e] 当然有 Fb-score,也就是

$$F_b = \frac{(1 + b)PR}{bP + R}$$

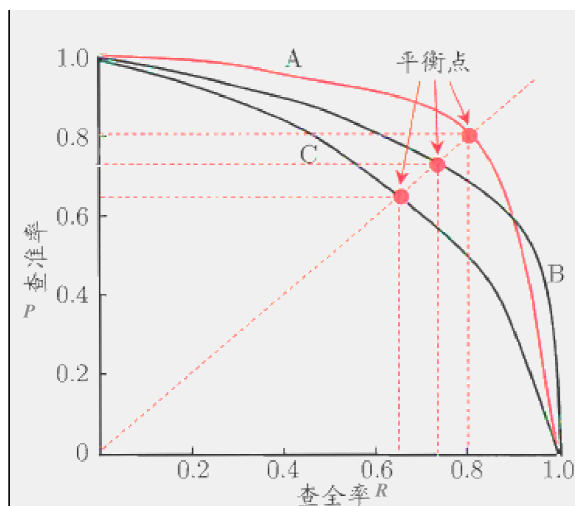
在有一堆混淆矩阵的时候,我们希望先平均再算总的 F1score,这里叫做 macro-F1.

Remark

- $TP + FP$: 所有被模型预测为“正”的样本总量.
- $TP + FN$: 样本集中实际存在的“正”样本总量.

3. PR,ROC 曲线

P-R 曲线：精度（Precision）与召回率（Recall）的曲线.



- 若一个学习器C的P-R曲线被另一个学习器A的P-R曲线完全**包住**，则称：A的性能优于C。若A和B的曲线发生了交叉，则谁的曲线下的面积大，谁的性能更优。
- “**平衡点**”（Break-Event Point, 简称BEP），即当 $P=R$ 时的取值，平衡点的取值越高，性能更优

图4 对测试集的 N 个样本,模型输出概率 $\{p_i\}$.然后设定一个阈值,大于之的认为是阳性,反之是阴性.这样我们能得到(这个阈值下的)P 和 R,把阈值从 0 变到 1,画出的所有 (R, P) 点连线.

ROC (Receiver Operating Characteristic) 曲线：接受者操作特性曲线.与 P-R 曲线类似，逐次设置阈值把样本预测为“正例”和“反例”，计算当前的真正例率和假正例率.ROC 曲线其与坐标轴围成的面积 AUC 越大性能越优.

- 纵轴：是“真正例率”（True Positive Rate, 简称TPR）；

$$TPR = \frac{TP}{TP + FN}, \quad (\text{灵敏度})$$

- 横轴为“假正例率”（False Positive Rate, 简称FPR）；

$$FPR = \frac{FP}{TN + FP} \cdot (1 - \text{特异性})$$

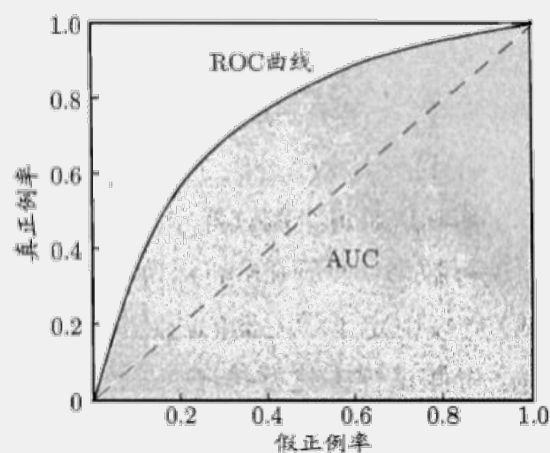


图5 同上,我们对测试集的 N 个样本描绘 N 个点.每当你把阈值跨过某一个样本的得分时,TP (真阳性) 或 FP (假阳性) 的数量就会发生变化,从而在图上产生一个新的点.

[f] 直观理解 ROC 曲线,就是模型区分能力: 区分能力越强,越能把正例设置为接近 1,反例设置为接近 0,那如何取阈值都不重要了.

4. 回归任务的性能度量指标

用于衡量模型预测值 $\hat{y}$ 与真实标签 y 之间的差异.

- MAE (平均绝对误差)

$$MAE = \frac{1}{n} \sum_{i=1}^n |\hat{y}_i - y_i|$$

特点：对离群点（Outliers）具有较好的容忍度.

- MSE (均方误差)

$$MSE = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

特点：放大较大误差,利于数学求导.

- RMSE (均方根误差)

$$\text{RMSE} = \sqrt{\text{MSE}}$$

特点：量纲与原始数据一致,更直观.

[g]

为什么回归任务中两者形式一致,而分类任务不同?

联系：两者都描述“偏差”.在回归中,我们希望最小化的目标 (Loss) 通常就是我们评估的标准 (Metric) .

区别：

- **可微性：**损失函数 (如 **L2 Loss**) 必须在整个参数空间内可导,以便通过梯度下降训练; 性能度量 (如准确率 Accuracy) 可以不可微.
- **目的：**Loss 用于“训练时收敛”,Metric 用于“训练后评价”.

分类预测的是概率 (连续),但评估的是类别 (离散).所以 Loss 用交叉熵 (衡量概率分布距离),Metric 用准确率 (衡量类别对错).在回归分析中,数据总体的波动可以分解为“模型能解释的”和“模型不能解释的”两部分.

$$\text{SST} = \text{SSR} + \text{SSE}$$

SST (Total Sum of Squares),总离差平方和: $\text{SST} = \sum (y_i - \bar{y})^2$, 反映原始数据总体的波动情况.

SSR (Regression Sum of Squares),回归平方和: $\text{SSR} = \sum (\hat{y}_i - \bar{y})^2$,反映模型捕捉到的“有规律”的变化.

SSE (Error Sum of Squares),残差平方和: $\text{SSE} = \sum (y_i - \hat{y}_i)^2$ 反映模型无法解释的“随机噪声”或误差.

判定系数 R^2 (R-Square):

$$R^2 = \frac{\text{SSR}}{\text{SST}} = 1 - \frac{\text{SSE}}{\text{SST}}$$

取值范围： [0, 1]

物理意义： R^2 越接近 1,表示模型解释数据变动的能力越强,拟合效果越好.

[h]

1. 为什么 L2 损失常加系数 $\frac{1}{2}$?

为了在求导时抵消掉平方项降下来的系数 2,使结果变为 1 (即 $(\frac{1}{2}x^2)' = x$),纯粹为了数学表达的简洁.

2. 交叉项为什么为 0?

在最小二乘法 (OLS) 的框架下,通过极值条件可以证明残差 ϵ 与预测值 $\hat{y}$ 是正交的 (不相关),因此交叉项相加为 0,方差分解公式才成立

1.6 模型比较检验

无非两种;1.列联表,看西瓜 P41.

2.t 分布.

- 模型A通过多次留出法或是交叉验证法等进行多次测试,会得到

k 个测试错误率 $\hat{\epsilon}_1, \hat{\epsilon}_2, \dots, \hat{\epsilon}_k$, 则平均测试错误率 μ 和方差 σ^2 为

$$\mu = \frac{1}{k} \sum_{i=1}^k \hat{\epsilon}_i, \quad \sigma^2 = \frac{1}{k-1} \sum_{i=1}^k (\hat{\epsilon}_i - \mu)^2$$

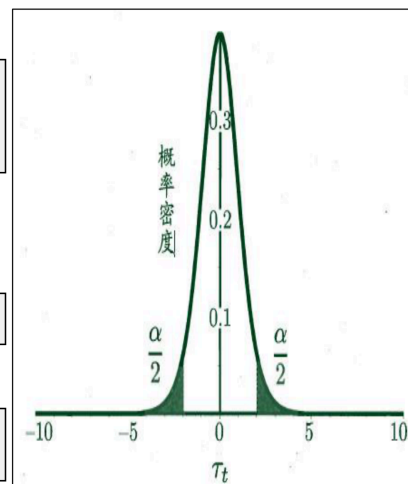
- 将这 k 个测试错误率看作泛化错误率 ϵ_0 的独立采样, 用“**t检验**”检验模型的泛化错误率是否满足:

$$H_0: \mu = \epsilon_0 \leftrightarrow H_1: \mu \neq \epsilon_0 \quad \text{双边检验}$$

- 构造统计检验量 τ_t 服从自由度为 $k-1$ 的t分布, 确定拒绝域

$$\tau_t = \frac{\sqrt{k}(\mu - \epsilon_0)}{\sigma} \sim t(k-1) \quad \text{拒绝域}\{|\tau_t| > \tau_{t, \alpha/2}(k-1)\}$$

显著性水平



2 线性回归

2.1 线性判别函数与感知机模型

2.1.1 基本概念

Definition 2.1.1

判别函数

判别函数 (discriminant function) 是用于对模式样本进行分类的准则函数. 它通过一些函数来表示和鉴别某个特征矢量属于哪个类别. 这些函数不是集群在特征空间形状的数学描述, 而是描述某一位置矢量属于某个类别的情况, 如属于某个类别的条件概率.

线性判别函数的一般形式为:

$$d(X) = w_1x_1 + w_2x_2 + \dots + w_nx_n + w_{n+1} = W_0^T X + w_{n+1}$$

如果令 $X = [x_1, x_2, \dots, x_n, 1]^T$ 为增广模式向量, $W = [w_1, w_2, \dots, w_n, w_{n+1}]^T$ 为增广权向量, 那么有:

$$d(X) = W^T X$$

对于两类的情况, 可以根据 $d(X)$ 的正负完成分类, 如果 $d(X) = 0$, 则属于不可判别情况, 可以随机分类, 也可以拒绝分类. 下面介绍对于多种分类问题, 该如何转化为多个二元分类问题:

1. $w_1/\bar{w}_1$ 是非两分法 (One-vs-Rest, OvR)

- **基本思路:** “每一个类别都问一句: 是不是这个类?” 为每一个类别 w_i 训练一个线性判别函数 $g_i(x)$, 这个判别函数的作用是将属于 w_i 的样本和所有不属于 w_i 的样本 (即 $\bar{w}_i$) 区分开.
- **判别规则:** 对于一个未知样本 x , 我们将其输入到所有 M 个判别函数中. 如果存在唯一的某个判别函数 $g_k(x)$ 的输出值大于 0, 其他所有判别函数的输出值都小于或等于 0, 那么我们就把 x 归为类别 w_k .

[i] 以下两种情况无法有效进行分类 (IR 区)

- **无响应:** 所有 M 个判别函数的输出都小于等于 0. 即, 没有任何一个类别“认领”这个样本.
- **多响应:** 有多个 (两个或以上) 判别函数的输出大于 0. 即, 有多个类别都“声称”这个样本属于自己.

2. w_i/w_j 成对两分法 (One-vs-One, OvO)

- 这种方法也常常被称为“一对一”法.
- **基本思路:** 为每两个类别的组合都训练一个线性判别函数. 例如: 为了区分 w_i 和 w_j , 我们训练一个判别函数 $g_{ij}(x)$. 共有 m 个类别, 则需要训练 $\frac{M(M-1)}{2}$ 个判别函数.
- **判别规则:** 采用“投票法”. 对于一个未知样本 x , 我们用所有的判别函数 $g_{ij}(x)$ 对它进行分类.
 - 若为正则投给 w_i .
 - 若为负则投给 w_j .
 - 最后统计每个类别获得的总票数, 将 x 归为得票最多的那个类别.

[j]

- 优点: 每个判别函数只负责区分两个类别, 因此通常比 OvR 中的判别函数更简单、更容易训练.
- 缺点: 需要训练的分类器数量随类别数急剧增加 $O(M^2)$, 计算和存储成本更高. 同样可能存在不确定区域 (IR 区) 例如当两个或多个类别获得相同的最高票数时.

3. w_i/w_j 成分两分法 (无 IR 区)

- 这是对第二种方法的改进, 旨在消除不确定区域.
- **基本思路:** 和方法二一样, 也需要为每一对类别训练一个判别函数 $g_{ij}(x)$. 但决策规则不同.
- **判别规则:** 使用连续的判别函数值而不是简单的“投票”.
 - 对于一个未知样本 x , 我们计算它对于某个类别 w_i 的“总得分”. 更常用的基于函数值大小的连续形式 (最大化判别函数值):

$$\text{若 } g_k(x) = \max_i g_i(x), \text{ 则判定 } x \in w_k$$

这里需要将成对的判别函数 g_{ij} 构成一个针对每个类别 w_i 的单一判别函数 g_i , 这可以通过一定的数学方法实现. 最终的决策是选择使 $g_i(x)$ 值最大的那个类别 i .

[k] 这里面提到的评分函数,是依靠感知机算法、向量机算法等方法先训练出二元分类器,接着再构造出每个类别的综合评分函数.这里会有两种常用方法:

1. 直接加和(简单有效)

$$g_i(x) = \frac{1}{M-1} \sum_{j \neq i} g_{ij}(x)$$

2. 先用 sigmoid 函数将输出转化为概率(鲁棒性好)

$$p_{ij}(x) = \frac{1}{1 + e^{-g_{ij}(x)}}$$

然后计算平均支持度:

$$g_i(x) = \frac{1}{M-1} \sum_{j \neq i} p_{ij}(x)$$

2.1.1.1 线性判别函数求解

考虑梯度求解.考察一个准则函数 $J(W, X)$, 在 J 的方向上更新梯度.则

$$W^{k+1} = W^k + c(-\nabla J) = W^k - c \left[\frac{\partial J(W, X)}{\partial W} \right]_{W=W^k}$$

[l] PPT 里没考虑 batchnorm,或者, batch=1 下面考虑一个例子.

2.1.1.2 固定增量法

$J(W, X) = \frac{1}{2}(|W^T X| - W^T X)$.

$$\begin{aligned} \frac{\partial(W^T X)}{\partial(W)} &= \frac{\partial}{\partial(W)} \left(\sum_{i=1}^n w_i x_i + w_{n+1} \right) \\ &= \left[\dots, \frac{\partial}{\partial(w_k)} \left(\sum_{i=1}^n w_i x_i + w_{n+1} \right), \dots \right]^T \\ &= [x_1, \dots, x_k, \dots, x_n, 1]^T = X \end{aligned}$$

[m]

$$\frac{dX}{dX^T} = \frac{dX^T}{dX} = I_{n \times n}$$

$$\therefore \frac{\partial(W^T X)}{\partial(W)} = I_{X_{(n+1) \times 1}} = X_{(n+1) \times 1}$$

当 $W^T X \leq 0$ 时, $\frac{\partial(|W^T X|)}{\partial(W)} = \frac{\partial(-W^T X)}{\partial(W)} = -X$

$$\Rightarrow \frac{\partial(|W^T X|)}{\partial(W)} = [\text{sgn}(W^T X)] \cdot X$$

[n] 其中

$$\text{sgn}(W^T X) = \begin{cases} +1 & \text{若 } W^T X > 0 \\ -1 & \text{若 } W^T X \leq 0 \end{cases}$$

$$\therefore \nabla J = \frac{\partial J(W, X)}{\partial(W)} = \frac{1}{2} [X \text{sgn}(W^T X) - X]$$

$$W(k+1) = W(k) - c \frac{1}{2} [X \text{sgn}(W^{T(k)} X) - X]$$

$$= W(k) + \begin{cases} 0 & \text{若 } W^{T(k)} X > 0 \\ cX & \text{若 } W^{T(k)} X \leq 0 \end{cases}$$

这就是固定增量算法.

[6] 可以看出,神经网络中的感知器算法是梯度方法的特例.梯度法是将感知器算法中联立不等式求解 W 的问题,转换为求目标函数极小值的问题.

分类算法是通过模式样本来确定判别函数的系数,因此必须采用有代表性的数据,这样训练出来的判别函数,能合理反映模式数据的区隔情况.

2.1.2 回归与分类

利用 sign 函数、sigmoid 函数.对数几率回归看下一章.

2.1.3 线性判别分析 (LDA)

线性判别分析 (Linear Discriminant Analysis, LDA) 主要用于降维和分类,关注“投影后不同类别的数据分得最开”.思想:

- **类内聚集**: 同一种类别的样本,投影后应该尽可能聚在一起;
- **类间分离**: 不同类别的样本中心,投影后应该尽可能离得远.

假设我们将数据投影到方向向量 w 上,对于二分类问题,我们希望找到一个 w ,使得投影后的两个类别的均值之差 $|\mu'_1 - \mu'_2|^2$ 最大,同时每个类内部的方差 $s_1'^2 + s_2'^2$ 最小.

2. 数学定义与符号

假设数据集 $D = \{(x_i, y_i)\}$, 其中 $x_i \in \mathbb{R}^d$.

- **均值向量**: 第 i 类样本的原始均值向量为 μ_i .
- **投影后的均值**: 投影方向为 w , 则投影后的均值为 $\mu'_i = w^T \mu_i$.
- **投影后的方差**: 第 i 类样本投影后的方差为 $\sigma_i'^2$.

为了实现 LDA 的目标,我们定义 类内散度矩阵 S_w (每个样本到自己类中心的距离,描述类内部有多散) 和 类间散度矩阵 S_b (描述两个类中心相差多远) ..

对于二分类问题:

$$S_w = \Sigma_1 + \Sigma_2 = \sum_{x \in C_1} (x - \mu_1)(x - \mu_1)^T + \sum_{x \in C_2} (x - \mu_2)(x - \mu_2)^T$$

$$S_b = (\mu_1 - \mu_2)(\mu_1 - \mu_2)^T$$

3. 目标函数: 广义瑞利商

我们要最大化“类间距离”与“类内紧凑度”的比值.定义目标函数 $J(w)$:

$$J(w) = \frac{|\mu'_1 - \mu'_2|^2}{\sigma_1'^2 + \sigma_2'^2}$$

将投影变换 w 代入展开,分子 (类间距离) 变为:

$$\begin{aligned} (w^T \mu_1 - w^T \mu_2)^2 &= (w^T (\mu_1 - \mu_2)) ((\mu_1 - \mu_2)^T w) \\ &= w^T (\mu_1 - \mu_2) (\mu_1 - \mu_2)^T w \\ &= w^T S_b w \end{aligned}$$

分母 (类内散度) 变为:

$$\sigma_1'^2 + \sigma_2'^2 = w^T \Sigma_1 w + w^T \Sigma_2 w = w^T (\Sigma_1 + \Sigma_2) w = w^T S_w w$$

因此,LDA 的优化目标即为广义瑞利商:

$$J(w) = \frac{w^T S_b w}{w^T S_w w}$$

4. 公式推导与求解

找 w 使得 $J(w)$ 最大.这是一个约束优化问题.固定分母 $w^T S_w w = 1$,则问题转化为:

$$\min_w (-w^T S_b w) \quad \text{s.t.} \quad w^T S_w w = 1$$

构造拉格朗日函数:

$$\mathcal{L}(w, \lambda) = -w^T S_b w + \lambda (w^T S_w w - 1)$$

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial w} &= -2S_b w + 2\lambda S_w w = 0 \\ \Rightarrow S_b w &= \lambda S_w w \end{aligned}$$

若 S_w 可逆,两边同乘 S_w^{-1} :

$$S_w^{-1} S_b w = \lambda w$$

这表明: 最佳投影方向 w 是矩阵 $S_w^{-1} S_b$ 的最大特征值对应的特征向量.

5. 二分类的特例（解析解）

在二分类中,我们有一个特殊的性质: $S_b w$ 的方向其实是固定的. 由定义 $S_b = (\mu_1 - \mu_2)(\mu_1 - \mu_2)^T$, 代入特征值方程:

$$S_w^{-1}(\mu_1 - \mu_2)(\mu_1 - \mu_2)^T w = \lambda w$$

注意 $(\mu_1 - \mu_2)^T w$ 是一个标量（实数）,不妨记为 k . 方程变为:

$$S_w^{-1}(\mu_1 - \mu_2)k = \lambda w$$

[p] 注意这里并不是说 $(\mu_1 - \mu_2)^T w$ 是不变的,只是说他并不是一个向量. 由于我们需要的是 w 的方向,标量系数 k 和 λ 不影响方向,因此可以直接得出:

$$w \propto S_w^{-1}(\mu_1 - \mu_2)$$

这就是二分类 LDA 最常用的快速计算公式,无需进行特征值分解. 用 S_w^{-1} 去除类内噪声,压低类内方差大的方向,“修正”类中心连线 $(\mu_1 - \mu_2)$. LDA 就是在做信噪比最大化.

Algorithm 2.1.2

输入: 数据集 $D = \{(x_1, y_1), \dots, (x_m, y_m)\}$, 其中样本 $x_i \in \mathbb{R}^n$, 类别 $y_i \in \{C_1, \dots, C_K\}$; 目标维度 d' (满足 $d' \leq K - 1$).

输出: 投影矩阵 W 及降维后的样本.

1. **计算均值:** 计算每个类别的均值向量 μ_k 以及所有样本的全局均值 μ .
2. **构造散度矩阵:**
 - 类内散度矩阵: $S_w = \sum_{k=1}^K \sum_{x \in C_k} (x - \mu_k)(x - \mu_k)^T$
 - 类间散度矩阵: $S_b = \sum_{k=1}^K N_k (\mu_k - \mu)(\mu_k - \mu)^T$
3. **特征值分解:** 求解矩阵 $S_w^{-1} S_b$ 的特征值和特征向量,即满足 $S_w^{-1} S_b w = \lambda w$.
4. **构造投影矩阵:** 将特征值从大到小排序,选取前 d' 个最大特征值对应的特征向量 $w_1, \dots, w_{d'}$, 组成矩阵 $W = [w_1, \dots, w_{d'}]$.
5. **数据转换:** 对于任意样本 x , 其降维后的表示为 $z = W^T x$.

Example

1. 数据集准备

假设存在两类数据 C_1 和 C_2 , 每类包含 2 个样本:

$$\begin{aligned} C_1 : x_1 &= \begin{pmatrix} 2 \\ 2 \end{pmatrix}, \quad x_2 = \begin{pmatrix} 3 \\ 1 \end{pmatrix} \\ C_2 : x_3 &= \begin{pmatrix} 4 \\ 4 \end{pmatrix}, \quad x_4 = \begin{pmatrix} 5 \\ 5 \end{pmatrix} \end{aligned}$$

2. 计算均值向量

首先计算每一类样本的均值中心 μ :

$$\begin{aligned} \mu_1 &= \frac{1}{2}(x_1 + x_2) = \frac{1}{2} \left(\begin{pmatrix} 2 \\ 2 \end{pmatrix} + \begin{pmatrix} 3 \\ 1 \end{pmatrix} \right) = \begin{pmatrix} 2.5 \\ 1.5 \end{pmatrix} \\ \mu_2 &= \frac{1}{2}(x_3 + x_4) = \frac{1}{2} \left(\begin{pmatrix} 4 \\ 4 \end{pmatrix} + \begin{pmatrix} 5 \\ 5 \end{pmatrix} \right) = \begin{pmatrix} 4.5 \\ 4.5 \end{pmatrix} \end{aligned}$$

3. 计算类内散度矩阵 S_w

类内散度矩阵定义为 $S_w = S_1 + S_2$, 其中 $S_i = \sum_{x \in C_i} (x - \mu_i)(x - \mu_i)^T$.

首先计算类别 1 的散度 S_1 . 将样本中心化:

$$\begin{aligned} x_1 - \mu_1 &= \begin{pmatrix} 2 - 2.5 \\ 2 - 1.5 \end{pmatrix} = \begin{pmatrix} -0.5 \\ 0.5 \end{pmatrix} \\ x_2 - \mu_1 &= \begin{pmatrix} 3 - 2.5 \\ 1 - 1.5 \end{pmatrix} = \begin{pmatrix} 0.5 \\ -0.5 \end{pmatrix} \end{aligned}$$

计算外积之和:

$$\begin{aligned} S_1 &= \begin{pmatrix} -0.5 \\ 0.5 \end{pmatrix} \begin{pmatrix} -0.5 \\ 0.5 \end{pmatrix}^T + \begin{pmatrix} 0.5 \\ -0.5 \end{pmatrix} \begin{pmatrix} 0.5 \\ -0.5 \end{pmatrix}^T \\ &= \begin{pmatrix} 0.5 & -0.5 \\ -0.5 & 0.5 \end{pmatrix} \end{aligned}$$

接着计算类别 2 的散度 S_2 . 将样本中心化:

$$x_3 - \mu_2 = \begin{pmatrix} -0.5 \\ -0.5 \end{pmatrix}, \quad x_4 - \mu_2 = \begin{pmatrix} 0.5 \\ 0.5 \end{pmatrix}$$

计算外积之和:

$$\begin{aligned} S_2 &= \begin{pmatrix} -0.5 \\ -0.5 \end{pmatrix} \begin{pmatrix} -0.5 \\ -0.5 \end{pmatrix}^T + \begin{pmatrix} 0.5 \\ 0.5 \end{pmatrix} \begin{pmatrix} 0.5 \\ 0.5 \end{pmatrix}^T \\ &= \begin{pmatrix} 0.5 & 0.5 \\ 0.5 & 0.5 \end{pmatrix} \end{aligned}$$

总的类内散度矩阵为:

$$S_w = S_1 + S_2 = \begin{pmatrix} 0.5 & -0.5 \\ -0.5 & 0.5 \end{pmatrix} + \begin{pmatrix} 0.5 & 0.5 \\ 0.5 & 0.5 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$$

这里计算出的 S_w 恰好为单位矩阵 I .

4. 计算最佳投影方向 w

根据 Fisher 判别准则的解析解:

$$w = S_w^{-1}(\mu_1 - \mu_2)$$

先计算均值差:

$$\mu_1 - \mu_2 = \begin{pmatrix} 2.5 \\ 1.5 \end{pmatrix} - \begin{pmatrix} 4.5 \\ 4.5 \end{pmatrix} = \begin{pmatrix} -2 \\ -3 \end{pmatrix}$$

由于 S_w 是单位矩阵, 其逆矩阵 S_w^{-1} 也是单位矩阵, 代入公式得:

$$w = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} -2 \\ -3 \end{pmatrix} = \begin{pmatrix} -2 \\ -3 \end{pmatrix}$$

结论

计算得到的最佳投影方向向量为 $w = (-2, -3)^T$. 这表明将数据投影到斜率为 $\frac{3}{2}$ 的直线上时, 能最大程度地将这两类数据区分开.

2.2 线性回归

2.2.1 基本概念

Definition 2.2.1

线性回归

线性回归 (linear regression) 是利用称为线性回归方程的最小二乘函数对一个或多个自变量和因变量之间关系进行建模的一种回归分析. 这种函数是一个或多个称为回归系数的模型参数的线性组合. 只有一个自变量的情况称为简单回归, 大于一个自变量情况的叫做多元回归 (multivariable linear regression).

给定数据集 $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$, 样本空间由 d 维向量张成, $x_i = (x_{i1}, x_{i2}, \dots, x_{id})^T$, $y_i \in \mathbb{R}$, 则:

线性回归方程为:

$$\hat{y}_i = g(x_i) = x_i^T w + w_0$$

$\hat{y}_i$ 是回归预测值, $w = (w_1, \dots, w_d)^T$ 和 w_0 是模型的参数.

最小化 MSE, 求解最优参数:

$$w^* = \arg \min_w \sum_{i=1}^m (\hat{y}_i - y_i)^2 = \arg \min_w (y - Xw)^T (y - Xw)$$

这里直接令

$$L(w) = (y - Xw)^T (y - Xw)$$

求导得

$$\begin{aligned} \frac{\partial L(w)}{\partial w} &= 2X^T(Xw - y) = 0 \\ w^* &= (X^T X)^{-1} X^T y \end{aligned}$$

所以得到

$$\hat{y}_i = g(x_i) = x_i^T (X^T X)^{-1} X^T y$$

这便是最小二乘法. 若 $X^T X$ 为满秩矩阵,则 $(X^T X)^{-1}$ 可求,最优参数 w^* 可直接解出,但是往往会遇到样本维数远大于样本个数(此时行数小于列数,一定不满秩)的情况,比如图像样本,则会导致 $X^T X$ 不满秩.若不可逆,则采用梯度下降法(后续展开)来训练参数.

Remark

这里的 w 是把 $x_i^T w + w_0$ 中的 w 与 w_0 结合成一个相同的向量了,这样形式更加简洁.
同时 X 矩阵也增加了一个全是 1 的列.

[q] 方法一: 展开求导法 (Standard Expansion)

这是最稳健的方法,核心是将矩阵运算展开成多项式形式,类比标量求导. 展开矩阵乘法: 利用转置性质 $(A - B)^T = A^T - B^T$,将式子展开:

$$\begin{aligned} L(w) &= (y^T - w^T X^T)(y - Xw) \\ &= y^T y - \underbrace{y^T Xw}_{\text{标量 A}} - \underbrace{X^T y}_{\text{标量 B}} + w^T X^T Xw \end{aligned}$$

合并同类项: 注意 $y^T Xw$ 的结果是一个实数(标量).实数的转置等于它自身:

$$(y^T Xw)^T = w^T X^T y$$

因此中间两项相等,可以合并:

$$L(w) = y^T y - 2w^T X^T y + w^T X^T Xw$$

逐项求导: 利用矩阵求导公式: $\nabla_{\{w\}}(w^T a) = a$ (类比 ax' 的导数为 a) $\nabla_{\{w\}}(w^T A w) = 2Aw$ (类比 ax^2 的导数为 $2ax$)
得: $\frac{\partial L(w)}{\partial w} = 0 - 2X^T y + 2X^T Xw$ 方法二: 链式法则法 (Chain Rule) 这是更快捷的方法,将 $(y - Xw)$ 看作一个整体,避免繁琐的展开. 令中间变量 $z = y - Xw$,则 $L(w) = z^T z$. 根据复合函数求导法则 (外层导数 $\times$ 内层导数): $\frac{\partial L}{\partial w} = \frac{\partial z}{(\partial w)^T} \cdot \frac{\partial L}{\partial z}$ 外层导数 (L 对 z): $L = z^T z = \|z\|^2$, 类比 x^2 , 导数为 $2z$. $\frac{\partial L}{\partial z} = 2z = 2(y - Xw)$ 内层导数 (z 对 w): $z = y - Xw$, 类比 $-ax$, 导数系数为 $-X$. $\frac{\partial z}{\partial w} = -X$ 组合: 注意维度匹配 (内层系数转置后左乘外层):

$$\frac{\partial L(w)}{\partial w} = (-X)^T \cdot 2(y - Xw) = -2X^T(y - Xw) = -2X^T y + 2X^T Xw$$

[r] 特别地,对于二元线性回归,有:

$$w_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2} w_0 = \bar{y} - w_1 \bar{x}$$

微分法是处理复杂表达式最高效的方法.首先计算标量函数的微分 df ,然后将其整理成与 dX 内积的形式,即 $df = \text{tr}\left(\left(\frac{\partial f}{\partial X}\right)^T dX\right)$,从而直接读出导数.

2.2.2 岭回归与拉索回归

岭回归 (Ridge regression) 和拉索回归 (Lasso regression) 是带有正则化的线性回归.
岭回归在线性回归代价函数上添加了 L_2 正则化项:

$$L(w) = \frac{1}{2m} \left[\sum_{i=1}^m (\hat{y}_i - y_i)^2 + \lambda \sum_{j=1}^d w_j^2 \right]$$

岭回归代价函数最小二乘优化的闭式解:

$$w^* = (X^T X + \lambda I)^{-1} X^T y$$

I 是单位矩阵,保证 $(X^T X + \lambda I)$ 可逆, w 具有闭式解.

特性维度	闭式解 (解析解)	梯度下降法 (迭代法)
计算效率	特征维度高时,计算逆矩阵的代价非常高 (复杂度约 $O(n^3)$)	通过迭代逼近,避免直接求逆,适合高维特征
内存需求	需要一次性将整个数据集矩阵加载到内存中计算	支持分批处理数据 (如随机梯度下降),对内存友好
适用问题	标准线性岭回归	适用范围更广,可用于其他不可微或更复杂的损失函数 (如 Lasso 回归的 L1 正则项)
灵活性	固定形式,调整正则化参数 λ 后需重新计算	灵活,便于在线学习 (模型可随新数据增量更新)

特性维度	闭式解（解析解）	梯度下降法（迭代法）
实现与稳定性	依赖矩阵求逆,当矩阵接近奇异时,即使有正则化,数值计算仍可能不稳定	实现相对直接,数值稳定性较好,尤其适合大规模问题

[s] 即使有了闭式解,梯度下降法仍然有必要性.

岭回归代价函数的梯度下降迭代:

$$w_k^{(i+1)} \leftarrow w_k^{(i)} - \eta \nabla_{w_k} L(w_k^{(i)}) = w_k^{(i)} - \frac{\eta}{m} \left(X^T (Xw - y) + \lambda w_k^{(i)} \right)$$

拉索回归在线性回归代价函数上添加了 L_1 正则化项（拉索回归没有通用的闭式解）:

$$L(w) = \frac{1}{2m} \left[\sum_{i=1}^m (\hat{y}_i - y_i)^2 + \lambda \sum_{j=1}^d |w_j| \right]$$

- [t]
- **L2 正则化（岭回归）**：惩罚项是权重的平方和.这是一个平滑的、凸的函数,其梯度在权重接近零时也会变小.因此,在优化过程中,权重会逐渐缩小,但除非正则化参数 λ 趋于无穷大,否则权重不会精确为零.这意味着所有特征都会被保留,只是权重值变小了.
 - **L1 正则化（Lasso 回归）**：惩罚项是权重的绝对值之和.这是一个非平滑的函数,在零点处不可导（有“尖点”）.当使用梯度下降或专用算法（如坐标下降）优化时,这些尖点使得最优解更容易出现在坐标轴上,即某些权重恰好为零.这称为**稀疏性**.

L2 正则化（圆形约束）	L1 正则化（棱形约束）
$w_1^2 + w_2^2 \leq C$	$ w_1 + w_2 \leq C$
约束区域是一个 圆 .圆形是光滑的,没有突出的顶点.	约束区域是一个 棱形 （正方形旋转）.它有尖锐的 顶点 ,且顶点位于坐标轴上.
等高线膨胀时,极大概率切在圆周的非轴位置（ $w_1 \neq 0, w_2 \neq 0$ ）.	等高线膨胀时,极易先碰到向外突出的 尖角 （即坐标轴 $w_i = 0$ 的位置）.
结论 : 参数变小,但很难为 0.	结论 : 参数容易卡在坐标轴上,变为 0.

表 2 L1 与 L2 约束区域的几何对比

场景	首选方法	理由
特征维度极高,噪音多	L1 (Lasso)	自带特征选择,能把无用特征系数置为 0,模型更简单.
需要模型解释性	L1 (Lasso)	得到稀疏解,容易解释哪几个特征起了决定性作用.
特征较少,且都很重要	L2 (Ridge)	保留所有特征信息,通常预测精度略高,计算更稳定.
特征间存在多重共线性	L2 (Ridge)	L1 倾向于随机保留相关特征中的一个; L2 会平摊权重,更稳健.

2.2.3 相关量化指标

[v] 当 $L(w)$ 不考虑正则化损失时,有:

$SST = SSR + SSE$

当考虑正则化损失式,等式退化为不等式:

$SST < SSR + SSE$

前文有,就不给证明了

[v] 这个等式的核心含义是：因变量 y 的总变异（SST）可以精确地分解为两部分：一部分是由回归模型解释的变异（SSR），另一部分是模型无法解释的剩余变异（SSE）.

普通最小二乘有完美的数学性质, 正则化是为了实用目的（防止过拟合）而进行的修改.

用于代价（损失）函数的 MAE、MSE 与用于回归性能度量的 MAE、MSE 的区别联系

方面	作为损失函数	作为性能度量
计算阶段	训练过程中不断计算	训练完成后一次性计算
数据使用	使用训练数据	使用验证集/测试集

方面	作为损失函数	作为性能度量
目的	指导参数优化	评估模型泛化能力
影响模型	直接影响权重更新	不影响模型内部参数

Theorem 2.2.2

在二元线性回归中, $R^2 = \rho_{XY}^2$.

$$\rho_{XY} = \frac{\text{Cov}(X, Y)}{\sqrt{\text{Var}(X)\text{Var}(Y)}} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2 \sum_{i=1}^n (y_i - \bar{y})^2}}$$

Proof: 只需要带入公式得到,此时

$$\text{SSR} = \sum_{i=1}^n (\hat{y}_i - \bar{y})^2 = w_1^2 \sum_{i=1}^n (x_i - \bar{x})^2 = \frac{\left(\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})\right)^2}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

其余只需要带入计算即可得到结论. □

[w] R^2 衡量的是因变量 Y 的方差中有多少比例可以被自变量 X 解释, ρ_{XY}^2 表示两个变量共享的方差比例, 定理将这两个看似不同的概念统一起来,使 R^2 有了更直观的统计意义.
这可以带来使得,在简单线性回归中,可以直接用相关系数计算 R^2 , 无需进行完整的回归分析, 提供了验证回归模型效果的快速方法.

2.2.4 广义线性回归

Definition 2.2.3

广义线性回归

广义线性回归的基本形式:

$$\hat{y}_i = \sigma(z_i) = \sigma(x_i^T w + w_0)$$

其中, σ 是单调可微的联系函数(link function), 也称为激活函数.

同时也存在多项式回归的情况,多项式回归方程:

$$\hat{y}_i = g(x_i) = w_0 + w_{11}x_{i1} + w_{12}x_{i2} + \dots + w_{1d}x_{id} + w_{21}x_{i1}^2 + \dots + w_n x_{i1}x_{i2} + \dots$$

Remark

包含偏置参数,一次项,高次项,交叉项等.这里要额外注意过拟合现象.将多项式回归中的高次项和交叉项视为新的特征变量,从而将非线性问题转化为线性问题.

通过一组基函数 $\Phi(\mathbf{x})$,可以将低维数据映射到高维特征空间,从而实现非线性回归:

$$g(\mathbf{x}) = \sum_{j=0}^M w_j \varphi_j(\mathbf{x}) = \mathbf{w}^T \Phi(\mathbf{x})$$

[x] 其中:

- $\mathbf{w} = [w_0, w_1, \dots, w_M]^T$ 是待求的 权重向量.
- $\Phi(\mathbf{x}) = [\varphi_0(\mathbf{x}), \varphi_1(\mathbf{x}), \dots, \varphi_M(\mathbf{x})]^T$ 是选定的一组 基函数向量.
- 通常令 $\varphi_0(\mathbf{x}) = 1$ 作为偏置项.

Remark

说人话: 利用已有的样本特征生成新的样本,或者叫 latent space?

Example

为了深入理解岭回归的计算过程及其对模型参数的影响,我们构建一个极简的一元线性回归数据集进行手算推导.
任务定义: 给定 3 个样本点,建立线性模型 $\hat{y} = w_0 + w_1x$. 设定正则化参数 $\lambda = 1$.
数据集:

样本 ID	特征 x	标签 y
1	1	2
2	2	3
3	3	5

为了使用闭式解 $w^* = (X^T X + \lambda I)^{-1} X^T y$, 我们需要先构建设计矩阵 X 和标签向量 y . 注意: 为了包含偏置项 w_0 , 我们需要在 X 中增加一列全为 1 的列.

$$X = \begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{pmatrix}, \quad y = \begin{pmatrix} 2 \\ 3 \\ 5 \end{pmatrix}, \quad w = \begin{pmatrix} w_0 \\ w_1 \end{pmatrix}$$

1. 计算 $X^T X$ (Gram Matrix):

$$\begin{aligned} X^T X &= \begin{pmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{pmatrix} \\ &= \begin{pmatrix} 1+1+1 & 1+2+3 \\ 1+2+3 & 1+4+9 \end{pmatrix} = \begin{pmatrix} 3 & 6 \\ 6 & 14 \end{pmatrix} \end{aligned}$$

2. 计算 $X^T y$:

$$\begin{aligned} X^T y &= \begin{pmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \end{pmatrix} \begin{pmatrix} 2 \\ 3 \\ 5 \end{pmatrix} \\ &= \begin{pmatrix} 2+3+5 \\ 2+6+15 \end{pmatrix} = \begin{pmatrix} 10 \\ 23 \end{pmatrix} \end{aligned}$$

我们设定 $\lambda = 1$. 此时正则化矩阵为 λI .

Remark

在实际工程中, 通常不对偏置项 w_0 进行正则化 (即 I 的第一行第一列设为 0), 因为偏置表示截距, 不影响模型的复杂度和方差. 但为了严格匹配上述闭式解公式 $w^* = (X^T X + \lambda I)^{-1} X^T y$, 我们在本例中对所有参数一视同仁地进行正则化.

1. 计算 $X^T X + \lambda I$:

$$\begin{aligned} A &= \begin{pmatrix} 3 & 6 \\ 6 & 14 \end{pmatrix} + 1 \cdot \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \\ &= \begin{pmatrix} 4 & 6 \\ 6 & 15 \end{pmatrix} \end{aligned}$$

2. 计算逆矩阵 A^{-1} : $A^{-1} = \frac{1}{24} \begin{pmatrix} 15 & -6 \\ -6 & 4 \end{pmatrix}$

1. 求解最优参数 w^* :

$$\begin{aligned} w^* &= A^{-1} (X^T y) \\ &= \frac{1}{24} \begin{pmatrix} 15 & -6 \\ -6 & 4 \end{pmatrix} \begin{pmatrix} 10 \\ 23 \end{pmatrix} \\ &= \frac{1}{24} \begin{pmatrix} 12 \\ 32 \end{pmatrix} = \begin{pmatrix} 0.5 \\ 1.333 \end{pmatrix} \end{aligned}$$

结果: 岭回归模型方程为 $\hat{y} = 0.5 + 1.333x$.

我们使用求得的岭回归模型计算预测值和误差.

1. 预测值:

$$\begin{aligned} \hat{y}_1 &= 0.5 + 1.333(1) = 1.833 \\ \hat{y}_2 &= 0.5 + 1.333(2) = 3.166 \\ \hat{y}_3 &= 0.5 + 1.333(3) = 4.500 \end{aligned}$$

2. 残差平方和 (SSE):

$$\begin{aligned} \text{SSE} &= (1.833 - 2)^2 + (3.166 - 3)^2 + (4.500 - 5)^2 \\ &\approx 0.028 + 0.028 + 0.25 = 0.306 \end{aligned}$$

3. 总离差平方和 (SST): y 的均值 $\bar{y} = \frac{2+3+5}{3} = 3.333$.

$$\begin{aligned} \text{SST} &= (2 - 3.333)^2 + (3 - 3.333)^2 + (5 - 3.333)^2 \\ &\approx 1.777 + 0.111 + 2.778 = 4.666 \end{aligned}$$

4. 决定系数 R^2 :

$$R^2 = 1 - \frac{\text{SSE}}{\text{SST}} = 1 - \frac{0.306}{4.666} \approx 0.934$$

评估: 模型的 R^2 为 0.934,说明该岭回归模型能解释约 93.4% 的数据变异,拟合效果良好且参数受到了约束. 我们沿用之前的极简数据集,演示如何用梯度下降法求解 Lasso.

Example

数据集回顾:

$$X = \begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{pmatrix}, \quad y = \begin{pmatrix} 2 \\ 3 \\ 5 \end{pmatrix}$$

参数设置:

- 正则化系数: $\lambda = 1$
- 学习率: $\eta = 0.1$

- **初始化策略:** 为了展示 Lasso 的收缩效果,我们**不从 0 开始**,而是从普通最小二乘法 (OLS) 的最优解开始.

$$w^{(0)} = \begin{pmatrix} 0.33 \\ 1.5 \end{pmatrix}$$

(这是我们在上个例子算出的 OLS 解)

梯度公式:

$$\nabla J(w) = \underbrace{-X^T(y - Xw)}_{\text{数据梯度}} + \underbrace{\lambda \cdot \text{sign}(w)}_{\text{正则梯度}}$$

迭代第 1 步:

1. **计算数据梯度:** 因为我们是从 OLS 最优解 $w^{(0)}$ 开始的,此时数据拟合误差最小 (导数为 0) .

$$\nabla_{\text{data}} \approx \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

(注: 实际上是 $X^T X w - X^T y$, OLS 解正好让它为 0).

2. **计算正则梯度 ($\lambda = 1$):**

$$w_0 = 0.33 > 0 \rightarrow \text{sign} = 1$$

$$w_1 = 1.5 > 0 \rightarrow \text{sign} = 1$$

$$\nabla_{\text{reg}} = \lambda \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

3. **更新权重:**

$$\begin{aligned} w^{(1)} &= w^{(0)} - \eta(\nabla_{\text{data}} + \nabla_{\text{reg}}) \\ &= \begin{pmatrix} 0.33 \\ 1.5 \end{pmatrix} - 0.1 \left(\begin{pmatrix} 0 \\ 0 \end{pmatrix} + \begin{pmatrix} 1 \\ 1 \end{pmatrix} \right) \\ &= \begin{pmatrix} 0.33 - 0.1 \\ 1.5 - 0.1 \end{pmatrix} \\ &= \begin{pmatrix} 0.23 \\ 1.4 \end{pmatrix} \end{aligned}$$

观察: Lasso 对两个参数都执行了“减法”,强制它们变小.

迭代第 2 步:

现在 w 变了,数据梯度不再是 0 了 (因为偏离了 OLS 最优解, MSE 变大了) .

1. 计算预测值与误差:

$$\hat{y} = Xw^{(1)} = \begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{pmatrix} \begin{pmatrix} 0.23 \\ 1.4 \end{pmatrix} = \begin{pmatrix} 1.63 \\ 3.03 \\ 4.43 \end{pmatrix}$$

$$e = y - \hat{y} = \begin{pmatrix} 2 \\ 3 \\ 5 \end{pmatrix} - \begin{pmatrix} 1.63 \\ 3.03 \\ 4.43 \end{pmatrix} = \begin{pmatrix} 0.37 \\ -0.03 \\ 0.57 \end{pmatrix}$$

2. 计算数据梯度 $-X^T e$:

$$\nabla_{\text{data}} = -\begin{pmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \end{pmatrix} \begin{pmatrix} 0.37 \\ -0.03 \\ 0.57 \end{pmatrix}$$

- 偏置项梯度: $-(0.37 - 0.03 + 0.57) = -0.91$
- x 项梯度: $-(1 \times 0.37 + 2 \times (-0.03) + 3 \times 0.57) = -(0.37 - 0.06 + 1.71) = -2.02$

$$\nabla_{\text{data}} = \begin{pmatrix} -0.91 \\ -2.02 \end{pmatrix}$$

(说明: 数据梯度是负的, 想要把 w 推大回去, 以减小误差).

3. 计算正则梯度 (w 仍大于 0):

$$\nabla_{\text{reg}} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

(说明: Lasso 依然坚定地想把 w 推小, 方向是正的).

4. 更新权重:

$$\begin{aligned} w^{(2)} &= w^{(1)} - \eta(\nabla_{\text{data}} + \nabla_{\text{reg}}) \\ &= \begin{pmatrix} 0.23 \\ 1.4 \end{pmatrix} - 0.1 \left(\begin{pmatrix} -0.91 \\ -2.02 \end{pmatrix} + \begin{pmatrix} 1 \\ 1 \end{pmatrix} \right) \\ &= \begin{pmatrix} 0.221 \\ 1.502 \end{pmatrix} \end{aligned}$$

[y] 分析:

- 对于 w_1 (特征权重): 数据梯度想加 0.202, Lasso 想减 0.1. 合力是加 0.102.
- 这说明在 $\lambda = 1$ 的情况下, 特征 x 非常重要, 数据梯度的力量超过了惩罚项, 权重出现了反弹.

思考: 如果我们将 λ 设得很大 (比如 10), 正则梯度就会变成 10, 远超数据梯度, 权重就会迅速被“减”到 0.

假设经过多轮迭代, 某个不重要的权重 w_j 变得很小, 比如 0.05, 且此时数据梯度也很小 (比如 -0.05).

- Lasso 更新:

$$w_{\text{new}} = 0.05 - \eta(-0.05 + 1) \approx 0.05 - 0.1 = -0.05$$

(它跨过了 0 点).

在专门的算法 (如软阈值法 Soft Thresholding 或 坐标下降) 中, 会包含一步判断:

$$S(w, \lambda) = \text{sign}(w) \cdot \max(|w| - \lambda, 0)$$

如果 $|w| < \lambda$, 直接置为 0.

2.3 逻辑斯蒂回归与最大熵模型

2.3.1 逻辑斯蒂回归模型/对数几率回归分类器

逻辑斯蒂回归是统计学习中的经典分类方法, 最大熵是概率模型学习的一个准则, 将其推广到分类问题得到最大熵模型. 逻辑斯蒂回归模型与最大熵模型都属于对数线性模型. 模型学习就是在给定的训练数据条件下对模型进行极大似然估计或正则化的极大似然估计.

Remark

有时会简称为逻辑模型, 但“逻辑”是音译“逻辑斯蒂 (logistic)”的缩写, 并不是说这个算法具有怎样的逻辑性.

比方说广告投入金额 x 和销售量 y 的关系,散点图如下,这种情况适用一元线性回归.但在许多实际问题中,因变量 y 是分类型,只取 0、1 两个值.如果用阶跃函数来拟合往往效果不错,但是这并不是一个连续函数.理想的情况,是像线性回归的函数一样, X 和 Y 之间的关系,是用一个可导的函数来描述的.

我们试图用一个模型,他输入是 $x \in R^n$,而输出是 $y \in \{0, 1\}$.这里其实输出的是 $P(Y = 0|x)$ 和 $P(Y = 1|x)$,如果沿用线性回归模型,会发现他的值域是 $(-\infty, +\infty)$,而概率则是 $(0, 1)$ 上的.我们首先定义几率为发生的概率与不发生的概率的商.即对于概率为 p 的几率为 $\frac{p}{1-p}$,这样范围就变成了 $(0, +\infty)$,再对这个式子取对数得到对数几率或 logic 函数是:

$$\text{logic}(p) = \log \frac{p}{1-p}$$

这样取值范围就变成 $(-\infty, +\infty)$ 了,这里就引入最关键的假设:

[z] 特征 X 的线性组合 $X\beta$ (或者 $w \cdot x + b$ 的形式),正好等于这个“对数几率”.

再根据这个假设我们反解出 P :

$$P = \frac{1}{1 + e^{-X\beta}}$$

Remark

而这个其实就是我们常见的 Sigmoid 函数了.

Definition 2.3.1

二阶逻辑斯蒂回归模型是如下条件的概率分布:

$$\begin{aligned} P(Y = 1|x) &= \frac{\exp(w \cdot x + b)}{1 + \exp(w \cdot x + b)} \\ P(Y = 0|x) &= 1 - P(Y = 1|x) \end{aligned}$$

这里, $x \in R^n$ 是输入, $Y \in \{0, 1\}$ 是输出, $w \in R^n$ 和 $b \in R$ 是参数, w 称为权值向量, b 称为偏置, $w \cdot x$ 为 w 和 x 的内积.

对于给定训练数据集 $T = \{(x_i, y_i) | x_i \in R^n, y_i \in \{0, 1\}\}$,可以应用极大似然估计法估计模型参数,从而得到逻辑斯蒂回归模型.设

$$P(Y = 1|x) = \pi(x), \quad P(Y = 0|x) = 1 - \pi(x)$$

似然函数为

$$\prod_{i=1}^N (\pi(x_i))^{y_i} (1 - \pi(x_i))^{1-y_i}$$

[aa] This is Cross Entropy.

对数似然函数为

$$\text{CE} = - \sum P(x_i) \log Q(x = x_i)$$

$$\begin{aligned} L(w) &= \sum_{i=1}^N [y_i \log \pi(x_i) + (1 - y_i) \log(1 - \pi(x_i))] \\ &= \sum_{i=1}^N \left[y_i \log \frac{\pi(x_i)}{1 - \pi(x_i)} + \log(1 - \pi(x_i)) \right] \\ &= \sum_{i=1}^N \{y_i(w \cdot x_i + b) - \log[1 + \exp(w \cdot x_i + b)]\} \end{aligned}$$

对 $L(w)$ 求极大值,得到 w 的估计值 $\hat{w}$.这样问题变成了以对数似然函数为目标函数的最优化问题.逻辑斯蒂回归学习中通常采用的方法是梯度下降法和拟牛顿法.

负对数似然函数定义为:

$$J(w) = -L(w) = - \sum_{i=1}^N y_i(w \cdot x_i + b) - \log \left[1 + \exp(w \cdot x_i + b) + \frac{\lambda}{2} \sum_{j=1}^d w_j^2 \right]$$

(后者可选的正则化) 所以有:

$$\nabla J(w) = \frac{\partial J(w)}{\partial w} = \sum_{i=1}^N x_i (\pi(x_i) - y_i)$$

$\pi(x_i) = \frac{1}{1 + \exp(-w \cdot x_i + b)}$ 是逻辑函数,表示样本 x_i 属于正类的概率.

梯度下降法具体的更新公式为：

$$w^{(k+1)} = w^{(k)} - \alpha \nabla J(w^{(k)}) = w^{(k)} - \alpha \sum_{i=1}^N x_i (\pi(x_i) - y_i)$$

其中 $\pi(x_i)$ 使用当前参数 $w^{(k)}$ 计算。

[ab] 在实际应用中,常使用随机梯度下降 (SGD) 或小批量梯度下降以提高计算效率.SGD 每次使用一个样本更新:

$$w^{(k+1)} = w^{(k)} - \alpha x_i (\pi(x_i) - y_i)$$

- 特征标准化可以加速收敛.
- 学习率 α 可能需要调整,如使用学习率衰减策略.

也可以将问题进一步推广到多项逻辑斯蒂回归模型中,用于多类分类.假设离散型随机变量 Y 的取值集合是 $\{1, 2, \dots, K\}$, 那么多项逻辑斯蒂回归模型是:

$$P(Y = k|x) = \frac{\exp(w_k \cdot x)}{1 + \sum_{k=1}^{K-1} \exp(w_k \cdot x)},$$

$$k = 1, 2, \dots, K-1$$

$$P(Y = K|x) = \frac{1}{1 + \sum_{k=1}^{K-1} \exp(w_k \cdot x)},$$

$$k = K$$

这里, $x \in R^{n+1}$, $w_k \in R^{n+1}$

这个其实就是 softmax 函数.将各个输出节点的输出值范围映射到 $[0, 1]$, 并且约束各个输出节点的输出值的和为 1.

Remark

hardmax 最大的特点就是只选出其中一个最大的值,即非黑即白.但是往往在实际中这种方式是不合情理的,比如对于文本分类来说,一篇文章或多或少包含着各种主题信息,我们更期望得到文章对于每个可能的文本类别的概率值(置信度),可以简单理解成属于对应类别的可信度.所以此时用到了 soft 的概念,Softmax 的含义就在于不再唯一的确定某一个最大值,而是为每个输出分类的结果都赋予一个概率值,表示属于每个类别的可能性.

[ac] 指数函数曲线呈现递增趋势,最重要的是斜率逐渐增大,也就是说在 x 轴上一个很小的变化,可以导致 y 轴上很大的变化.这种函数曲线能够将输出的数值拉开距离.经过使用指数形式的 Softmax 函数能够将差距大的数值距离拉的更大.在深度学习中通常使用反向传播求解梯度进而使用梯度下降进行参数更新的过程,而指数函数在求导的时候比较方便. $((e^x)' = e^x)$

当使用 Softmax 函数作为输出节点的激活函数的时候,一般使用交叉熵作为损失函数.由于 Softmax 函数的数值计算过程中,很容易因为输出节点的输出值比较大而发生数值溢出的现象,在计算交叉熵的时候也可能会出现数值溢出的问题.现在可以构建比较复杂的神经网络模型,最重要的原因之一得益于反向传播算法.反向传播算法从输出端也就是损失函数开始向输入端基于链式法则计算梯度,然后通过计算得到的梯度,应用梯度下降算法迭代更新待优化参数.这样对于 softmax 函数求导就至关重要了.

这里我们把 softmax 函数形式记为

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{c=1}^C e^{z_c}} \triangleq p_i$$

对于 z_i 对应的输出 y_i , 会受到 $z_1, z_2, \dots, z_i, \dots, z_C$ 的影响,需要分别求出对于每个 z_j 的偏导数.显然对于 $j = i$ 情况下的偏导数 $\frac{\partial y_i}{\partial z_i}$ 与 $j \neq i$ 的其他情况是不一样的.所以需要分情况讨论.最终答案为:

$$\frac{\partial y_i}{\partial z_j} = \begin{cases} p_i(1 - p_j) & j = i \\ -p_j \cdot p_i & j \neq i \end{cases}$$

Algorithm 2.3.2

逻辑斯蒂回归模型

输入: 训练数据集 $T = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$, 其中 $x_i \in R^n$, $y_i \in \{0, 1\}$; 学习率 α ; 最大迭代次数 K 或收敛阈值 ϵ .
输出: 模型参数 w 和 b .

1. **数据预处理:** 为了计算方便,将权重向量和输入向量加以扩充.令 $\hat{w} = (w^T, b)^T$, 对每个样本令 $\hat{x}_i = (x_i^T, 1)^T$. 此时模型简化为 $P(Y = 1|x) = \pi(\hat{x}) = \frac{1}{1 + e^{-\hat{w} \cdot \hat{x}}}$.
2. **初始化:** 选择初始权重 $\hat{w}^{(0)}$ (通常初始化为零向量或微小的随机向量), 设置迭代计数器 $k = 0$.
3. **迭代优化:** 重复以下步骤,直到满足停止条件 (如梯度模长 $\|\nabla J(\hat{w})\| < \epsilon$ 或 $k \geq K$):
 - **计算预测概率:** 对于所有 $i = 1, \dots, N$, 利用当前参数计算:

$$\pi(\hat{x}_i) = \frac{1}{1 + \exp(-\hat{w}^{(k)} \cdot \hat{x}_i)}$$

- **计算梯度**: 基于负对数似然函数计算梯度 (批量梯度):

$$\nabla J(\hat{w}^{(k)}) = \sum_{i=1}^N (\pi(\hat{x}_i) - y_i) \hat{x}_i$$

- **更新参数**: 沿负梯度方向更新权重:

$$\hat{w}^{(k+1)} = \hat{w}^{(k)} - \alpha \nabla J(\hat{w}^{(k)})$$

- **更新计数**: $k \leftarrow k + 1$

4. **输出结果**: 得到最终参数 $\hat{w}^*$, 将其拆分为权重向量 w^* 和偏置 b^* , 构建最终分类模型:

$$f(x) = \begin{cases} 1 & \text{if } P(Y=1|x) > 0.5 \\ 0 & \text{otherwise} \end{cases}$$

[ad] 如果你很奇怪, 为什么没有任何 b 的更新? 这是因为, 吸收偏置的技巧: 令 $w = [w_1, w_2, \dots, w_d, b]^T \in \mathbb{R}^{d+1}$, $x = [x_1, x_2, \dots, x_d, 1]$. (注意维度多了 1). 这样不需要算两个梯度, 只需要算一个梯度即可. 只有这个算法框里用了这个技巧. 当然, 直接对 b 求一个梯度也能解决问题, 只是记号不同了.

2.3.2 最大熵模型

假设离散随机变量 X 的概率分布是 $P(X)$, 我们前面介绍过其熵是 $H(P) = -\sum_x P(x) \log P(x)$ 且有不等式 $0 \leq H(P) \leq \log |X|$ 成立. $|X|$ 表示 X 的取值个数. 当且仅当 X 满足均匀分布的时候, 右边的等号成立, 即 X 服从均匀分布时, 熵最大.

[ae] 最大熵原理认为要选择的概率模型首先必须满足已有的约束条件, 在没有更多信息的情况下, 那些不确定的部分都是等可能的. 最大熵原理通过熵的最大化来表示等可能, 形而上的“等可能”不容易实现, 但是熵可以作为一个数值指标.

假设分类模型是一个条件概率分布 $P(Y|X)$, $X \in \mathcal{X} \subseteq \mathbb{R}^n$ 表示输入, $Y \in \mathcal{Y}$ 表示输出, $\mathcal{X}$ 和 $\mathcal{Y}$ 表示输入和输出的集合. 这个模型表示的是对于给定的输入 X , 以条件概率 $P(Y|X)$ 输出 Y .

给定一个训练数据集 T , 学习的目标是用最大熵原理选择最好的分类模型. 首先考虑模型应该满足的条件, 给定训练数据集, 可以确定联合分布 $P(X, Y)$ 的经验分布和边缘分布 $P(X)$ 的经验分布, 分别以 $\tilde{P}(X, Y)$ 和 $\tilde{P}(X)$ 表示. 这里计算其实就是古典概型中的频率计算. 用特征函数 $f(x, y)$ 描述输入 x 和输出 y 之间的某一个事实. 定义是:

$$f(x, y) = \begin{cases} 1 & x \text{ 与 } y \text{ 满足某一事实} \\ 0 & \text{otherwise} \end{cases}$$

特征函数 $f(x, y)$ 关于经验分布 $\tilde{P}(x, y)$ 的期望值用 $E_{\tilde{P}}(f)$ 表示, 而关于模型 $P(Y|X)$ 与经验分布 $\tilde{P}(X)$ 的期望值用 $E_P(f)$ 表示. 如果模型能够获取训练数据中的信息, 那么就可以假设两个期望相等:

$$E_P(f) = E_{\tilde{P}}(f)$$

即:

$$\sum_{x,y} \tilde{P}(x) P(y|x) f(x, y) = \sum_{x,y} \tilde{P}(x, y) f(x, y)$$

Remark

这就是模型学习的约束条件. 加入有 n 个特征函数 $f_i(x, y)$, $i = 1, 2, \dots, n$, 那么就有 n 个约束条件.

Definition 2.3.3

假设满足所有约束条件的模型集合为

$$\mathcal{C} \equiv \{P \in \mathcal{P} | E_P(f_i) = E_{\tilde{P}}(f_i), i = 1, 2, \dots, n\}$$

定义在条件概率分布 $P(X|Y)$ 上的条件熵为:

$$H(P) = -\sum_{x,y} \tilde{P}(x) P(y|x) \log P(y|x)$$

则模型集合 $\mathcal{C}$ 中条件熵 $H(P)$ 最大的模型称为最大熵模型.

Remark

最大熵模型的学习过程就是求解最大熵模型的过程, 最大熵模型的学习可以形式化为约束最优化问题.

对于给定的训练数据集 T 以及特征函数 $f_i(x, y)$, $i = 1, 2, \dots, n$, 最优化问题习惯上把求最大值问题改写为等价求最小值问题:

$$\begin{aligned} \min_{P \in \mathcal{C}} \quad & -H(P) = \sum_{x,y} \tilde{P}(x) P(y|x) \log P(y|x) \\ \text{s.t.} \quad & E_{P(f_i)} - E_{\tilde{P}(f_i)} = 0, \quad i = 1, 2, \dots, n \\ & \sum_y P(y|x) = 1 \end{aligned}$$

我们将约束最优化的原始问题转换为无约束最优化的对偶问题,通过求解对偶问题求解原始问题.引入拉格朗日乘子 $w_0, w_1, \dots, w_n$,定义拉格朗日函数 $L(P, w)$:

$$L(P, w) \equiv -H(P) + w_0 \left(1 - \sum_y P(y|x) \right) + \sum_{i=1}^n w_i (E_{\tilde{P}(f_i)} - E_P(f_i))$$

最优化原始问题是 $\min_{P \in \mathcal{C}} \max_w L(P, w)$,对偶问题是 $\max_w \min_{P \in \mathcal{C}} L(P, w)$.

Remark

由于拉格朗日函数 $L(P, w)$ 是 P 的凸函数,所以原始问题的解和对偶问题的解是等价的.

首先,求解对偶问题内部的极小化问题, $\min_{P \in \mathcal{C}} L(P, w)$ 是 w 的函数,将其记作

$$\Psi(w) = \min_{P \in \mathcal{C}} L(P, w) = L(P_w, w)$$

$\Psi(w)$ 称为对偶函数,同时将其解记作:

$$P_w = \arg \min_{P \in \mathcal{C}} L(P, w) = P_w(y|x)$$

具体地,求 $L(P, w)$ 对 $P(y|x)$ 的偏导数:

$$\begin{aligned} \frac{\partial L(P, w)}{\partial P(y|x)} &= \sum_{x,y} \tilde{P}(x) (\log P(y|x) + 1) - \sum_y w_0 - \sum_{x,y} \left(\tilde{P}(x) \sum_{i=1}^n w_i f_i(x, y) \right) \\ &= \sum_{x,y} \tilde{P}(x) \left(\log P(y|x) + 1 - w_0 - \sum_{i=1}^n w_i f_i(x, y) \right) \end{aligned}$$

令偏导数为 0,在 $\tilde{P}(x) > 0$ 的情况下,解得:

$$P(y|x) = \exp \left(\sum_{i=1}^n w_i f_i(x, y) + w_0 - 1 \right) = \frac{\exp \left(\sum_{i=1}^n w_i f_i(x, y) \right)}{\exp(1 - w_0)}$$

由于 $\sum_y P(y|x) = 1$,得:

$$P_w(y|x) = \frac{1}{Z_w(x)} \exp \left(\sum_{i=1}^n w_i f_i(x, y) \right)$$

其中,

$$Z_w(x) = \sum_y \exp \left(\sum_{i=1}^n w_i f_i(x, y) \right)$$

$Z_w(x)$ 称为规范化因子, $f_i(x, y)$ 是特征函数, w_i 是特征的权重.

之后求解对偶问题外部的极大化问题:

$$\max_w \Psi(w)$$

其解记作 w^* ,即:

$$w^* = \arg \max_w \Psi(w)$$

最大熵模型的学习最终归结为对偶函数 $\Psi(w)$ 的极大化.

Remark

常见方法是对 w_i 分别求偏导令偏导数为 0 得到答案.

Theorem 2.3.4

对偶函数的极大化等价于最大熵模型的极大似然估计.

Proof:

已知训练数据的经验概率分布 $\tilde{P}(X, Y)$, 条件概率分布 $P(Y|X)$ 的对数似然函数表示为:

$$L_{\tilde{P}}(P_w) = \log \prod_{x,y} [\tilde{P}(x, y)]^{P(y|x)} = \sum_{x,y} \tilde{P}(x, y) \log P(y|x)$$

当条件概率分布是最大熵模型时, 对数似然函数是:

$$\begin{aligned} L_{\tilde{P}}(P_w) &= \sum_{x,y} \tilde{P}(x, y) \log P(y|x) \\ &= \sum_{x,y} \tilde{P}(x, y) \sum_{i=1}^n w_i f_i(x, y) - \sum_{x,y} \tilde{P}(x, y) \log Z_w(x) \\ &= \sum_{x,y} \tilde{P}(x, y) \sum_{i=1}^n w_i f_i(x, y) - \sum_x \tilde{P}(x) \log Z_w(x) \end{aligned}$$

而对于对偶函数有:

$$\begin{aligned} \Psi(w) &= \sum_{x,y} \tilde{P}(x) P_w(y|x) \log P_w(y|x) + \sum_{i=1}^n w_i \left(\sum_{x,y} \tilde{P}(x, y) f_i(x, y) - \sum_{x,y} \tilde{P}(x) P_w(y|x) f_i(x, y) \right) \\ &= \sum_{x,y} \tilde{P}(x, y) \sum_{i=1}^n w_i f_i(x, y) + \sum_{x,y} \tilde{P}(x) P_w(y|x) \left(\log P_w(y|x) - \sum_{i=1}^n w_i f_i(x, y) \right) \\ &= \sum_{x,y} \tilde{P}(x, y) \sum_{i=1}^n w_i f_i(x, y) - \sum_{x,y} \tilde{P}(x) P_w(y|x) \log Z_w(x) \\ &= \sum_{x,y} \tilde{P}(x, y) \sum_{i=1}^n w_i f_i(x, y) - \sum_x \tilde{P}(x) \log Z_w(x) \quad \left(\sum_y P(y|x) = 1 \right) \end{aligned}$$

所以有:

$$\Psi(w) = L_{\tilde{P}}(P_w)$$

□

可以将最大熵模型写成更一般的形式:

$$\begin{aligned} P_w(y|x) &= \frac{1}{Z_w(x)} \exp \left(\sum_{i=1}^n w_i f_i(x, y) \right) \\ Z_w(x) &= \sum_y \exp \left(\sum_{i=1}^n w_i f_i(x, y) \right) \end{aligned}$$

这里 $x \in R^n$ 为输入, $y \in \{1, 2, \dots, K\}$ 为输出, $w \in R^n$ 为权重向量, $f_i(x, y), i = 1, 2, \dots, n$ 为任意实值特征函数.

Algorithm 2.3.5**最大熵模型**

输入: 训练数据集 $T = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$; 特征函数集合 $\{f_1, f_2, \dots, f_n\}$.

输出: 最大熵模型 $P_w(y|x)$, 即确定最优参数 w^* .

1. **计算经验期望:** 对于每个特征函数 $f_i (i = 1, \dots, n)$, 根据训练数据计算其关于经验分布的期望值:

$$E_{\tilde{P}}(f_i) = \sum_{x,y} \tilde{P}(x, y) f_i(x, y) = \frac{1}{N} \sum_{j=1}^N f_i(x_j, y_j)$$

这一步只需要计算一次, 作为后续优化的目标约束.

2. **初始化参数:** 初始化权重向量 $w^{(0)} = (w_1, w_2, \dots, w_n)^T$ (通常设为 0), 设置迭代计数 $k = 0$.
3. **迭代优化:** 重复以下步骤, 直到收敛 (如参数变化量小于阈值 ϵ):

- **计算规范化因子:** 对于训练集中出现的每个 x , 计算当前参数下的 $Z_{w(x)}$:

$$Z_{w(x)} = \sum_y \exp \left(\sum_{i=1}^n w_i f_i(x, y) \right)$$

- **计算模型期望:** 基于当前模型 $P_{w(y|x)}$ 计算特征函数的期望:

$$E_{P(f_i)} = \sum_{x,y} \tilde{P}(x) P_{w(y|x)} f_i(x, y) = \frac{1}{N} \sum_{j=1}^N \sum_y P_{w(y|x_j)} f_i(x_j, y)$$

其中 $P_{w(y|x)} = \frac{1}{Z_{w(x)}} \exp\left(\sum_{i=1}^n w_i f_i(x, y)\right)$.

- **更新参数**: 利用梯度上升法 (或 GIS/IIS 算法) 更新 w . 对数似然函数的梯度为 $g_i = E_{\hat{P}}(f_i) - E_{P(f_i)}$.

$$w_i^{(k+1)} = w_i^{(k)} + \alpha \cdot (E_{\hat{P}}(f_i) - E_{P(f_i)})$$

(注: 若是 GIS/IIS 算法, 更新步长会有特定的解析解形式, 而非固定学习率 α).

4. **输出模型**: 得到最优参数 w^* , 输出最终的最大熵模型:

$$P_{w^*}(y|x) = \frac{1}{Z_{w^*}(x)} \exp\left(\sum_{i=1}^n w_i^* f_i(x, y)\right)$$

3 贝叶斯分类器

3.1 贝叶斯决策论 (Bayesian Decision Theory)

若将样本 x 判定为类别 c_i , 其期望损失为:

$$R(c_i|x) = \sum_{j=1}^N \lambda_{ij} P(c_j|x)$$

其中 λ_{ij} 为将真实类别 c_j 误判为 c_i 的损失. 如果分类正确, 通常 $\lambda_{ii} = 0$; 错误分类则 $\lambda_{ij} > 0$. 想学一个 $h: \mathcal{X} \mapsto \mathcal{Y}$, 来最小化 $\mathbb{E}_x[R(h(x)|x)]$. 所以我们只需在每个样本上选择能使条件风险最小的类别:

$$h^*(x) = \arg \min_{c \in \mathcal{Y}} R(c|x) = \arg \max_{c \in \mathcal{Y}} \mathbb{P}(c|x)$$

.第二个式子是直观的, 用 λ_{ij} 的定义可以得到.

这里 h^* 就是贝叶斯最优分类器, 总体风险就是贝叶斯风险. 它在概率框架下引入了 **损失 (Loss)** 的概念, 旨在找出一个判定准则以最小化分类的 **期望风险**.

核心思想: 分类不仅仅是找概率最大的类 (MAP), 而是找 **代价最小** 的决策. 当误判损失相等时, 最小化风险等价于最大化后验概率.

3.2 极大似然估计 (Maximum Likelihood Estimation, MLE)

在贝叶斯分类器中, 我们需要估计类条件概率 $p(x|c)$, 是在“给定类别 c ”的条件下, x 的概率密度. 由于真实分布未知, 通常采用 **参数化 (Parametric)** 策略: 先 **假设** 分布形式, 再用数据估计参数. 直观地说, 假设 $p(x|c)$ 具有确定的概率分布形式 (如高斯分布), 且由参数 θ_c 唯一确定. 我们在所有可能的参数取值中, 寻找能使当前观测数据 D_c 出现的“可能性”最大的那一组参数 $\hat{\theta}_c$. 即:

$$\hat{\theta}_c = \arg \max_{\theta_c} P(D_c|\theta_c)$$

$$L(\theta_c) = \log P(D_c|\theta_c) = \sum_{x \in D_c} \log P(x|\theta_c)$$

由于 $\log$ 是单调递增函数, 最小化负对数似然 (NLL) 等价于最大化原始似然.

[af] 每个样本都对总损失函数贡献了一项 $\log P(x_i|\theta)$, 每个样本权重是完全一样的. MLE 没有机制去区分“这是真实的罕见规律”还是“这是噪声/异常值”. MLE 会忠实反映数据集的分布, 它默认所有数据都是真实且重要的.

若假设 $p(x|c) \sim \mathcal{N}(\mu_c, \sigma_c^2)$, 通过对 $L(\theta_c)$ 分别关于 μ 和 σ^2 求偏导并令其为 0, 可得:

- **均值估计**: $\hat{\mu}_c = \frac{1}{|D_c|} \sum_{x \in D_c} x$ (样本均值)
- **方差估计**: $\hat{\sigma}_c^2 = \frac{1}{|D_c|} \sum_{x \in D_c} (x - \hat{\mu}_c)(x - \hat{\mu}_c)^T$ (样本方差)

3.3 贝叶斯法

Definition 3.3.1

朴素贝叶斯法

朴素贝叶斯法 (naive Bayes) 法是基于 **贝叶斯定理与特征条件独立假设** 的分类方法. 对于给定的训练数据集, 首先基于特征条件独立假设, **[ag]** 就是 $P(c|x) = \frac{P(c)}{P(x)} \prod P(x_i|c)$ 学习输入输出的联合概率分布; 然后基于此模型, 对给定的输入 x , 利用贝叶斯定理求出后验概率最大的输出 y .

3.3.1 朴素贝叶斯法的学习与分类

设输入空间 $\mathcal{X} \subseteq \mathbb{R}^n$ 为 n 维向量的集合, 输出空间为类标记集合 $\mathcal{Y} = \{c_1, c_2, \dots, c_K\}$. 输入为特征向量 $x \in \mathcal{X}$, 输出为类标记 $y \in \mathcal{Y}$. X 是定义在输入空间 $\mathcal{X}$ 上的随机向量, Y 是定义在输出空间 $\mathcal{Y}$ 上的随机变量, $P(X, Y)$ 是 X 和 Y 的联合概率分布. 训练数据集 T 由 $P(X, Y)$ 独立同分布产生.

朴素贝叶斯法通过训练数据集学习联合概率分布 $P(X, Y)$. 具体地, 从先验概率和条件概率分布

$$P(Y = c_k), P(X = x|Y = c_k), \quad k = 1, 2, \dots, K$$

学习联合概率分布.而条件概率分布 $P(X=x|Y=c_k)$ 有指数级数量的参数,其估计实际上是不可行的.

[ah] 朴素贝叶斯法对条件概率分布作了**条件独立性的假设**.由于这是一个较强的假设,朴素贝叶斯法由此得名.参数数量从 $S^d - 1$ 降低到了 $d(S - 1)$,实现了从指数级到线性级的降维.

[ai] 如果不做任何假设,我们需要计算的是所有特征联合起来的概率:

$$P(X^{(1)} = x^{(1)}, \dots, X^{(n)} = x^{(n)} | Y = c_k)$$

输入特征向量 x 有 S^n 种可能,我们需要为每一种组合都分配一个概率值.由于所有概率之和必须为 1,只需要考察 $S^n - 1$ 个概率,参数个数为 $S^n - 1$.

[aj] 朴素贝叶斯假设特征之间**相互独立**:

$$\begin{aligned} P(X|Y = c_k) &= \\ P(X^{(1)}|Y = c_k) \times \dots \times P(X^{(n)}|Y = c_k) \end{aligned}$$

单独为每个特征需要 $S - 1$ 个参数,所以一共是 $n(S - 1)$.

朴素贝叶斯法实际上是**学习到生成数据**的机制,所以属于**生成模型**.朴素贝叶斯法分类时,对给定的输入 x ,通过学习到的模型计算后验概率分布 $P(Y = c_k | X = x)$,将后验概率最大的类作为 x 的类输出.后验概率计算根据贝叶斯定理进行:

$$\begin{aligned} P(Y = c_k | X = x) &= \frac{P(X = x | Y = c_k) P(Y = c_k)}{\sum_k P(X = x | Y = c_k) P(Y = c_k)} \\ &= \frac{P(Y = c_k) \prod_j P(X^{(j)} = x^{(j)} | Y = c_k)}{\sum_k P(Y = c_k) \prod_j P(X^{(j)} = x^{(j)} | Y = c_k)} \end{aligned}$$

所以朴素贝叶斯分类器可表示为:

$$\begin{aligned} y &= f(x) \\ &= \arg \max_{c_k} P(Y = c_k | X = x) \\ &= \arg \max_{c_k} \frac{P(Y = c_k) \prod_j P(X^{(j)} = x^{(j)} | Y = c_k)}{\sum_k P(Y = c_k) \prod_j P(X^{(j)} = x^{(j)} | Y = c_k)} \end{aligned}$$

而我们注意到,分母对所有 c_k 都是相同的,所以:

$$y = \arg \max_{c_k} P(Y = c_k) \prod_j P(X^{(j)} = x^{(j)} | Y = c_k)$$

Theorem 3.3.2

朴素贝叶斯法将实例分到后验概率最大的类中,这等价于期望风险最小化.

Proof:

假设选择 0-1 损失函数, $f(X)$ 表示分类决策函数,那么期望风险函数为:

$$R_{\text{exp}}(f) = E[L(Y, f(X))]$$

期望是对联合分布 $P(X, Y)$ 取的,由此取条件期望

$$R_{\text{exp}}(f) = E_X \sum_{k=1}^K [L(Y, f(X))] P(c_k | X)$$

为了使期望风险最小化,只需对 $X = x$ 逐个极小化,由此得到:

$$\begin{aligned} f(x) &= \arg \min_{y \in \mathcal{Y}} \sum_{k=1}^K L(c_k, y) P(c_k | X = x) \\ &= \arg \min_{y \in \mathcal{Y}} \sum_{k=1}^K P(y \neq c_k | X = x) \\ &= \arg \max_{y \in \mathcal{Y}} P(y = c_k | X = x) \end{aligned}$$

这样一来,根据期望风险最小化准则就得到了后验概率最大化准则,即朴素贝叶斯法所采用的原理. □

自 然 的 , 先 验 概 率 是 好 估 计 的 .

[ak] 少样本的时候需要使用拉普拉斯修正.随着训练集变大,修正所引入的影响会逐渐变小、可忽略.修正公式:

$$\hat{P}(c) = \frac{|D_c| + 1}{|D| + N}$$

$$\hat{P}(x_i | c) = \frac{|D_{c,x_i}| + 1}{|D_c| + N_i}$$

令 D_c 表示训练集 D 中第 c 类样本组成的集合, 若有充足的独立同分布样本, 则可容易地估计出类先验概率

$$P(c) = \frac{|D_c|}{|D|} \quad \text{公式 (7.16)}$$

对离散属性而言, 令 D_{c,x_i} 表示 D_c 中在第 i 个属性上取值为 x_i 的样本组成的集合, 则条件概率 $P(x_i | c)$ 可估计为

$$P(x_i | c) = \frac{|D_{c,x_i}|}{|D_c|} \quad \text{公式 (7.17)}$$

对连续属性而言可考虑概率密度函数, 假定 $p(x_i | c) \sim \mathcal{N}(\mu_{c,i}, \sigma_{c,i}^2)$, 其中 $\mu_{c,i}$ 和 $\sigma_{c,i}^2$ 分别是第 c 类样本在第 i 个属性上取值的均值和方差, 则有

$$p(x_i | c) = \frac{1}{\sqrt{2\pi} \sigma_{c,i}} \exp\left(-\frac{(x_i - \mu_{c,i})^2}{2\sigma_{c,i}^2}\right)$$

自 强 弘 毅 求 是 拓 新

图 6 朴素贝叶斯分类器

Algorithm 3.3.3

朴素贝叶斯法

输入: 训练数据集 $T = \{(x_1, c_1), (x_2, c_2), \dots, (x_N, c_N)\}$, 平滑参数 α , 新样本 X_{new} .

输出: 新样本的预测类别 $\hat{y} = c_{\text{new}}$.

1. 估计先验概率: 计算每个类别在训练集中出现的频率.

$$P(Y = c_k) = \frac{\text{类别为 } c_k \text{ 的样本数量}}{\text{训练集总样本数量}}$$

2. 估计条件概率: 对每个特征 x_i (假设为离散特征, 有 S_i 个可能的取值)

$$P(x_i = \text{value} | Y = c_k) = \frac{\text{在类别 } c_k \text{ 中 } x_i = \text{value} \text{ 的数量}}{\text{类别 } c_k \text{ 的总数}}$$

3. 拉普拉斯平滑: 为了避免训练集中未出现的情况导致概率为 0 (使得整个连乘为 0), 我们通常使用修正后的公式, 此时计算概率 $P(Y = c_k)$ 时也同样需要修正:

$$P(x_i = \text{value} | Y = c_k) = \frac{\text{计数} + \alpha}{\text{类别总数} + \alpha \times S_i}$$

其中, α 是一个大于 0 的平滑参数, 当 $\alpha = 1$ 时, 也称为加一平滑.

4. 预测最终结果: 计算新样本 $X_{\text{new}} = (x_1, x_2, \dots, x_n)$ 对于每个可能的类别的得分 (即后验概率的分子), 从而确定得分最高的类别 c_k 为新样本 X_{new} 的预测类别.

$$\hat{y} = \arg \max_{c_k} P(Y = c_k) \prod_j P(X^{(j)} = x^{(j)} | Y = c_k)$$

3.3.2 半朴素贝叶斯分类器

半朴素贝叶斯分类器的基本思想是适当考虑一部分属性间的相互依赖信息, 避免完全联合概率计算的高复杂性, 同时不彻底忽略较强的属性依赖关系.

最常用的策略是**独依赖估计**(ODE [a1] 哈哈,注意区分常微分方程.),即假设每个属性在类别之外最多仅依赖于一个其他属性.其核心公式可表示为:

$$P(c|\mathbf{x}) \propto P(c) \prod_{i=1}^d P(x_i|c, pa_i)$$

其中 pa_i 是属性 x_i 所依赖的父属性.
常见的半朴素贝叶斯分类器主要有以下几种:

- 1. **SPODE(超父独依赖估计)**: 假设所有属性都依赖于同一个属性,该属性称为“超父”.通过交叉验证等模型选择方法来确定超父属性.这种方法结构简单,但超父的选择至关重要,且假设所有属性共同依赖一个超父可能仍过于简化.
- 2. **TAN(树增强朴素贝叶斯)**: 通过以下步骤构建属性间的依赖关系树形结构
 - 计算任意两个属性间的**条件互信息**,以刻画它们在已知类别下的相关性.

$$I(X, Y|Z) = \sum_{z \in Z} p_Z(z) \sum_{y \in Y} \sum_{x \in X} p_{X,Y|Z}(x, y|z) \log \frac{p_{X,Y|Z}(x, y|z)}{p_{X|Z}(x|z)p_{Y|Z}(y|z)}$$

- 以属性为结点,条件互信息为边权构建**完全图**.
- 构建此完全图的**最大带权生成树**.
- 选定根节点并方向,加入类别节点指向所有属性.

Remark

TAN 方法保留了强相关属性间的依赖性,通常能获得比朴素贝叶斯和 SPODE 更好的分类性能.构造出来的这个生成树,就表达着每个结点的依赖关系.

- 最终仍然是计算联合概率取最大值确定具体的类别,只不过是考虑到条件概率进行计算罢了.
- 1. **AODE(平均独依赖估计)**:与其寻找一个最优的“超父”特征(如 SPODE),不如将每个特征都作为超父来构建 SPODE 模型,然后集成所有这些模型的结果. 对于一个测试实例 $\mathbf{x} = (x_1, x_2, \dots, x_d)$, AODE 的分类决策公式为:

$$\hat{y} = \arg \max_{c \in C} \sum_{\substack{i=1 \\ |D_{x_i}| \geq m'}} P(c, x_i) \prod_{j=1}^d P(x_j|c, x_i)$$

- $|D_{x_i}|$ 是在训练集中特征 i 取值为 x_i 的样本数量
- m' 是阈值(通常设置为 30),用于确保有足够的数据支持概率估计
- $P(c, x_i)$ 是类别 c 和特征 x_i 的联合概率
- $P(x_j|c, x_i)$ 是在给定类别 c 和超父特征 x_i 的条件下特征 x_j 的概率.

Remark

AODE 是基于 SPODE,通过集成学习得到的一个独依赖分类器,与 SPODE 的不同在于,SPODE 是选择一个模型来进行预测,而在 AODE 中,每个模型都进行一次预测(设置了门槛),然后将预测结果进行平均后得到最终的预测结果.不难看出,AODE 也是一种惰性学习方法,且易于实现增量学习.

[am] 这里对于概率的计算,也需要考虑用拉普拉斯平滑.

Remark

以上方法如果进一步地拓展到 kDE, 随着 k 增大,虽然可以一定程度上增加泛化能力,但是还是会免不了陷入指数爆炸的泥淖中.

Example

TAN (Tree Augmented Naive Bayes) 计算实例

- 假设我们有一个贷款违约预测问题:
- **类别 Y**: 是否违约 ($y = 1$ 违约, $y = 0$ 不违约).
 - **特征 X**: 三个二值特征 X_1 (收入高), X_2 (资产多), X_3 (已婚).

假设我们基于训练数据计算出的互信息值如下(单位: 比特):

特征对	条件互信息 $I(X_i, X_j Y)$	说明
(X_1, X_2)	0.52	强相关 (收入与资产通常挂钩)
(X_1, X_3)	0.12	弱相关

特征对	条件互信息 $I(X_i, X_j Y)$	说明
(X_2, X_3)	0.08	弱相关

我们需要构建一个图,节点是 X_1, X_2, X_3 ,边权是上面的互信息值.

- **完全图**: 所有节点两两相连.
- **生成树**: 我们要选出边权最大的边,且不构成回路.
- 选定 X_1 为树的根节点.
- 边的方向: $X_1 \rightarrow X_2, X_1 \rightarrow X_3$.
- 加入类别节点 Y ,让 Y 指向所有特征.

基于上图结构,联合概率分布 $P(X_1, X_2, X_3, Y)$ 分解为:

$$P(y, x_1, x_2, x_3) = P(y) \cdot \underbrace{P(x_1|y)}_{\text{根节点}} \cdot \underbrace{P(x_2|x_1, y)}_{\text{依赖父节点}} \cdot \underbrace{P(x_3|x_1, y)}_{\text{依赖父节点}}$$

Remark

对比朴素贝叶斯 (NB): NB 会假设 $P(x_2|x_1, y) = P(x_2|y)$. TAN 则利用 $P(x_2|x_1, y)$ 捕捉了 X_1 和 X_2 的强相关性.

计算示例: 来了一个新样本 $x_{\text{new}} = \{X_1 = 1, X_2 = 0, X_3 = 1\}$,我们要判断它是否违约 ($Y = 1$?).

我们需要比较 $P(Y = 1|x)$ 和 $P(Y = 0|x)$.以 $Y = 1$ 为例,计算分子的 Score:

$$\text{Score}_1 = P(Y = 1) \cdot P(X_1 = 1|Y = 1) \cdot P(X_2 = 0|X_1 = 1, Y = 1) \cdot P(X_3 = 1|X_1 = 1, Y = 1)$$

假设从训练数据查表得到的概率表如下:

- $P(Y = 1) = 0.5$
- $P(X_1 = 1|Y = 1) = 0.8$ (违约的人通常收入看起来不低?)
- $P(X_2 = 0|X_1 = 1, Y = 1) = 0.1$ (**关键点**: 收入高但资产少且违约的概率)
- $P(X_3 = 1|X_1 = 1, Y = 1) = 0.6$

代入计算:

$$\begin{aligned}\text{Score}_1 &= 0.5 \times 0.8 \times 0.1 \times 0.6 \\ &= 0.024\end{aligned}$$

最后对 $Y = 0$ 做同样的计算得到 Score_0 ,然后归一化即可分类:

$$P(Y = 1|x) = \frac{\text{Score}_1}{\text{Score}_1 + \text{Score}_0}$$

3.3.3 贝叶斯网络

3.3.3.1 贝叶斯网络的定义: 结构+参数

贝叶斯网络 (Bayesian network) 是一种概率图模型. 由结构 G 和参数 Θ 组成, G 有向无环, 每个点都是一个 **属性**. 边代表他们的依赖关系 (由 Θ 描述). 我们规定不连边的两个属性是独立的.

[an] 结构就是画图纸. 决定谁影响谁. 比如: 感冒 $\rightarrow$ 咳嗽. 不能画成圈 (无环), 因为因果不能循环 (你不能说“咳嗽导致感冒, 感冒又导致咳嗽”作为直接因果链).

参数就是填数据. 具体有多大的影响? 比如: $P(\text{咳嗽}|\text{感冒}) = 0.8$ (如果感冒了, 80% 概率会咳嗽).

多个事件通过条件概率连接构成有向无环图, 边的权重可理解为概率大小. 如果看到“给定节点的值”的表述, 这是指: 我们在做推断 (inference) 时, 把某些变量的值“钉死”了, 作为已知证据.

Example

假设节点 E 直接影响到节点 H , 即 $E \rightarrow H$, 则边权用条件概率 $P(H|E)$ 来表示.

形式上的, $\Theta \supseteq \theta_{x_i|\pi_i} \triangleq \mathbb{P}_B(x_i|\pi_i)$, 其中 π_i 是 x_i 的父节点集.

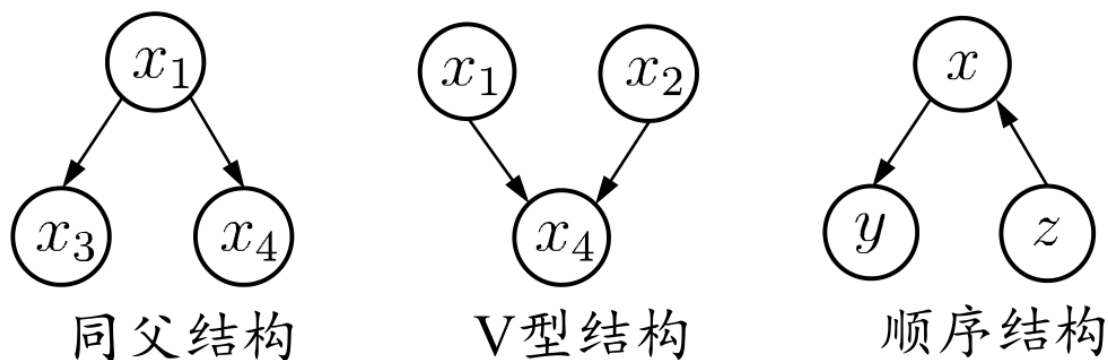


图 7 贝叶斯网中三个变量之间的典型依赖关系.条件独立性的例子:左一, x_3, x_4 ;左二, x_1, x_2 ;右一, y, z .

[ao] 总结: 什么是“阻断”的数学本质? 从数学推导可以看出,“阻断”的本质就是联合概率分布能否因式分解.

结构类型	结构图示	给定中间节点 Z ?	数学结果
链式	$X \rightarrow Z \rightarrow Y$	是	$P(X, Y Z) = P(X Z)P(Y Z)$
分叉	$X \leftarrow Z \rightarrow Y$	是	$P(X, Y Z) = P(X Z)P(Y Z)$
V-结构	$X \rightarrow Z \leftarrow Y$	否	$P(X, Y) = P(X)P(Y)$
V-结构	$X \rightarrow Z \leftarrow Y$	是	$P(X, Y Z) \neq P(X Z)P(Y Z)$

核心逻辑:

- 在链式和分叉结构中, Z 包含了 X 和 Y 之间所有的相关信息.一旦 Z 已知, X 对 Y 就没有额外的信息量了(条件独立).
- 在 V-结构中, X 和 Y 原本独立,但它们共同影响了 Z .一旦 Z 已知, X 和 Y 就必须“协同工作”来解释 Z 的状态,从而产生了依赖关系.

在这个图里,用 V 型结构做例子.我们能得到

$$\begin{aligned}
 P(x_1, x_2) &= \sum_{x_4} P(x_1, x_2, x_4) \\
 &= \sum_{x_4} P(x_4 | x_1, x_2) P(x_1) P(x_2) \\
 &= P(x_1) P(x_2)
 \end{aligned}$$

这被称为边际独立性, $x_1 \perp\!\!\!\perp x_2$.

3.3.3.2 贝叶斯网络的训练

贝叶斯网络的训练主要是**结构学习**和**参数学习**两部分.

1. 首要任务:学结构(评分搜索)

我们需要一个标准来给每个可能的结构打分,这就是评分函数.我们认为一个好的模型应该能像高效的压缩软件一样,用最短的编码来“描述”你的训练数据.而编码长度包含两部分:

- 描述模型本身的代价:** 存储这个贝叶斯网络的结构 G 和参数 Θ 需要多少比特? 网络越复杂(节点和边越多),这部分代价就越高.
- 描述数据所需的代价:** 用这个贝叶斯网络的概率分布来编码(表示)整个训练数据集 D 需要多少比特? 如果网络能很好地拟合数据,那么数据在该网络下的概率就会很高,根据信息论,概率高的事件所需的编码长度就短.

评分函数:

$$s(B|D) = f(\theta)|B| - \mathbb{L}(B|D)$$

- $|B|$: 网络 B 的参数个数,代表模型的复杂度.
- $f(\theta)$: 描述每个参数需要的平均编码位数.
- $\mathbb{L}(B|D)$: $\mathbb{L}$ 是对数似然,衡量网络 B 对数据 D 的拟合程度.值越大,拟合效果越好.

为什么要学结构?如果 G 固定,优化 $s(B|D)$ 等价于估计 Θ ,这是比较容易在训练集上得到的.

网络结构的可能数量是随变量个数指数级增长的,我们不可能穷举所有结构.因此,我们需要使用搜索算法(例如:爬山法、遗传算法等)在巨大的结构空间中进行**启发式搜索**,逐步调整网络结构(如增加、删除、反转边),朝着评分降低的方向前进.

3.3.4 吉布斯采样

如果是进行精密计算的话,那么在计算某个概率时,可能会因为全概率公式展开后求和项指数级增长,以至于非常难以计算.因此我们采用近似推断的方法,这里介绍的是**吉布斯采样**,这是一种马尔科夫链蒙特卡洛(MCMC)方法.

1. **初始化**: 随机生成一个与证据 $E = e$ 一致的样本作为起点.例如,证据是“地湿=是”,那么初始样本中这些变量的值必须固定为此,其他未知变量的值可以随机初始化.
2. **迭代采样(随机漫步)**: 从初始样本开始,进行多轮迭代.在每一轮中:
 1. 选择一个非证据变量(比如“是否下雨”)作为当前要更新的目标.
 2. 保持所有其他变量的取值不变(包括证据变量和除目标外的其他非证据变量).
 3. 根据这个“局部环境”(即该变量在贝叶斯网络中的马尔可夫毯——当前节点的父节点、子节点、子节点的其他父节点),计算出该目标变量取各种值的条件概率.
 4. 像扔骰子一样,按照这个计算出的概率分布,随机地为该目标变量赋予一个新值.
3. 重复步骤二很多次(比如 $T = 10000$ 次). 在采样过程中,我们会记录下每次采样后,查询变量 Q 的取值情况.例如,我们想知道 $Q = q$ 的概率.
 - 经过 T 次采样后,我们统计出查询变量的取值与 q 一致的样本共有 n_q 个.
 - 根据大数定律,我们可以近似估算出后验概率: $P(Q = q | E = e) \approx \frac{n_q}{T}$

[ap]

1. 它是一个马尔可夫链: 每一步采样只依赖于前一步的状态,符合马尔可夫性质.
2. 收敛到目标分布: 在满足一定条件(如遍历性)时,无论从哪个初始状态开始,其状态分布会收敛到一个唯一的平稳分布.对于吉布斯采样,这个平稳分布恰好就是我们想求的后验分布 $P(Q | E = e)$.

算法利用了贝叶斯网络的局部相关性:对于变量 X_i ,其在给定其他所有变量 X_{-i} 下的条件概率仅与其**马尔可夫毯**(也就是, X_i 的父节点、子节点、配偶节点)相关:

$$P(X_i | X_{-i}) = P(X_i | mb(X_i))$$

设证据变量为 $E = e$,待查询变量及其他变量为 Q :

1. **初始化**: 随机生成一个满足证据约束的初始样本 $q^{(0)}$.
2. **迭代采样**: 在第 t 步迭代中,依次对 Q 中的每个分量 X_i 进行更新:

$$X_i^{(t)} \sim P\left(X_i | X_1^{(t)}, \dots, X_{i-1}^{(t)}, X_{i+1}^{(t-1)}, \dots, X_n^{(t-1)}, e\right)$$

3. **收敛与平稳**: 经过足够次数的迭代(Burn-in 阶段)后,生成的样本将服从真实的后验分布 $P(Q | E = e)$.

假定总采样次数为 T ,其中与目标状态 q 一致的样本数量为 n_q ,则后验概率的近似值为:

$$P(q | E = e) \approx \frac{n_q}{T}$$

. (111)

4 支持向量机

4.1 引言

支持向量机(support vector machine, SVM)是一种二类分类模型,它的基本模型是定义在特征空间上的**间隔最大**的线性分类器,间隔最大使他有别于感知机;支持向量机还包括核技巧,这使它成为实质上的非线性分类器.

当**训练数据线性可分**时,通过硬间隔最大化,学习一个线性可分支持向量机,又称**硬间隔支持向量机**;当**训练数据近似线性可分**时,通过软间隔最大化,也学习一个线性的分类器,即**线性支持向量机**,又称为**软间隔支持向量机**;当**训练数据线性不可分**时,通过使用**核技巧**及**软间隔最大化**,学习**非线性支持向量机**.

[aq] 当输入空间为欧式空间或离散集合、特征空间为希尔伯特空间时,核函数表示将输入从输入空间映射到特征空间得到的特征向量之间的内积.通过使用核函数可以学习非线性支持向量机,等价于隐式地在高维的特征空间中学习线性支持向量机.这样的方法称为核技巧.核方法是比支持向量机更为一般的机器学习方法.

Remark

按照套路算出 α^* , 然后检查一下是否在约束范围之内, 最后用 alpha 算出 w 和 b。

硬间隔: 支持向量只能严格落在间隔边界(虚线)上。它们到分离超平面的几何间隔全部精确等于 1。

软间隔: 由于松弛变量 ξ_i 的存在, 对应的支持向量(即 $\alpha_i^* > 0$ 的点)分布非常广泛:

1. 若 $\alpha_i^* < C$, 则 $\xi_i = 0$, 点恰好落在间隔边界上。
2. 若 $\alpha_i^* = C, 0 < \xi_i < 1$, 点在间隔边界与分离超平面之间(分类正确但踩线)。

3. 若 $\alpha_i^* = C, \xi_i = 1$, 点恰好在分离超平面上。
4. 若 $\alpha_i^* = C, \xi_i > 1$, 点位于分离超平面误分的一侧。

4.2 线性可分支持向量机与硬间隔最大化

感知机只是为了能够找到正确划分的超平面而已,超平面会有无数个而且对于新样本的划分质量不高,所以评判的标准从**正确划分训练集**变成了**以最大把握划分训练集**,因而提出了**函数间隔和几何间隔**的概念.进一步地,由于间隔值取值本身不影响最优化问题的答案,所以最后得到的优化问题是:

$$\min_{w,b} \frac{1}{2} \|w\|^2, \text{ s.t. } y_i(w \cdot x_i + b) - 1 \geq 0, \quad i = 1, 2, \dots, N$$

由此得到分离超平面:

$$w^* \cdot x + b^* = 0$$

分类决策函数:

$$f(x) = \text{sign}(w^* \cdot x + b^*)$$

进一步地利用拉格朗日对偶性,转化为了:

$$\begin{aligned} \min_{\alpha} \quad & \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) - \sum_{i=1}^N \alpha_i \\ \text{s.t.} \quad & \sum_{i=1}^N \alpha_i y_i = 0, \alpha_i \geq 0, i = 1, 2, \dots, N \end{aligned}$$

进而求得原始最优优化问题的解 w^*, b^* :

$$\begin{aligned} w^* &= \sum_{i=1}^N \alpha_i^* y_i x_i \\ b^* &= y_j - \sum_{i=1}^N \alpha_i^* y_i (x_i \cdot x_j) \end{aligned}$$

4.2.0.1 线性可分支持向量机

考虑一个二分类问题,假设输入空间与特征空间为两个不同的空间,输入空间为欧式空间或离散集合,特征空间为欧式空间或希尔伯特空间.线性可分支持向量机、线性支持向量机假设这两个空间的元素一一对应,并将输入空间中的输入映射为特征空间中的特征向量.非线性支持向量机利用一个从输入空间到特征空间的非线性映射将输入映射为特征向量.所有输入都由输入空间转换到特征空间,支持向量机的学习是在特征空间进行的。

[ar] 输入空间是数据的“原始形式”或“自然状态”,而特征空间是经过精心设计和转换后、更适合机器学习模型理解的“表示形式”。

假设给定一个特征空间上的训练数据集

$$T = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$$

其中, $x_i \in \mathcal{X} = R^n, y_i \in \mathcal{Y} = \{+1, -1\}, i = 1, 2, \dots, N$. x_i 为第 i 个特征向量,也称为实例, y_i 为 x_i 的类标记.当 $y_i = +1$ 时,称 x_i 为正例;否则称为负例. (x_i, y_i) 称为样本点. **假设训练数据集是线性可分的.**

学习的目标是在特征空间中找到一个分离超平面,能将实例分到不同的类.分离超平面对应于方程 $w \cdot x + b = 0$, 它由法向量 w 和截距 b 决定,可以用 (w, b) 来表示.分离超平面将特征划分为两部分,一部分是正类,一部分是负类.法向量指向的一侧为正类,另一侧为负类.

一般地,当训练数据集线性可分时,存在无穷个分离超平面可将两类数据正确分开.感知机利用误分类最小的策略求得分离超平面,但是这样的解是无穷多个;线性可分支持向量机采用间隔最大化求最优分离超平面,这时解唯一。

Remark

这里来讲讲**希尔伯特空间**是指什么。

1. **线性空间**: 定义了加法和数乘,空间中的元素可以通过加法和数乘进行线性组合。
2. **度量空间**: 一个定义了距离的空间我们把它称为度量空间,而距离的定义需要满足:
 - 非负性、同一性: $d(x, y) \geq 0$ 当且仅当 $x = y$ 时取等号;
 - 对称性: $d(x, y) = d(y, x)$
 - 三角不等式: $d(x, z) \leq d(x, y) + d(y, z)$
3. **赋范空间**: 赋范空间就是定义了范数的空间.而范数的定义要满足:
 - 非负性: $\|x\| \geq 0$
 - 数乘性: $\|ax\| = |a| \|x\|$
 - 三角不等式: $\|x + y\| \leq \|x\| + \|y\|$

可以发现,有了范数一定能利用范数定义距离,但是有距离不能定义范数。

4. **线性赋范空间**: 如题,线性,而且要有范数.
5. **巴拿赫空间**: 就是完备的赋范空间,即: 如果一个空间是完备的,那么该空间中的任何一个柯西序列都收敛在该空间之内.柯西序列就是随着序数增加,值之间的距离越来越小的序列.(其实就是学高数的时候柯西判别数列收敛的方法) 完备要求,其实就隐含了有**距离**的概念.所以巴拿赫空间满足距离、范数、完备.
6. **内积空间**: 在一个线性空间上定义了内积运算的空间.定义了内积才能定义角度,定义正则.内积运算需要满足:
 - 严格正定性: $(x, x) \geq 0$,等号成立当且仅当 $x = 0$
 - 共轭对称性: $(x, y) = \overline{(y, x)}$
 - 对第一变量线性: $(ax + bz, y) = a(x, y) + b(z, y)$ (等价于对第二变量共轭线性)
7. **欧氏空间**: 日常空间,定义了内积的有限维实线性空间.
8. **希尔伯特空间**: 完备的内积空间.希尔伯特空间中的元素一般是函数,因为一个函数可以视为一个无穷维的向量.这个空间的基底就是一组无限多的函数.
9. **再生核希尔伯特空间**: 核技巧将不能线性可分的数据集映射到高维空间,使其变得线性可分.我们定义了一种核函数(例如径向基函数),就定义了一个希尔伯特空间,而这个核函数的再生性使得我们可以不去计算高维特征空间中的内积,而只需计算核函数,降低了大量的计算量.

Definition 4.2.1**线性可分支持向量机**

给定线性可分训练数据集,通过间隔最大化或等价地求解相应的凸二次规划问题学习得到的分离超平面为:

$$w^* \cdot x + b^* = 0$$

以及对应的分类决策函数

$$f(x) = \text{sign}(w^* \cdot x + b^*)$$

称为线性可分支持向量机.

下面我们来叙述有关间隔最大与相应约束最优化的问题.

4.2.1 函数间隔和几何间隔

一般来说,一个点距离分离超平面的远近可以表示分类预测的确信程度.在超平面 $w \cdot x + b = 0$ 确定的情况下, $|w \cdot x + b|$ 能够相对地表示点 x 距离超平面的远近.而 $w \cdot x + b$ 的符号和类标记 y 的符号是否一致能够表示分类是否正确,所以可用量 $y(w \cdot x + b)$ 来表示分类的正确性及确信度,这就是**函数间隔(functional margin)**的概念.

Definition 4.2.2**函数间隔**

对于给定的训练数据集 T 和超平面 (w, b) ,定义超平面 (w, b) 关于样本点 (x_i, y_i) 的函数间隔为:

$$\hat{\gamma}_i = y_i(w \cdot x_i + b)$$

定义超平面 (w, b) 关于训练数据集 T 的函数间隔为超平面 (w, b) 关于 T 中所有样本点 (x_i, y_i) 的函数间隔之最小值,即:

$$\hat{\gamma} = \min_{i=1,2,\dots,N} \hat{\gamma}_i$$

Definition 4.2.3**几何间隔**

但选择超平面时,只有函数间隔还不够.因为只要成比例地改变 w 和 b ,超平面并不会改变,而函数间隔却变成了两倍,因而要对 w, b 进行规范化,这时函数间隔就变成了几何间隔(geometric margin).

对于给定训练数据集 T 和超平面 (w, b) ,定义超平面 (w, b) 关于样本点的几何间隔为

$$\gamma_i = y_i \left(\frac{w}{\|w\|} \cdot x_i + \frac{b}{\|w\|} \right)$$

定义超平面 (w, b) 关于训练数据集 T 的几何间隔为超平面 (w, b) 关于 T 中所有样本点 (x_i, y_i) 的几何间隔之最小值,即:

$$\gamma = \min_{i=1,2,\dots,N} \gamma_i$$

4.2.2 间隔最大化

间隔最大化的直观解释是: 对训练数据集找到**几何间隔最大的超平面**,意味着以充分大的确信度对训练数据进行分类.也就是说,不仅将正负实例点分开,而且对最难分的实例点(离超平面最近的点)也有足够大的确信度将他们分开.这样的超平面对于未知新实例有很好的分类预测能力.

具体地,这个问题可以表示为下面的约束最优化问题:

$$\begin{aligned} \max_{w,b} \gamma \\ \text{s.t. } y_i \left(\frac{w}{\|w\|} \cdot x_i + \frac{b}{\|w\|} \right) \geq \gamma, \quad i = 1, 2, \dots, N \end{aligned}$$

我们把问题转换到函数间隔上:

$$\begin{aligned} \max_{w,b} \frac{\hat{\gamma}}{\|w\|} \\ \text{s.t. } y_i(w \cdot x_i + b) \geq \hat{\gamma}, \quad i = 1, 2, \dots, N \end{aligned}$$

而函数间隔 $\hat{\gamma}$ 的取值其实对上面的不等式约束是没有影响的(它是标量),所以可用取 $\hat{\gamma} = 1$,而如果最大化 $\frac{1}{\|w\|}$,其实和最小化 $\frac{1}{2}\|w\|^2$ 是等价的

Remark

这里能够把 $\frac{1}{\|w\|}$ 变成 $\frac{1}{2}\|w\|^2$ 其实就是依据的是单调性;而之所以会选择 $\frac{1}{2}\|w\|^2$,其实是因为他凸且光滑,求导为 w 便于使用梯度下降或拉格朗日对偶方法,是出于数学便利性.

而 $\|w\|$ 在 $w = 0$ 处不可导.

而且 $\frac{1}{2}\|w\|^2$ 是一个凸函数,且有最小值 0,具有良好的数学性质.

,所以问题转换为

$$\begin{aligned} \min_{w,b} \frac{1}{2}\|w\|^2 \\ \text{s.t. } y_i(w \cdot x_i + b) - 1 \geq 0, \quad i = 1, 2, \dots, N \end{aligned}$$

这是一个凸二次规划问题.

Theorem 4.2.4

若训练数据集 T 线性可分,则可将训练数据集中的样本点完全正确分开的最大间隔分离超平面存在且唯一.

Proof:

1. 存在性

由于训练数据集线性可分,所以算法中的最优化问题一定存在可行解.又由于目标函数有下界,所以优化问题必有解,记作 (w^*, b^*) .由于训练数据集中既有正类点又有负类点,所以 $(w, b) = (0, b)$ 不是最优化问题的可行解,因而最优解 (w^*, b^*) 必满足 $w^* \neq 0$.由此得知分离超平面的存在性.

2. 唯一性

首先证明解中 w^* 的唯一性.假设问题存在两个最优解 (w_1^*, b_1^*) 和 (w_2^*, b_2^*) .显然 $\|w_1^*\| = \|w_2^*\| = c$,其中 c 是一个常数.令 $w = \frac{w_1^* + w_2^*}{2}$, $b = \frac{b_1^* + b_2^*}{2}$,易知 (w, b) 是问题的可行解,即满足 $y_i(w \cdot x_i + b) - 1 \geq 0, \quad i = 1, 2, \dots, N$,从而有:

$$c \leq \|w\| \leq \frac{1}{2}\|w_1^*\| + \frac{1}{2}\|w_2^*\| = c$$

这样的话, $\|w\| = \|w_1^*\| = \|w_2^*\|$, $w_1^* = \lambda w_2^*$, $|\lambda| = 1$.若 $\lambda = -1$,则 $w = 0$, (w, b) 不是问题的可行解,出现矛盾;所以 $\lambda = 1$,这样就有 $w_1^* = w_2^*$,我们就证明出了 w^* 的唯一性.

下面我们证明 $b_1^* = b_2^*$.设 x_1' 和 x_2' 是集合 $\{x_i \mid y_i = +1\}$ 中分别对应于 (w^*, b_1^*) 和 (w^*, b_2^*) 使得问题的不等式等号成立的点, x_1'' 和 x_2'' 是集合 $\{x_i \mid y_i = -1\}$ 中分别对应于 (w^*, b_1^*) 和 (w^*, b_2^*) 使得问题的不等式等号成立的点,则有:

$$b_1^* - b_2^* = -\frac{1}{2}w^* \cdot (x_1' + x_1'' - x_2' - x_2'')$$

又因为:

$$\begin{aligned} w^* \cdot x_2' + b_1^* &\geq 1 = w^* \cdot x_1' + b_1^* \\ w^* \cdot x_1' + b_2^* &\geq 1 = w^* \cdot x_2' + b_2^* \end{aligned}$$

所以, $w^* \cdot (x_1' - x_2') = 0$,同理有 $w^* \cdot (x_1'' - x_2'') = 0$.因此

$$b_1^* = b_2^*$$

所以解是唯一的.

综上,解存在且唯一. □

Definition 4.2.5

支持向量

在线性可分情况下,训练数据集的样本点中,与分离超平面距离最近的样本点的实例称为**支持向量**(support vector).支持向量是使约束条件等号成立的点,即:

$$y_i(w \cdot x_i + b) - 1 = 0$$

对 $y_i = +1$ 的正实例点,支持向量在超平面

$$H_1 : w \cdot x + b = 1$$

上,对 $y_i = -1$ 的负例点,支持向量在超平面

$$H_2 : w \cdot x + b = -1$$

上.在 H_1 和 H_2 上的点就是**支持向量**.

Remark

其实就是卡住最大间隔超平面的那些点,被称为支持向量.注意 SV 是点.

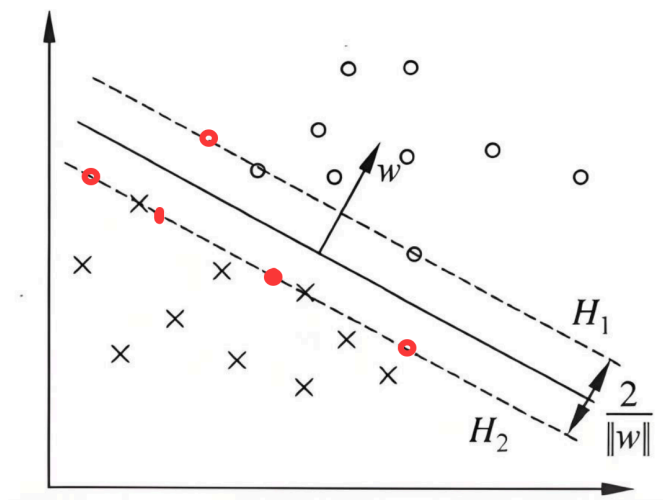


图 8 红色的东西就是支持向量.线(无论是虚线还是实线)都是超平面.

H_1 与 H_2 平行,并且没有实例点落在他们中间,在二者之间形成一个长带,分离超平面与他们平行且位于他们中央.长带的宽度,即 H_1 与 H_2 之间的距离称为**间隔 (margin)**.间隔的长度为 $\frac{2}{\|w\|}$, H_1 和 H_2 称为**间隔边界**.

[as] 由于支持向量在确定分离超平面中起着决定性作用,所以将这种分类模型称为支持向量机.支持向量的个数一般很少,所以支持向量机由很少的“重要的”训练样本确定.

4.2.3 求解: 线性可分支持向量机的最优化问题

上面的问题为原始最优化问题,应用拉格朗日对偶性,通过求解对偶问题得到原始问题的最优解,就是线性可分支持向量机的对偶算法(dual algorithm).

Remark

这样做的优点有:

1. 对偶问题往往容易求解.
2. 自然引入核函数,从而推广到非线性分类问题.

首先构建拉格朗日函数,为此对每一个不等式约束引入拉格朗日乘子 $\alpha_i \geq 0, i = 1, 2, \dots, N$, 定义拉格朗日函数:

$$L(w, b, \alpha) = \frac{1}{2} \|w\|^2 - \sum_{i=1}^N \alpha_i y_i (w \cdot x_i + b) + \sum_{i=1}^N \alpha_i$$

其中, $\alpha = (\alpha_1, \alpha_2, \dots, \alpha_N)^T$ 为拉格朗日乘子向量.

根据拉格朗日对偶性,原始问题的对偶问题是极大极小问题:

[at] 直观理解: 外层 $\max_{\alpha}$ 寻找使拉格朗日函数最大的乘子(惩罚违反约束的情形), 内层 $\min_{w, b}$ 在给定乘子下最小化目标.若约束 $h_{i(x)} \geq 0$ 被违反,拉格朗日乘子 α_i 会使对应项趋于无穷大,迫使 $\min$ 在优化时回到可行域内.

$$\begin{aligned}
& \max_{\alpha} \min_{w,b} L(w, b, \alpha) \\
& \nabla_w L(w, b, \alpha) = w - \sum_{i=1}^N \alpha_i y_i x_i = 0 \\
& \nabla_b L(w, b, \alpha) = - \sum_{i=1}^N \alpha_i y_i = 0 \\
& \Rightarrow w = \sum_{i=1}^N \alpha_i y_i x_i \\
& \Rightarrow \sum_{i=1}^N \alpha_i y_i = 0
\end{aligned}$$

带入到拉格朗日函数,得到:

$$\begin{aligned}
L(w, b, \alpha) &= \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) - \sum_{i=1}^N \alpha_i y_i \left[\left(\sum_{j=1}^N \alpha_j y_j x_j \right) \cdot x_i + b \right] + \sum_{i=1}^N \alpha_i \\
&= -\frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) + \sum_{i=1}^N \alpha_i
\end{aligned}$$

[au] 我们仍然习惯上把极大问题转换为极小问题,得到

$$\begin{aligned}
& \min_{\alpha} \quad \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) - \sum_{i=1}^N \alpha_i \\
& s.t. \quad \sum_{i=1}^N \alpha_i y_i = 0, \alpha_i \geq 0, i = 1, 2, \dots, N
\end{aligned}$$

现在对偶问题就转换为了

$$\begin{aligned}
& \max_{\alpha} \quad -\frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) + \sum_{i=1}^N \alpha_i \\
& s.t. \quad \sum_{i=1}^N \alpha_i y_i = 0, \alpha_i \geq 0, i = 1, 2, \dots, N
\end{aligned}$$

原始问题满足拉格朗日对偶条件,所以存在 w^*, α^* ,使 w^* 是原始问题的解, α^* 是对偶问题的解.

Theorem 4.2.6

设 $\alpha^* = (\alpha_1^*, \alpha_2^*, \dots, \alpha_l^*)^T$ 是对偶最优化问题的解,则存在下标 j ,使得 $\alpha_j^* > 0$,并可按下式求得原始最优化问题的解 w^*, b^* :

$$\begin{aligned}
w^* &= \sum_{i=1}^N \alpha_i^* y_i x_i \\
b^* &= y_j - \sum_{i=1}^N \alpha_i^* y_i (x_i \cdot x_j)
\end{aligned}$$

Proof: 易知:

$$\begin{aligned}
& \nabla_w L(w^*, b^*, \alpha^*) = w^* - \sum_{i=1}^N \alpha_i^* y_i x_i = 0 \\
& \nabla_b L(w^*, b^*, \alpha^*) = - \sum_{i=1}^N \alpha_i^* y_i = 0 \\
& \alpha_i^* (y_i (w^* \cdot x_i + b^*) - 1) = 0, \quad i = 1, 2, \dots, N
\end{aligned}$$

[av] KKT 条件的互补松弛性: 上面这个式子,称为互补松弛条件.对于每个约束,拉格朗日乘子与约束的“松弛”的乘积为 0.这意味着:

- 如果 $\alpha_i^* > 0$,则必须有 $y_i (w^* \cdot x_i + b^*) - 1 = 0$,即样本点 (x_i, y_i) 恰好落在分类间隔的边界上(这些点称为支持向量).
- 如果 $y_i (w^* \cdot x_i + b^*) - 1 > 0$ (即点被正确分类且不在边界上),则必须有 $\alpha_i^* = 0$,表示这些点对最终分类器没有贡献.

这个条件确保了最优解只有支持向量(边界上的点)影响模型参数.

$$\begin{aligned}
& y_i (w^* \cdot x_i + b^*) - 1 \geq 0, \quad i = 1, 2, \dots, N \\
& \alpha_i^* \geq 0, \quad i = 1, 2, \dots, N
\end{aligned}$$

由此得：

$$w^* = \sum_i \alpha_i^* y_i x_i$$

其中至少有一个 $\alpha_j^* > 0$ (用反证法,假设 $\alpha^* = 0$,那么 $w^* = 0$,而这不是原始最优化问题的解,矛盾),对此 j 有:

$$y_j(w^* \cdot x_j + b^*) - 1 = 0$$

而且:

$$y_j^2 = 1$$

所以有:

$$b^* = y_j - \sum_{i=1}^N \alpha_i^* y_i (x_i \cdot x_j)$$

□

那么分离超平面可以写成

$$\sum_{i=1}^N \alpha_i^* y_i (x \cdot x_i) + b^* = 0$$

分类决策函数可以写成

$$f(x) = \text{sign} \left[\sum_{i=1}^N \alpha_i^* y_i (x \cdot x_i) + b^* \right]$$

Remark

这就是说,分类决策函数只依赖于输入 x 和所有训练样本输入的内积.在预测阶段,新点 x 的预测,只需要计算它与支持向量的内积即可.

综上所述,我们得到以下算法

Algorithm 4.2.7

线性可分支持向量机

输入: 训练数据集 $T = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$, 其中, $x_i \in \mathcal{X} = R^n, y_i \in \mathcal{Y} = \{-1, +1\}, i = 1, 2, \dots, N$.

输出: 分离超平面和分类决策函数.

1. 构造并求解凸二次规划问题,求得最优解 $\alpha^* = (\alpha_1^*, \alpha_2^*, \dots, \alpha_N^*)^T$:

$$\begin{aligned} \min_{\alpha} \quad & \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) - \sum_{i=1}^N \alpha_i \\ \text{s.t.} \quad & \sum_{i=1}^N \alpha_i y_i = 0, \alpha_i \geq 0, i = 1, 2, \dots, N \end{aligned}$$

2. 计算 $w^* = \sum_{i=1}^N \alpha_i^* y_i x_i$
选择 α^* 的一个满足条件 $\alpha_j^* > 0$ 的分量 α_j^* , 计算

$$b^* = y_j - \sum_{i=1}^N y_i \alpha_i^* (x_i \cdot x_j)$$

或者用: $b^* = y_j - w^* \cdot x_j$ 亦可。

3. 求得分离超平面

$$\sum_{i=1}^N \alpha_i^* y_i (x \cdot x_i) + b^* = 0$$

分类决策函数

$$f(x) = \text{sign} \left[\sum_{i=1}^N \alpha_i^* y_i (x \cdot x_i) + b^* \right]$$

在现实问题中,训练数据集往往线性不可分,样本中出现噪声或特一点,此时有更一般的学习算法.

Example

1. 准备数据

假设有 3 个样本点 (2 个正例, 1 个负例):

- 正例 ($y = +1$): $x_1 = (3, 3)^T, x_2 = (4, 3)^T$
- 负例 ($y = -1$): $x_3 = (1, 1)^T$

2. 第一步: 计算 Gram 矩阵 (内积矩阵)

我们需要计算所有样本两两之间的内积 $x_i \cdot x_j$:

$$G = [x_i \cdot x_j] = \begin{pmatrix} 18 & 21 & 6 \\ 21 & 25 & 7 \\ 6 & 7 & 2 \end{pmatrix}$$

计算细节示例: $x_1 \cdot x_1 = 3 \times 3 + 3 \times 3 = 18$.

3. 第二步: 构建对偶问题

对偶问题 (极小值形式) 的目标函数为:

$$\min_{\alpha} \frac{1}{2} \sum_{i=1}^3 \sum_{j=1}^3 \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) - \sum_{i=1}^3 \alpha_i$$

约束条件:

$$\sum_{i=1}^3 \alpha_i y_i = 0 \rightarrow \alpha_1 + \alpha_2 - \alpha_3 = 0 \rightarrow \alpha_3 = \alpha_1 + \alpha_2$$

$$\alpha_i \geq 0, \quad i = 1, 2, 3$$

将 Gram 矩阵和 y 值代入目标函数, 并利用约束消去 α_3 , 得到关于 α_1, α_2 的函数 $S(\alpha_1, \alpha_2)$:

$$S(\alpha_1, \alpha_2) = 4\alpha_1^2 + \frac{13}{2}\alpha_2^2 + 10\alpha_1\alpha_2 - 2\alpha_1 - 2\alpha_2$$

4. 第三步: 求解最优 α^*

对 α_1, α_2 求偏导并令其为 0:

$$\begin{cases} \frac{\partial S}{\partial \alpha_1} = 8\alpha_1 + 10\alpha_2 - 2 = 0 \\ \frac{\partial S}{\partial \alpha_2} = 13\alpha_2 + 10\alpha_1 - 2 = 0 \end{cases}$$

联立解得 $\alpha_1 = 1.5, \alpha_2 = -1$. 由于 $\alpha_2 = -1 < 0$ 违反约束, 说明极值在边界上. 放弃那个越界变量的偏导方程, 令 $\alpha_2 = 0$, 代入第一个方程 $4\alpha_1 = 1$, 得 $\alpha_1 = 0.25$. 此时 $\alpha_3 = \alpha_1 + \alpha_2 = 0.25$.

故 最 优 解 为 : $\alpha^* = (0.25, 0, 0.25)^T$

[aw] 固定 α_2 后, 我们失去了在 α_2 方向调节的自由, 所以不能要求 α_2 方向的导数为 0; 但我们在 α_1 方向依然是自由的, 必须保证该方向的导数为 0. 因而这里必须带入第 1 个方程求得 α_1 , 而不能代入第 2 个方程.

5. 第四步: 计算原始参数 w^* 和 b^*

计算 w^* :

$$w^* = \sum_{i=1}^3 \alpha_i^* y_i x_i = 0.25 \cdot x_1 + 0 \cdot x_2 + 0.25 \cdot (-1) \cdot x_3$$

$$w^* = 0.25 \begin{pmatrix} 3 \\ 3 \end{pmatrix} - 0.25 \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 0.5 \\ 0.5 \end{pmatrix}$$

计算 b^* : 选择支持向量 x_1 (因为 $\alpha_1^* > 0$), 利用 $b^* = y_1 - w^* \cdot x_1$:

$$b^* = 1 - (0.5 \times 3 + 0.5 \times 3) = 1 - 3 = -2$$

(注: 若使用 x_3 验证, 结果也是 -2)

6. 最终结果

分离超平面方程: $0.5x^{(1)} + 0.5x^{(2)} - 2 = 0$ 整理得: $x^{(1)} + x^{(2)} - 4 = 0$

分类决策函数: $f(x) = \text{sign}(0.5x^{(1)} + 0.5x^{(2)} - 2)$

此例直观展示了 SVM 的稀疏性: x_2 对应的 $\alpha_2^* = 0$, 它不是支持向量, 对最终模型没有影响.

4.3 线性支持向量机与软间隔最大化

4.3.1 线性支持向量机与软间隔最大化

4.3.1.1 线性支持向量机

通常训练数据集不是线性可分的,意味着某些样本点 (x_i, y_i) 不能满足函数间隔大于等于1的约束条件.为了解决这个问题,可以对每个样本点 (x_i, y_i) 引入一个松弛变量 $\xi_i \geq 0$,使函数间隔加上松弛变量大于等于1.这样,约束条件变为

$$y_i(w \cdot x_i + b) \geq 1 - \xi_i$$

同时,对于每个松弛变量 ξ_i ,支付一个代价 ξ_i .目标函数变成:

$$\frac{1}{2} \|w\|^2 + C \sum_{i=1}^N \xi_i$$

Remark

这里面的代价形式其实是不一定的,可以是二次的,对数的等等.

这里, $C > 0$ 称为惩罚参数,一般由应用问题决定, C 值大时对误分类的惩罚增大, C 值小时对误分类的惩罚减小.

最小化目标函数有两层含义: 使 $\frac{1}{2} \|w\|^2$ 尽量小即间隔尽量大,同时使误分类点的个数尽量少, C 是调和二者的系数.这称为**软间隔最大化**.

线性不可分的线性支持向量机的学习问题变成如下凸二次规划问题 (原始问题):

$$\begin{aligned} \min_{w, b, \xi} \quad & \frac{1}{2} \|w\|^2 + C \sum_{i=1}^N \xi_i \\ \text{s.t.} \quad & y_i(w \cdot x_i + b) \geq 1 - \xi_i, \quad i = 1, 2, \dots, N \\ & \xi_i \geq 0, \quad i = 1, 2, \dots, N \end{aligned}$$

这是一个凸二次规划问题,因而关于 (w, b, ξ) 的解是存在的.可以证明 w 的解是唯一的,而 b 的解可能不唯一,而是存在于一个区间.

设原始问题的解是 w^*, b^* ,于是可以得到分离超平面 $w^* \cdot x + b^* = 0$ 及分类决策函数 $f(x) = \text{sign}(w^* \cdot x + b^*)$,称这样的模型为**训练样本线性不可分时的线性支持向量机**,简称为**线性支持向量机**.显然,线性支持向量机包含线性可分支持向量机.

Definition 4.3.1

线性支持向量机

对于给定的线性不可分的训练数据集,通过求解凸二次规划问题,即软间隔最大化问题,得到的分离超平面为

$$w^* \cdot x + b^* = 0$$

以及对应的分类决策函数

$$f(x) = \text{sign}(w^* \cdot x + b^*)$$

称为线性支持向量机.

4.3.2 学习的对偶算法

为了构建拉格朗日函数,我们将不等式约束对应到乘子:

- 针对分类间隔约束 $y_i(w \cdot x_i + b) - 1 + \xi_i \geq 0$, 引入乘子 $\alpha_i \geq 0$.
- 针对松弛变量非负约束 $\xi_i \geq 0$, 引入乘子 $\mu_i \geq 0$.

根据拉格朗日对偶性 (对于 $g(x) \geq 0$ 的约束, 构建 $L = f(x) - \sum \lambda g(x)$), 我们在目标函数中减去乘子与约束的乘积:

$$L(w, b, \xi, \alpha, \mu) = \underbrace{\frac{1}{2} \|w\|^2 + C \sum_{i=1}^N \xi_i}_{\text{原始目标}} - \underbrace{\sum_{i=1}^N \alpha_i [y_i(w \cdot x_i + b) - 1 + \xi_i]}_{\text{间隔约束}} - \underbrace{\sum_{i=1}^N \mu_i \xi_i}_{\text{非负}}$$

$$\alpha_i \geq 0, \mu_i \geq 0$$

对偶问题是拉格朗日函数的极大极小问题,首先求极小,由

$$\nabla_w L(w, b, \xi, \alpha, \mu) = w - \sum_{i=1}^N \alpha_i y_i x_i = 0$$

$$\nabla_b L(w, b, \xi, \alpha, \mu) = - \sum_{i=1}^N \alpha_i y_i = 0$$

$$\nabla_{\xi_i} L(w, b, \xi, \alpha, \mu) = C - \alpha_i - \mu_i = 0$$

得:

$$\begin{aligned}
w &= \sum_{i=1}^N \alpha_i y_i x_i \\
\sum_{i=1}^N \alpha_i y_i &= 0 \\
C - \alpha_i - \mu_i &= 0
\end{aligned}$$

带回可得:

$$\min_{w, b, \xi} L(w, b, \xi, \alpha, \mu) = -\frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) + \sum_{i=1}^N \alpha_i$$

再对其求 α 的极大,即得对偶问题:

$$\begin{aligned}
\max_{\alpha} \quad & -\frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) + \sum_{i=1}^N \alpha_i \\
s.t. \quad & \sum_{i=1}^N \alpha_i y_i = 0, C - \alpha_i - \mu_i = 0, \alpha_i \geq 0, \mu_i \geq 0, \quad i = 1, 2, \dots, N
\end{aligned}$$

Theorem 4.3.2

设 $\alpha^* = (\alpha_1^*, \alpha_2^*, \dots, \alpha_N^*)^T$ 是对偶问题的一个解.若存在 α^* 的一个分量 $\alpha_j^*, 0 < \alpha_j^* < C$,则原始问题的解 w^*, b^* 可以按下式求解:

$$\begin{aligned}
w^* &= \sum_{i=1}^N \alpha_i^* y_i x_i \\
b^* &= y_j - \sum_{i=1}^N y_i \alpha_i^* (x_i \cdot x_j)
\end{aligned}$$

Proof: 原始问题是凸二次规划问题,解满足 KKT 条件,即得:

$$\begin{aligned}
\nabla_w L(w^*, b^*, \xi^*, \alpha^*, \mu^*) &= w^* - \sum_{i=1}^N \alpha_i^* y_i x_i = 0 \\
\nabla_b L(w^*, b^*, \xi^*, \alpha^*, \mu^*) &= - \sum_{i=1}^N \alpha_i^* y_i = 0 \\
\nabla_{\xi} L(w^*, b^*, \xi^*, \alpha^*, \mu^*) &= C - \alpha^* - \mu^* = 0 \\
\alpha_i^* (y_i (w^* \cdot x_i + b^*) - 1 + \xi_i^*) &= 0
\end{aligned}$$

[ax] 这个式子的作用与前面线性可分支持向量机的是一致的.

$$\mu_i^* \xi_i^* = 0$$

[ay]

- 这个式子与上面的式子类似,如果松弛变量 $\xi_i^* > 0$,说明该样本点被允许违反间隔约束,则必须有 $\mu_i^* = 0$
- 反之,如果拉格朗日乘子 $\mu_i^* > 0$,说明对松弛惩罚是活跃的,则必须有 $\xi_i^* = 0$,说明该点严格遵守了间隔约束.

$$\begin{aligned}
y_i (w^* \cdot x_i + b^*) - 1 + \xi_i^* &\geq 0 \\
\xi_i^* \geq 0, \alpha_i^* \geq 0, \mu_i^* \geq 0, \quad i &= 1, 2, \dots, N
\end{aligned}$$

若存在 $\alpha_j^*, 0 < \alpha_j^* < C$,则 $y_i (w^* \cdot x_i + b^*) - 1 = 0$,则有:

$$\begin{aligned}
w^* &= \sum_{i=1}^N \alpha_i^* y_i x_i \\
b^* &= y_j - \sum_{i=1}^N y_i \alpha_i^* (x_i \cdot x_j)
\end{aligned}$$

□

由此可知,分离超平面可以写成

$$\sum_{i=1}^N \alpha_i^* y_i (x \cdot x_i) + b^* = 0$$

分类决策函数可以写成

$$f(x) = \text{sign} \left[\sum_{i=1}^N \alpha_i^* y_i (x \cdot x_i) + b^* \right]$$

综上所述,我们得到以下算法

Algorithm 4.3.3

线性支持向量机

输入: 训练数据集 $T = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$, 其中, $x_i \in \mathcal{X} = R^n, y_i \in \mathcal{Y} = \{-1, +1\}, i = 1, 2, \dots, N$.

输出: 分离超平面和分类决策函数.

1. 选择惩罚参数 $C > 0$, 构造并求解凸二次规划问题, 求得最优解 $\alpha^* = (\alpha_1^*, \alpha_2^*, \dots, \alpha_N^*)^T$:

$$\begin{aligned} \min_{\alpha} \quad & \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) - \sum_{i=1}^N \alpha_i \\ \text{s.t.} \quad & \sum_{i=1}^N \alpha_i y_i = 0, 0 \leq \alpha_i \leq C, i = 1, 2, \dots, N \end{aligned}$$

2. 计算 $w^* = \sum_{i=1}^N \alpha_i^* y_i x_i$
选择 α^* 的一个分量 α_j^* 满足条件 $0 < \alpha_j^* < C$, 计算

$$b^* = y_j - \sum_{i=1}^N y_i \alpha_i^* (x_i \cdot x_j)$$

3. 求得分离超平面

$$\sum_{i=1}^N \alpha_i^* y_i (x \cdot x_i) + b^* = 0$$

分类决策函数

$$f(x) = \text{sign} \left[\sum_{i=1}^N \alpha_i^* y_i (x \cdot x_i) + b^* \right]$$

理论上 b 的解可能是不唯一的, 但实际意义中, 往往只会出现算法叙述的情况.

Example

虽然该数据是线性可分的, 但通过设定参数 C , 完全可以演示软间隔算法的通用步骤. 当 C 足够大时, 软间隔解将收敛于硬间隔解.

1. 输入数据与参数设定

我们将惩罚参数设定为 $C = 1$. 训练数据 T 包含 3 个样本点:

- 正例 ($y = +1$): $x_1 = (3, 3)^T, x_2 = (4, 3)^T$
- 负例 ($y = -1$): $x_3 = (1, 1)^T$

2. 第一步: 构造并求解对偶问题

我们需要求解以下带约束的凸二次规划问题:

$$\min_{\alpha} \frac{1}{2} \sum_{i=1}^3 \sum_{j=1}^3 \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) - \sum_{i=1}^3 \alpha_i$$

约束条件变为 (注意增加了 $\alpha_i \leq C$):

$$\begin{cases} \sum_{i=1}^3 \alpha_i y_i = \alpha_1 + \alpha_2 - \alpha_3 = 0 \\ 0 \leq \alpha_i \leq 1, \quad i = 1, 2, 3 \end{cases}$$

代入 Gram 矩阵 (内积) 并利用 $\alpha_3 = \alpha_1 + \alpha_2$ 消元后, 目标函数 $S(\alpha_1, \alpha_2)$ 与硬间隔情况一致:

$$S(\alpha_1, \alpha_2) = 4\alpha_1^2 + \frac{13}{2}\alpha_2^2 + 10\alpha_1\alpha_2 - 2\alpha_1 - 2\alpha_2$$

我们对 α_1, α_2 求偏导并令其为 0, 联立解得无约束极值点:

$$\alpha_1 = 1.5, \quad \alpha_2 = -1$$

这不满足 $0 \leq \alpha_i$ 的约束. 最小值必然在边界上.

经过边界分析 (令 $\alpha_2 = 0$), 我们得到最优解:

$$\alpha_1^* = 0.25, \quad \alpha_2^* = 0$$

从而推导 $\alpha_3^* = \alpha_1^* + \alpha_2^* = 0.25$.

关键检查：验证约束条件 $0 \leq \alpha_i^* \leq C$ 此时解为 $\alpha^* = (0.25, 0, 0.25)^T$. 由于我们要保证 $\alpha_i^* \leq 1$ (即 $C = 1$), 显然 $0.25 \leq 1$ 成立. 所以该解在可行域内, 是本问题的最优解.

3. 第二步：计算 w^* 和 b^*

计算 w^* :

$$w^* = \sum_{i=1}^3 \alpha_i^* y_i x_i = 0.25x_1 - 0.25x_3 = \begin{pmatrix} 0.5 \\ 0.5 \end{pmatrix}$$

计算 b^* : 根据算法步骤, 我们需要选择一个分量 α_j^* , 满足 **严格不等式**:

$$0 < \alpha_j^* < C$$

检查我们的解:

- $\alpha_1^* = 0.25$, 满足 $0 < 0.25 < 1$.
- $\alpha_3^* = 0.25$, 满足 $0 < 0.25 < 1$.

我们可以选择 $j = 1$ (即利用支持向量 x_1) 来计算 b^* :

$$\begin{aligned} b^* &= y_1 - \sum_{i=1}^3 y_i \alpha_i^* (x_i \cdot x_1) \\ &= 1 - [1 \cdot 0.25 \cdot (18) + 1 \cdot 0 \cdot (21) + (-1) \cdot 0.25 \cdot (6)] \\ &= 1 - [4.5 - 1.5] \\ &= 1 - 3 = -2 \end{aligned}$$

(注: 如果选取 $j = 3$ 计算, 结果同样是 -2 . 这体现了 KKT 条件的一致性.)

4. 第三步：输出结果

分离超平面:

$$0.5x^{(1)} + 0.5x^{(2)} - 2 = 0$$

分类决策函数:

$$f(x) = \text{sign}(0.5x^{(1)} + 0.5x^{(2)} - 2)$$

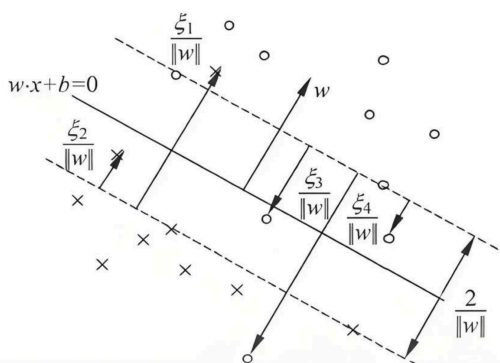
结果分析: 在这个例子中, 因为数据本身线性可分且我们给的 C 比较宽松 ($C = 1 > 0.25$), 所有的支持向量 (x_1, x_3) 都落在间隔边界上 ($\xi_i = 0$), 所以 $0 < \alpha_i < C$.

如果我们将 C 设得极小 (例如 $C = 0.1$), 那么 α_1, α_3 将会被强制截断在 0.1 处, 此时 $\alpha_i = C$, 这些点就会变成”误分类”或”间隔内”的支持向量.

4.3.3 支持向量

在线性不可分的情况下, 将对偶问题的解 $\alpha^* = (\alpha_1^*, \alpha_2^*, \dots, \alpha_N^*)^T$ 中对应于 $\alpha_i^* > 0$ 的样本点 (x_i, y_i) 的实例 x_i 称为支持向量 (软间隔的支持向量). 软间隔的支持向量 x_i 或者在间隔边界上, 或者在间隔边界与分离超平面之间, 或者在分离超平面误分一侧.

实例 x_i , 到间隔边界的距离 $\frac{\xi_i}{\|w\|}$ 。



- (一) $\alpha^* = 0$, 位于本侧支持面的外侧, 分类正确 (非支持向量)
- (二) $0 < \alpha^* < C$, 则有 $\xi^* = 0$, 落在支持面上, 分类正确
- (三) $\alpha^* = C$: (1) $0 < \xi^* < 1$, 落在缓冲区内, 分类正确
- (2) $\xi^* = 1$, 落在分隔面上, 分类任意
- (3) $\xi^* > 1$, 落在分隔面错误的一侧, 分类错误 (越过分隔面)

[az]

- 若 $\alpha_i^* < C$, 则 $\xi_i = 0$, 支持向量 x_i 恰好落在间隔边界上;
- 若 $\alpha_i^* = C, 0 < \xi_i < 1$, 则分类正确, x_i 在间隔边界与分离超平面之间;
- 若 $\alpha_i^* = C, \xi_i = 1$, 则 x_i 在分离超平面上;
- 若 $\alpha_i^* = C, \xi_i > 1$, 则 x_i 位于分离超平面误分一侧。

图 9 一个漂亮的图例子

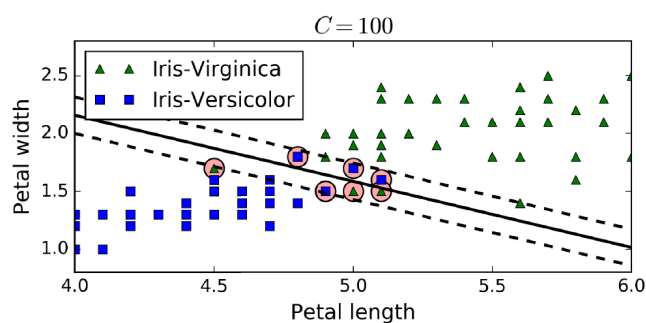


图 10 使用高 C 值 SVM, 间隔违例较少, 但是间隔也较小

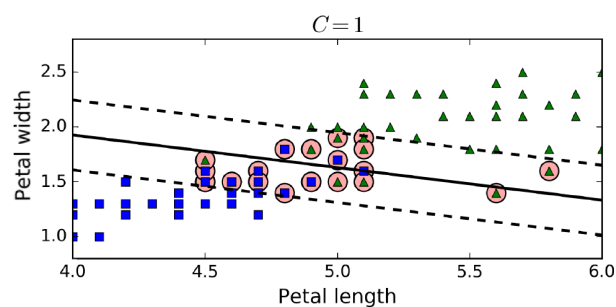


图 11 使用低 C 值 SVM, 间隔违例较多, 但是间隔较大, 泛化能力更强

4.3.4 合页损失函数

线性支持向量机学习还有另一种解释, 就是最小化以下目标函数:

$$\sum_{i=1}^N [1 - y_i(w \cdot x_i + b)]_+ + \lambda \|w\|^2$$

目标函数的第 1 项是经验损失或经验风险, 函数

$$L(y(w \cdot x + b)) = [1 - y(w \cdot x + b)]_+$$

称为合页损失函数(hinge loss function). 下标“+”表示以下取正值的函数:

$$[z]_+ = \begin{cases} z & z > 0 \\ 0 & z \leq 0 \end{cases}$$

这就是说, 当样本点 (x_i, y_i) 被正确分类且函数间隔 (确信度) $y_i(w \cdot x_i + b) > 1$ 时, 损失是 0; 否则损失为 $1 - y_i(w \cdot x_i + b)$; 目标函数的第 2 项是系数是 λ 的 w 的 L_2 范数, 是正则化项。

Theorem 4.3.4

线性支持向量机原始最优化问题

$$\begin{aligned} \min_{w,b,\xi} \quad & \frac{1}{2}\|w\|^2 + C \sum_{i=1}^N \xi_i \\ \text{s.t.} \quad & y_i(w \cdot x_i + b) \geq 1 - \xi_i, \xi_i \geq 0, \quad i = 1, 2, \dots, N \end{aligned}$$

等价于最优化问题

$$\min_{w,b} \sum_{i=1}^N [1 - y_i(w \cdot x_i + b)]_+ + \lambda \|w\|^2$$

Proof: 令

$$[1 - y_i(w \cdot x_i + b)]_+ = \xi_i$$

则 $\xi_i \geq 0$. 当 $1 - y_i(w \cdot x_i + b) > 0$ 时, 有 $y_i(w \cdot x_i + b) = 1 - \xi_i$; 当 $1 - y_i(w \cdot x_i + b) \leq 0$ 时, $\xi_i = 0$, 所以 $y_i(w \cdot x_i + b) \geq 1 - \xi_i$; 所以优化式就转化成了

$$\min_{w,b} \sum_{i=1}^N \xi_i + \lambda \|w\|^2$$

若取 $\lambda = \frac{1}{2C}$, 则:

$$\min_{w,b} \frac{1}{C} \left(\frac{1}{2}\|w\|^2 + C \sum_{i=1}^N \xi_i \right)$$

与原始优化问题等价. □

合页损失函数是 0-1 损失函数的上界. 由于 0-1 损失函数不是连续可导的, 直接优化由其构成的目标函数比较困难, 可以认为线性支持向量机是优化由 0-1 损失函数的上界构成的目标函数. 这时的上界损失函数又称为代理损失函数(surrogate loss function).

Remark

当样本点 (x_i, y_i) 被正确分类时, 损失是 0, 否则损失是 $-y_i(w \cdot x_i + b)$, 相比之下, 合页损失函数不仅要求分类正确, 而且确信度足够高时损失才是 0. 也就是说, 合页损失函数对学习有更高的要求.

4.4 非线性支持向量机与核函数

非线性分类问题是指通过利用非线性模型才能很好地进行分类的问题. [ba] 醍醐灌顶! 一般来说, 如果能用 R^n 中的一个超曲面将 (之前提到的) 正负例正确分开, 则称这个问题为非线性可分问题.

Remark

所采取的方法是进行一个非线性变换, 将非线性问题变换为线性问题, 通过解变换后的线性问题的方法求解原来的非线性问题.

设原空间为 $\mathcal{X} \subset R^2, x = (x^{(1)}, x^{(2)})^T \in \mathcal{X}$, 新空间 $\mathcal{Z} \subset R^2, z = (z^{(1)}, z^{(2)})^T \in \mathcal{Z}$, 定义在原空间为 $\mathcal{X} \subset R^2$ 变换为新空间 $\mathcal{Z} \subset R^2$, 原空间中的点相应地变换为新空间中的点, 原空间中的椭圆

$$w_1(x^{(1)})^2 + w_2(x^{(2)})^2 + b = 0$$

变换成为新空间中的直线

$$w_1 z^{(1)} + w_2 z^{(2)} + b = 0$$

这样, 原空间的非线性可分问题就变成了新空间的线性可分问题.

1. 使用一个变换将原空间的数据映射到新空间.
2. 新空间里用线性分类学习方法进行学习.

Remark

核技巧属于这种方法. 通过一个非线性变换将输入空间 (欧氏空间 R^n 或离散集合) 对应于一个特征空间 (希尔伯特空间 $\mathcal{H}$), 使得在输入空间 R^n 中的超曲面模型对应于特征空间 $\mathcal{H}$ 中的超平面模型 (支持向量机). 根据 Remark 4.2.7, 我们能知道 SVM 的关键是算内积, 所以我们不需要知道变换本身 (参考下面的定义, 我们只需要 K , 甚至不需要知道 ϕ).

Definition 4.4.1

设 $\mathcal{X}$ 是输入空间 (欧氏空间 R^n 的子集或离散集合), 又设 $\mathcal{H}$ 为特征空间 (希尔伯特空间), 如果存在一个从 $\mathcal{X}$ 到 $\mathcal{H}$ 的映射

$$\phi(x) : \mathcal{X} \rightarrow \mathcal{H}$$

使得对所有 $x, z \in \mathcal{X}$, 函数 $K(x, z)$ 满足条件

$$K(x, z) = \phi(x) \cdot \phi(z)$$

则称 $K(x, z)$ 为核函数, $\phi(x)$ 为映射函数, 式中的运算为内积.

核函数一定有

$$K(x, z) = K(z, x)$$

Remark

虽然在推导中使用的是 $\phi(x)$, 但是实际我们需要计算的是 K , 学习在隐式地在特征空间进行的, 不需要显式地定义特征空间和映射函数, 这就是核技巧.

Example

假设输入空间是 $\mathbb{R}^2$, 核函数是 $K(x, z) = (x \cdot z)^2$, 试找出其相关的特征空间 $\mathcal{H}$ 和映射 $\phi(x) : \mathbb{R}^2 \rightarrow \mathcal{H}$.

取特征空间 $\mathcal{H} = \mathbb{R}^3$, 记 $x = (x^{(1)}, x^{(2)})^T$, $z = (z^{(1)}, z^{(2)})^T$, 由于

$$(x \cdot z)^2 = (x^{(1)}z^{(1)} + x^{(2)}z^{(2)})^2 = (x^{(1)}z^{(1)})^2 + 2x^{(1)}z^{(1)}x^{(2)}z^{(2)} + (x^{(2)}z^{(2)})^2$$

所以可以取映射

$$\phi(x) = \left((x^{(1)})^2, \sqrt{2}x^{(1)}x^{(2)}, (x^{(2)})^2 \right)^T$$

容易验证 $\phi(x) \cdot \phi(z) = (x \cdot z)^2 = K(x, z)$ 仍取 $\mathcal{H} = \mathbb{R}^3$ 以及

$$\phi(x) = \frac{1}{\sqrt{2}} \left((x^{(1)})^2 - (x^{(2)})^2, 2x^{(1)}x^{(2)}, (x^{(1)})^2 + (x^{(2)})^2 \right)^T$$

还可以取 $\mathcal{H} = \mathbb{R}^4$ 和

$$\phi(x) = \left((x^{(1)})^2, x^{(1)}x^{(2)}, x^{(1)}x^{(2)}, (x^{(2)})^2 \right)^T$$

[bb] 对于多项式核都是这样的套路, 展开多项式、识别特征项、确定系数.

对偶问题的目标函数中的内积 $x_i \cdot x_j$ 可以用核函数 $K(x_i, x_j) = \phi(x_i) \cdot \phi(x_j)$ 来代替, 对偶问题的目标函数成为:

$$W(\alpha) = \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j K(x_i, x_j) - \sum_{i=1}^N \alpha_i$$

同样, 分类决策函数中的内积也可以用核函数代替, 分类决策函数式成为

$$\begin{aligned} f(x) &= \text{sign} \left(\sum_{i=1}^{N_s} \alpha_i^* y_i \phi(x_i) \cdot \phi(x) + b^* \right) \\ &= \text{sign} \left(\sum_{i=1}^{N_s} \alpha_i^* y_i K(x_i, x) + b^* \right) \end{aligned}$$

通常所说的核函数就是**正定核函数**. 一个函数 K 是正定核函数, 当且仅当 Gram 矩阵

$$\mathbb{K} \triangleq [K(x_i, x_j)]_{m \times m}$$

是半正定的. 证明见附录.

[bc] 这其实就是对于正定核的等价定义. 对于一个具体函数验证它是否为正定核函数并不容易, 因为要求对任意有限输入集 $\{x_1, x_2, \dots, x_m\}$ 验证 K 对应的 Gram 矩阵是半正定矩阵. 在实际问题中往往应用已有的核函数.

名称	表达式
线性核	$\kappa(x_i, x_j) = x_i^T x_j$
多项式核	$\kappa(x_i, x_j) = (x_i^T x_j)^d$

高斯核	$\kappa(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\frac{\ \mathbf{x}_i - \mathbf{x}_j\ ^2}{2\sigma^2}\right)$
拉普拉斯核	$\kappa(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\frac{\ \mathbf{x}_i - \mathbf{x}_j\ }{\sigma}\right)$
Sigmoid 核	$\kappa(\mathbf{x}_i, \mathbf{x}_j) = \tanh(\beta \mathbf{x}_i^T \mathbf{x}_j + \theta)$

假设 $K(x, z)$ 是定义在 $\mathcal{X} \times \mathcal{X}$ 上的对称函数, 并且对任意的 $x_1, x_2, \dots, x_m \in \mathcal{X}$, $K(x, z)$ 关于 $x_1, x_2, \dots, x_m$ 的 Gram 矩阵是半正定的. 可以依据函数 $K(x, z)$ 构成一个希尔伯特空间, 步骤是: 首先定义映射 ϕ 并构成向量空间 $\mathcal{S}$, 然后在 $\mathcal{S}$ 上定义内积构成内积空间, 最后将 $\mathcal{S}$ 完备化构成希尔伯特空间.

Remark

一个对称矩阵 A 是半正定的, 如果对于任意实向量 $c = (c_1, c_2, \dots, c_m)^T$, 都有:

$$c^T A c \geq 0$$

这意味着二次型 $\sum_{i=1}^m \sum_{j=1}^m c_i c_j A_{ij} \geq 0$ 在核函数语境下, 我们要求 Gram 矩阵是半正定的, 即

$$\sum_{i=1}^m \sum_{j=1}^m c_i c_j K(x_i, x_j) \geq 0$$

这个条件确保了核函数是正定核, 保证了核函数对于内积具有非负性.

先定义映射

$$\phi: x \mapsto K(\cdot, x)$$

根据这一映射, 对任意 $x_i \in \mathcal{X}$, $\alpha_i \in \mathbb{R}$, $i = 1, 2, \dots, m$, 定义线性组合

$$f(\cdot) = \sum_{i=1}^m \alpha_i K(\cdot, x_i)$$

考虑由线性组合为元素的集合 $\mathcal{S}$, 由于集合 $\mathcal{S}$ 对加法和数乘运算是封闭的, 所以 $\mathcal{S}$ 构成一个向量空间. 接着在 $\mathcal{S}$ 上定义一个运算 $*$: 对任意 $f, g \in \mathcal{S}$, 有:

$$\begin{aligned} f(\cdot) &= \sum_{i=1}^m \alpha_i K(\cdot, x_i) \\ g(\cdot) &= \sum_{j=1}^l \beta_j K(\cdot, z_j) \\ f * g &= \sum_{i=1}^m \sum_{j=1}^l \alpha_i \beta_j K(x_i, z_j) \end{aligned}$$

证明运算 $*$ 是空间 $\mathcal{S}$ 的内积, 就要证明:

$$\begin{aligned} (cf) * g &= c(f * g), c \in \mathbb{R} \\ (f + g) * h &= f * h + g * h, h \in \mathcal{S} \\ f * g &= g * f \\ f * f &\geq 0, f * f = 0 \Leftrightarrow f = 0 \end{aligned}$$

现在我们证明: $f * f \geq 0, f * f = 0 \Leftrightarrow f = 0$

由 Gram 矩阵的半正定性知非负性, 下面证明下面的等价关系. 充分性显然, 下证必要性: 对于任意的 $x \in \mathcal{X}$, 有

$$K(\cdot, x) * f = \sum_{i=1}^m \alpha_i K(x, x_i) = f(x)$$

Remark

$K(\cdot, x)$ 可以看作一个特殊的函数 $g \in \mathcal{S}$, 他对应于点 x 的映射. 具体地, $g(\cdot) = K(\cdot, x)$, 这等价于 g 是一个线性组合 (只有一项, 系数为 1, 点为 x):

$$g(\cdot) = 1 \cdot K(\cdot, x)$$

即, $l = 1, \beta_1 = 1, z_1 = x$.

于是,

$$|f(x)|^2 = |K(\cdot, x) * f|^2$$

由柯西不等式有:

$$\begin{aligned} |K(\cdot, x) * f|^2 &\leq (K(\cdot, x) * K(\cdot, x))(f * f) \\ &= K(x, x)(f * f) \end{aligned}$$

所以有:

$$|f(x)|^2 \leq K(x, x)(f * f)$$

此式表明,当 $f * f = 0$ 时,对任意的 x 都有 $|f(x)| = 0$.

至此,证明了 $*$ 为向量空间 $\mathcal{S}$ 的内积,因此 $\mathcal{S}$ 是一个内积空间.我们仍然用 $\cdot$ 来表示内积运算.

再根据内积得到范数

$$\|f\| = \sqrt{f \cdot f}$$

得到赋范向量空间,进一步可以再完备化得到希尔伯特空间,这样的希尔伯特空间称为再生核希尔伯特空间,因为核 K 具有再生性.

$$K(\cdot, x) \cdot f = f(x)$$

及

$$K(\cdot, x) \cdot K(\cdot, z) = K(x, z)$$

称为再生核.

Definition 4.4.2

非线性支持向量机

从非线性分类训练集,通过核函数与软间隔最大化或凸二次规划学习得到的分类决策函数

$$f(x) = \text{sign} \left(\sum_{i=1}^{N_s} \alpha_i^* y_i K(x, x_i) + b^* \right)$$

称为非线性支持向量机, $K(x, z)$ 是正定核函数.

Algorithm 4.4.3

非线性支持向量机学习算法

输入: 训练数据集 $T = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$, 其中 $x_i \in \mathcal{X} = \mathbb{R}^2, y_i \in \mathcal{Y} = \{-1, +1\}, i = 1, 2, \dots, N$

输出: 分类决策函数.

1. 选取适当的核函数 $K(x, z)$ 和适当的参数 C , 构造并求解最优化问题, 求得最优解 $\alpha^* = (\alpha_1^*, \alpha_2^*, \dots, \alpha_N^*)^T$

$$\begin{aligned} \min_{\alpha} \quad & \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j K(x_i, x_j) - \sum_{i=1}^N \alpha_i \\ \text{s.t.} \quad & \sum_{i=1}^N \alpha_i y_i = 0, 0 \leq \alpha_i \leq C, i = 1, 2, \dots, N \end{aligned}$$

2. 选择 α^* 的一个正分量 $0 < \alpha_j^* < C$, 计算

$$b^* = y_j - \sum_{i=1}^N \alpha_i^* y_i K(x_i, x_j)$$

3. 构造决策函数:

$$f(x) = \text{sign} \left(\sum_{i=1}^{N_s} \alpha_i^* y_i K(x, x_i) + b^* \right)$$

4.5 序列最小最优化算法

如何高效地实现支持向量机学习是一个重要的问题,现在我们学习序列最小最优化(sequential minimal optimization, SMO)算法,于1998年由Platt提出.

SMO算法要解如下凸二次规划的对偶问题:

$$\begin{aligned} \min_{\alpha} \quad & \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j y_i y_j K(x_i, x_j) - \sum_{i=1}^N \alpha_i \\ \text{s.t.} \quad & \sum_{i=1}^N \alpha_i y_i = 0, 0 \leq \alpha_i \leq C, i = 1, 2, \dots, N \end{aligned}$$

在这个问题中,变量是拉格朗日乘子,一个变量 α_i 对应于一个样本点 (x_i, y_i) ; 变量的总数等于训练样本容量 N .

SMO 要解决什么? 解决样本量太大时,全变量对偶问题算不动的算力危机.每次优化两个变量.一般而言一个是违反 KKT 条件最严重的一个,另一个由约束条件自动确定.

整个 SMO 算法包括两个部分: **求解两个变量二次规划的解析方法**和**选择变量的启发式方法**.

不失一般性,假设选择的两个变量是 α_1, α_2 ,其他变量 $\alpha_i (i = 3, 4, \dots, N)$ 是固定的.于是 SMO 的最优化问题的子问题可以写成

$$\begin{aligned} \min_{\alpha_1, \alpha_2} \quad & W(\alpha_1, \alpha_2) = \frac{1}{2} K_{11} \alpha_1^2 + \frac{1}{2} K_{22} \alpha_2^2 + y_1 y_2 K_{12} \alpha_1 \alpha_2 - \\ & (\alpha_1 + \alpha_2) + y_1 \alpha_1 \sum_{i=3}^N y_i \alpha_i K_{i1} + y_2 \alpha_2 \sum_{i=3}^N y_i \alpha_i K_{i2} \\ \text{s.t.} \quad & \alpha_1 y_1 + \alpha_2 y_2 = - \sum_{i=3}^N y_i \alpha_i = \varsigma, \quad 0 \leq \alpha_i \leq C, i = 1, 2 \end{aligned}$$

其中, $K_{ij} = K(x_i, x_j), i, j = 1, 2, \dots, N$, ς 是常数.

注意,子问题的两个变量中只有一个是自由变量. α_1, α_2 确定其一就可以确定另外一个,所以这其实是一个单变量的最优化问题,不妨考虑为变量 α_2 的最优化问题.

当 $y_1 \neq y_2$ 时, $\alpha_1 - \alpha_2 = k$, 因为 $0 \leq \alpha_1, \alpha_2 \leq C$, 所以有 $\alpha_2 \in [k, C+k] \cap [0, C]$, 所以定义 $L = \max(0, k) = \max(0, \alpha_2 - \alpha_1)$, $H = \min(C, C + \alpha_2 - \alpha_1)$, 则有 $L \leq \alpha_2^{\text{new}} \leq H$, 对于 $y_1 = y_2$ 的情况,同理有 $L = \max(0, \alpha_2 + \alpha_1 - C)$, $H = \min(C, \alpha_2 + \alpha_1)$.

Theorem 4.5.1

设未经不等式取值范围约束时的 α_2 的最优解为 $\alpha_2^{\text{new, unc}}$,然后再求剪辑后 α_2 的解 α_2^{new} .记

$$\begin{aligned} g(x) &= \sum_{i=1}^N \alpha_i y_i K(x_i, x) + b \\ E_i &= g(x_i) - y_i, \quad i = 1, 2 \end{aligned}$$

E_i 为函数 $g(x)$ 对输入 x_i 的预测值与真实输出 y_i 之差.

前述最优化问题沿着约束方向未经剪辑时的解是

$$\alpha_2^{\text{new, unc}} = \alpha_2^{\text{old}} + \frac{y_2(E_1 - E_2)}{\eta}$$

其中,

$$\eta = K_{11} + K_{22} - 2K_{12} = \|\Phi(x_1) - \Phi(x_2)\|^2$$

$\Phi(x)$ 是输入空间到特征空间的映射.

经剪辑后 α_2 的解是

$$\alpha_2^{\text{new}} = \begin{cases} H, & \alpha_2^{\text{new, unc}} > H \\ \alpha_2^{\text{new, unc}}, & L \leq \alpha_2^{\text{new, unc}} \leq H \\ L, & \alpha_2^{\text{new, unc}} < L \end{cases}$$

由 α_2^{new} 求得 α_1^{new} 是

$$\alpha_1^{\text{new}} = \alpha_1^{\text{old}} + y_1 y_2 (\alpha_2^{\text{old}} - \alpha_2^{\text{new}})$$

Theorem 4.5.2

在每一步更新完 α_1 和 α_2 后,需要更新 b .

若 $0 < \alpha_1^{\text{new}} < C$,由 α_1 的支持向量条件可得 b_1^{new} :

$$b_1^{\text{new}} = -E_1 - y_1 K_{11} (\alpha_1^{\text{new}} - \alpha_1^{\text{old}}) - y_2 K_{21} (\alpha_2^{\text{new}} - \alpha_2^{\text{old}}) + b^{\text{old}}$$

若 $0 < \alpha_2^{\text{new}} < C$,由 α_2 的支持向量条件可得 b_2^{new} :

$$b_2^{\text{new}} = -E_2 - y_1 K_{12} (\alpha_1^{\text{new}} - \alpha_1^{\text{old}}) - y_2 K_{22} (\alpha_2^{\text{new}} - \alpha_2^{\text{old}}) + b^{\text{old}}$$

阈值 b 的更新

最终的 b^{new} 更新规则为:

$$b^{\text{new}} = \begin{cases} b_1^{\text{new}}, & \text{若 } 0 < \alpha_1^{\text{new}} < C \\ b_2^{\text{new}}, & \text{若 } 0 < \alpha_2^{\text{new}} < C \\ \frac{b_1^{\text{new}} + b_2^{\text{new}}}{2}, & \text{否则} \end{cases}$$

SMO 算法在每个子问题中选择两个变量优化,其中至少一个变量是违反 KKT 条件的.

SMO 称选择第 1 个变量的过程为外层循环.外层循环在训练样本中选取违反 KKT 条件最严重的样本点,并将其对应的变量作为第 1 个变量.具体地,

$$\begin{aligned} \alpha_i = 0 &\Leftrightarrow y_i g(x_i) \geq 1 \\ 0 < \alpha_i < C &\Leftrightarrow y_i g(x_i) = 1 \\ \alpha_i = C &\Leftrightarrow y_i g(x_i) \leq 1 \end{aligned}$$

其中, $g(x) = \sum_{i=1}^N \alpha_i y_i K(x_i, x) + b$.

该检验是在 ε 范围内进行的,在检验过程中,外层循环首先遍历所有满足条件 $0 < \alpha_i < C$ 的样本点,即在间隔边界上的支持向量点,检验它们是否满足 KKT 条件.如果这些样本点都满足 KKT 条件,那么遍历整个训练集,检验他们是否都满足 KKT 条件.

变量 2 的选择策略.

Algorithm 4.5.3

SMO 算法

输入: 训练数据集 $T = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$, 其中, $x_i \in \mathcal{X} = R^n$, $y_i \in \mathcal{Y} = \{-1, +1\}$, $i = 1, 2, \dots, N$, 精度 ε , 惩罚参数 C .

输出: 近似解 $\hat{\alpha}$, $\hat{b}$.

1. 初始化: 取初值 $\alpha^{(0)} = (0, 0, \dots, 0)^T$, $b^{(0)} = 0$, 令 $k = 0$;
2. 变量选择 (外层循环——选择 α_1):
 - 首先遍历所有满足 $0 < \alpha_i < C$ 的样本点 (间隔边界上的支持向量), 检查是否满足 KKT 条件:

$$\begin{aligned} \alpha_i = 0 &\Leftrightarrow y_i g(x_i) \geq 1, \\ 0 < \alpha_i < C &\Leftrightarrow y_i g(x_i) = 1, \\ \alpha_i = C &\Leftrightarrow y_i g(x_i) \leq 1 \end{aligned}$$

其中 $g(x) = \sum_{j=1}^N \alpha_j y_j K(x_j, x) + b$.

- 若所有支持向量点均在 ε 内满足 KKT 条件, 则遍历整个训练集.
 - 选取 KKT 条件违反最严重的样本点对应的变量作为 α_1 .
3. 变量选择 (内层循环——选择 α_2):
 - 在剩余变量中选择 $|E_1 - E_i|$ 最大的 α_i 作为 α_2 , 其中 $E_i = g(x_i) - y_i$.
 - 若上述选择未能使目标函数有足够下降, 则先遍历 $0 < \alpha_i < C$ 的样本, 再遍历全集, 选取能使 α_2 有最大步长的变量.
 4. 解析求解子问题:
 - 计算 $\eta = K_{11} + K_{22} - 2K_{12} = \|\Phi(x_1) - \Phi(x_2)\|^2$.
 - 计算未经剪辑的 α_2 最优解:

$$\alpha_2^{\text{new, unc}} = \alpha_2^{\text{old}} + \frac{y_2(E_1 - E_2)}{\eta}$$

- 根据 η 的正负确定取值范围边界 (L, H) :
 - 若 $y_1 \neq y_2$: $L = \max(0, \alpha_2^{\text{old}} - \alpha_1^{\text{old}})$, $H = \min(C, C + \alpha_2^{\text{old}} - \alpha_1^{\text{old}})$
 - 若 $y_1 = y_2$: $L = \max(0, \alpha_1^{\text{old}} + \alpha_2^{\text{old}} - C)$, $H = \min(C, \alpha_1^{\text{old}} + \alpha_2^{\text{old}})$
- 剪辑得到 α_2 的更新值:

$$\alpha_2^{\text{new}} = \begin{cases} H, & \alpha_2^{\text{new, unc}} > H \\ \alpha_2^{\text{new, unc}}, & L \leq \alpha_2^{\text{new, unc}} \leq H \\ L, & \alpha_2^{\text{new, unc}} < L \end{cases}$$

- 更新 α_1 :

$$\alpha_1^{\text{new}} = \alpha_1^{\text{old}} + y_1 y_2 (\alpha_2^{\text{old}} - \alpha_2^{\text{new}})$$

5. 更新阈值 b :

- 若 $0 < \alpha_1^{\text{new}} < C$:

$$b_1^{\text{new}} = -E_1 - y_1 K_{11} (\alpha_1^{\text{new}} - \alpha_1^{\text{old}}) - y_2 K_{21} (\alpha_2^{\text{new}} - \alpha_2^{\text{old}}) + b^{\text{old}}$$

- 若 $0 < \alpha_2^{\text{new}} < C$:

$$b_2^{\text{new}} = -E_2 - y_1 K_{12} (\alpha_1^{\text{new}} - \alpha_1^{\text{old}}) - y_2 K_{22} (\alpha_2^{\text{new}} - \alpha_2^{\text{old}}) + b^{\text{old}}$$

- 取最终值 b^{new} :

$$b^{\text{new}} = \begin{cases} b_1^{\text{new}}, & \text{若 } 0 < \alpha_1^{\text{new}} < C \\ b_2^{\text{new}}, & \text{若 } 0 < \alpha_2^{\text{new}} < C \\ \frac{b_1^{\text{new}} + b_2^{\text{new}}}{2}, & \text{否则} \end{cases}$$

6. 停机条件检验: 若在精度 ε 范围内满足以下 KKT 条件, 则转步骤 7, 否则令 $k = k + 1$, 转步骤 2:

$$\sum_{i=1}^N \alpha_i y_i = 0, \quad 0 \leq \alpha_i \leq C, \quad i = 1, 2, \dots, N$$

$$y_i g(x_i) = \begin{cases} \geq 1, & \{x_i | \alpha_i = 0\} \\ = 1, & \{x_i | 0 < \alpha_i < C\} \\ \leq 1, & \{x_i | \alpha_i = C\} \end{cases}$$

7. 输出: 取 $\hat{\alpha} = \alpha^{(k+1)}$, $\hat{b} = b^{\text{new}}$.

Example

1. 输入数据与参数设定

我们将惩罚参数设定为 $C = 1$. 训练数据 T 包含 3 个样本点:

- 正例 ($y = +1$): $x_1 = (3, 3)^T, x_2 = (4, 3)^T$
- 负例 ($y = -1$): $x_3 = (1, 1)^T$

1. 初始化

设初始拉格朗日乘子全为 0, 偏置 $b = 0$:

$$\alpha^{(0)} = (0, 0, 0)^T, \quad b^{(0)} = 0$$

2. 变量选择 (Heuristic)

SMO 需要选择两个变量 α_i, α_j 进行优化. 根据 KKT 条件检验, 所有点目前的函数间隔都是 $y(w \cdot x + b) = 0 < 1$, 都违反了约束. 我们假设外层循环选中了 x_1 (违反 KKT 最严重), 内层循环根据误差最大原则选中了 x_3 .

选定优化对: x_1 (索引 1) 和 x_3 (索引 2). 即我们要更新 α_1 和 α_3 . (注意: 为了方便公式对应, 下文称 α_1 为第一个变量, α_3 为第二个变量).

3. 计算预测值与误差 E

当前模型 $f(x) = 0$ (因为 $\alpha = 0$). 计算选定样本的误差 $E_k = f(x_k) - y_k$:

$$E_1 = 0 - y_1 = -1$$

$$E_3 = 0 - y_3 = 0 - (-1) = +1$$

4. 计算核函数值与 eta

计算所需的内积 (Gram 矩阵元素):

$$K_{11} = x_1 \cdot x_1 = 18$$

$$K_{33} = x_3 \cdot x_3 = 2$$

$$K_{13} = x_1 \cdot x_3 = 6$$

计算二阶导数 η (即目标函数关于 α_3 的二阶导数):

$$\eta = K_{11} + K_{33} - 2K_{13} = 18 + 2 - 2(6) = 8$$

5. 计算未经剪辑的 $\alpha_3^{\text{new, unc}}$

根据 SMO 更新公式:

$$\begin{aligned} \alpha_3^{\text{new, unc}} &= \alpha_3^{\text{old}} + \frac{y_3(E_1 - E_3)}{\eta} \\ &= 0 + \frac{(-1) \cdot (-1 - 1)}{8} \\ &= \frac{2}{8} = 0.25 \end{aligned}$$

6. 剪辑边界 L 与 H

因为 $y_1 n = y_3$ ($1n = -1$), 限制条件为:

$$L = \max(0, \alpha_3^{\text{old}} - \alpha_1^{\text{old}}) = \max(0, 0) = 0$$

$$H = \min(C, C + \alpha_3^{\text{old}} - \alpha_1^{\text{old}}) = \min(1, 1) = 1$$

$\alpha_3^{\text{new, unc}} = 0.25$ 落在 $[0, 1]$ 之间,无需剪辑:

$$\alpha_3^{\text{new}} = 0.25$$

7. 更新 α_1

根据 $\sum \alpha y = 0$ 的约束同步更新 α_1 :

$$\begin{aligned}\alpha_1^{\text{new}} &= \alpha_1^{\text{old}} + y_1 y_3 (\alpha_3^{\text{old}} - \alpha_3^{\text{new}}) \\ &= 0 + (1)(-1)(0 - 0.25) \\ &= 0.25\end{aligned}$$

此时 $\alpha = (0.25, 0, 0.25)^T$.

8. 更新阈值 b

我们需要分别计算 b_1 和 b_2 :

$$\begin{aligned}b_1 &= b^{\text{old}} - E_1 - y_1(\alpha_1^{\text{new}} - \alpha_1^{\text{old}})K_{11} - y_3(\alpha_3^{\text{new}} - \alpha_3^{\text{old}})K_{13} \\ &= 0 - (-1) - 1(0.25)(18) - (-1)(0.25)(6) \\ &= 1 - 4.5 + 1.5 = -2 \\ b_2 &= b^{\text{old}} - E_3 - y_1(\alpha_1^{\text{new}} - \alpha_1^{\text{old}})K_{13} - y_3(\alpha_3^{\text{new}} - \alpha_3^{\text{old}})K_{33} \\ &= 0 - 1 - 1(0.25)(6) - (-1)(0.25)(2) \\ &= -1 - 1.5 + 0.5 = -2\end{aligned}$$

因为 $\alpha_1^{\text{new}}, \alpha_3^{\text{new}} \in (0, C)$, 所以 $b_1 = b_2$. 更新 $b^{\text{new}} = -2$.

9. 结果验证

经过这一轮迭代,我们得到:

$$\begin{aligned}w &= 0.25x_1 - 0.25x_3 = (0.5, 0.5)^T \\ b &= -2\end{aligned}$$

这正好就是我们之前用全局代数法求出的精确解! 说明对于这个简单数据集,SMO 一步就收敛了.

Proof of Theorems

Theorem 4.5.4

设 $K: \mathcal{X} \times \mathcal{X} \rightarrow R$ 是对称函数, 则 $K(x, z)$ 为正定核函数的充要条件是对任意 $x_i \in \mathcal{X}, i = 1, 2, \dots, m, K(x, z)$ 对应的 Gram 矩阵是半正定矩阵.

Proof:

必要性. 由于 $K(x, z)$ 是 $\mathcal{X} \times \mathcal{X}$ 上的正定核, 所以存在从 $\mathcal{X}$ 到希尔伯特空间 $\mathcal{H}$ 的映射 ϕ , 使得

$$K(x, z) = \phi(x) \cdot \phi(z)$$

于是, 对任意 $x_1, x_2, \dots, x_m$, 构造 $K(x, z)$ 关于 $x_1, x_2, \dots, x_m$ 的 Gram 矩阵:

$$[K_{ij}]_{m \times m} = [K(x_i, x_j)]_{m \times m}$$

对任意 $c_1, c_2, \dots, c_m \in R$, 有:

$$\begin{aligned}\sum_{i,j=1}^m c_i c_j K(x_i, x_j) &= \sum_{i,j=1}^m c_i c_j (\phi(x_i) \cdot \phi(x_j)) \\ &= \left(\sum_i c_i \phi(x_i) \right) \cdot \left(\sum_j c_j \phi(x_j) \right) \\ &= \left\| \sum_i c_i \phi(x_i) \right\|^2 \geq 0\end{aligned}$$

表明 $K(x, z)$ 关于 $x_1, x_2, \dots, x_m$ 的 Gram 矩阵是半正定的.

充分性. 对任意 $x_1, x_2, \dots, x_m \in \mathcal{X}$, 已知对称函数 $K(x, z)$ 关于 $x_1, x_2, \dots, x_m$ 的 Gram 矩阵是半正定的, 对于给定的 $K(x, z)$, 可以构造从 $\mathcal{X}$ 到某个希尔伯特空间 $\mathcal{H}$ 的映射:

$$\phi: x \rightarrow K(\cdot, x)$$

易知:

$$K(\cdot, x) \cdot f = f(x)$$

$$K(\cdot, x) \cdot K(\cdot, z) = K(x, z)$$

表明 $K(x, z)$ 是 $\mathcal{X} \times \mathcal{X}$ 上的核函数.

□

5 决策树

本章主要讨论用于分类的决策树.其主要优点是模型具有可读性,分类速度快.决策树的学习通常包括 3 个步骤: **特征选择**、**决策树的生成**和**决策树的修剪**.

Remark

生长阶段: 遍历当前可用的特征集 A 与所有可能的切分点 s , 每步挑选出最优

- **ID3 算法**: 信息增益 $g(D, A)$ 最大化 (缺点: 极度偏爱多值特征)。
- **C4.5 算法**: 信息增益率 $g_{R(D, A)}$ 最大化 (通过特征取值分布熵 $H_{A(D)}$ 作为分母进行惩罚校正)。
- **CART 分类树**: 划分后的 **基尼指数 (Gini Index)** $Gini(D, A)$ 最小化。
- **CART 回归树**: 划分后左右子集各自回归到均值时的 **总平方误差** 最小化。

剪枝阶段: **总损失 = 在训练集上的犯错代价 $C(T)$ + 树长得太复杂的罚款 $(\alpha |T|)$** , 看哪个少。

5.1 决策树模型与学习

分类决策树模型是一种描述对实例进行分类的属性结构.决策树由结点和有向边组成.结点由两种类型: 内部结点和叶结点.**内部结点表示一个特征或属性**,**叶结点表示一个类**.

假设 X 为表示特征的随机变量, Y 为表示类的随机变量, 那么这个条件概率分布可以表示为 $P(Y|X)$. 各叶结点 (单元) 上的条件概率往往属于某一类的概率较大, 决策树分类时将该结点的实例强行分到条件概率大的那一类去.

假设给定训练数据集

$$D = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$$

其中, $x_i = (x_i^{(1)}, x_i^{(2)}, \dots, x_i^{(n)})$ 为输入实例 (特征向量), n 为特征个数, $y_i \in \{1, 2, \dots, K\}$ 为类标记, $i = 1, 2, \dots, N$, N 为样本容量.

[bd] 决策树学习本质上是从训练数据集中归纳出一组分类规则.能对训练数据正确分类的决策树可能有多,也可能一个都没有.我们需要的是一个与训练数据矛盾较小的决策树.同时具有很好的泛化能力.从另一个角度看,决策树学习是由训练数据集估计条件概率模型.我们选择的条件概率模型应该不仅对训练数据有很好的拟合,而且对未知数据有很好的预测.

决策树学习的损失函数通常是正则化的极大似然函数.因为从所有可能的决策树中选取最优决策树是 **NP 完全问题**,所以现实中决策树学习算法通常采用**启发式方法**,近似求解这一最优化问题. **[be]** Trivial. 这样得到的决策树是次最优的.

决策树学习算法包括特征选择、决策树生成与决策树的剪枝过程.由于决策树表示一个条件概率分布,所以深浅不同的决策树对应不同复杂度的概率模型.决策树的生成对应于模型的局部选择,决策树的剪枝对应于模型的全局选择.决策树的生成只考虑局部最优,而决策树的剪枝则考虑全局最优.

Remark

决策树学习常见的算法有 ID3、C4.5 与 CART.

5.2 特征选择

特征选择在于选取对训练数据具有分类能力的特征,这样可以提高决策树学习的效率.如果利用一个特征进行分类的结果与随机分类的结果没有很大差别,则称这个特征是没有分类能力的.经验上扔掉这样的特征对决策树学习的精度影响不大.通常特征选择的准则是**信息增益**或**信息增益比**.

Definition 5.2.1

随机变量的熵

在信息论与概率统计中,熵是表示随机变量不确定性的度量.设 X 是一个取有限个值的离散随机变量,其概率分布为

$$P(X = x_i) = p_i, \quad i = 1, 2, \dots, n$$

则随机变量的熵定义为

$$H(X) = - \sum_{i=1}^n p_i \log(p_i)$$

通常对数会以 2 或者 e 为底.熵只依赖于 X 的分布,而与 X 的取值无关,所以也可将 X 的熵记作 $H(p)$,即:

$$H(p) = - \sum_{i=1}^n p_i \log(p_i)$$

Theorem 5.2.2

$$0 \leq H(p) \leq \log n$$

[bf] Jensen 不等式可以直接解决.考虑函数 $f(x) = x \log x$,它在 $x > 0$ 上是凸函数.那么:

$$\begin{aligned} \sum_{i=1}^n \frac{1}{n} f(p_i) &\geq f\left(\sum_{i=1}^n \frac{1}{n} p_i\right) \\ &= f\left(\frac{1}{n} \sum p_i\right) = f\left(\frac{1}{n}\right) \end{aligned}$$

即:

$$\begin{aligned} \frac{1}{n} \sum p_i \log p_i &\geq \frac{1}{n} \log \frac{1}{n} \\ \Rightarrow \sum p_i \log p_i &\geq \log \frac{1}{n} = -\log n \\ \Rightarrow -\sum p_i \log p_i &\leq \log n \end{aligned}$$

Proof: 构造出以下优化问题:

$$\max_{p_i} -\sum_{i=1}^n p_i \log p_i \quad s.t. \sum_{i=1}^n p_i = 1$$

引入拉格朗日乘子 λ ,构造拉格朗日函数为

$$L(p_i, \lambda) = -\sum_{i=1}^n p_i \log p_i + \lambda \left(\sum_{i=1}^n p_i - 1 \right)$$

求解:

$$\begin{cases} \frac{\partial L(p_i, \lambda)}{\partial p_i} = 0 \\ \sum_{i=1}^n p_i = 1 \end{cases}$$

最终可知 $p_i = e^{\lambda-1} = \frac{1}{n}$ 时,原式取到最大值 $\log n$. □

Definition 5.2.3**条件熵**

设有随机变量 (X, Y) ,其联合概率分布为

$$P(X = x_i, Y = y_j) = p_{ij}, \quad i = 1, 2, \dots, n \quad j = 1, 2, \dots, m$$

条件熵 $H(Y|X)$ 表示在已知随机变量 X 的条件下随机变量 Y 的不确定性.定义为(条件概率分布的熵对 X 的数学期望):

$$H(Y|X) = \sum_{i=1}^n P(X = x_i) H(Y|X = x_i)$$

当熵和条件熵中的概率由数据估计(特别是极大似然估计)得到时,所对应的熵与条件熵分别成为经验熵和经验条件熵.

信息增益表示得知特征 X 的信息而使得类 Y 的信息的不确定性减少的程度.

Definition 5.2.4**信息增益**

特征 A 对训练数据集 D 的信息增益 $g(D, A)$ 定义为:

$$g(D, A) = H(D) - H(D|A)$$

[bg] 决策树学习中的信息增益等价于训练数据集中类与特征的互信息.

决策树学习用信息增益选择特征.给定训练数据集 D 和特征 A ,经验熵 $H(D)$ 表示对数据集 D 进行分类的不确定性.而经验条件熵 $H(D|A)$ 表示在特征 A 给定条件下,对数据集 D 进行分类的不确定性.它们的差即信息增益,就表示由于特征 A 而使得对数据集 D 的分类不确定性减少的程度.信息增益大的特征具有更强的分类能力.

设训练数据集为 D , $|D|$ 表示样本个数.设有 K 个类 $C_k, k = 1, 2, \dots, K, |C_k|$ 为属于类 C_k 的样本个数, $\sum_{k=1}^K |C_k| = |D|$.

设特征 A 有 n 个不同的取值 $\{a_1, a_2, \dots, a_n\}$, 根据特征 A 的取值将 D 划分为 n 个子集 $D_1, D_2, \dots, D_n$, $|D_i|$ 为 D_i 的样本个数, $\sum_{i=1}^n |D_i| = |D|$. 记 D_i 中属于类 C_k 的样本的集合为 D_{ik} , 即 $D_{ik} = D_i \cap C_k$, $|D_{ik}|$ 为 D_{ik} 的样本个数.
(说人话就是先按照特征分, 再在小类里面算类别的熵) 所以有信息增益算法如下:

Algorithm 5.2.5**计算信息增益**

输入: 训练数据集 D 和特征 A .

输出: 特征 A 对训练数据集 D 的信息增益 $g(D, A)$

1. 计算数据集 D 的经验熵 $H(D)$

$$H(D) = - \sum_{k=1}^K \frac{|C_k|}{|D|} \log_2 \frac{|C_k|}{|D|}$$

2. 计算特征 A 对数据集 D 的经验条件熵 $H(D|A)$

$$H(D|A) = \sum_{i=1}^n \frac{|D_i|}{|D|} H(D_i) = - \sum_{i=1}^n \frac{|D_i|}{|D|} \sum_{k=1}^K \frac{|D_{ik}|}{|D_i|} \log_2 \frac{|D_{ik}|}{|D_i|}$$

3. 计算信息增益

$$g(D, A) = H(D) - H(D|A)$$

以信息增益作为划分训练数据集的特征, 存在偏向于选择取值较多的特征的问题, 使用信息增益比可以对这个问题进行校正, 也是特征选择的一个准则.

Definition 5.2.6**信息增益率**

特征 A 对训练数据集 D 的信息增益比 $g_R(D, A)$ 定义为其信息增益 $g(D, A)$ 与训练数据集 D 关于特征 A 的值的熵 $H_A(D)$ 之比, 即

$$g_R(D, A) = \frac{g(D, A)}{H_A(D)}$$

其中, $H_A(D) = - \sum_{i=1}^n \frac{|D_i|}{|D|} \log_2 \frac{|D_i|}{|D|}$, n 是特征 A 取值的个数, $D^{i(i=1,2,\dots,n)}$ 是指按照特征 A 的不同取值划分出的集合. $H_A(D)$ 是特征取值的分布熵. 如果特征本身出现频率很均匀, $H_A(D)$ 就会很大.

5.3 ID3 算法

ID3 算法的核心是选择信息增益最大的特征作为结点的特征不断的构造新节点.

[bh] 选了信息增益最大的, 剩下的熵就最小了. 其实也相当于用极大似然法进行概率模型的选择.

Algorithm 5.3.1**ID3 算法生成决策树**

输入: 训练数据集 D , 特征集 A 阈值 ϵ .

输出: 决策树 T .

1. 若 D 中所有实例属于同一类 C_k , 则 T 为单结点树, 并将类 C_k 作为该结点的类标记, 返回 T .
2. 若 $A = \emptyset$, 则 T 为单结点树, 并将 D 中实例最大的类 C_k 作为该结点的类标记, 返回 T .
3. 否则, 按照前述方法计算 A 中各特征对 D 的信息增益, 选择信息增益最大的特征 A_g ;
4. 如果 A_g 的信息增益小于阈值 ϵ , 则置 T 为单结点树, 并将 D 中实例数量最大的类 C_k 作为该结点的类标记, 返回 T (分不清楚就用频率最多的类糊弄);
5. 否则, 对 A_g 的每一可能值 a_i , 依 $A_g = a_i$ 将 D 分割成若干非空子集 D_i , 将 D_i 中实例数最大的类作为标记, 构建子结点, 由结点及其子结点构成树 T , 返回 T ;
6. 对第 i 个子结点, 以 D_i 为训练集, 以 $A - \{A_g\}$ 为特征集, 递归地调用步骤 1-5, 得到子树 T_i , 返回 T_i ;

Example

ID3 算法手算实例: 贷款申请分类

我们使用一个简化的数据集, 包含 5 个样本. 目标是根据特征判断是否批准贷款 (类别 Y).

1. 数据集准备

ID	年龄 (A_1)	有工作 (A_2)	有房子 (A_3)	类别: 批准? (Y)
1	青年	否	否	否
2	青年	是	否	是

ID	年龄 (A_1)	有工作 (A_2)	有房子 (A_3)	类别: 批准? (Y)
3	老年	否	是	是
4	老年	是	是	是
5	老年	否	否	否

2. 计算根结点的经验熵 $H(D)$

数据集 D 中共有 5 个样本:

- 类别“是” ($Y = 1$): 3 个样本
- 类别“否” ($Y = 0$): 2 个样本

$$H(D) = - \sum_{k=1}^K p_k \log_2 p_k = - \left(\frac{3}{5} \log_2 \frac{3}{5} + \frac{2}{5} \log_2 \frac{2}{5} \right) \approx 0.971$$

3. 计算各特征的信息增益

我们要分别计算 A_1 (年龄)、 A_2 (有工作)、 A_3 (有房子) 的信息增益.

(1) 特征 A_1 : 年龄

- 取值“青年” (D_1): 2 个样本 (1 否, 1 是). $H(D_1) = - \left(\frac{1}{2} \log_2 \frac{1}{2} + \frac{1}{2} \log_2 \frac{1}{2} \right) = 1.0$
- 取值“老年” (D_2): 3 个样本 (1 否, 2 是). $H(D_2) = - \left(\frac{1}{3} \log_2 \frac{1}{3} + \frac{2}{3} \log_2 \frac{2}{3} \right) \approx 0.918$

经验条件熵:

$$H(D|A_1) = \frac{2}{5} H(D_1) + \frac{3}{5} H(D_2) = 0.4 \times 1.0 + 0.6 \times 0.918 = 0.951$$

信息增益:

$$g(D, A_1) = H(D) - H(D|A_1) = 0.971 - 0.951 = 0.020$$

(2) 特征 A_2 : 有工作

- 取值“是” (D_1): 2 个样本 (2 是). 纯度最高, $H(D_1) = 0$
- 取值“否” (D_2): 3 个样本 (2 否, 1 是). $H(D_2) \approx 0.918$

经验条件熵:

$$H(D|A_2) = \frac{2}{5} \times 0 + \frac{3}{5} \times 0.918 = 0.551$$

信息增益:

$$g(D, A_2) = H(D) - H(D|A_2) = 0.971 - 0.551 = 0.420$$

(3) 特征 A_3 : 有房子

- 取值“是” (D_1): 2 个样本 (2 是). $H(D_1) = 0$
- 取值“否” (D_2): 3 个样本 (2 否, 1 是). $H(D_2) \approx 0.918$

信息增益 (计算过程同上):

$$g(D, A_3) = 0.971 - (0.4 \times 0 + 0.6 \times 0.918) = 0.420$$

4. 构建决策树 (第一层划分)

比较信息增益:

$$g(D, A_2) = g(D, A_3) > g(D, A_1)$$

我们发现“有工作”和“有房子”的信息增益相同且最大.ID3 算法可以任选一个.这里我们选择特征 A_3 (有房子) 作为根结点.

- 右分支 (有房子=是): 对应样本 {3, 4}, 类别全为“是”.这是一个叶结点, 标记为 “是”.
- 左分支 (有房子=否): 对应样本 $D' = \{1, 2, 5\}$.
 - 样本 1: 青年, 没工作 -> 否
 - 样本 2: 青年, 有工作 -> 是
 - 样本 5: 老年, 没工作 -> 否

此时 D' 中还有混合类别, 需要递归. 计算 D' 中剩余特征 (A_1 年龄, A_2 工作) 的增益: 显然可以看出, 在 D' 中, 只要“有工作”就是“是”, 没工作就是“否”. A_2 (工作) 将把数据完美分开, 增益最大.

[bi] 在这个例子中,因为数据量较小,第二步递归时特征“有工作”直接得到了纯度为 0 的子集,因此算法终止,生成了深度为 2 的树。

5.4 C4.5 算法

5.4.1 使用信息增益比作为判准

ID3 算法只有树的生成,所以该算法生成的树容易产生过拟合.C4.5 算法与 ID3 算法相似,C4.5 算法对 ID3 算法进行了改进,生成过程中选择信息增益比来选择特征。

可以避免 ID3 算法偏爱那些待选数量多的特征(比如 100 个学号)。

5.4.2 最佳属性划分: 连续值与缺失值处理

5.4.2.1 连续属性离散化

若属性取值是连续值,不能直接根据数值进行结点划分,需要使用连续属性离散化技术.C4.5 决策树采用二分法(bi-partition)对连续属性进行处理.具体流程如下:

给定训练数据集 D 和连续属性 a ,假定 a 在 D 上出现了 n 个不同的取值,将这些值从小到大排序,记为 $\{a_1, a_2, \dots, a_n\}$.基于划分点 t 可将 D 分为子集 D_t^- 和 D_t^+ ,其中 D_t^- 包含那些在属性 a 上取值不大于 t 的样本, D_t^+ 包含那些在属性 a 上取值大于 t 的样本.对相邻的属性取值 a_i 与 a_{i+1} 来说, t 在区间 $[a_i, a_{i+1}]$ 中取任意值所产生的划分结果相同.因此,对连续属性 a ,我们可以考察包含 $n-1$ 个元素的候选划分点集合:

$$T_a = \left\{ \frac{a_i + a_{i+1}}{2} \mid 1 \leq i \leq n-1 \right\}$$

即把区间 $[a_i, a_{i+1}]$ 的中位点 $\frac{a_i + a_{i+1}}{2}$ 作为候选划分点.然后,像处理离散属性值一样,考察这些划分点,选取最优的划分点进行样本集合的划分:

$$\text{Gain}(D, a) = \max_{t \in T_a} \text{Gain}(D, a, t) = \max_{t \in T_a} H(D) - \sum_{\lambda \in \{-, +\}} \frac{|D_t^\lambda|}{|D|} H(D_t^\lambda)$$

其中 $\text{Gain}(D, a, t)$ 是样本集 D 基于划分点 t 二分后的信息增益.我们只需计算每个候选划分点的信息增益,选择使信息增益最大的 t 作为划分点,将属性 a 以该划分点为界离散化。

[bj] 与离散属性不同,若当前结点划分属性为连续属性,该属性还可以作为其后代结点的划分属性继续使用.例如在父结点以“密度 ≤ 0.381 ”划分后,子结点中仍然可以使用“密度 ≤ 0.294 ”再次划分。

5.4.2.2 缺失值处理

C4.5 算法提供了内置的缺失值处理机制。

给定训练集 D 和属性 a ,令 $\tilde{D}$ 表示 D 中在属性 a 上没有缺失值的样本子集.对于属性 a ,解决两个问题: (1) 如何在属性值缺失的情况下进行划分属性选择? (2) 给定划分属性,若样本在该属性上的值缺失,如何对样本进行划分?

对于问题(1),仅可根据 $\tilde{D}$ 来判断属性 a 的优劣.具体地,以属性不缺失的样本计算信息增益,再乘以无缺失样本占该属性所有样本的比例作为权重系数:

$$\text{Gain}(D, a) = \frac{|\tilde{D}|}{|D|} \text{Gain}(\tilde{D}, a)$$

也就是说,用 $\tilde{D}$ 上的信息增益乘以该属性上无缺失样本的比例,作为最终的信息增益.这样可以使得缺失值比例较高的属性在划分选择中处于不利地位。

对于问题(2),若样本 x 在划分属性 a 上的取值已知,则将 x 划入与其取值对应的子结点,且样本权值在子结点中保持为 w_x .若样本 x 在划分属性 a 上的取值未知,则将 x 同时划入所有子结点,且样本权值在与属性值 a_v 对应的子结点中调整为 $\tilde{w}_x = \tilde{r}_v \cdot w_x$,其中 $\tilde{r}_v$ 是无缺失样本中属性 a 上取值为 a_v 的样本所占的比例.直观地说,就是让同一个样本以不同的概率同时进入不同的子结点。

5.5 决策树的剪枝

Definition 5.5.1

剪枝 (Pruning)

在决策树学习中将已生成的树进行简化的过程称为剪枝(pruning). 具体地,剪枝从已生成的树上裁掉一些子树或叶结点,并将其根结点或父结点作为新的叶结点,从而简化分类树模型。

5.6 决策树的剪枝: 从数学符号到核心逻辑

决策树剪枝的本质是引入正则化惩罚项,在“训练集拟合程度”与“模型复杂度”之间寻找最佳平衡,从而有效防止模型过拟合。以下采用自底向上的依赖分析,层层拆解剪枝公式的物理意义。

5.6.1 底层依赖概念拆解

1. 单张叶子节点的“混乱度” $H_t(T)$ 对于决策树 T 的任意一个叶节点 t , 其内部数据的混乱程度通过经验熵来度量:

$$H_t(T) = - \sum_k \frac{N_{tk}}{N_t} \log \frac{N_{tk}}{N_t}$$

N_t 是该叶节点包含的样本总数; N_{tk} 是其中属于第 k 类的样本数。

• **物理意义**: 若叶节点内部全是同一类样本 (纯度极高), 则 $H_t(T) = 0$; 若各类别分布极不均匀, 则 $H_t(T)$ 较大。

2. 整棵树的“总犯错代价” $C(T)$ 将所有叶节点的混乱度进行加权求和, 得到模型对训练数据的整体预测误差:

$$C(T) = \sum_{t=1}^{|T|} N_t H_t(T) = - \sum_{t=1}^{|T|} \sum_{k=1}^K N_{tk} \log \frac{N_{tk}}{N_t}$$

• **物理意义**: 衡量了整棵树对训练集的拟合程度。随着树的生长分支变多, 每个叶节点越来越纯, $C(T)$ 会单调递减。若任由树无限生长, $C(T)$ 可降至 0, 但会引发严重的过拟合。

5.6.2 核心机制: 损失函数 $C_\alpha(T)$ 的天平博弈

为遏制树的无限制膨胀, 引入叶节点个数 $|T|$ 作为模型复杂度的惩罚项, 构成整体损失函数:

$$C_\alpha(T) = C(T) + \alpha |T|$$

- **天平左端 $C(T)$** : 代表模型在训练集上的**犯错代价**。树越复杂, 该项越小。
- **天平右端 $\alpha |T|$** : 代表对模型**臃肿程度**的**罚款**。 $|T|$ 是叶节点总数, $\alpha \geq 0$ 是调和系数。
- **α 的调和逻辑**:
 - 当 $\alpha = 0$ 时, 不施加任何复杂度惩罚, 完全生长树是最优解。
 - 当 $\alpha \rightarrow +\infty$ 时, 惩罚极重, 只包含根节点的单节点树是最优解。
 - 剪枝的目标, 就是在给定的 α 下, 找出使 $C_\alpha(T)$ 达到全局最小的子树模型。

5.6.3 局部的“Before 与 After”对决机制

在实际操作中, 算法无需每次都进行全局扫描。由于损失函数的局部可加性, 剪枝决策完全可以在局部节点独立推演。

假设一组叶节点回缩到其父节点之前与之后的整体树分别是 T_B (Before) 与 T_A (After):

- **剪枝前 (T_B)**: 保持子树原样。虽然局部误差 $C(T_B)$ 较小, 但由于叶节点较多, 需要承担较高的复杂度罚款 $\alpha |T_B|$ 。
- **剪枝后 (T_A)**: 将子树强行收缩为单个叶节点。虽然导致局部混乱度上升 ($C(T_A) > C(T_B)$), 但节省了叶节点数量, 免去了相应的罚款。

根据最优化原则, 剪枝的判定条件为:

$$C_\alpha(T_A) \leq C_\alpha(T_B)$$

逻辑说明: 将子树砍掉虽然在训练集上多犯了一些错, 但省下来的复杂度罚款足够多, 使得综合整体损失反而更低 (或持平)。此时, 应当果断执行剪枝, 将父节点变为新的叶节点。因为其他未剪枝的分支在前后减法对比中会自动抵消, 该计算天然支持局部动态规划的高效实现。

Remark

决策树算法只考虑两个数的损失函数的差, 所以计算是可以在局部进行的, 因而可以由一种动态规划算法实现。

5.7 CART 算法

Remark

ID3 和 C4.5 算法, 生成的决策树是多叉树, 只能处理分类不能处理回归。而 CART (classification and regression tree) 分类回归树算法, 既可用于分类也可用于回归。分类树的输出是样本的类别, 回归树的输出是一个实数。

Definition 5.7.1

CART

CART 是在给定输入随机变量 X 条件下输出随机变量 Y 的条件概率分布的学习方法。CART 假设决策树是**二叉树**, 内部结点特征的取值为“是”和“否”, 左分支是取值为“是”的分支, 右分支是取值为“否”的分支。这样的决策树等价于递归地二分每个特征, 将输入空间即特征空间划分为有限个单元, 并在这些单元上确定预测的概率分布, 也就是在输入给定的条件下输出的条件的概率分布。

1. **决策树生成**: 基于训练数据集生成决策树, 生成的决策树要尽量大。
2. **决策树剪枝**: 用验证数据集对已生成的树进行剪枝并选择最优子树, 这时用损失函数最小作为剪枝的标准。

5.8 CART 生成

决策树的生成就是递归地构建二叉决策树的过程, 对回归树用平方误差最小化准则, 对分类树用基尼指数 (Gini index) 最小化准则, 进行特征选择, 生成二叉树。

1. 回归树生成

每次划分时逐一考察当前集合中所有特征的所有取值, 根据平方误差最小化准则选择最优划分点。

Definition 5.8.1

假设 X 与 Y 分别为输入和输出变量,并且 Y 是连续变量.给定数据集 D ,考虑如何生成回归树.一棵回归树对应着输入空间 (即特征空间) 的一个划分以及在划分的单元上的输出值.假设已将输入空间划分为 M 个单元 $R_1, R_2, \dots, R_M$,并且在每个单元 R_m 上有一个固定的输出值 c_m ,于是回归树模型可表示为:

$$f(x) = \sum_{m=1}^M c_m I(x \in R_m)$$

当输入空间的划分确定时,可以用平方误差 $\sum_{x_i \in R_m} (y_i - f(x_i))^2$ 来表示回归树对训练数据的预测误差,用平方误差最小的准则求解每个单元上的最优输出值.

[bkk] 易知(可以通过求导方法证明),单元 R_m 上, c_m 的最优值 $\hat{c}_m$ 是 R_m 上的所有输入实例 x_i 对应的输出 y_i 的均值,即:

$$\hat{c}_m = \text{ave}(y_i | x_i \in R_m)$$

问题是怎样对输入空间进行划分.采用启发式的方法,选择第 j 个变量 $x^{(j)}$ 和它的取值 s 作为切分变量 (splitting variable) 和切分点 (splitting point),并定义两个区域:

$$R_1(j, s) = \{x | x^{(j)} \leq s\}, \quad R_2(j, s) = \{x | x^{(j)} > s\}$$

然后寻找最优切分变量 j 和最优切分点 s .具体地,求解

$$\min_{j,s} \left[\min_{c_1} \sum_{x_i \in R_1(j,s)} (y_i - c_1)^2 + \min_{c_2} \sum_{x_i \in R_2(j,s)} (y_i - c_2)^2 \right]$$

对固定的输入变量 j 可以找到最优切分点 s .

$$\hat{c}_1 = \text{ave}(y_i | x_i \in R_1(j, s)) \quad \hat{c}_2 = \text{ave}(y_i | x_i \in R_2(j, s))$$

遍历所有输入变量,找到最优切分变量 j ,构成一个对 (j, s) ,依次将输入空间划分为两个区域.接着对每个区域重复上述划分过程,直到满足停止条件为止.这样就生成一颗回归树.这样的回归树通常称为**最小二乘树**.

Example

假设我们有 5 套二手房的房源数据,特征 X 为面积 (单位: 十平米),目标标签 Y 为售价 (单位: 百万元)。我们希望构建一棵回归树,通过面积来预测房价。

1. 基础数据准备

房源 ID	面积 X (特征)	售价 Y (标签)
1	1	1.5
2	2	2.5
3	3	6.0
4	4	7.0
5	5	8.0

2. 确定候选切分点 s

因为输入特征 X 是连续值,我们需要采用二分法来寻找切分点。首先将 X 的取值从小到大排序 (这里已经是排好序的: 1, 2, 3, 4, 5),然后取相邻两个数值的中点作为候选切分点:

候选点集合 $T_X = \{1.5, 2.5, 3.5, 4.5\}$

接下来,算法需要遍历这 4 个候选切分点,看看哪一刀切下去能让两边的总平方误差最小。

3. 核心计算演示: 以 $s = 2.5$ 为例

我们将特征空间以 $s = 2.5$ 为界一分为二,定义两个区域 R_1 和 R_2 :

- 区域 $R_1(X \leq 2.5)$: 包含房源 {1, 2}, 对应的真实售价 $Y = \{1.5, 2.5\}$ 。
- 区域 $R_2(X > 2.5)$: 包含房源 {3, 4, 5}, 对应的真实售价 $Y = \{6.0, 7.0, 8.0\}$ 。

3.1 步骤 A: 计算各区域的预测常数 (均值)

根据导数优化结论，每个区域的最优预测值就是该区域内所有标签的平均值：

$$\hat{c}_1 = \text{ave}(Y \in R_1) = \frac{1.5 + 2.5}{2} = 2.0$$
$$\hat{c}_2 = \text{ave}(Y \in R_2) = \frac{6.0 + 7.0 + 8.0}{3} = 7.0$$

3.2 步骤 B：计算两边的平方误差之和
现在我们来查看如果用均值作为预测值，产生的误差有多大：
 R_1 的平方误差：

$$SE_1 = (1.5 - 2.0)^2 + (2.5 - 2.0)^2 = 0.25 + 0.25 = 0.5$$

R_2 的平方误差：

$$SE_2 = (6.0 - 7.0)^2 + (7.0 - 7.0)^2 + (8.0 - 7.0)^2 = 1.0 + 0.0 + 1.0 = 2.0$$

总平方误差 = $SE_1 + SE_2 = 0.5 + 2.0 = 2.5$

4. 遍历所有候选切分点的结果对比

使用同样的方法，算法会把其余三个候选点全部算一遍，得到如下的成绩单：

候选切分点 s	左侧区域 R_1 的均值 $\hat{c}_1$	右侧区域 R_2 的均值 $\hat{c}_2$	总平方误差 "SE"_1 + "SE"_2
$s = 1.5$	1.5	5.875	$0 + 17.188 = 17.188$
$s = 2.5$	2.0	7.0	$0.5 + 2.0 = 2.5$ （最优）
$s = 3.5$	3.333	7.5	$11.167 + 0.5 = 11.667$
$s = 4.5$	4.25	8.0	$21.25 + 0 = 21.25$

5. 做出最终切分决策

通过对比可以清晰地看到，在 $s = 2.5$ 处切一刀，系统的总平方误差最小（仅为 2.5）。因此，CART 回归树会选择 $X \leq 2.5$ 作为根节点的划分标准。
此时，回归树的第一层构建完毕：
1. 只要新来一套房子，其面积 ≤ 2.5 （即小于 25 平米），树模型就会直接预测其价格为 2.0 百万元。
2. 如果面积 > 2.5 ，则预测其价格为 7.0 百万元。

提示：如果数据量很大，算法会对 R_1 和 R_2 两个子区域继续递归调用这个过程，直到每个小格子里的误差降到满意范围内为止。

2. 分类树生成

Definition 5.8.2

基尼指数

分类树用基尼指数选择最优特征,同时决定该特征的最优二值切分点.
分类问题中,假设有 K 个类,样本点属于第 k 类的概率为 p_k ,则概率分布的基尼指数定义为

$$\text{Gini}(D) = \sum_{k=1}^K p_k(1 - p_k) = 1 - \sum_{k=1}^K p_k^2$$

对于给定的样本集合 D 和属性 $a \in A$,其基尼指数为

$$\text{Gini_index}(D, a) = \sum_{v=1}^V \text{Gini}(D^v)$$

[b1] 基尼指数 $\text{Gini}(D)$ 表示集合 D 的不确定性,基尼指数 $\text{Gini}(D, a)$ 表示经 $A = a$ 分割后集合 D 的不确定性.

Remark

CART 生成算法停止计算的条件是**结点中样本个数小于预定阈值**或**样本集的基尼指数小于预定阈值** (样本基本属于同一类), 或者没有更多特征.

Algorithm 5.8.3

CART 生成算法

输入: 训练数据集 D , 停止计算的条件.
输出: CART 决策树.
根据训练数据集, 从根结点开始, 递归地对每个结点进行以下操作, 构建二叉决策树:

1. 设结点的训练数据集为 D , 计算现有特征对该数据集的基尼指数. 此时, 对每一个特征 A , 对其可能取的每个值 a , 根据样本点对 $A = a$ 的测试为“是”或“否”, 将 D 分割成 D_1 和 D_2 两部分, 计算 $A = a$ 时的基尼指数:

$$\text{Gini}(D, a) = \frac{|D_1|}{|D|} \text{Gini}(D_1) + \frac{|D_2|}{|D|} \text{Gini}(D_2)$$
2. 在所有可能的特征 A 以及它们所有可能的切分点 a 中, 选择**基尼指数最小的特征**及其对应的切分点作为最优特征与最优切分点. 依最优特征和最优切分点, 从现结点生成两个子结点, 将训练数据集依特征分配到两个子结点中去.
3. 对两个子结点递归地调用 (1), (2), 直至满足停止条件.
4. 生成 CART 决策树.

算法停止计算的条件是**结点中的样本个数小于预定阈值**, 或**样本集的基尼指数小于预定阈值** (样本基本属于同一类), 或者没有更多特征.

Example

ID	有房 (A_1)	婚姻 (A_2)	年薪 (万) (A_3)	类别 (Y)
1	是	单身	12.5	0 (正常)
2	否	已婚	10.0	0 (正常)
3	否	单身	7.0	1 (拖欠)
4	是	已婚	12.0	0 (正常)
5	否	离婚	9.5	1 (拖欠)

表 3 贷款申请数据集

- 数据摘要:
- 总样本 $|D| = 5$.
 - 类别分布: $Y = 0$ (3 个), $Y = 1$ (2 个).
 - 根节点的基尼指数:

$$\text{Gini}(D) = 1 - \left(\left(\frac{3}{5} \right)^2 + \left(\frac{2}{5} \right)^2 \right) = 1 - (0.36 + 0.16) = 0.48$$

- (1) 特征 A_1 : 有房
- 切分点: {是} vs {否}
 - D_1 (有房): ID {1, 4} (全 0). $\text{Gini} = 0$.
 - D_2 (没房): ID {2, 3, 5} (0, 1, 1). $\text{Gini} = 1 - \left(\left(\frac{1}{3} \right)^2 + \left(\frac{2}{3} \right)^2 \right) \approx 0.444$.
 - 加权 Gini:

$$\text{Gini}(D, A_1) = \frac{2}{5}(0) + \frac{3}{5}(0.444) \approx 0.267$$

- (2) 特征 A_2 : 婚姻 (多值离散) CART 需要将其转为二值逻辑.
- 切分 A: {已婚} vs {单身, 离婚}
 - D_1 (已婚): ID {2, 4} (全 0). $\text{Gini} = 0$.
 - D_2 (其他): ID {1, 3, 5} (0, 1, 1). $\text{Gini} = 0.444$.
 - 结果: $\frac{2}{5}(0) + \frac{3}{5}(0.444) \approx 0.267$.
 - 切分 B: {单身} vs {已婚, 离婚}
 - D_1 (单身): ID {1, 3} (0, 1). $\text{Gini} = 0.5$.
 - D_2 (其他): ID {2, 4, 5} (0, 0, 1). $\text{Gini} = 0.444$.
 - 结果: $\frac{2}{5}(0.5) + \frac{3}{5}(0.444) = 0.2 + 0.267 = 0.467$.

(3) 特征 A_3 : 年薪 (连续值) 先排序: 7.0, 9.5, 10.0, 12.0, 12.5. 取相邻中点作为候选切分点: 8.25, 9.75, 11.0, 12.25. 我们测试切分点 9.75:

- D_1 (≤ 9.75): ID {3, 5} (全 1, 拖欠). Gini = 0.
- D_2 (> 9.75): ID {1, 2, 4} (全 0, 正常). Gini = 0.
- 加权 Gini:

$$\text{Gini}(D, A_3 = 9.75) = 0$$

(完美划分)

比较发现,特征 A_3 (年薪 ≤ 9.75) 的基尼指数最小 (0.0). 因此将其作为根节点.由于划分后的子集纯度均为 0,树的生长停止.每一层节点都是一场独立的比赛,胜者通吃 (成为切分点),败者只能等待下一轮 (子节点) 的机会.如果比赛在这一轮就彻底结束了 (完美分类),败者就没有上场机会了.

5.9 CART 剪枝

CART 剪枝算法的目标是剪掉那些算了很多步骤但是对 Gini 的收益不是太大的分支.

由两步组成: 首先从生成算法产生的决策树 T_0 底端开始不断剪枝,直到 T_0 的根节点,形成一个子树序列 $\{T_0, T_1, \dots, T_n\}$; 然后通过交叉验证法在独立的验证数据集上对子树序列进行测试,从中选择最优子树.

[bm] 第一步是贪心地生成一系列候选树,第二步用交叉验证从候选树里挑最好的.

在剪枝过程中,计算子树的损失函数:

$$C_\alpha(T) = C(T) + \alpha|T|$$

其中, T 为任意子树, $C(T)$ 为对训练数据的预测误差 (如基尼指数), $|T|$ 为子树的叶结点个数, $\alpha \geq 0$ 为参数, $C_\alpha(T)$ 为参数是 α 时子树 T 的整体损失.参数 α 权衡训练数据的拟合程度与模型的复杂度.对固定的 α ,一定存在使损失函数 $C_\alpha(T)$ 最小的子树,将其表示为 T_α . T_α 在损失函数 $C_\alpha(T)$ 最小的意义下是最优的.

Theorem 5.9.1

CART 剪枝算法中,当 α 确定时,存在唯一的最小子树 T_α 使损失函数 $C_\alpha(T)$ 最小.

Proof: 因为 $C_\alpha(T) = C(T) + \alpha|T|$,而 $C(T)$ 随节点数 $|T|$ 增加而递减, $\alpha|T|$ 随 $|T|$ 增加而递增,因此 $C_\alpha(T)$ 是关于 $|T|$ 的连续分段线性函数.在所有可能的子树中,存在一个最优子树 T_α 使得 $C_\alpha(T_\alpha)$ 最小; 由于损失函数的单调性和连续性,最优子树是唯一的. $\square$

Breiman 等人证明: 可以用递归的方法对树进行剪枝.将 α 从小增大, $0 = \alpha_0 < \alpha_1 < \dots < \alpha_n < +\infty$, 产生一系列的区间 $[\alpha_i, \alpha_{i+1})$, $i = 0, 1, \dots, n$; 剪枝得到的子树序列对应着区间的最优子树序列 $\{T_0, T_1, \dots, T_n\}$, 序列中的子树是嵌套的.

具体地,从整体树 T_0 开始剪枝,对 T_0 的任意内部结点 t ,以 t 为单结点的损失函数是

$$C_\alpha(t) = C(t) + \alpha$$

以 t 为根结点的子树 T_t 的损失函数是

$$C_\alpha(T_t) = C(T_t) + \alpha|T_t|$$

当 $\alpha = 0$ 及 α 充分小的时候,有:

$$C_\alpha(T_t) < C_\alpha(t)$$

当 α 增大时,在某一 α 有

$$C_\alpha(T_t) = C_\alpha(t)$$

只要 $\alpha = \frac{C(t) - C(T_t)}{|T_t| - 1}$, T_t 与 t 有相同的损失函数值,而 t 的结点数少,要对 T_t 进行剪枝.为此,对 T_0 中每一内部结点 t ,计算

$$g(t) = \frac{C(t) - C(T_t)}{|T_t| - 1}$$

他表示剪枝后整体损失函数减少的程度.在 T_0 中剪去 $g(t)$ 最小的 T_t ,将得到的子树作为 T_1 ,同时将最小的 $g(t)$ 设为 α_1 . T_1 为区间 $[\alpha_1, \alpha_2)$ 的最优子树.

Theorem 5.9.2

CART 剪枝算法中,求出的子树序列 $\{T_0, T_1, \dots, T_n\}$ 分别是区间 $\alpha \in [\alpha_i, \alpha_{i+1})$ 的最优子树 T_α , 这里 $i = 0, 1, \dots, n$, $0 = \alpha_0 < \alpha_1 < \dots < \alpha_n < +\infty$

Proof: 已知 T_0 是完整的决策树, T_i 是剪枝掉 T_{i+1} 的一个分支得到的子树.因为 T_0 在 $\alpha = 0$ 时最优,假设 T_k 在 $[\alpha_k, \alpha_{k+1})$ 内最优,由于剪枝操作连续,那么 T_{k+1} 在 $[\alpha_{k+1}, \alpha_{k+2})$ 内最优. $\square$

这种方法的优点是可以自动生成一系列复杂度不同的树,便于通过交叉验证选择最佳 α 和对应子树.具体地,利用独立的验证数据集,测试子树序列 $T_0, T_1, \dots, T_n$ 中各棵子树的平方误差或基尼指数.平方误差或基尼指数最小的决策树被认为是最优的决策树.在子树序列中,每棵子树 $T_1, T_2, \dots, T_n$ 都对应一个参数 $\alpha_1, \alpha_2, \dots, \alpha_n$.所以,当最优子树 T_k 确定时,对应的 α_k 也确定了,即得到最优决策树 T_α .

Algorithm 5.9.3

CART 剪枝算法

输入: CART 算法生成的决策树 T_0

输出: 最优决策树 T_α

1. 设 $k = 0, T = T_0$
2. $\alpha = +\infty$
3. 自下而上地计算各内部结点 t 的 $C(T_t), |T_t|. g(t) = \frac{C(t) - C(T_t)}{|T_t| - 1}, \alpha = \min(\alpha, g(t))$.
4. 对 $g(t) = \alpha$ 的内部结点 t 进行剪枝,并对叶结点 t 以多数表决法决定其类,得到树 T .
5. 设 $k = k + 1, \alpha_k = \alpha, T_k = T$
6. 如果 T_k 不是由根结点及两个叶结点构成的树,回到步骤 2; 否则令 $T_k = T_n$, 得到 $\{T_0, T_1, \dots, T_n\}$ 一系列剪枝后的子树
7. 子树序列 $T_0, T_1, \dots, T_n$ 拿到验证集上全部跑一遍, 选取最优子树 T_α

6 集成学习

6.1 定义与概念

集成学习 (Ensemble Learning) 通过构建并结合多个学习器来完成学习任务,通常能够获得比单一学习器显著优越的泛化性能.每个个体学习器可以是不同的.

[bn] 集成学习:每个基学习器通常处理相同的特征空间,只是通过数据采样(Bagging)、序列纠错(Boosting)或随机化(Random Forest)产生差异;差异性来源是模型/数据的随机性,而非视图的语义差异.

Multi-View + Mixture of Expert:核心思想是不同视图对应不同的语义信息源,目标是利用数据的多模态/多表示结构.每个“专家”对应一个语义上不同的视图(如图像、文本、音频;或不同传感器;或不同特征子集的语义分组);差异性来源是数据本身的结构异质性.学习目标往往包含视图一致性约束(co-training、CCA 等),这是集成学习没有的

集成学习主要分为两大类:一类是利用基学习器之间的依赖关系进行串行生成的 Boosting (如 AdaBoost, GBDT); 另一类是利用基学习器之间的独立性进行并行生成的 Bagging, 以及其特例**随机森林**(Random Forest).

6.1.1 同质集成/异质集成

- **同质集成 (Homogeneous Ensemble)**: 集成中只包含同种类型的个体学习器,此时个体学习器也被称为“**基学习器**(base learner)”,相应的学习算法称为“**基学习算法**”.
- **异质集成 (Heterogeneous Ensemble)**: 只要是使用多个学习器来解决问题,就可以认为符合集成学习的广义形式.个体学习器由不同的学习算法生成,不存在“基学习算法”.相应地,个体学习器一般不称为“基学习器”.

6.1.2 多专家组合(multiexpert combination)

个体学习器并行工作.

- **全局(Global)方法**: 又称为融合(fusion),给定一个样本输入,所有的个体学习器都产生一个输出,并且所有这些输出都要使用(通过平均、投票或堆叠来产生最终结果)
- **局部(Local)方法**: 又称为选择(selection),如混合专家(mixture of expert),有一个门控模型动态考察输入,并选择一小部分学习器来产生输出

6.1.3 多级组合(multistage combination)

个体学习器按**复杂度递增**排序,构成一个**串行的组合**,其中后一个基学习器在前一个基学习器预测不够准确的样本上进行训练,或者对前一个学习器的预测残差、梯度进行调整.

Example

简单的投票分类器

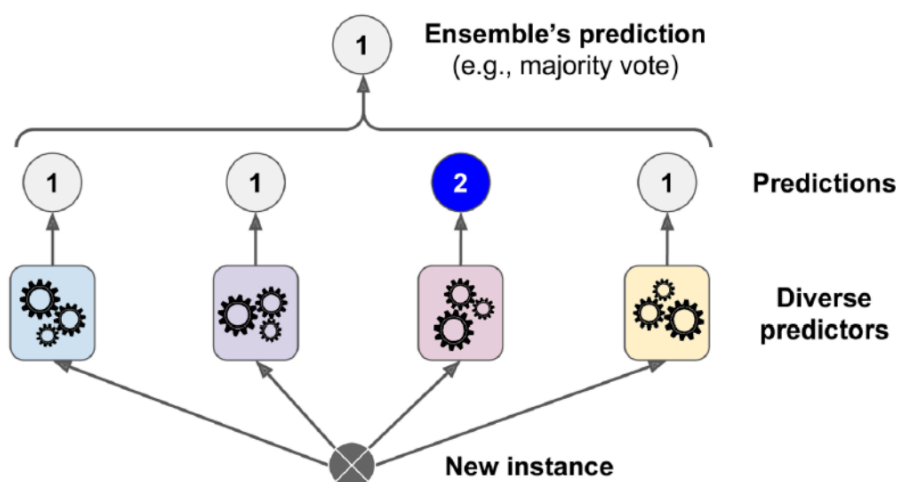


图 12 硬投票分类器：聚合每个分类器的预测,然后将得票最多的结果作为预测类别。

集成学习可能起作用,或者不起作用,甚至起反作用。

6.2 集成学习理论基础

6.2.1 Why Ensemble?

1. 统计分析视角 → 降低“单一选择风险”

虽然能在训练集上找到表现最好的分类器,但它在未来数据上的表现很难保证.集成多个“还不错”的分类器,通过平均/投票等方式组合,更有可能落在“好分类器”区域内,从而降低选错的风险.

2. 算法求解视角 → 避免陷入局部极小值

很多算法使用梯度下降,容易卡在局部最优,导致泛化性能差.多个模型分布在不同的位置,综合起来更接近全局最优.

3. 模型表达视角 → 扩展假设空间,逼近真实函数

某些任务的真实最佳分类器不在当前算法能表达的范围内.单个模型无效,但多个模型线性组合后可能逼近真实分类器.

6.2.2 How to Ensemble?

- Combination level (集成层面): 设计不同的“组合器”(Combiner),比如投票、加权平均、Stacking 元学习器等.
 - ▶ ☒ 举例: 同样是 5 个模型,你可以用“多数投票”,也可以用“按准确率加权”,还可以用另一个模型来学习怎么组合它们 —— 这就是在“集成层面”制造多样性.
- Classifier level (模型层面): 使用不同的基分类器 (Base Classifiers),比如有的用决策树,有的用 SVM,有的用神经网络.
 - ▶ ☒ 举例: 异质集成 (如 Stacking) 就是在这个层面做文章 —— 让不同算法的模型一起工作.
- Feature level (特征层面): 给每个模型提供不同的特征子集 (Feature Subsets).
 - ▶ ☒ 举例: 随机森林中每棵树只看到部分特征; 或者人工给不同模型分配不同列的数据.
- Data level (数据层面): 给每个模型提供不同的数据子集 (Data Subsets),比如自助采样 (Bootstrap Sampling).
 - ▶ ☒ 举例: Bagging / 随机森林的核心 —— 每棵树训练在不同的 bootstrap 样本上.

6.3 Boosting

通过改变训练样本的权重学习多个分类器,并将这些分类器进行线性组合,提高分类的性能.

6.3.1 AdaBoost 算法

Boosting 从弱学习算法出发,反复学习,得到一系列弱分类器 (又称为基本分类器),然后组合这些弱分类器,构成一个强分类器.

大多数的 Boosting 都是改变训练数据的概率分布 (训练数据的权值分布),针对不同的训练数据分布调用弱学习算法学习一系列弱分类器.

两个问题:

1. 每一轮如何改变训练数据的权值或概率分布;
2. 如何将弱分类器组合成一个强分类器.

AdaBoost 的做法是**提高那些被前一轮弱分类器错误分类样本的权值,而降低那些被正确分类样本的权值**,和采取**加权多数表决**的方法.具体而言,加大分类误差率小的弱分类器的权值,使其在表决中起较大的作用;减小分类误差率大的弱分类器的权值,使其在表决中起较小的作用.

Remark

Boosting 算法必须在每一轮改变训练数据的权值,主要有以下三个核心原因:

1. 迫使模型“关注”难分类样本 (核心机制)
2. 避免生成重复模型 (保证多样性)
3. 数学本质: 梯度下降

Algorithm 6.3.1

AdaBoost 算法

输入: 二类分类的训练集 $T = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$, 其中 $x_i \in \mathcal{X} \subseteq \mathbb{R}^n, y_i \in \mathcal{Y} = \{-1, +1\}$; 弱学习算法.

输出: 最终分类器 $G(x)$.

1. 初始化训练数据的权值分布

$$D_1 = (w_{11}, \dots, w_{1i}, \dots, w_{1N}), \quad w_{1i} = \frac{1}{N}, \quad i = 1, 2, \dots, N$$

[bo] w_{mi} 表示第 m 轮中第 i 个实例的权值. 假设每个训练样本在基本分类器的学习中作用相同, 保证第 1 步能够在原始数据上学习基本分类器.

2. 对

$m = 1, 2, \dots, M$:

[bp] 反复学习基本分类器:

- (a) 使用当前分布 D_m 加权的训练数据集学习基本分类器 $G_m(x)$.
- (b) $G_m(x)$ 在加权的训练数据集上的分类误差率是被误分类样本的权值之和.
- (c) 当 $e_m \leq \frac{1}{2}$ 时, $\alpha_m \geq 0$, 并且 α_m 随着 e_m 的减小而增大, 所以分类误差率越小的基本分类器在对结果的影响越大.
- (d) 更新训练数据的权值分布为下一轮作准备.
如果预测正确, y_i 和 $G_m(x_i)$ 同号, 乘积为 +1, 指数部分 $\exp(-\alpha_m) < 1$, 权重减小. 被误分类的则乘以 $\exp(\alpha_m) > 1$, 权重增大. 误分类样本的权值被放大 $e^{2\alpha_m} = \frac{1-e_m}{e_m}$ 倍. 因此, 误分类样本在下一轮学习中起更大的作用.

1. 使用具有权值分布 D_m 的训练数据集学习, 得到基本分类器:

$$G_m(x) : \mathcal{X} \rightarrow \{-1, +1\}$$

2. 计算 $G_m(x)$ 在训练数据集上的分类误差率:

$$e_m = \sum_{i=1}^N P(G_m(x_i) \neq y_i) = \sum_{i=1}^N w_{mi} \mathbb{I}(G_m(x_i) \neq y_i)$$

$$\sum_{i=1}^N w_{mi} = 1$$

3. 计算 $G_m(x)$ 的系数 α_m , 表示 $G_m(x)$ 在最终分类器中的重要性:

$$\alpha_m = \frac{1}{2} \ln \frac{1 - e_m}{e_m}$$

4. 更新训练数据集的权值分布:

$$D_{m+1} = (w_{m+1,1}, \dots, w_{m+1,i}, \dots, w_{m+1,N})$$

$$w_{m+1,i} = \frac{w_{mi}}{Z_m} \exp(-\alpha_m y_i G_m(x_i)), \quad i = 1, 2, \dots, N$$

这里 Z_m 是归一化因子, 确保 $\sum_{i=1}^N w_{mi} = 1$:

$$Z_m = \sum_{i=1}^N w_{mi} \exp(-\alpha_m y_i G_m(x_i))$$

3. 构 建 基 本 分 类 器 的 线 性 组 合

[bq] 实现 M 个基本分类器的加权表决. 所有 α_m 之和不为 1. $f(x)$ 的符号决定实例 x 的类, $f(x)$ 的绝对值表示分类的确信度.

$$f(x) = \sum_{m=1}^M \alpha_m G_m(x)$$

4. 得到最终分类器

$$G(x) = \text{sign}(f(x)) = \text{sign}\left(\sum_{m=1}^M \alpha_m G_m(x)\right)$$

[br] 不改变所给的训练数据,而不断改变训练数据权值的分布,使得训练数据在基本分类器的学习中起不同的作用,这是 AdaBoost 的一个特点.

利用基本分类器的线性组合构建最终分类器是 AdaBoost 的另一特点.

Example

假设有 10 个训练样本,输入为 x ,输出为 $y \in \{-1, +1\}$.

索引 i	1	2	3	4	5	6	7	8	9	10
x_i	0	1	2	3	4	5	6	7	8	9
y_i	1	1	1	-1	-1	-1	1	1	1	-1

我们将使用最简单的“决策桩”(Decision Stump)作为弱分类器,即形如 $G(x) = v$ 或 $G(x) = -v$ 的阈值切分函数.

2. AdaBoost 迭代过程

第 1 轮迭代 ($m = 1$)

首先初始化权值分布,认为所有样本同等重要:

$$D_1 = (w_{1,1}, \dots, w_{1,10}) = (0.1, 0.1, \dots, 0.1)$$

在权值均等的情况下,我们需要找到一个切分点 v ,使得分类误差率最低.

1. 经过遍历,选取阈值 $v = 2.5$.
2. 规则设为: $x < 2.5$ 取 1,否则取 -1.
3. 基本分类器为:

$$G_1(x) = \begin{cases} 1, & x < 2.5 \\ -1, & x > 2.5 \end{cases}$$

4. 误差分析: x 为 6, 7, 8 的样本被错分 (真实为 1, 预测为 -1) .
5. 误差率: $e_1 = 0.1 + 0.1 + 0.1 = 0.3$.
6. 分类器权重:

$$\alpha_1 = \frac{1}{2} \ln \left(\frac{1 - e_1}{e_1} \right) \approx 0.4236$$

更新权值 D_2 : 被分错的样本 6, 7, 8 权值将由 0.1 升至约 0.1667, 分对的样本降至约 0.0714.

第 2 轮迭代 ($m = 2$)

此时样本 6, 7, 8 的权重很大,算法必须优先考虑分对它们.

1. 为了修正上一轮的错误,选取阈值 $v = 8.5$.
2. 规则设为: $x < 8.5$ 取 1,否则取 -1.
3. 基本分类器为:

$$G_2(x) = \begin{cases} 1, & x < 8.5 \\ -1, & x > 8.5 \end{cases}$$

4. 误差分析: 此时 6, 7, 8 分对了,但 3, 4, 5 被错分 (真实为 -1, 预测为 1) .
5. 误差率: 计算的是加权误差, $e_2 = 3 \times 0.0714 \approx 0.2143$.
6. 分类器权重:

$$\alpha_2 = \frac{1}{2} \ln \left(\frac{1 - e_2}{e_2} \right) \approx 0.6496$$

更新权值 D_3 : 被分错的样本 3, 4, 5 的权值在下一轮将显著增大.

第 3 轮迭代 ($m = 3$)

此时样本 3, 4, 5 的权重最大.

1. 选取阈值 $v = 5.5$.
2. 规则设为: $x > 5.5$ 取 1,否则取 -1 (注意方向变了) .
3. 基本分类器为:

$$G_3(x) = \begin{cases} 1, & x > 5.5 \\ -1, & x < 5.5 \end{cases}$$

4. 误差分析: 3, 4, 5 被正确分出,此时错分样本为权重较小的 0, 1, 2, 9.
5. 误差率: $e_3 \approx 0.1820$.
6. 分类器权重:

$$\alpha_3 = \frac{1}{2} \ln \left(\frac{1 - e_3}{e_3} \right) \approx 0.7514$$

3. 最终分类器构建与验证

将三个基本分类器线性组合：

$$f(x) = 0.4236G_1(x) + 0.6496G_2(x) + 0.7514G_3(x)$$

最终判别函数为 $G(x) = \text{sign}(f(x))$.

验证最难分的点 ($x = 5$)：该点真实值为 -1 . 让我们看三个分类器的表现：

1. $G_1(5) = -1$ (正确)
2. $G_2(5) = 1$ (错误, 因为 $5 < 8.5$)
3. $G_3(5) = -1$ (正确, 因为 $5 < 5.5$)

代入公式计算：

$$\begin{aligned} f(5) &= 0.4236 \times (-1) + 0.6496 \times (1) + 0.7514 \times (-1) \\ &= -0.4236 + 0.6496 - 0.7514 \\ &= -0.5254 \end{aligned}$$

$$\text{sign}(-0.5254) = -1$$

结果与真实值一致. AdaBoost 通过 G_1 和 G_3 的“合力”, 修正了 G_2 的错误, 实现了正确分类.

Theorem 6.3.2

AdaBoost 的训练误差界

AdaBoost 算法最终分类器的训练误差界为

$$\frac{1}{N} \sum_{i=1}^N I(G(x_i) \neq y_i) \leq \frac{1}{N} \sum_i \exp(-y_i f(x_i)) = \prod_m Z_m$$

Proof:

当 $G(x_i) \neq y_i$ 时, $y_i f(x_i) < 0$, 因而 $\exp(-y_i f(x_i)) \geq 1$.

$$\begin{aligned} \frac{1}{N} \sum_i \exp(-y_i f(x_i)) &= \frac{1}{N} \sum_i \exp \left(- \sum_{m=1}^M \alpha_m y_i G_m(x_i) \right) \\ &= \sum_i w_{1i} \prod_{m=1}^M \exp(-\alpha_m y_i G_m(x_i)) \\ &= Z_1 \sum_i w_{2i} \prod_{m=2}^M \exp(-\alpha_m y_i G_m(x_i)) \\ &= Z_1 Z_2 \sum_i w_{3i} \prod_{m=3}^M \exp(-\alpha_m y_i G_m(x_i)) \\ &\quad \vdots \\ &= Z_1 Z_2 \cdots Z_{M-1} \sum_i w_{Mi} \exp(-\alpha_M y_i G_M(x_i)) \\ &= \prod_{m=1}^M Z_m \end{aligned}$$

□

6.3.2 AdaBoost 算法的数学本质

AdaBoost 的所有公式 (系数 α_m 的计算、权值 w 的更新) 并非凭空捏造, 其本质是在进行 **前向分步加法建模**, 旨在最小化 **指数损失函数** (前文未提到):

$$\mathbb{L}(y, f(x)) = \exp(-yf(x))$$

其中 $y \in \{-1, +1\}$, $f(x) = \sum_{m=1}^M \alpha_m G_m(x)$ 是强分类器的输出值.

直观理解: 如果 y 和 $f(x)$ 符号相同 (分类正确) 且 $|f(x)|$ 很大 (置信度高), 则 $-yf(x)$ 为很大的负数, 损失 $L \rightarrow 0$; 反之, 损失会呈指数级爆炸增长.

整个算法遵循 **贪心策略 (Greedy Strategy)**: 为了避免求解全局最优的 NP-hard 困境, 算法每次只考虑 **当前这一步** 如何让损失下降得最快 (Line Search), 一旦确定了 f_{m-1} , 就不再去修改之前的参数.

[bs]

1. 为什么 $\alpha_m = \frac{1}{2} \ln\left(\frac{1-e_m}{e_m}\right)$?

这个公式是损失函数的 **解析解 (Analytical Solution)**. 假设在第 m 轮要学习强分类器 f_m , 已有 $m-1$ 轮的强分类器 $f_{m-1}(x)$, 现在寻找最佳的 α 和 $G(x)$ 以构建更强的 f_m . 代入公式 $f_m(x_i) = f_{m-1}(x_i) + \alpha G(x_i)$:

$$\begin{aligned} L(\alpha, G) &= \sum_{i=1}^N \exp(-y_i(f_{m-1}(x_i) + \alpha G(x_i))) \\ &= \sum_{i=1}^N \tilde{w}_{mi} \exp(-y_i \alpha G(x_i)) \end{aligned}$$

这里的 $\tilde{w}_{mi} = \exp(-y_i f_{m-1}(x_i))$ 即第 m 轮的样本权重 (非归一化). 假设已找到最优 $G_m(x)$, 将样本分为“正确”和“错误”两组求解 α :

$$L(\alpha) = (1 - e_m)e^{-\alpha} + e_m e^{\alpha}$$

对 α 求导并令其为 0, 即得:

$$\alpha = \frac{1}{2} \ln\left(\frac{1-e_m}{e_m}\right)$$

这保证了在 G_m 选定后, α_m 能让指数损失沿该方向达到极小值.

1. 权值更新的本质: 函数空间梯度下降

权值更新公式 $w_{m+1,i} = w_{m,i} \exp(-\alpha_m y_i G_m(x_i))$ 递归展开后为:

$$w_{m+1,i} = \exp(-y_i f_m(x_i))$$

这意味着样本权值在数值上正比于该样本在当前模型下的 **指数损失**. 每一轮的弱分类器 G_m , 实际上是在拟合损失函数对于当前模型 f_{m-1} 的 **负梯度方向**. 这是一种 **函数空间 (Function Space) 中的梯度下降**, 而不仅仅是参数调整.

6.3.3 提升树

提升树 (Boosting Tree) 是以分类树或回归树为基本分类器的 Boosting 方法.

提升树 = 把 AdaBoost 中的“样本权重调整”换成“拟合残差”, 并把基学习器固定为决策树 (通常是 CART), 从而能用于回归和分类, 且更稳定、更通用.

1. 加法模型: 基函数的线性组合,

$$f_M(x) = \sum_{m=1}^M T(x; \Theta_m)$$

其中, $T(x; \Theta_m)$ 表示决策树, M 为树的个数. 参数 $\Theta_m = \{(R_1, c_1), \dots, (R_J, c_J)\}$ 包含了树的区域划分 R_j 以及各区域上的输出常数 c_j . 这体现了决策树“区域化”的特性: 将输入空间 $\mathcal{X}$ 划分为 J 个互不相交的区域, 并在每个区域确定输出的常数.

2. 前向分步算法: 首先确定初始提升树 $f_0(x) = 0$, 第 m 步的模型为:

$$f_m(x) = f_{m-1}(x) + T(x; \Theta_m)$$

在这一步中, 我们需要通过经验风险极小化来确定当前这棵树的参数 $\hat{\Theta}_m$:

$$\hat{\Theta}_m = \arg \min_{\Theta_m} \sum_{i=1}^N L(y_i, f_{m-1}(x_i) + T(x_i; \Theta_m))$$

针对不同的问题, 提升树主要区别在于损失函数 L 的选择. 对于回归问题, 通常使用 **平方误差损失函数** $L(y, f(x)) = (y - f(x))^2$. 此时, 算法的优化目标变得非常直观. 将损失函数展开:

$$\begin{aligned} L(y, f_{m-1}(x) + T(x; \Theta_m)) &= (y - f_{m-1}(x) - T(x; \Theta_m))^2 \\ &= (r - T(x; \Theta_m))^2 \end{aligned}$$

这里 $r = y - f_{m-1}(x)$ 即为当前模型拟合数据的 **残差 (Residual)**.

这一推导揭示了回归问题的提升树算法的本质: **每一轮只需要简单地拟合当前模型的残差**. 通过不断地添加新的树去拟合上一轮遗留的误差, 模型就能逐步逼近真实值.

Algorithm 6.3.3

回归问题的提升树算法

输入: 训练数据集 $T = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$, $x_i \in \mathcal{X} \subseteq \mathbb{R}^n$, $y_i \in \mathcal{Y} \subseteq \mathbb{R}$.

输出: 提升树 $f_M(x)$.

1. 初始化 $f_0(x) = 0$.

2. 对 $m = 1, 2, \dots, M$,

1. 计算残差:

$$r_{mi} = y_i - f_{m-1}(x_i), \quad i = 1, 2, \dots, N$$

2. 拟合残差 r_{mi} 学习一个回归树, 得到 $T(x; \Theta_m)$.

3. 更新 $f_m(x) = f_{m-1}(x) + T(x; \Theta_m)$.

3. 得到回归问题提升树:

$$f_M(x) = \sum_{m=1}^M T(x; \Theta_m)$$

Example

已知如表所示的训练数据, x 的取值范围为区间 $[0.5, 10.5]$, y 的取值范围为区间 $[5.0, 10.0]$, 学习这个回归问题的提升树模型, 考虑只用树桩作为基函数.

x_i	1	2	3	4	5	6	7	8	9	10
y_i	5.56	5.70	5.91	6.40	6.80	7.05	8.90	8.70	9.00	9.05

解: 按照回归问题的提升树算法进行迭代.

第 1 步: 求 $f_1(x)$ 即回归树 $T_1(x)$.

首先通过以下优化问题求解训练数据的切分点 s :

$$\min_s \left[\min_{c_1} \sum_{x_i \in R_1} (y_i - c_1)^2 + \min_{c_2} \sum_{x_i \in R_2} (y_i - c_2)^2 \right]$$

其中 $R_1 = \{x \mid x \leq s\}$, $R_2 = \{x \mid x > s\}$. 容易求得使平方损失误差达到最小值的 c_1, c_2 为各区域的均值:

$$c_1 = \frac{1}{N_1} \sum_{x_i \in R_1} y_i, \quad c_2 = \frac{1}{N_2} \sum_{x_i \in R_2} y_i$$

根据所给数据, 考虑如下切分点: 1.5, 2.5, 3.5, 4.5, 5.5, 6.5, 7.5, 8.5, 9.5. 例如, 当 $s = 1.5$ 时, $R_1 = \{1\}$, $R_2 = \{2, 3, \dots, 10\}$, 则 $c_1 = 5.56$, $c_2 = 7.50$. 此时的误差为:

$$m(s) = \sum_{x_i \in R_1} (y_i - c_1)^2 + \sum_{x_i \in R_2} (y_i - c_2)^2 = 0 + 15.72 = 15.72$$

现将 s 及 $m(s)$ 的所有计算结果列于下表.

s	1.5	2.5	3.5	4.5	5.5	6.5	7.5	8.5	9.5
$m(s)$	15.72	12.07	8.36	5.78	3.91	1.93	8.01	11.73	15.74

表 5 计算数据表 ($m(s)$)

由上表可知, 当 $s = 6.5$ 时 $m(s)$ 达到最小值. 此时 $R_1 = \{1, 2, \dots, 6\}$, $R_2 = \{7, 8, 9, 10\}$, $c_1 = 6.24$, $c_2 = 8.91$. 所以回归树 $T_1(x)$ 为:

$$T_1(x) = \begin{cases} 6.24, & x < 6.5 \\ 8.91, & x \geq 6.5 \end{cases}$$

此时 $f_1(x) = T_1(x)$. 用 $f_1(x)$ 拟合训练数据的残差 $r_{2i} = y_i - f_1(x_i)$ 见表 8.4.

x_i	1	2	3	4	5	6	7	8	9	10
r_{2i}	-0.68	-0.54	-0.33	0.16	0.56	0.81	-0.01	-0.21	0.09	0.14

表 6 残差表 (r_{2i})

此时用 $f_1(x)$ 拟合训练数据的平方损失误差为 $L(y, f_1(x)) = \sum (y_i - f_1(x_i))^2 = 1.93$.

第 2 步: 求 $T_2(x)$. 方法与求 $T_1(x)$ 一样, 只是拟合的数据变成了表 8.4 中的残差. 经计算, 当 $s = 3.5$ 时误差最小, 此时:

$$T_2(x) = \begin{cases} -0.52, & x < 3.5 \\ 0.22, & x \geq 3.5 \end{cases}$$

更新模型 $f_2(x) = f_1(x) + T_2(x)$:

$$f_2(x) = \begin{cases} 5.72, & x < 3.5 \\ 6.46, & 3.5 \leq x < 6.5 \\ 9.13, & x \geq 6.5 \end{cases}$$

此时的平方损失误差降为 $L(y, f_2(x)) = 0.79$.

后续步骤： 重复上述过程,继续拟合残差,求得:

$$T_3(x) = \begin{cases} 0.15, & x < 6.5 \\ -0.22, & x \geq 6.5 \end{cases} \quad L(y, f_3(x)) = 0.47$$

$$T_4(x) = \begin{cases} -0.16, & x < 4.5 \\ 0.11, & x \geq 4.5 \end{cases} \quad L(y, f_4(x)) = 0.30$$

$$T_5(x) = \begin{cases} 0.07, & x < 6.5 \\ -0.11, & x \geq 6.5 \end{cases} \quad L(y, f_5(x)) = 0.23$$

$$T_6(x) = \begin{cases} -0.15, & x < 2.5 \\ 0.04, & x \geq 2.5 \end{cases}$$

最终得到的提升树模型 $f_6(x) = f_5(x) + T_6(x)$ 为:

$$f_6(x) = \begin{cases} 5.63, & x < 2.5 \\ 5.82, & 2.5 \leq x < 3.5 \\ 6.56, & 3.5 \leq x < 4.5 \\ 6.83, & 4.5 \leq x < 6.5 \\ 8.95, & x \geq 6.5 \end{cases}$$

此时用 $f_6(x)$ 拟合训练数据的平方损失误差是:

$$L(y, f_6(x)) = \sum_{i=1}^{10} (y_i - f_6(x_i))^2 = 0.17$$

假设此时已满足误差要求,那么 $f(x) = f_6(x)$ 即为所求提升树.

Remark

一棵**深树**是通过**不断切分空间**来细化,而**提升树**是通过**叠加函数**来细化.这一区别导致了完全不同的结果.它选择使用**高偏差、低方差**的弱模型(浅树).

通过不断叠加,它在逐步降低整体的**偏差**,同时因为每个基模型都很简单,整体模型的**方差**并不会像单棵深树那样指数级爆炸.这样既不会欠拟合,也不会过拟合.

6.3.4 梯度提升树 (GBDT)

提升树利用加法模型与前向分步算法实现学习的优化过程.当损失函数是平方损失和指数损失函数时,每一步优化是很简单的.但对一般损失函数而言,往往每一步优化并不那么容易.

针对这一问题,Freidman 提出了梯度提升 (gradient boosting) 算法.这是利用最速下降法的近似方法,其关键是利用损失函数的负梯度在当前模型的值

$$-\left[\frac{\partial L(y, f(x_i))}{\partial f(x_i)}\right]_{f(x)=f_{m-1}(x)}$$

作为回归问题提升树算法中的残差的近似值,拟合一个回归树.

Remark

“提升树”= 拟合残差 (仅适用于平方损失); “梯度提升树 (GBDT)”= 拟合负梯度 (适用于任何可微损失函数).

Algorithm 6.3.4

梯度提升算法

输入： 训练数据集 $T = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$, $x_i \in \mathcal{X} \subseteq \mathbb{R}^n$, $y_i \in \mathcal{Y} \subseteq \mathbb{R}$; 损失函数 $L(y, f(x))$.

输出： 回归树 $\hat{f}(x)$.

(1) 初始化

$$f_0(x) = \arg \min_c \sum_{i=1}^N L(y_i, c)$$

(2) 对 $m = 1, 2, \dots, M$

(a) 对 $i = 1, 2, \dots, N$, 计算对于每个样本的负梯度:

$$r_{mi} = - \left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)} \right]_{f(x)=f_{m-1}(x)}$$

(b) 对 r_{mi} 拟合一个回归树, 得到第 m 棵树的叶结点区域 $R_{mj}, j = 1, 2, \dots, J$.

(c) 对 $j = 1, 2, \dots, J$, 计算每个叶结点权重:

$$c_{mj} = \arg \min_c \sum_{x_i \in R_{mj}} L(y_i, f_{m-1}(x_i) + c)$$

(d) 更新 $f_m(x) = f_{m-1}(x) + \sum_{j=1}^J c_{mj} I(x \in R_{mj})$

(3) 得到回归树:

$$\hat{f}(x) = f_M(x) = \sum_{m=1}^M \sum_{j=1}^J c_{mj} I(x \in R_{mj})$$

算法第 1 步初始化, 估计使损失函数极小化的常数值, 它是只有一个根结点的树. 第 2(a) 步计算损失函数的负梯度在当前模型的值, 将它作为残差的估计. 对于平方损失函数, 它就是通常所说的残差; 对于一般损失函数, 它就是残差的近似值. 第 2(b) 步估计回归树叶结点区域, 以拟合残差的近似值. 第 2(c) 步利用线性搜索估计叶结点区域的值, 使损失函数极小化. 第 2(d) 步更新回归树. 第 3 步得到输出的最终模型 $\hat{f}(x)$.

Example

- **任务:** 预测用户是否购买商品 (二分类问题, $y = 1$ 为购买, $y = 0$ 为未购买) .
- **输入:** 用户停留时间 x (单位: 秒) .
- **模型输出:** $f(x)$, 表示对数几率 (Log-Odds).
- **预测概率:**

$$p(x) = \frac{1}{1 + e^{-f(x)}}$$

- **损失函数** (Log Loss):

$$L(y, f(x)) = -y \ln(p) - (1 - y) \ln(1 - p)$$

- **负梯度** (用于拟合的“伪残差”):

$$r_i = - \frac{\partial L}{\partial f(x_i)} = y_i - p_i$$

即: **真实标签 - 当前预测概率.**

样本 ID	停留时间 x	标签 y
1	10s	0 (没买)
2	20s	0 (没买)
3	50s	1 (买了)

寻找一个常数 c 最小化损失. 对于 Log Loss, 最优常数是 $\ln\left(\frac{P_{\text{Pos}}}{P_{\text{Neg}}}\right)$. 本例中有 1 个正样本, 2 个负样本.

$$f_0(x) = \ln\left(\frac{1}{2}\right) \approx -0.69$$

对应的初始概率为:

$$p_0 = \frac{1}{1 + e^{-(-0.69)}} \approx 0.33$$

Step A. 计算负梯度 (Pseudo-residuals)

计算公式: $r_{1i} = y_i - p_0$

- $x_1: 0 - 0.33 = -0.33$
- $x_2: 0 - 0.33 = -0.33$
- $x_3: 1 - 0.33 = 0.67$

Step B. 拟合回归树

使用回归树拟合残差集合 $\{-0.33, -0.33, 0.67\}$. 最佳切分点为 $x < 35$:

- 左叶子节点 R_{11} : 包含样本 x_1, x_2 (停留时间短).
- 右叶子节点 R_{12} : 包含样本 x_3 (停留时间长).

Step C. 计算叶结点数值 (关键步骤)

使用 Newton-Raphson 近似公式计算叶子权重 c_{mj} :

$$c_{mj} \approx \frac{\sum_{x_i \in R_{mj}} r_i}{\sum_{x_i \in R_{mj}} p_i(1-p_i)} = \frac{\text{梯度之和}}{\text{二阶导(Hessian)之和}}$$

- 对于叶结点 $R_{11} (x_1, x_2)$:

$$\text{分子} = -0.33 + (-0.33) = -0.66$$

$$\text{分母} = [0.33 \times (1 - 0.33)] + [0.33 \times (1 - 0.33)] \approx 0.44$$

$$c_{11} = -\frac{0.66}{0.44} = -1.5$$

- **物理意义**: 对于不买的人群,大幅降低其 Log-Odds 得分.

- 对于叶结点 $R_{12} (x_3)$:

$$\text{分子} = 0.67$$

$$\text{分母} = 0.33 \times (1 - 0.33) \approx 0.22$$

$$c_{12} = \frac{0.67}{0.22} \approx 3.0$$

- **物理意义**: 对于购买的人群,大幅提升其 Log-Odds 得分.

Step D. 更新模型

更新公式: $f_1(x) = f_0(x) + \sum c_{1j}I(x \in R_{1j})$

- x_1, x_2 (短停留) 新得分:

$$-0.69 + (-1.5) = -2.19 \rightarrow \text{新概率 } p \approx 0.10 \quad (\text{更接近 } 0)$$

- x_3 (长停留) 新得分:

$$-0.69 + 3.0 = 2.31 \rightarrow \text{新概率 } p \approx 0.91 \quad (\text{更接近 } 1)$$

后面还可以进一步地重复,这里就不再展开了.

6.4 Bagging 与 随机森林**6.4.1 自主采样(Bootstrap Aggregating)与 Bagging**

Bagging 是并行式集成学习方法的典型代表.它的核心思想是通过**自助采样法**(Bootstrap Sampling)构建出具有差异性的训练数据集,分别训练基学习器,最后通过聚合策略(如多数投票或平均法)得到最终结果.Bagging 的主要目的是**降低方差**(Variance),从而提高模型的稳定性,能够有效减少过拟合.

Definition 6.4.1**自助采样 (Bootstrap Sampling)**

给定包含 m 个样本的数据集 D ,对它采样产生数据集 D' : 每次随机从 D 中挑选一个样本,将其拷贝放入 D' ,然后再将该样本放回初始数据集 D 中,使得该样本在下次采样时仍有可能被采到.

重复执行 m 次后,我们就得到了一个包含 m 个样本的数据集 D' .

在自助采样中,初始数据集 D 中约有 36.8% 的样本未出现在采样数据集 D' 中.这些未使用的样本称为**袋外样本**(Out-of-Bag, OOB),可用于验证模型性能.

Proof: 假设数据集 D 中有 m 个样本.每次采样选中某个特定样本的概率是 $\frac{1}{m}$,不被选中的概率是 $1 - \frac{1}{m}$. 经过 m 次采样,该样本始终未被选中的概率为:

$$p = \left(1 - \frac{1}{m}\right)^m$$

取极限 $m \rightarrow \infty$:

$$\lim_{m \rightarrow \infty} \left(1 - \frac{1}{m}\right)^m = \frac{1}{e} \approx 0.368$$



基于自助采样, Bagging 的基本流程如下:

1. **采样**: 从原始训练集 D 中使用 Bootstrap 方法采样 T 次, 得到 T 个含 m 个训练样本的采样集 $D_1, D_2, \dots, D_T$.
2. **训练**: 基于每个采样集 D_t , 训练出一个基学习器 $h_t(x)$.
3. **聚合**:
 - **分类任务**: 使用简单投票法 $H(x) = \arg \max_y \sum_{t=1}^T \mathbb{I}(h_t(x) = y)$.
 - **回归任务**: 使用简单平均法, $H(x) = \frac{1}{T} \sum_{t=1}^T h_t(x)$.

Bagging 对于不稳定的基学习器 (如不剪枝的决策树、神经网络) 效果最好, 因为这些模型对数据分布的微小变化非常敏感, 而 Bagging 通过平滑这种敏感性来降低方差.

Remark

虽然 Bagging 框架本身是通用的, 但在实际应用 (尤其是随机森林) 中, 基学习器的训练方法有非常具体的最佳实践. 简单来说: 通常使用 CART 决策树, 且不进行剪枝.

原因: Bagging 主要用于降低方差. 它最适合那些对数据分布敏感的、**不稳定** (Unstable) 的算法.

- **不稳定**: 训练数据稍微变动一点, 模型结构就会发生剧烈变化. 决策树就是典型的不稳定算法.
- **稳定**: 如 k-NN、线性回归. 对它们做 Bagging 效果很差, 甚至可能因为重采样导致有效数据变少而起反作用.

这是随机森林训练中最反直觉的一点: **我们希望基学习器“过拟合”**.

在训练每个 $h_t(x)$ 时, 通常采取以下策略:

- **完全生长**: 让决策树一直分裂下去, 直到叶子节点只有 1 个样本, 或者纯度达到 100%.
- **不剪枝**: 传统决策树为了防止过拟合会进行剪枝, 但在 Bagging/RF 中, 我们通常**不做剪枝**.

原理分析 (偏差-方差权衡):

- **单棵树**: 由于不剪枝, 单棵树的复杂度很高, 表现为**低偏差** (Low Bias), 但是**高方差** (High Variance, 即过拟合).
- **集成后**: Bagging 的平均/投票机制可以极大地消除方差.
- **最终结果**: 我们利用单棵树保证了整体的“低偏差”, 利用聚合机制解决了“高方差”, 从而得到了一个双优模型.

6.4.2 随机森林 (Random Forest)

随机森林 (Random Forest, RF) 是用决策树构建的 Bagging. RF 在以决策树为基学习器构建 Bagging 集成的基础上, 进一步在决策树的训练过程中引入了**随机属性选择**. 在随机自主采样的 m 个样本上训练决策树; 分裂节点时, 只考察总特征数 d 的一个随机子集来划分. 一般来说, $m = N, k = \lceil \log_2 d + 1 \rceil$.

[bt] 随机森林包含两个层面的随机性, 这使得基学习器之间的相关性进一步降低:

- (1) **样本扰动**: 通过 Bootstrap 采样, 每个基决策树使用的训练集不同.
- (2) **属性扰动**: 在分裂节点时, 只考察特征的一个随机子集 (通常取 $k = \log_2 d + 1$ 或 $k = \sqrt{d}$).

Algorithm 6.4.2

随机森林算法

输入: 训练集 $D = \{(x_1, y_1), \dots, (x_m, y_m)\}$, 属性集合 A , 基学习器个数 T .

过程: For $t = 1, 2, \dots, T$:

- 对训练集 D 进行 Bootstrap 采样, 得到 D_t .
- 基于 D_t 训练一棵决策树 h_t .
- 在树的每个节点分裂时:
 - 随机从所有属性 A 中选择 k 个属性构成子集 A_k .
 - 在 A_k 中选择最优划分属性进行分裂.
 - 树生长到最大深度, 通常**不剪枝**.

输出: 集成模型 $H(x)$.

- 分类: $H(x) = \text{MajorityVote}\left(\{h_{t(x)}\}_{t=1}^T\right)$
- 回归: $H(x) = \frac{1}{T} \sum_{t=1}^T h_{t(x)}$

为什么随机森林比单纯的 Bagging 更好?

这可以用方差的分解来解释. 假设我们要计算 T 个独立同分布的随机变量均值的方差. 如果变量完全独立, 方差为 $\frac{\sigma^2}{T}$. 但在 Bagging 中, 由于使用的是重叠的数据集, 树之间存在正相关性.

设单棵树的方差为 σ^2 , 任意两棵树之间的相关系数为 ρ . 则集成模型 (T 棵树均值) 的方差为:

$$\text{Var}\left(\frac{1}{T} \sum h_i(x)\right) = \rho\sigma^2 + \frac{1-\rho}{T}\sigma^2$$

Proof:

假设我们有 T 个随机变量（代表 T 棵树的预测输出） $X_1, X_2, \dots, X_T$. 我们要计算的是它们的均值 $\bar{X} = \frac{1}{T} \sum_{i=1}^T X_i$ 的方差. 根据方差的性质, $\text{Var}(aX) = a^2 \text{Var}(X)$. 这里 $a = \frac{1}{T}$, 所以:

$$\text{Var}(\bar{X}) = \text{Var}\left(\frac{1}{T} \sum_{i=1}^T X_i\right) = \frac{1}{T^2} \text{Var}\left(\sum_{i=1}^T X_i\right)$$

展开求和项的方差:

$$\text{Var}\left(\sum_{i=1}^T X_i\right) = \sum_{i=1}^T \text{Var}(X_i) + \sum_{i \neq j} \text{Cov}(X_i, X_j)$$

1. **方差项** (对角线元素): 共有 T 个项. 假设每棵树的方差相同, 即 $\text{Var}(X_i) = \sigma^2$.

$$\sum_{i=1}^T \text{Var}(X_i) = T\sigma^2$$

2. **协方差项** (非对角线元素): 共有 $T \times T - T = T(T-1)$ 个项. 根据相关系数定义 $\rho_{ij} = \text{Cov} \frac{X_i, X_j}{\sigma_i \sigma_j}$. 假设所有树两两之间的相关系数均为 ρ , 且方差均为 σ^2 , 则 $\text{Cov}(X_i, X_j) = \rho\sigma^2$.

$$\sum_{i \neq j} \text{Cov}(X_i, X_j) = T(T-1)\rho\sigma^2$$

将这两部分代回原式:

$$\begin{aligned} \text{Var}(\bar{X}) &= \frac{1}{T^2} [T\sigma^2 + T(T-1)\rho\sigma^2] \\ &= \frac{1}{T}\sigma^2 + \frac{T-1}{T}\rho\sigma^2 \\ &= \frac{1}{T}\sigma^2 + \left(1 - \frac{1}{T}\right)\rho\sigma^2 \\ &= \frac{1}{T}\sigma^2 + \rho\sigma^2 - \frac{1}{T}\rho\sigma^2 \\ &= \rho\sigma^2 + \frac{1}{T}(1-\rho)\sigma^2 \end{aligned}$$

□

Remark

在现实中, Tree 1 和 Tree 2 的相关性肯定不等同于 Tree 1 和 Tree 3. 公式里用同一个 ρ 因为: 这里的 ρ 实际上代表的是“**平均相关系数**” (Average Correlation).

- **第一项 $\rho\sigma^2$** : 当 $T \rightarrow \infty$ 时, 这一项保持不变. 这意味着仅仅增加树的数量不能无限降低方差, 瓶颈在于树之间的相关性 ρ .
- **第二项 $\frac{1-\rho}{T}\sigma^2$** : 随着 T 增加, 这一项趋向于 0.

Bagging 通过样本扰动降低了 ρ . 而随机森林通过**属性扰动**, 强制让树长得“不一样”, 进一步大幅降低了 ρ , 从而在第一项上取得了更好的减方差效果. 虽然这可能会轻微增加单棵树的偏差 (因为只能用部分特征), 但整体泛化误差通常更低.

在分类问题中, 除了硬投票 (Hard Voting, 直接看类别), 随机森林也常用软投票 (Soft Voting), 即平均所有树对各类的预测概率, 通常效果更好.

6.4.3 OOB 误差估计示例

由于 Bootstrap 的特性, 我们可以直接利用袋外样本 (OOB) 来估计泛化误差, 而无需专门划分验证集.

1. 对于每个样本 $(\mathbf{x}_i, y_i)$, 找出那些**没有**使用该样本进行训练的决策树集合, 记为 H_i^{OOB} .
2. 仅利用 H_i^{OOB} 中的树对 $\mathbf{x}_i$ 进行预测, 得到预测结果 $\hat{y}_i^{\text{OOB}}$.
3. OOB 误差率计算为:

$$E_{\text{OOB}} = \frac{1}{m} \sum_{i=1}^m I(\hat{y}_i^{\text{OOB}} \neq y_i)$$

这种估计在 m 足够大时, 与 k 折交叉验证的结果非常接近.

6.4.4 随机森林的优缺点分析

为了更全面地理解随机森林, 我们将其性质总结如下:

优点:

1. **准确性高**: 在许多数据集上表现优异, 通常不需要复杂的调参.
2. **抗过拟合**: 由于双重随机性 (样本和特征), 极大地降低了方差.

- 3. 处理高维数据: 能够处理成千上万的输入变量,且不需要做特征选择 (它自己会做) .
- 4. 特征重要性: 训练完成后,可以给出哪些特征比较重要 (Feature Importance) .
- 5. 并行化: 树与树之间互不依赖,训练速度可以高度并行化,比 Boosting 快.

缺点:

- 1. 模型解释性差: 相比于单棵决策树 (白盒模型) ,几百棵树的集成变成了“黑盒”.
- 2. 对噪声过大的数据: 如果数据噪声特别大,可能会陷入过拟合 (虽然比单棵树好) .
- 3. 计算资源: 需要存储大量的树模型,预测时需要遍历所有树,实时性要求极高的场景 (微秒级) 可能受限.

6.4.5 代码实现: 特征重要性评估

随机森林的一个重要副产物是特征重要性评估.主要有两种方法:

- 1. 基于基尼指数 (MDI) : 在训练过程中,记录每个特征在所有树的分裂节点上减少的杂质 (Gini Impurity) 总和.减少得越多,特征越重要.
- 2. 基于 OOB 数据置换 (MDA) :
 - 用训练好的森林在 OOB 数据上计算误差 E_0 .
 - 保持其他列不变,随机打乱 OOB 数据中特征 j 的数值,再次计算误差 E_j .
 - 特征重要性 $Imp_j = E_j - E_0$.如果打乱特征 j 后误差大幅上升,说明该特征很重要.

Algorithm 6.4.3

OOB 特征重要性计算 (MDA)

输入: 训练好的随机森林 RF,袋外数据 D_{oob} . 输出: 特征重要性向量 I .

- 1. 计算原始 OOB 准确率: $acc_0 = Evaluate(RF, D_{oob})$
- 2. For 每个特征 $j \in \{1, ..., p\}$:
 - 复制 D_{oob} 为 D'_j .
 - 随机打乱 D'_j 中第 j 列的特征.
 - 计算打乱后的准确率: $acc_j = Evaluate(RF, D'_j)$
 - 计算重要性: $I_j = acc_0 - acc_j$
- 3. 对 I 进行归一化处理.

这为特征选择提供了一种非常实用的非参数统计方法.

Example

任务定义:

- 1. 分类 (y_c): 判断水果是否是“苹果” (1=是,0=否) .
- 2. 回归 (y_r): 预测水果的“甜度” (0-10) .

数据集 D (4 个样本):

ID	特征 1: 颜色 (x_1)	特征 2: 大小 (x_2)	分类目标 (y_c)	回归目标 (y_r)
1	1 (红)	1 (小)	1 (是)	7
2	1 (红)	2 (大)	1 (是)	9
3	0 (绿)	1 (小)	0 (否)	3
4	0 (绿)	2 (大)	0 (否)	1

表 7 原始训练集

森林参数设置:

- 树的数量 $T = 2$.
- 特征子集大小 $k = 1$ (为了演示特征扰动,强制每个节点只随机看 1 个特征).
- 最大深度 = 2.

任务一: 分类问题 (Classification)

我们使用 Gini 指数 作为分裂标准.

第一步: 构建第一棵树 (Tree 1)

- 1. Bootstrap 采样: 假设随机采样得到 ID 序列: {1, 2, 2, 3}.
 - 训练集 D_1 : {(红, 小, 1), (红, 大, 1), (红, 大, 1), (绿, 小, 0)}.
 - 注意: 样本 4 没有被选中,它是 OOB 样本.
- 2. 根节点分裂:

- **特征随机选择**: 假设随机抽到了 x_1 (颜色).
- **计算分裂**:
 - 左分支 (红): $\{1, 2, 2\}$ -> 类别 $\{1, 1, 1\}$. $Gini = 1 - \left(\frac{3}{3}\right)^2 - 0 = 0$ (纯度最高).
 - 右分支 (绿): $\{3\}$ -> 类别 $\{0\}$. $Gini = 0$ (纯度最高).
- **结论**: Tree 1 极其简单, 规则为 “红色是苹果, 绿色不是”.

第二步: 构建第二棵树 (Tree 2)

1. **Bootstrap 采样**: 假设随机采样得到 ID 序列: $\{3, 4, 1, 4\}$.
 - 训练集 D_2 : $\{(绿, 小, 0), (绿, 大, 0), (红, 小, 1), (绿, 大, 0)\}$.
 - **注意**: 样本 2 是 OOB 样本.
2. **根节点分裂**:
 - **特征随机选择**: 假设这次随机抽到了 x_2 (大小).
 - **计算分裂**:
 - 左分支 (小): $\{3, 1\}$ -> 类别 $\{0, 1\}$.

$$Gini_{left} = 1 - \left(\frac{1}{2}\right)^2 - \left(\frac{1}{2}\right)^2 = 0.5$$

- 右分支 (大): $\{4, 4\}$ -> 类别 $\{0, 0\}$.

$$Gini_{right} = 0$$

- 加权 Gini: $\frac{2}{4} \times 0.5 + \frac{2}{4} \times 0 = 0.25$.

虽然不完美, 但这是我们唯一的选择 (因为只选了 x_2).

- 左节点 (小) 包含样本 $\{3, 1\}$. 我们需要继续分裂.
3. **左子节点分裂**:
 - 数据: $\{3(绿, 小, 0), 1(红, 小, 1)\}$.
 - **特征随机选择**: 假设抽到了 x_1 (颜色).
 - **分裂**:
 - 红 -> 1 (苹果)
 - 绿 -> 0 (非苹果)
 - 完美分开.

第三步: 集成预测

现在来了一个新水果: 样本 A (颜色=绿, 大小=小). 它实际上是样本 3.

1. **Tree 1 预测**: 看颜色 -> 绿 -> 0 (非苹果).
2. **Tree 2 预测**:
 - 看大小 -> 小 -> 进入左子树.
 - 看颜色 -> 绿 -> 0 (非苹果).
3. **随机森林投票**: 0 票 vs 0 票 -> 最终预测 0.

任务二: 回归问题 (Regression)

我们使用 **均方误差 (MSE)** 或 **方差** 作为分裂标准. 预测值为叶子节点均值.

第一步: 构建第一棵树 (Tree 1)

1. **Bootstrap 采样**: $\{1, 2, 2, 3\}$. 甜度值为 $\{7, 9, 9, 3\}$.
2. **根节点分裂**:
 - 随机选特征 x_1 (颜色).
 - 左 (红): $\{7, 9, 9\}$. 均值 $|y|_L = 8.33$.

$$MSE_L = \frac{1}{3} [(7 - 8.33)^2 + (9 - 8.33)^2 + (9 - 8.33)^2] \approx 0.89$$

- 右 (绿): $\{3\}$. 均值 $|y|_R = 3$. $MSE_R = 0$.
- **Tree 1 结构**: 如果是红, 预测 8.33; 如果是绿, 预测 3.

第二步: 构建第二棵树 (Tree 2)

1. **Bootstrap 采样**: $\{3, 4, 1, 4\}$. 甜度值为 $\{3, 1, 7, 1\}$.
2. **根节点分裂**:
 - 随机选特征 x_2 (大小).
 - 左 (小): $\{3, 7\}$ (样本 3, 1). 均值 $|y|_L = 5$.
 - 右 (大): $\{1, 1\}$ (样本 4, 4). 均值 $|y|_R = 1$.
 - **Tree 2 结构**: 如果是小, 预测 5; 如果是大, 预测 1. (假设停止分裂).

第三步: 集成预测

预测 样本 2 (红, 大), 真实甜度 9.

1. Tree 1 预测: 颜色是红 -> 预测 8.33.
2. Tree 2 预测: 大小是大 -> 预测 1.0. (注意: 这里 Tree 2 预测得很差, 因为它训练数据里只有绿色的“大”水果, 没见过红色的“大”水果, 这就是欠拟合).
3. 随机森林均值:

$$H(x) = \frac{8.33 + 1.0}{2} = 4.665$$

任务三: 袋外误差评估 (OOB Evaluation)

我们不需要单独的测试集, 直接用 OOB 样本评估模型的泛化能力. 这里演示 分类问题 的 OOB 计算.

1. 样本 1: 出现在 Tree 1, Tree 2 的训练集中.-> 没有 OOB 预测.
2. 样本 2 (红, 大, 1):
 - 在 Tree 1 中是训练样本.
 - 在 Tree 2 中是 OOB 样本 (没被抽中).
 - OOB 预测: 仅用 Tree 2 预测样本 2. Tree 2 规则: 看大小(大) -> 预测 0.
 - 误差: 真实是 1, 预测是 0. 错!
3. 样本 3: 出现在 Tree 1, Tree 2.-> 无 OOB.
4. 样本 4 (绿, 大, 0):
 - 在 Tree 1 中是 OOB 样本.
 - 在 Tree 2 中是训练样本.
 - OOB 预测: 仅用 Tree 1 预测样本 4. Tree 1 规则: 看颜色(绿) -> 预测 0.
 - 误差: 真实是 0, 预测是 0. 对!

OOB 错误率计算: 我们只有样本 2 和样本 4 有 OOB 预测结果.

- 样本 2: 错.
- 样本 4: 对.

$$E_{\text{OOB}} = \frac{1}{2} = 50\%$$

Remark

因为数据太少, 且模型非常简单, 误差看起来很大, 但逻辑是通用的.

6.4.6 堆叠

首先在原始数据中训练初级学习器, 然后生成一个“新数据集”训练次级学习器. 一般 在这个“新数据集”中, 初级学习器的输出被当作样例特征, 原始样本的标记仍被当作样例标记.

一般来说, 初级学习器的输出是预测概率向量 (无论 hard/soft), 而不是拿到模型倒数第二层的 embedding 向量再去训练新分类器

[bu] 这种做法更接近 迁移学习 / 表示学习, 比如拿 ResNet 倒数第二层的 embedding 向量作为特征再训练一个新分类器. Stacking 里不这么做, 原因很简单: 初级学习器可能是随机森林、SVM、XGBoost 等完全不同架构的模型, 根本没有统一的“中间层特征”概念. 即便都是神经网络, 暴露内部表示会让元学习器过度依赖某个特定模型的内部结构, 泛化性反而更差. — By Claude

6.5 多样性

Definition 6.5.1

对某个特定的学习器 $h_i(x)$:

$$A(h_i|x) \triangleq (h_i(x) - H(x))^2$$

, $H(x)$ 是最终集成后的输出.

分歧

$$(h_i - y)^2 = [(h_i - H) + (H - y)]^2$$

展开得到:

$$\underbrace{(h_i(x) - y)^2}_{\text{个体误差 } e_i} = \underbrace{(h_i(x) - H(x))^2}_{\text{分歧 } A_i} + \underbrace{(H(x) - y)^2}_{\text{集成误差 } E} + 2(h_i(x) - H(x))(H(x) - y)$$

对 T 个学习器取均值, 交叉项因为 $\sum_i (h_i - H) = 0$ 而消掉, 得到非常优美的结论:

$$E = \bar{E} - \bar{A}$$

集体的错误 = 大家的平均错误 - 大家之间的分歧.

教材里通常给出一张针对**分类器**的多样性度量表,核心是用一个 **2x2 列联表** 来刻画两个学习器 h_i 和 h_j 之间的一致性与差异性.
两分类器的 2x2 列联表

	h_j 正确	h_j 错误
h_i 正确	a	b
h_i 错误	c	d

a, d = 同时对/错 (一致); b, c = 分歧 (不一致); $m = a + b + c + d$

6.5.1 多样性度量 (五大指标)

不合度 (disagreement)

$$dis = \frac{b+c}{m}$$

越大越多样

相关系数 ρ

$$\rho = \frac{ad-bc}{\sqrt{(a+b)(a+c)(c+d)(b+d)}}$$

Q 统计量

$$Q = \frac{ad-bc}{ad+bc} \in [-1, 1]$$

κ 统计量 (kappa)

$$\kappa = \frac{p_1 - p_2}{1 - p_2}, p_1 = \frac{a+d}{m}, p_2 = \frac{(a+b)(a+c)+(c+d)(b+d)}{m^2}$$

双误差率 (double-fault)

$$df = \frac{d}{m}$$

两者同时出错比例

Q 和 ρ 符号相同; $|Q| \geq |\rho|$; κ 越小表示多样性越高

[bv]

- 第一,这五个度量都是**两两之间的** (pairwise),要描述整个集成的多样性,需要对所有 $\binom{T}{2}$ 对求平均,或者画 $\frac{Q}{\rho}$ 的散点图 (纵轴 κ , 横轴个体准确率) 来分析多样性-准确率权衡.
- 第二,多样性难以直接优化.上述 $E = \bar{E} - \bar{A}$ 的推导虽然优美,但分歧 $\bar{A}$ 和个体误差 $\bar{E}$ **并不独立** —— 强行增大分歧 (如引入大量噪声) 往往同时增大个体误差,净效果未必好.这是集成学习的核心难题.
- 第三,Bagging 靠子采样引入多样性,Boosting 靠重加权引入多样性,Stacking 靠异构学习器引入多样性,本质上都是在用不同方式增大 $\bar{A}$.

[bw] 不是我请问了,这和高中知识有区别吗

7 降维与度量学习

7.1 低维嵌入

核心直觉:空间扩张速度远超数据填充速度.在高维情形下出现的数据样本稀疏、距离计算困难等问题,是机器学习方法共同面临的严重障碍,被称为维数灾难 (维数爆炸).

为了缓解维数灾难,需要把原始高维度属性转换为一个低维度子空间,也就是学一个 embedding.

Definition 7.1.1

多维缩放 (Multiple Dimensional Scaling, MDS)

要求数据在降维后 (低维地图上) 的欧氏距离,尽量等于降维前 (原始高维空间中) 的距离.

我们设 m 个样本在 d 维度里,距离矩阵是 $D \in \mathbb{R}^{m \times m}$,目标是获得一个在 $d' < d$ 维度的表示 $Z \in \mathbb{R}^{d' \times m}$,使得在这个低维度里的距离和原来的距离一样.也就是

$$\begin{aligned} d_{ij}^2 &= \|z_i\|^2 + \|z_j\|^2 - 2z_i^T z_j \\ &= b_{ii} + b_{jj} - 2b_{ij} \end{aligned} \quad (1)$$

其中 $B = Z^T Z \in \mathbb{R}^{m \times m}$. 将低维空间的坐标中心点 (原点) 放在所有样本的几何中心上: $\sum_{i=1}^m z_i = 0$. 则内积矩阵 B 的每一行之和、每一列之和都变成了 0.

现在尝试求解 B 的每一个元素.注意到

$$b_{ij} = -\frac{1}{2}(d_{ij}^2 - d_{i.}^2 - d_{.j}^2 + d_{..}^2) \quad (2)$$

所以分析 B 的前 d' 特征值我们就能分析出 Z . B 半正定, 于是

$$\begin{aligned} B &= V \Lambda V^T \\ Z &= \Lambda_*^{\frac{1}{2}} V_*^T \end{aligned} \quad (3)$$

当然实际操作中选前 d' 大的特征值即可.

Algorithm 7.1.2

MDS

输入 $D \in \mathbb{R}^{m \times m}$, 元素是距离 d_{ij} ; 输入低维度数目 d' .

1. 计算 B 的每一个元素. 注意到

$$b_{ij} = -\frac{1}{2}(d_{ij}^2 - d_{i.}^2 - d_{.j}^2 + d_{..}^2) \quad (4)$$

, 其中 $d_{i.} = \frac{1}{m} \sum_{j=1}^m d_{ij}^2$, $d_{.j} = \frac{1}{m} \sum_{i=1}^m d_{ij}^2$, $d_{..} = \frac{1}{m^2} \sum_{i=1}^m \sum_{j=1}^m d_{ij}^2$

2. 获取 B 的特征值和特征向量, 取 $\tilde{\Lambda} \triangleq \text{diag}\{\lambda_1, \dots, \lambda_{d'}\}$ 为 d' 个最大特征值构成的对角矩阵, $\tilde{V} \triangleq [v_1, \dots, v_{d'}]$ 为对应的特征向量矩阵.

3. return $\tilde{V} \tilde{\Lambda}^{\frac{1}{2}}$. 对一个对角矩阵开平方根, 就是对对角线上的数字做平方根.

7.2 样本主成分分析: 基于 (样本) 协方差矩阵

[bx] 总体主成分分析 (上帝视角): 假设你拥有全知全能的视角, 完全掌握了数据产生背后的绝对真理 (即真实的概率分布). 这是一种纯理论的数学推导.

样本主成分分析 (凡人视角): 现实中我们永远无法获得 “所有” 数据, 只能通过观测得到一个有限的数据集. 我们利用这批数据去估计主成分. 我们在机器学习代码里跑的 PCA, 100% 都是样本主成分分析.

主成分分析 (principal component analysis, PCA) 是一种常用的无监督学习方法, 这一方法利用正交变换把由线性相关变量表示的观测数据转换为少数几个由线性无关变量表示的数据, 线性无关的变量称为主成分. 主成分的个数通常小于原始变量的个数, 所以主成分分析属于降维方法. 主成分分析主要用于发现数据中的基本结构, 即数据中变量之间的关系, 是数据分析的有力工具, 也用于其他机器学习方法的前处理.

7.2.1 算法直觉与流程

PCA 的几何本质是通过对原坐标系进行正交变换 (旋转), 寻找一组新的线性无关的坐标轴. 这组新轴能够最大化投影方差 (保留最多信息), 或等价地最小化投影误差, 从而揭示数据的 “基本结构” 并实现降维.

[by] 值得注意的是, 这里的中心化是对特征中心化. 例如, 有样本 $X = [x_1, x_2, \dots, x_n]^T$ (注意 transpose 了, 这意味着 x_i 是每个样本), 那么这个中心化就不是对 x_i 做, 而是对每个样本的第 i 个特征做 (也就是, $X = [f_1, \dots, f_n]$ 的 f_i 中心化).

1. 预处理: 数据规范化

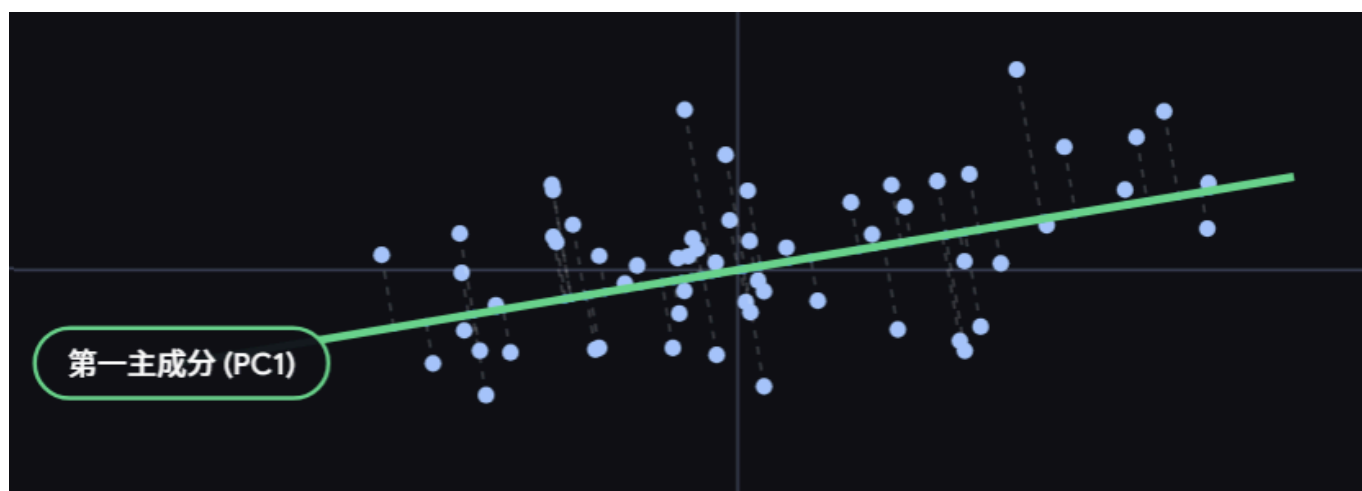
在进行坐标变换前, 需对实数空间中的原始数据进行均值归 0, 方差归 1.

1. 几何直觉: 坐标系旋转与方差最大化

高维数据的分布通常呈现超椭圆球体. 对数据进行 PCA, 等价于在原点附近旋转坐标系:

- **第一主成分 (y_1)**: 选取方差最大的方向作为新坐标系的第一轴 (对应椭圆的长轴). 此方向上保留了最大的数据信息.
- **第二主成分 (y_2)**: 在与 y_1 正交的约束下, 选取方差次之的方向作为第二轴 (对应椭圆的短轴).

经过正交变换后, 新坐标系中的各主成分 ($y_1, y_2, \dots$) 彼此线性无关.



Algorithm 7.2.1**样本主成分分析: 基于 (样本) 协方差矩阵的特征值分解**

输入: 样本集 $D = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m\}^T$; 低维空间维数 d' .

输出: 投影矩阵 $\mathbf{W} = (\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_{d'})$.

过程:

1. **样本中心化** (消除平移偏差, 将数据整体移动到坐标系原点附近):

$$\mathbf{x}_i \rightarrow \mathbf{x}_i - \frac{1}{m} \sum_{i=1}^m \mathbf{x}_i \quad (5)$$

2. **计算样本协方差矩阵** (提取各维度间的相关性与变异程度. 设 $\mathbf{X}$ 为中心化后的 $d \times m$ 样本矩阵):

$$\mathbf{C} = \frac{1}{m-1} \mathbf{X} \mathbf{X}^T \quad (6)$$

3. **特征值分解** (在原始空间中寻找一组全新的正交坐标轴, 使得数据投影后的方差被逐级隔离): 对协方差矩阵 $\mathbf{X} \mathbf{X}^T$ 做特征值分解, 求得特征值与对应的特征向量.

4. **提取主成分进行降维** (丢弃包含极小方差的“噪声”维度, 保留包含最大信息量的前 d' 个方向): 取最大的 d' 个特征值所对应的特征向量 $\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_{d'}$.

Example

我们使用一组均值非零的数据. 假设有 $n = 3$ 个样本, $p = 2$ 个特征:

$$\mathbf{X} = \begin{pmatrix} 3 & 2 \\ 2 & 3 \\ 1 & 1 \end{pmatrix} \quad (7)$$

第一步: 数据中心化

首先计算每个特征 (列) 的均值:

$$\begin{aligned} \bar{x}_1 &= \frac{3+2+1}{3} = 2 \\ \bar{x}_2 &= \frac{2+3+1}{3} = 2 \end{aligned} \quad (8)$$

均值向量为 $\boldsymbol{\mu} = (2, 2)$.

计算中心化矩阵 $\mathbf{B} = \mathbf{X} - \boldsymbol{\mu}$, 即原始数据减去均值:

$$\mathbf{B} = \begin{pmatrix} 3-2 & 2-2 \\ 2-2 & 3-2 \\ 1-2 & 1-2 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ -1 & -1 \end{pmatrix} \quad (9)$$

第二步: 计算协方差矩阵

公式为 $\boldsymbol{\Sigma} = \frac{1}{n-1} \mathbf{B}^T \mathbf{B}$. 这里 $n = 3$, 分母为 2. 先计算内积矩阵 $\mathbf{B}^T \mathbf{B}$:

$$\mathbf{B}^T \mathbf{B} = \begin{pmatrix} 1 & 0 & -1 \\ 0 & 1 & -1 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ -1 & -1 \end{pmatrix} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix} \quad (10)$$

除以 $n-1 = 2$, 得到协方差矩阵:

$$\boldsymbol{\Sigma} = \frac{1}{2} \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix} = \begin{pmatrix} 1 & 0.5 \\ 0.5 & 1 \end{pmatrix} \quad (11)$$

第三步: 计算特征值与贡献度

求解特征方程 $\det(\boldsymbol{\Sigma} - \lambda \mathbf{I}) = 0$:

$$\det \left(\begin{pmatrix} 1-\lambda & 0.5 \\ 0.5 & 1-\lambda \end{pmatrix} \right) = (1-\lambda)^2 - 0.5^2 = 0 \quad (12)$$

$$(1-\lambda)^2 = 0.25 \Rightarrow 1-\lambda = \pm 0.5 \quad (13)$$

解得特征值: $\lambda_1 = 1.5$ (最大特征值, 方差最大)

$\lambda_2 = 0.5$ (次大特征值)

贡献度分析: 总方差 $= 1.5 + 0.5 = 2.0$. 第一主成分贡献率 $= \frac{1.5}{2.0} = 75\%$. 这说明如果只保留第一主成分, 虽然去除了噪声 (或次要信息), 但也损失了 25% 的原始数据差异.

第四步: 计算特征向量

对于 $\lambda_1 = 1.5$,求解方程组 $(\Sigma - 1.5I)v = 0$:

$$\begin{pmatrix} 1-1.5 & 0.5 \\ 0.5 & 1-1.5 \end{pmatrix} \begin{pmatrix} v_1 \\ v_2 \end{pmatrix} = \begin{pmatrix} -0.5 & 0.5 \\ 0.5 & -0.5 \end{pmatrix} \begin{pmatrix} v_1 \\ v_2 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \tag{14}$$

由 $-0.5v_1 + 0.5v_2 = 0$ 整理得 $v_1 = v_2$. 取基础解向量 $v = (1, 1)^T$. 进行单位化 (除以模长 $\sqrt{1^2 + 1^2} = \sqrt{2}$):

$$\alpha_1 = \begin{pmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{pmatrix} \approx \begin{pmatrix} 0.707 \\ 0.707 \end{pmatrix} \tag{15}$$

第五步：降维投影

将中心化后的矩阵 B 投影到第一主成分方向 α_1 上. 注意：一定要用**中心化后**的数据乘以特征向量.

$$Y_{\text{new}} = B \cdot \alpha_1 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ -1 & -1 \end{pmatrix} \begin{pmatrix} 0.707 \\ 0.707 \end{pmatrix} \tag{16}$$

结果总结

样本	原始坐标 (x_1, x_2)	PCA 得分 (y_1)
Sample 1	(3, 2)	0.707
Sample 2	(2, 3)	0.707
Sample 3	(1, 1)	-1.414

注：样本 1 和样本 2 在原始数据中有明显区别 (一个 3,2 一个 2,3),但在第一主成分看来,它们的“综合得分”是一样的 (都是高于平均水平).它们之间的细微区别 (25% 的信息) 被丢弃了.

Remark

如果保留所有的主成分,PCA 就仅仅是一个正交变换 (Orthogonal Transformation),或者更通俗地说,就是一次坐标轴旋转.

1. 为什么数据中心化操作不会失真?

Remark

中心化 ($x_i - \mu$) 在几何上等同于坐标系的“平移”,而不是“缩放”或“扭曲”. 数据的内部结构 (点与点之间的相对距离) 和形状 (数据的分布形态) 在平移后完全保持不变.做这一步是为了让后续计算 (特别是协方差公式) 变得极其简洁.

Proof: 根据方差的定义: $\text{Var}(x) = E[(x - \mu)^2]$. 如果我们构造新变量 $x' = x - \mu$ (即中心化),那么新变量的均值为 0. 新方差 $\text{Var}(x') = E[(x' - 0)^2] = E[(x - \mu)^2]$.

可见 $\text{Var}(x') = \text{Var}(x)$,数据的波动性 (信息量) 完全没有丢失. □

2. 为什么“确定投影方向”等价于“确定系数向量”?

Remark

几何上的“方向”和代数上的“系数”是同一事物的两种表达. 投影直线在空间中的指向,由一个单位向量 α 决定.而线性变换公式中的系数 α_1, α_2 ,恰好就是这个单位向量在坐标轴上的分量.

Proof: 假设有一个单位方向向量 $\alpha = (\alpha_1, \alpha_2)^T$. 数据点 $x = (x_1, x_2)^T$ 在该方向上的投影长度 (标量) y 由点积公式给出:

$$y = \alpha \cdot x = \alpha^T x \tag{17}$$

展开后即为线性变换形式:

$$y = \alpha_1 x_1 + \alpha_2 x_2 \tag{18}$$

因此,寻找最佳的线性组合系数,就是在寻找最佳的空间投影方向. □

3. 为什么协方差矩阵能算出方差最大的方向?

Remark

协方差矩阵 Σ 本质上是一个“方差计算器”.它不仅记录了数据的形状,还决定了二次型 $\alpha^T \Sigma \alpha$ 的值. 通过拉格朗日乘数法可以证明,使得这个二次型 (投影方差) 取极值的方向,必然是 Σ 的特征向量方向.

Proof: 我们的目标是：在约束 $\alpha^T \alpha = 1$ 下,最大化方差 $f(\alpha) = \alpha^T \Sigma \alpha$. 构造拉格朗日函数：

$$L(\alpha, \lambda) = \alpha^T \Sigma \alpha - \lambda(\alpha^T \alpha - 1) \quad (19)$$

对 α 求导并令其为 0：

$$\frac{\partial L}{\partial \alpha} = 2\Sigma \alpha - 2\lambda \alpha = 0 \quad (20)$$

化简得特征方程定义式：

$$\Sigma \alpha = \lambda \alpha \quad (21)$$

这证明了：最优投影方向 α 必须是协方差矩阵的特征向量. 同时,将上式左乘 α^T ,可得最大方差值恰好等于特征值：

$$\text{Var}(y) = \alpha^T (\Sigma \alpha) = \alpha^T (\lambda \alpha) = \lambda(\alpha^T \alpha) = \lambda \quad (22)$$

□

4. 协方差矩阵与原始数据的关系：为什么最大化二次型就是最大化原始方差？

Remark

协方差矩阵 Σ 可以被视为原始海量数据 X 的一个“高保真压缩包”. 它已经预先计算并存储了数据在各个维度上的离散程度和相关性. 公式 $f(\alpha) = \alpha^T \Sigma \alpha$ 并不是一个抽象定义的函数,它实际上就是**投影后新变量方差**的精确计算公式. 因此,不需要每次都对所有原始数据点进行投影计算,直接利用矩阵 Σ 即可瞬间算出任意方向上的方差.

Proof: 假设原始数据向量为 x (已中心化,即 $E[x] = 0$). 我们将数据投影到单位方向 α 上,得到新变量 (标量) y ：

$$y = \alpha^T x \quad (23)$$

我们的目标是计算 y 的方差. 因为均值为 0,方差等于平方的期望：

$$\text{Var}(y) = E[y^2] \quad (24)$$

将 y 的定义代入：

$$\text{Var}(y) = E[(\alpha^T x)^2] \quad (25)$$

利用标量的转置等于其自身 $(\alpha^T x)^T = x^T \alpha$,将平方展开为“自己乘自己”：

$$\text{Var}(y) = E[(\alpha^T x)(x^T \alpha)] \quad (26)$$

根据矩阵乘法的结合律,写成：

$$\text{Var}(y) = E[\alpha^T (x x^T) \alpha] \quad (27)$$

在期望运算 $E[\cdot]$ 中, α 是常数向量,只有 x 是随机变量. 因此可以将 α 提取到期望符号外面：

$$\text{Var}(y) = \alpha^T \cdot \underbrace{E[x x^T]}_{\text{协方差矩阵 } \Sigma} \cdot \alpha \quad (28)$$

最终得证：

$$\text{Var}(y) = \alpha^T \Sigma \alpha \quad (29)$$

这证明了：最大化二次型 $\alpha^T \Sigma \alpha$,在数学上严格等价于最大化原始数据投影后的方差. □

Theorem 7.2.2

m 维随机变量 $y = (y_1, y_2, \dots, y_m)^T$ 的分量依次是 x 的第一主成分到第 m 主成分的充要条件是：

(1) $y = A^T x$, A 为正交矩阵：

$$A = \begin{bmatrix} \alpha_{11} & \alpha_{12} & \dots & \alpha_{1m} \\ \alpha_{21} & \alpha_{22} & \dots & \alpha_{2m} \\ \vdots & \vdots & & \vdots \\ \alpha_{m1} & \alpha_{m2} & \dots & \alpha_{mm} \end{bmatrix} \quad (30)$$

(2) y 的协方差矩阵为对角矩阵：

$$\begin{aligned} \text{cov}(y) &= \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_m) \\ \lambda_1 &\geq \lambda_2 \geq \dots \geq \lambda_m \end{aligned} \quad (31)$$

其中, λ_k 是 Σ 的第 k 个特征值, α_k 是对应的单位特征向量, $k = 1, 2, \dots, m$.

7.2.2 深入理解 PCA：数学推导、性质与应用考量

1. 特征值分解与协方差矩阵对角化

设 λ_k 是协方差矩阵 Σ 的第 k 个特征值, α_k 是对应的单位特征向量,即满足:

$$\Sigma \alpha_k = \lambda_k \alpha_k, \quad k = 1, 2, \dots, m \quad (32)$$

将所有特征向量组合成矩阵 $A = [\alpha_1, \alpha_2, \dots, \alpha_m]$, 将特征值组合成对角矩阵 $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_m)$, 上述关系可写为矩阵形式:

$$\Sigma A = A \Lambda \quad (33)$$

由于 A 是正交矩阵, 满足 $A^T A = A A^T = I$. 在等式两边左乘 A^T , 得到谱分解公式:

$$A^T \Sigma A = \Lambda \quad \text{和} \quad \Sigma = A \Lambda A^T \quad (34)$$

因此, 总体主成分 $y = A^T x$ 的协方差矩阵为:

$$\text{cov}(y) = \text{cov}(A^T x) = A^T \Sigma A = \Lambda \quad (35)$$

这证明了主成分之间互不相关, 且方差为特征值.

2. 总方差守恒性质

利用矩阵迹 (trace) 的性质, 总体主成分 y 的方差之和等于随机变量 x 的方差之和:

$$\sum_{i=1}^m \text{var}(x_i) = \text{tr}(\Sigma) = \text{tr}(A \Lambda A^T) = \text{tr}(A^T A \Lambda) = \text{tr}(\Lambda) = \sum_{i=1}^m \lambda_i \quad (36)$$

3. 因子负荷量 (Factor Loading) 的推导

第 k 个主成分 y_k 与变量 x_i 的相关系数 $\rho(y_k, x_i)$ 称为因子负荷量.

首先计算 y_k 与 x_i 的协方差. 令 e_i 为基本单位向量 (第 i 个分量为 1), 则 $x_i = e_i^T x, y_k = \alpha_k^T x$.

$$\text{cov}(y_k, x_i) = \text{cov}(\alpha_k^T x, e_i^T x) = \alpha_k^T \Sigma e_i = e_i^T \Sigma \alpha_k = \lambda_k e_i^T \alpha_k = \lambda_k \alpha_{ik} \quad (37)$$

代入相关系数公式:

$$\rho(y_k, x_i) = \frac{\text{cov}(y_k, x_i)}{\sqrt{\text{var}(y_k) \text{var}(x_i)}} = \frac{\lambda_k \alpha_{ik}}{\sqrt{\lambda_k} \sqrt{\sigma_{ii}}} = \frac{\sqrt{\lambda_k} \alpha_{ik}}{\sqrt{\sigma_{ii}}} \quad (38)$$

4. 因子负荷量的性质

(1) 第 k 个主成分 y_k 与 m 个变量的因子负荷量加权平方和满足:

$$\sum_{i=1}^m \sigma_{ii} \rho^2(y_k, x_i) = \sum_{i=1}^m \sigma_{ii} \frac{\lambda_k \alpha_{ik}^2}{\sigma_{ii}} = \lambda_k \sum_{i=1}^m \alpha_{ik}^2 = \lambda_k \alpha_k^T \alpha_k = \lambda_k \quad (39)$$

(2) m 个主成分与第 i 个变量 x_i 的因子负荷量平方和为 1: 由于 x_i 可表示为 $y_1, \dots, y_m$ 的线性组合, 且 y_k 互不相关, 故 x_i 与 $y_1, \dots, y_m$ 的相关系数平方和为 1:

$$\sum_{k=1}^m \rho^2(y_k, x_i) = 1 \quad (40)$$

主成分分析的主要目的是降维, 所以一般选择 k ($k \ll m$) 个主成分 (线性无关变量) 来代替 m 个原有变量 (线性相关变量), 使问题得以简化, 并能保留原有变量的大部分信息. 这里所说的信息是指原有变量的方差. 为此, 先给出一个定理, 说明选择 k 个主成分是最优选择.

Theorem 7.2.3

对任意正整数 q , $1 \leq q \leq m$, 考虑正交线性变换

$$y = B^T x \quad (41)$$

其中, y 是 q 维向量, B^T 是 $q \times m$ 矩阵, 令 y 的协方差矩阵为

$$\Sigma_y = B^T \Sigma B \quad (42)$$

则 Σ_y 的迹 $\text{tr}(\Sigma_y)$ 在 $B = A_q$ 时取得最大值, 其中矩阵 A_q 由正交矩阵 A 的前 q 列组成.

Remark

“最优”是指: 在约束为 k 维的前提下, PCA 提供了数学上信息损失最小的线性解.

在实际问题中, 不同变量可能有不同的量纲, 直接求主成分有时会产生不合理的结果. 为了消除这个影响, 常常对各个随机变量实施规范化, 使其均值为 0, 方差为 1.

设 $x = (x_1, x_2, \dots, x_m)^T$ 为 m 维随机变量, x_i 为第 i 个随机变量, $i = 1, 2, \dots, m$, 令

$$x_i^* = \frac{x_i - E(x_i)}{\sqrt{\text{var}(x_i)}}, \quad i = 1, 2, \dots, m \quad (43)$$

其中, $E(x_i)$, $\text{var}(x_i)$ 分别是随机变量 x_i 的均值和方差, 这时 x_i^* 就是 x_i 的规范化随机变量.

显然, 规范化随机变量的协方差矩阵就是相关矩阵 $\mathbf{R}$. 主成分分析通常在规范化随机变量的协方差矩阵即相关矩阵上进行. 对照总体主成分的性质可知, 规范化随机变量的总体主成分有以下性质:

(1) 规范化变量主成分的协方差矩阵是

$$\mathbf{\Lambda}^* = \text{diag}(\lambda_1^*, \lambda_2^*, \dots, \lambda_m^*) \quad (44)$$

其中, $\lambda_1^* \geq \lambda_2^* \geq \dots \geq \lambda_m^* \geq 0$ 为相关矩阵 $\mathbf{R}$ 的特征值.

(2) 协方差矩阵的特征值之和为 m :

$$\sum_{k=1}^m \lambda_k^* = m \quad (45)$$

(3) 规范化随机变量 x_i^* 与主成分 y_k^* 的相关系数 (因子载荷) 为

$$\rho(y_k^*, x_i^*) = \sqrt{\lambda_k^*} e_{ik}^*, \quad k, i = 1, 2, \dots, m \quad (46)$$

其中, $\mathbf{e}_k^* = (e_{1k}^*, e_{2k}^*, \dots, e_{mk}^*)^\top$ 为矩阵 $\mathbf{R}$ 对应于特征值 λ_k^* 的单位特征向量.

(4) 所有规范化随机变量 x_i^* 与主成分 y_k^* 的相关系数的平方和等于 λ_k^* :

$$\sum_{i=1}^m \rho^2(y_k^*, x_i^*) = \sum_{i=1}^m \lambda_k^* e_{ik}^{*2} = \lambda_k^*, \quad k = 1, 2, \dots, m \quad (47)$$

(5) 规范化随机变量 x_i^* 与所有主成分 y_k^* 的相关系数的平方和等于 1:

$$\sum_{k=1}^m \rho^2(y_k^*, x_i^*) = \sum_{k=1}^m \lambda_k^* e_{ik}^{*2} = 1, \quad i = 1, 2, \dots, m \quad (48)$$

Remark

这其实相对于前面只是多了一个标准化操作, 其他方面没有区别.

相关系数也可以看成协方差: 一种剔除了两个变量量纲影响、标准化后的特殊协方差, 它消除了两个变量变化幅度的影响, 而只是单纯反应两个变量每单位变化时的相似程度.

Example

$$\mathbf{X} = \begin{pmatrix} 3 & 2 \\ 2 & 3 \\ 1 & 1 \end{pmatrix} \quad (49)$$

我们使用之前的数据 $\mathbf{X}$ 来验证上述五条性质.

0. 数据准备

计算得到特征 (列) 均值均为 2, 样本方差均为 1. 因此, 规范化后的数据 x^* 与中心化数据数值相同. 计算得到相关矩阵 (即规范化数据的协方差矩阵):

$$\mathbf{R} = \begin{pmatrix} 1 & 0.5 \\ 0.5 & 1 \end{pmatrix} \quad (50)$$

(1) 验证特征值排序 求解 $|\mathbf{R} - \lambda \mathbf{I}| = 0$, 得到特征值:

$$\lambda_1^* = 1.5, \quad \lambda_2^* = 0.5 \quad (51)$$

满足 $\lambda_1^* > \lambda_2^*$, 构成对角阵:

$$\mathbf{\Lambda}^* = \begin{pmatrix} 1.5 & 0 \\ 0 & 0.5 \end{pmatrix} \quad (52)$$

(2) 验证特征值之和 特征值之和为 $1.5 + 0.5 = 2$. 变量个数 $m = 2$.

$$\sum_{k=1}^2 \lambda_k^* = 2 = m \quad (53)$$

验证成立.

(3) 计算因子载荷 (相关系数) 首先求单位特征向量:

• 对应 $\lambda_1^* = 1.5$: $\mathbf{e}_1^* = \left(\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}\right)^\top \approx (0.707, 0.707)^\top$

- 对应 $\lambda_2^* = 0.5$: $\mathbf{e}_2^* = \left(\frac{1}{\sqrt{2}}, -\frac{1}{\sqrt{2}}\right)^\top \approx (0.707, -0.707)^\top$

根据公式 $\rho(y_k^*, x_i^*) = \sqrt{\lambda_k^*} e_{ik}^*$ 计算:

- 第一主成分 ($k = 1$):

$$\rho(y_1^*, x_1^*) = \sqrt{1.5} \cdot 0.707 = \sqrt{0.75} \approx 0.866 \quad (54)$$

$$\rho(y_1^*, x_2^*) = \sqrt{1.5} \cdot 0.707 = \sqrt{0.75} \approx 0.866 \quad (55)$$

- 第二主成分 ($k = 2$):

$$\rho(y_2^*, x_1^*) = \sqrt{0.5} \cdot 0.707 = \sqrt{0.25} = 0.5 \quad (56)$$

$$\rho(y_2^*, x_2^*) = \sqrt{0.5} \cdot (-0.707) = -0.5 \quad (57)$$

(4) 验证性质: 行平方和 (对主成分贡献) 验证 $\sum_{i=1}^m \rho^2(y_k^*, x_i^*) = \lambda_k^*$:

- 对于第一主成分 ($k = 1$):

$$0.866^2 + 0.866^2 = 0.75 + 0.75 = 1.5 = \lambda_1^* \quad (58)$$

- 对于第二主成分 ($k = 2$):

$$0.5^2 + (-0.5)^2 = 0.25 + 0.25 = 0.5 = \lambda_2^* \quad (59)$$

验证成立.

(5) 验证性质: 列平方和 (变量的信息解释) 验证 $\sum_{k=1}^m \rho^2(y_k^*, x_i^*) = 1$:

- 对于变量 x_1 ($i = 1$):

$$\rho^2(y_1^*, x_1^*) + \rho^2(y_2^*, x_1^*) = 0.75 + 0.25 = 1 \quad (60)$$

- 对于变量 x_2 ($i = 2$):

$$\rho^2(y_1^*, x_2^*) + \rho^2(y_2^*, x_2^*) = 0.75 + 0.25 = 1 \quad (61)$$

验证成立.

7.3 样本主成分分析

Definition 7.3.1

样本主成分

给定样本矩阵 $\mathbf{X}$. 一般地, 样本第 i 主成分 $y_i = \mathbf{a}_i^\top \mathbf{x}$ 是在 $\mathbf{a}_i^\top \mathbf{a}_i = 1$ 和 $\mathbf{a}_i^\top \mathbf{x}_j$ 与 $\mathbf{a}_k^\top \mathbf{x}_j (k < i, j = 1, 2, \dots, n)$ 的样本协方差 $\mathbf{a}_k^\top \mathbf{S} \mathbf{a}_i = 0$ 条件下, 使 $\mathbf{a}_i^\top \mathbf{x}_j (j = 1, 2, \dots, n)$ 的样本方差 $\mathbf{a}_i^\top \mathbf{S} \mathbf{a}_i$ 最大的 $\mathbf{x}$ 的线性变换.

样本主成分与总体主成分具有同样的性质, 这从样本主成分的定义容易看出. 只要以样本协方差矩阵 $\mathbf{S}$ 代替总体协方差矩阵 Σ 即可. 样本主成分的性质不再重述.

在使用样本主成分时, 一般假设样本数据是规范化的, 即对样本矩阵作如下变换:

$$x_{ij}^* = \frac{x_{ij} - \bar{x}_i}{\sqrt{s_{ii}}}, \quad i = 1, 2, \dots, m, \quad j = 1, 2, \dots, n \quad (62)$$

其中,

$$\bar{x}_i = \frac{1}{n} \sum_{j=1}^n x_{ij}, \quad i = 1, 2, \dots, m \quad (63)$$

$$s_{ii} = \frac{1}{n-1} \sum_{j=1}^n (x_{ij} - \bar{x}_i)^2, \quad i = 1, 2, \dots, m \quad (64)$$

为了方便, 将规范化变量 x_{ij}^* 仍记作 x_{ij} , 规范化的样本矩阵仍记作 $\mathbf{X}$. 这时, 样本协方差矩阵 $\mathbf{S}$ 就是样本相关矩阵 $\mathbf{R}$:

$$\mathbf{R} = \frac{1}{n-1} \mathbf{X} \mathbf{X}^\top \quad (65)$$

样本协方差矩阵 $\mathbf{S}$ 是总体协方差矩阵 Σ 的无偏估计, 样本相关矩阵 $\mathbf{R}$ 是总体相关矩阵的无偏估计, $\mathbf{S}$ 的特征值和特征向量是 Σ 的特征值和特征向量的极大似然估计.

Algorithm 7.3.2

样本主成分分析: 基于相关矩阵的特征值分解

给定样本矩阵 $\mathbf{X}$, 利用数据的样本协方差矩阵或者样本相关矩阵的特征值分解进行主成分分析. 具体步骤如下:

- (1) 对观测数据按式 (16.48) 进行规范化处理, 得到规范化数据矩阵, 仍以 $\mathbf{X}$ 表示.
- (2) 依据规范化数据矩阵, 计算样本相关矩阵 $\mathbf{R}$:

$$\boldsymbol{R} = [r_{ij}]_{m \times m} = \frac{1}{n-1} \boldsymbol{X} \boldsymbol{X}^\top \tag{66}$$

其中,

$$r_{ij} = \frac{1}{n-1} \sum_{l=1}^n x_{il} x_{lj}, \quad i, j = 1, 2, \dots, m \tag{67}$$

(3) 求样本相关矩阵 $\boldsymbol{R}$ 的 k 个特征值和对应的 k 个单位特征向量. 求解 $\boldsymbol{R}$ 的特征方程

$$|\boldsymbol{R} - \lambda \boldsymbol{I}| = 0 \tag{68}$$

得 $\boldsymbol{R}$ 的 m 个特征值:

$$\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_m \tag{69}$$

求方差贡献率 $\sum_{i=1}^k \eta_i$ 达到预定值的主成分个数 k . 求前 k 个特征值对应的单位特征向量:

$$\boldsymbol{a}_i = (a_{1i}, a_{2i}, \dots, a_{mi})^\top, \quad i = 1, 2, \dots, k \tag{70}$$

(4) 求 k 个样本主成分 以 k 个单位特征向量为系数进行线性变换, 求出 k 个样本主成分:

$$\boldsymbol{y}_i = \boldsymbol{a}_i^\top \boldsymbol{x}, \quad i = 1, 2, \dots, k \tag{71}$$

(5) 计算 k 个主成分 $\boldsymbol{y}_j$ 与原变量 $\boldsymbol{x}_i$ 的相关系数 $\rho(\boldsymbol{x}_i, \boldsymbol{y}_j)$, 以及 k 个主成分对原变量 $\boldsymbol{x}_i$ 的贡献率 ν_i .

(6) 计算 n 个样本的 k 个主成分值 将规范化样本数据代入 k 个主成分式 (16.50), 得到 n 个样本的主成分值. 第 j 个样本 $\boldsymbol{x}_j = (x_{1j}, x_{2j}, \dots, x_{mj})^\top$ 的第 i 主成分值是

$$y_{ij} = (a_{1i}, a_{2i}, \dots, a_{mi})(x_{1j}, x_{2j}, \dots, x_{mj})^\top = \sum_{l=1}^m a_{li} x_{lj}, \tag{72}$$

$$i = 1, 2, \dots, m, \quad j = 1, 2, \dots, n \tag{73}$$

主成分分析得到的结果可以用于其他机器学习方法的输入. 比如, 将样本点投影到以主成分为坐标轴的空间中, 然后应用聚类算法就可以对样本点进行聚类.

Example

学生成绩的主成分分析

假设有 n 个学生参加四门课程 (语文 $\boldsymbol{x}_1$ 、外语 $\boldsymbol{x}_2$ 、数学 $\boldsymbol{x}_3$ 、物理 $\boldsymbol{x}_4$) 的考试. 将学生的考试成绩看作随机变量的取值, 对考试成绩数据进行标准化处理, 得到样本相关矩阵 $\boldsymbol{R}$, 如下表所示.

课程	语文 ($\boldsymbol{x}_1$)	外语 ($\boldsymbol{x}_2$)	数学 ($\boldsymbol{x}_3$)	物理 ($\boldsymbol{x}_4$)
语文	1.00	0.44	0.29	0.33
外语	0.44	1.00	0.35	0.32
数学	0.29	0.35	1.00	0.60
物理	0.33	0.32	0.60	1.00

表 8 样本相关矩阵 $\boldsymbol{R}$

解: 对样本相关矩阵进行特征值分解, 得到相关矩阵的特征值, 并按大小排序:

$$\lambda_1 = 2.17, \quad \lambda_2 = 0.87, \quad \lambda_3 = 0.57, \quad \lambda_4 = 0.39 \tag{74}$$

这些特征值就是各主成分的方差贡献率. 假设要求主成分的累计方差贡献率大于 75%, 那么只需取前两个主成分即可 ($k = 2$), 因为:

$$\frac{\lambda_1 + \lambda_2}{\sum_{i=1}^4 \lambda_i} = \frac{2.17 + 0.87}{4} = 0.76 > 75\% \tag{75}$$

求出对应于特征值 λ_1, λ_2 的单位特征向量, 列于下表.

项目	$\boldsymbol{x}_1$	$\boldsymbol{x}_2$	$\boldsymbol{x}_3$	$\boldsymbol{x}_4$	方差贡献率
$\boldsymbol{y}_1$	0.460	0.476	0.523	0.537	0.543
$\boldsymbol{y}_2$	0.574	0.486	-0.476	-0.456	0.218

表 9 单位特征向量和主成分的方差贡献率

由此可得第一、第二主成分的表达式:

$$y_1 = 0.460x_1 + 0.476x_2 + 0.523x_3 + 0.537x_4$$

(76)

$$y_2 = 0.574x_1 + 0.486x_2 - 0.476x_3 - 0.456x_4$$

(77)

接下来由特征值和单位特征向量求出第一、第二主成分的因子负荷量 $\rho(y_k, x_i) = \sqrt{\lambda_k}a_{ik}$,如下表所示.

项目	x_1	x_2	x_3	x_4
y_1	0.678	0.701	0.770	0.791
y_2	0.536	0.453	-0.444	-0.425
y_1, y_2 对 x_i 的贡献率	0.747	0.697	0.790	0.806

表 10 主成分的因子负荷量和贡献率

结果分析:

1. 第一主成分 y_1 : 所有因子负荷量均为正数,且数值相近 (物理 x_4 略大).这说明各门课程成绩提高都会使 y_1 提高.因此, y_1 反映了学生的**整体成绩**.
2. 第二主成分 y_2 : 因子负荷量有正有负.正的是语文和外语 (文科),负的是数学和物理 (理科).这说明文科好理科差会使 y_2 升高.因此, y_2 反映了学生的**文科与理科成绩的差异** (文科倾向) .

下图将原变量在由两个主成分构成的平面坐标系中表示出来.

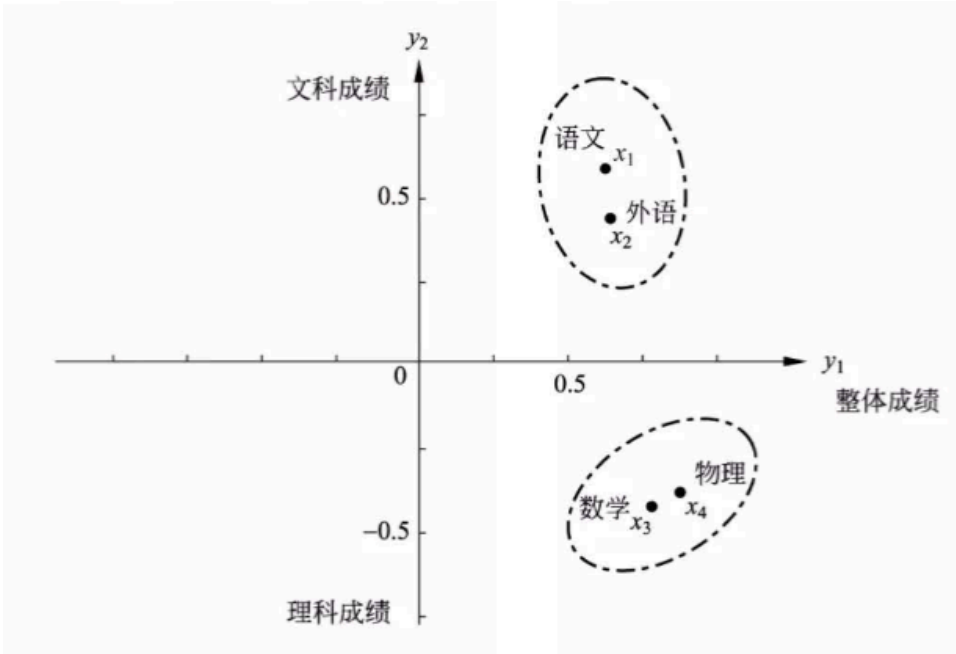


图 13 因子负荷量分布图

给定样本矩阵 X ,利用数据矩阵奇异值分解进行主成分分析.具体过程如下,这里假设有 k 个主成分.

对于 $m \times n$ 实矩阵 A ,假设其秩为 $r, 0 < k < r$,则可以将矩阵 A 进行截断奇异值分解:

$$A \approx U_k \Sigma_k V_k^T$$

(78)

式中 U_k 是 $m \times k$ 矩阵, V_k 是 $n \times k$ 矩阵, Σ_k 是 k 阶对角矩阵; U_k, V_k 分别由取 A 的完全奇异值分解的矩阵 U, V 的前 k 列, Σ_k 由取 A 的完全奇异值分解的矩阵 Σ 的前 k 个对角线元素得到.

定义一个新的 $n \times m$ 矩阵 X' :

$$X' = \frac{1}{\sqrt{n-1}}X^T$$

(79)

X' 的每一列均值为零.不难得知:

$$\begin{aligned} X'^T X' &= \left(\frac{1}{\sqrt{n-1}}X^T \right)^T \left(\frac{1}{\sqrt{n-1}}X^T \right) \\ &= \frac{1}{n-1}X X^T \end{aligned}$$

(80)

即 $X'^T X'$ 等于 X 的协方差矩阵 S_X :

$$S_X = X'^T X'$$

(81)

主成分分析归结于求协方差矩阵 S_X 的特征值和对应的单位特征向量,所以问题转化为求矩阵 $X'^T X'$ 的特征值和对应的单位特征向量.

假设 X' 的截断奇异值分解为 $X' = U\Sigma V^T$,那么 V 的列向量就是 $S_X = X'^T X'$ 的单位特征向量.因此, V 的列向量构成 X 的主成分的正交直角坐标系.于是,求 X 主成分可以通过求 X' 的奇异值分解来实现.具体算法如下.

Algorithm 7.3.3

样本主成分分析: 基于样本的奇异值分解

输入: $m \times n$ 样本矩阵 X ,其每一行元素的均值为零.

输出: $k \times n$ 样本主成分矩阵 Y .

参数: 主成分个数 k .

(1) 构造新的 $n \times m$ 矩阵:

$$X' = \frac{1}{\sqrt{n-1}} X^T \quad (82)$$

X' 每一列的均值为零.

(2) 对矩阵 X' 进行截断奇异值分解,得到:

$$X' = U\Sigma V^T \quad (83)$$

有 k 个奇异值、奇异向量.矩阵 V^T 和 X 的乘积构成样本主成分矩阵.

(3) 求 $k \times n$ 样本主成分矩阵:

$$Y = V^T X \quad (84)$$

Example

为了展示上述算法的具体计算过程,假设有一个 $m = 2$ (两个特征), $n = 3$ (三个样本) 的数据矩阵 X .

(1) 输入数据与预处理

给定样本矩阵 X ,其中每一行代表一个特征,每一列代表一个样本.假设数据如下 (已满足每一行均值为零的要求):

$$X = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \end{pmatrix} \quad (85)$$

这里 $m = 2, n = 3$.

(2) 构造矩阵 X'

根据公式 $X' = \frac{1}{\sqrt{n-1}} X^T$,其中 $\frac{1}{\sqrt{n-1}} = \frac{1}{\sqrt{2}}$:

$$X' = \frac{1}{\sqrt{2}} \begin{pmatrix} -1 & -2 \\ 0 & 0 \\ 1 & 2 \end{pmatrix} \quad (86)$$

(3) 对 X' 进行截断奇异值分解

Remark

截断奇异值分解是指只保留前 k 个最大的奇异值 ($k < r$),将剩余的 $r - k$ 个奇异值置为 0.

首先计算 $X'^T X'$,即协方差矩阵 S_X :

$$\begin{aligned} S_X = X'^T X' &= \left(\frac{1}{\sqrt{2}} \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \end{pmatrix} \right) \left(\frac{1}{\sqrt{2}} \begin{pmatrix} -1 & -2 \\ 0 & 0 \\ 1 & 2 \end{pmatrix} \right) \\ &= \frac{1}{2} \begin{pmatrix} 2 & 4 \\ 4 & 8 \end{pmatrix} = \begin{pmatrix} 1 & 2 \\ 2 & 4 \end{pmatrix} \end{aligned} \quad (87)$$

求解 S_X 的特征值和特征向量: 特征方程 $|\lambda I - S_X| = (\lambda - 1)(\lambda - 4) - 4 = \lambda^2 - 5\lambda = 0$. 解得特征值 $\lambda_1 = 5, \lambda_2 = 0$. 对应的奇异值 $\sigma_1 = \sqrt{5}, \sigma_2 = 0$.

求对应的单位特征向量 (即矩阵 V 的列向量): 对于 $\lambda_1 = 5$,解方程 $(S_X - 5I)v_1 = 0$,归一化后得 $v_1 = \frac{1}{\sqrt{5}}(1, 2)^T$. 对于 $\lambda_2 = 0$,解方程 $S_X v_2 = 0$,归一化后得 $v_2 = \frac{1}{\sqrt{5}}(2, -1)^T$.

于是得到正交矩阵 V :

$$V = \frac{1}{\sqrt{5}} \begin{pmatrix} 1 & 2 \\ 2 & -1 \end{pmatrix} \quad (88)$$

(4) 求主成分矩阵 Y

假设我们需要保留 1 个主成分 ($k = 1$). 取 V 的前 1 列得到 V_k (也就是截断 V):

$$V_k = \frac{1}{\sqrt{5}} \begin{pmatrix} 1 \\ 2 \end{pmatrix} \quad (89)$$

最后,根据公式 $Y = V_k^T X$ 计算 1×3 的主成分矩阵:

$$\begin{aligned} Y &= \left(\frac{1}{\sqrt{5}} (1 \ 2) \right) \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \end{pmatrix} \\ &= (-\sqrt{5} \ 0 \ \sqrt{5}) \end{aligned} \quad (90)$$

这就是降维后的数据矩阵,原有的 2 维数据被压缩到了 1 维,且保留了最大的方差信息.

7.4 核化线性降维

面对卷曲的流形,只用直线去投影,会把不同层的数据强行“压”在一起,导致颜色混杂,丢失了流形原本的结构.

假设存在一个非线性映射函数 Φ ,能把原始的低维样本 $\mathbf{x}_i$ 变成高维空间的样本 $\mathbf{z}_i$,即 $\mathbf{z}_i = \Phi(\mathbf{x}_i)$. 如果在高维空间里做 PCA,我们原本要求解的目标依然是特征值问题:

$$\left(\sum_{i=1}^m \mathbf{z}_i \mathbf{z}_i^T \right) \mathbf{W} = \lambda \mathbf{W} \quad (91)$$

$\mathbf{W}$ 就是高维空间里的主成分方向.

令 $\alpha_i = \frac{1}{\lambda} \mathbf{z}_i^T \mathbf{W}$,

$$\mathbf{W} = \frac{1}{\lambda} \left(\sum_{i=1}^m \mathbf{z}_i \mathbf{z}_i^T \right) \mathbf{W} = \sum_{i=1}^m \mathbf{z}_i \frac{\mathbf{z}_i^T \mathbf{W}}{\lambda} = \sum_{i=1}^m \mathbf{z}_i \alpha_i = \mathbf{Z} \alpha \quad (92)$$

将 $\mathbf{W}$ 的表示代入:

$$(\mathbf{Z} \mathbf{Z}^T)(\mathbf{Z} \alpha) = \lambda (\mathbf{Z} \alpha) \quad (93)$$

等式两边同时左乘一个 $\mathbf{Z}^T$:

$$\mathbf{Z}^T \mathbf{Z} (\mathbf{Z}^T \mathbf{Z}) \alpha = \lambda \mathbf{Z}^T \mathbf{Z} \alpha \quad (94)$$

$\mathbf{Z}^T \mathbf{Z}$ 是一个 $m \times m$ 的方阵,里面的每一个元素刚好就是 $\mathbf{z}_i^T \mathbf{z}_j$. 故可以用核函数直接计算的核矩阵 $\mathbf{K}$.

把 $\mathbf{K} = \mathbf{Z}^T \mathbf{Z}$ 代进去:

$$\mathbf{K} \mathbf{K} \alpha = \lambda \mathbf{K} \alpha \quad (95)$$

等式两边消去一个 $\mathbf{K}$ (简化说明,严格证明涉及特征空间分解):

$$\mathbf{K} \alpha = \lambda \alpha \quad (96)$$

$\mathbf{K}$ 是一个 $m \times m$ 的常数矩阵,靠低维样本代入核函数 $\kappa(\mathbf{x}_i, \mathbf{x}_j)$ 瞬间算完. λ 是标量,相当于一个新的特征值. α 就是已知矩阵 $\mathbf{K}$ 的特征向量.

模型训练好了 (求出了 α), 现在来了一个新样本 $\mathbf{x}$, 我们怎么把它降维呢? 按理说, 应该是新样本的高维坐标 $\Phi(\mathbf{x})$ 乘以高维主成分 $\mathbf{w}_j$:

$$z_j = \mathbf{w}_j^T \Phi(\mathbf{x}) \quad (97)$$

把第 2 步中 $\mathbf{w}_j = \sum \alpha_i^j \Phi(\mathbf{x}_i)$ 代入进去, 再次出现了内积, 并立刻用核函数替换:

$$z_j = \sum_{i=1}^m \alpha_i^j \kappa(\mathbf{x}_i, \mathbf{x}) \quad (98)$$

Remark

为了求出一个新样本降维后的坐标, 你需要用核函数去计算新样本 $\mathbf{x}$ 和所有的训练样本 $\mathbf{x}_i$ 的关系并求和. 这与线性 PCA 完全不同 (线性 PCA 只需要把新样本乘上一个投影矩阵就结束了). 所以, KPCA 虽然能处理复杂的非线性结构, 但代价是极高的预测 (推理) 计算开销.

[bz] 用 mitex 的都是 sb, 可读性和 gemini 做一桌

7.5 流形学习

流形是在局部与欧氏空间同胚的空间. 抛开微分几何而直观地说, 在一个小邻域里, 我们能欧式距离来刻画. 想在局部建立降维映射关系, 然后再设法将局部映射关系推广到全局. [ca] 我讨厌微分几何

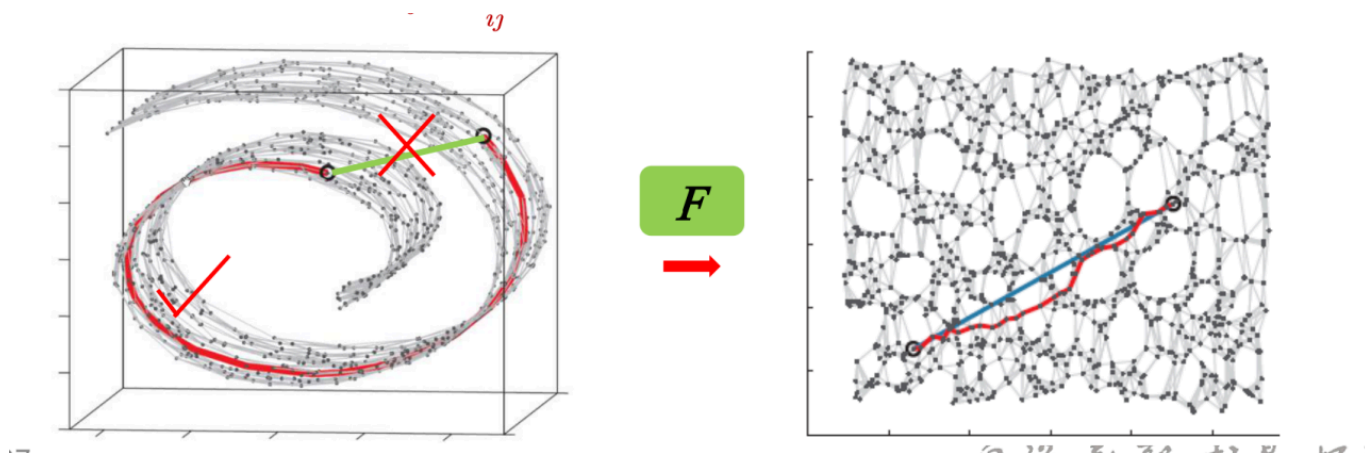


图 14 一个流形学习的例子.显然我们不希望距离算成绿色的线段,而是根据红色的线走;所以我们需要通过 k 近邻.这就是著名的 ISOMAP 算法.他通过算 k 近邻来计算距离矩阵,然后再输入到之前说过的 MDS 酱味算法.

7.5.1 局部线性嵌入, LLE

局部线性嵌入试图保持邻域内的线性关系,并使得该线性关系在降维后的空间中继续保持.为每个样本 x_i 找到近邻 Q ,然后计算线性重构的系数 w_i ,也就是

$$\min_{i \in J} \sum_{i=1}^m \|x_i - \sum_{j \in Q_i} w_{ij} x_j\|^2 \text{ s.t. } \sum_{j \in Q_i} w_{ij} = 1 \quad (99)$$

得到解

$$w_{ij} = \frac{\sum_{k \in Q_i} C_{jk}^{-1}}{\sum_{l, s \in Q_i} C_{ls}^{-1}}, C_{jk} \triangleq (x_i - x_j)^T (x_i - x_k) \quad (7.5.101)$$

[cb] 哪个傻逼记的.注意 C 并不是内积!

当然我们想得到一个更低维度的嵌入,而且 LLE 规定 w 是不变的.所以,只需要考虑把低维度坐标 z 替换 x 即可.令

$$\begin{aligned} Z &\triangleq (z_1, \dots, z_m) \in \mathbb{R}^{d' \times m}, \\ M &\triangleq (I - W)^T (I - W) \end{aligned} \quad (102)$$

则式 100 可以被重写为

$$\min_Z \text{tr}(Z^T M Z) \wedge Z Z^T = I. \quad (103)$$

Algorithm 7.5.1

ISOMAP

样本集 $X = \{x_1, \dots, x_m\}$;

1. 确定 x_i 的 k 近邻,并且 x_i 与 k 近邻之间的距离设置为欧式距离.其他的都是 ∞ .
2. 计算 d_{ij} ,填充距离矩阵 D ;(当然,这里用 dij 或者 floyd 等算法了,而不是 ML 里的算法)
3. return MDS 算法的输出

Algorithm 7.5.2

LLE

样本集 $X = \{x_1, \dots, x_m\}$;

1. 确定 x_i 的 k 近邻.记近邻集合为 Q_i .
2. 在 Q_i 内算

$$w_{ij} = \frac{\sum_{k \in Q_i} C_{jk}^{-1}}{\sum_{l, s \in Q_i} C_{ls}^{-1}}, C_{jk} \triangleq (x_i - x_j)^T (x_i - x_k) \quad (104)$$

.其他的都是 0.得到矩阵 W .

3. $M \triangleq (I - W)^T (I - W)$.然后,特征值分解 M ,return 最小 d' 个特征向量构成的矩阵 $Z \triangleq [v_1, \dots, v_{d'}]$

7.6 度量学习

Definition 7.6.1

度量学习的一般框架

度量学习的核心是学习一个半正定对称矩阵 M ,使得:

- 相似样本在该度量下距离更近
- 不相似样本在该度量下距离更远

不同的度量学习方法针对不同目标获得“好”的半正定对称距离度量矩阵 $\mathbf{M}$.

7.6.1 研究动机

在机器学习中,对高维数据进行降维的主要目的是希望找到一个合适的低维空间,在此空间中进行学习能比原始空间性能更好.事实上,每个空间对应了在样本属性上定义的一个距离度量,而寻找合适的空间,实质上就是在寻找一个合适的距离度量.那么,为何不直接尝试“学习”出一个合适的距离度量呢?——这就是度量学习的核心思想.

[cc] 传统降维方法(如 PCA、MDS)是先假设一个距离度量(通常是欧氏距离),然后寻找保持该度量的低维表示.而度量学习则反其道而行之:直接学习距离度量本身,让数据告诉我们什么样的距离才是“好”的距离.

7.6.2 度量学习基础

7.6.2.1 平方欧氏距离

欲对距离度量进行学习,必须有一个便于学习的距离度量表达形式.对两个 d 维样本 $\mathbf{x}_i$ 和 $\mathbf{x}_j$,它们之间的平方欧氏距离可写为:

$$\begin{aligned}\text{dist}_{\text{ed}}^2(\mathbf{x}_i, \mathbf{x}_j) &= \|\mathbf{x}_i - \mathbf{x}_j\|_2^2 = \\ &= \text{dist}_{ij,1}^2 + \text{dist}_{ij,2}^2 + \cdots + \text{dist}_{ij,d}^2\end{aligned}\quad (105)$$

其中 $\text{dist}_{ij,k}$ 表示 $\mathbf{x}_i$ 与 $\mathbf{x}_j$ 在第 k 维上的距离.

7.6.2.2 加权距离(属性权重)

若假定不同属性的重要性不同,则可引入属性权重 $\mathbf{w}$,得到加权欧氏距离:

$$\begin{aligned}\text{dist}_{\text{wed}}^2(\mathbf{x}_i, \mathbf{x}_j) &= \|\mathbf{x}_i - \mathbf{x}_j\|_2^2 \\ &= w_1 \cdot \text{dist}_{ij,1}^2 + \cdots + w_d \cdot \text{dist}_{ij,d}^2 \\ &= (\mathbf{x}_i - \mathbf{x}_j)^T \mathbf{W} (\mathbf{x}_i - \mathbf{x}_j)\end{aligned}\quad (106)$$

其中 $w_i \geq 0$, $\mathbf{W} = \text{diag}(\mathbf{w})$ 是一个对角矩阵, $(\mathbf{W})_{ii} = w_i$.

Remark

此时权重矩阵 $\mathbf{W}$ 的非对角元素均为零,这意味着坐标轴是正交的,即属性之间无关.但现实问题中往往不是这样——例如考虑西瓜的“重量”和“体积”这两个属性,它们显然是正相关的,其对应的坐标轴不再正交.

7.6.2.3 度量矩阵推广

为此将 $\mathbf{W}$ 替换为一个普通的半正定对称矩阵 $\mathbf{M}$,于是得到了马氏距离 (Mahalanobis distance):

$$\text{dist}_{\text{mah}}^2(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i - \mathbf{x}_j)^T \mathbf{M} (\mathbf{x}_i - \mathbf{x}_j) = \|\mathbf{x}_i - \mathbf{x}_j\|_{\mathbf{M}}^2 \quad (107)$$

其中 $\mathbf{M}$ 亦称度量矩阵,而度量学习则是对 $\mathbf{M}$ 进行学习.

7.6.2.4 度量矩阵的性质

注意到为了保持距离非负且对称, $\mathbf{M}$ 必须是(半)正定对称矩阵,即必有正交基 $\mathbf{P}$ 使得 $\mathbf{M}$ 能写为:

$$\mathbf{M} = \mathbf{P} \mathbf{P}^T \quad (108)$$

[cd] 半正定矩阵 (Positive Semidefinite Matrix)

一个 $n \times n$ 的实对称矩阵 $\mathbf{A}$ 称为半正定矩阵,如果对于任意非零向量 $\mathbf{x} \in \mathbb{R}^n$,都满足:

$$\mathbf{x}^T \mathbf{A} \mathbf{x} \geq 0 \quad (109)$$

半正定矩阵的主要性质:

- **特征值非负**: 所有特征值 $\lambda_i \geq 0$
- **可分解性**: 任意半正定矩阵 $\mathbf{A}$ 都可分解为 $\mathbf{A} = \mathbf{P} \mathbf{P}^T$, 其中 $\mathbf{P}$ 是某个矩阵
- **距离非负**: 若度量矩阵 $\mathbf{M}$ 是半正定的,则对任意两点 $\mathbf{x}_i, \mathbf{x}_j$, 马氏距离 $\|\mathbf{x}_i - \mathbf{x}_j\|_{\mathbf{M}}^2 \geq 0$ 恒成立

正交基 (Orthogonal Basis)

一组向量 $\{\mathbf{p}_1, \mathbf{p}_2, \dots, \mathbf{p}_k\}$ 称为正交基,如果满足:

- **两两正交**: $\mathbf{p}_i^T \mathbf{p}_j = 0$ (当 $i \neq j$)
- **单位长度**: $\|\mathbf{p}_i\|_2 = 1$

若矩阵 $\mathbf{P}$ 的列向量构成正交基,则 $\mathbf{P}$ 是正交矩阵,满足 $\mathbf{P}^T \mathbf{P} = \mathbf{I}$.

两者关系

根据谱分解定理,任意半正定对称矩阵 $\mathbf{M}$ 都可以进行特征值分解:

$$\mathbf{M} = \mathbf{Q} \mathbf{\Lambda} \mathbf{Q}^T \quad (110)$$

其中 $\mathbf{Q}$ 是正交矩阵 (列向量为标准正交特征向量), $\mathbf{\Lambda}$ 是对角矩阵 (对角线为非负特征值).

令 $\mathbf{P} = \mathbf{Q} \mathbf{\Lambda}^{1/2}$, 则 $\mathbf{M} = \mathbf{P} \mathbf{P}^T$. 这就是度量矩阵分解的理论依据.

对 $\mathbf{M}$ 进行学习当然要设置一个目标.假定我们是希望提高近邻分类器的性能,则可将 $\mathbf{M}$ 直接嵌入到近邻分类器的评价指标中去,通过优化该性能指标相应地求得 $\mathbf{M}$.

[ce] 在新空间算“欧氏距离”(用 $\mathbf{P}$ 投影),在数学上完全等价于在原空间算“加权距离”(用 $\mathbf{M}$ 做中间件).这就是 $\mathbf{M} = \mathbf{P} \mathbf{P}^T$ 的来历.

下面以近邻成分分析为例讨论.

7.6.3 近邻成分分析 (NCA)

近邻成分分析 (Neighbourhood Component Analysis, NCA) 相对于 K means 有两点不同.

1. K means 进行判别时通常使用**多数投票法**,邻域中的每个样本投 1 票,邻域外的样本投 0 票.不妨将其替换为**概率投票法**.
2. 直接算欧氏距离,模型会平等地对待每一个维度.但如果某些特征用处不大,欧氏距离就会失效.NCA 学习一个矩阵 $\mathbf{P}$,把原始数据**通过 $\mathbf{P}$ 投影**后,在新的空间里,同类样本会聚集在一起.

对于任意样本 $\mathbf{x}_j$,它对 $\mathbf{x}_i$ 分类结果影响的概率为:

$$p_{ij} = \frac{\exp(-\|\mathbf{x}_i - \mathbf{x}_j\|_{\mathbf{M}}^2)}{\sum_l \exp(-\|\mathbf{x}_i - \mathbf{x}_l\|_{\mathbf{M}}^2)} \quad \text{式 (10.35)} \quad (111)$$

当 $i = j$ 时, p_{ij} 最大.显然, $\mathbf{x}_j$ 对 $\mathbf{x}_i$ 的影响随着它们之间距离的增大而减小.

若以**留一法 (LOO) 正确率的最大化**为目标,则可计算 $\mathbf{x}_i$ 的留一法正确率,即它被自身之外的所有样本正确分类的概率为:

$$p_i = \sum_{j \in \Omega_i} p_{ij} \quad (112)$$

其中 Ω_i 表示与 $\mathbf{x}_i$ 同类的样本集合.

整个样本集上的留一法正确率为:

$$\sum_{i=1}^m p_i = \sum_{i=1}^m \sum_{j \in \Omega_i} p_{ij} \quad (113)$$

将式 (10.35) 带入上式.

再考虑到 $\mathbf{M} = \mathbf{P} \mathbf{P}^T$, 则 NCA 的优化目标为:

$$\min_{\mathbf{P}} 1 - \sum_{i=1}^m \sum_{j \in \Omega_i} \frac{\exp(-\|\mathbf{P}^T \mathbf{x}_i - \mathbf{P}^T \mathbf{x}_j\|_2^2)}{\sum_l \exp(-\|\mathbf{P}^T \mathbf{x}_i - \mathbf{P}^T \mathbf{x}_l\|_2^2)} \quad (114)$$

求解即可得到**最大化近邻分类器 LOO 正确率**的距离度量矩阵 $\mathbf{M}$.

Algorithm 7.6.2**近邻成分分析 (NCA)**

输入: 样本集 $D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_m, y_m)\}$; 目标维度 d' .

输出: 度量矩阵 $\mathbf{M}$ 或降维矩阵 $\mathbf{P} \in \mathbb{R}^{d \times d'}$.

过程:

1. **初始化**: 初始化降维矩阵 $\mathbf{P}$ (如随机初始化或使用 PCA 结果).

2. **计算概率**: 对于每对样本 (i, j) , 计算概率投票:

$$p_{ij} = \frac{\exp(-\|P^T x_i - P^T x_j\|_2^2)}{\sum_l \exp(-\|P^T x_i - P^T x_l\|_2^2)} \quad (115)$$

3. **计算目标函数**: 计算留一法正确率:

$$f(P) = \sum_{i=1}^m \sum_{j \in \Omega_i} p_{ij} \quad (116)$$

4. **梯度优化**: 通过梯度上升法更新 P , 最大化目标函数 $f(P)$.

5. **输出结果**: 返回优化后的 P , 度量矩阵为 $M = PP^T$.

Remark

NCA 的目标函数是 **1 减去留一法正确率**, 因此是最小化问题. 这等价于最大化留一法正确率. 优化过程通常使用梯度下降法 (或梯度上升法, 取决于符号约定).

7.6.4 度量学习的扩展

实际上, 我们**不仅**能把**错误率**这样的监督学习目标作为度量学习的优化目标, 还能在度量学习中引入**领域知识**.

若已知某些样本相似、某些样本不相似, 则可定义**必连** (must-link) 约束集合 $\mathcal{M}$ 与**勿连** (cannot-link) 约束集合 $\mathcal{C}$:

- $(x_i, x_j) \in \mathcal{M}$ 表示 x_i 与 x_j 相似
- $(x_i, x_k) \in \mathcal{C}$ 表示 x_i 与 x_k 不相似

显然, 我们希望相似的样本之间距离较小, 不相似的样本之间距离较大.

于是可通过求解下面这个凸优化问题获得适当的度量矩阵:

$$\begin{aligned} \min_{\mathbf{M}} \quad & \sum_{(x_i, x_j) \in \mathcal{M}} \|x_i - x_j\|_{\mathbf{M}}^2 \\ \text{s.t.} \quad & \sum_{(x_i, x_k) \in \mathcal{C}} \|x_i - x_k\|_{\mathbf{M}}^2 \geq 1 \\ & \mathbf{M} \succeq 0 \end{aligned} \quad (117)$$

其中约束 $\mathbf{M} \succeq 0$ 表明 $\mathbf{M}$ 必须是半正定的 [cf] $v^T M v \leq 0 \forall v$. 上式要求在不相似样本间的距离不小于 1 的前提下, 使相似样本间的距离尽可能小.

7.6.5 度量学习与降维的关系

不同的度量学习方法针对不同目标获得“好”的半正定对称距离度量矩阵 $\mathbf{M}$. 若 $\mathbf{M}$ 是一个**低秩矩阵**, 则通过对 $\mathbf{M}$ 进行特征值分解, 总找到一组正交基, 其正交基数目为矩阵 $\mathbf{M}$ 的秩 $\text{rank}(\mathbf{M})$, 小于原属性数 d .

于是, 度量学得的结果可衍生出一个**降维矩阵**:

$$\mathbf{P} \in \mathbb{R}^{d \times \text{rank}(\mathbf{M})} \quad (118)$$

能用于降维的目的.

Remark

度量学习与降维之间存在紧密联系:

- 度量学习学得一个度量矩阵 $\mathbf{M}$
- 若 $\mathbf{M}$ 是低秩的, 则其自然蕴含了降维
- 通过对 $\mathbf{M}$ 进行特征值分解 $\mathbf{M} = \mathbf{P}\mathbf{P}^T$, 可得降维矩阵 $\mathbf{P}$
- 原始样本 $\mathbf{x}$ 经 $\mathbf{P}^T \mathbf{x}$ 变换后即降维后的表示

因此, 度量学习不仅可以改善距离度量, 还能同时实现降维, 一举两得.

7.7 k 近邻法

k 近邻法的**输入**为**实例的特征向量**, 对应于特征空间的点; **输出**为**实例的类别**, 可以取多类.

k 近邻法假设给定一个训练数据集, 其中实例类别已定. 分类时找出距离 x 最近的 k 个点, 看这 k 个点里面哪个类的最多.

利用训练数据集对特征向量空间进行划分, 并作为其分类的模型. k 值的选择、距离度量及分类决策规则是 k 近邻法的 3 个基本要素.

7.7.1 k 近邻算法

Algorithm 7.7.1

输入: 训练数据集

$$T = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$$

k 近邻算法

其中, $x_i \in \mathcal{X} \subseteq R^n$ 为实例的特征向量, $y_i \in \mathcal{Y} = \{c_1, c_2, \dots, c_K\}$ 为实例的类别, $i = 1, 2, \dots, N$.

输出: 实例 x 所属的类 y .

找出距离 x 最近的 k 个点, 看这 k 个点里面哪个类的最多.

1. 根据给定的距离度量, 在训练集 T 中找出与 x 最邻近的 k 个点, 涵盖这 k 个点的 x 的邻域记作 $N_k(x)$.
2. 在 $N_k(x)$ 中根据分类决策规则 (如多数表决) 决定 x 的类别 y :

$$y = \arg \max_{c_j} \sum_{x_i \in N_k(x)} I(y_i = c_j), \quad i = 1, 2, \dots, N, j = 1, 2, \dots, K$$

Remark

$k = 1$ 的时候, k 近邻法变成**最近邻算法**, 将训练数据集中与 x 最邻近点的类作为 x 的类.

k 近邻法没有显式的学习过程. k 近邻法的“学习”就是纯粹的记忆, 它不会从数据中提炼出任何超越数据本身的规则或模型. 正因为如此, 它也被归类为“懒惰学习”或“基于实例的学习”

7.7.2 k 近邻模型

k 近邻法使用的模型实际上对应于对特征空间的划分. 模型由 3 个基本要素——**距离度量**、 **k 值的选择**和**分类决策规则**决定.

7.7.2.1 模型

特征空间中, 对每个训练实例点 x_i , 距离该点比其他点更近的所有点组成一个区域, 叫作**单元**. 每个训练实例点拥有一个单元, 所有训练实例点的单元构成对特征空间的一个划分. 最近邻法将实例 x_i 的类 y_i 作为其单元中所有点的类标记. 这样, 每个单元的实例点的类别是确定的.

7.7.2.2 距离度量

一般会使用欧氏距离, 也可能用 L_p 距离或 Minkowski 距离.

设特征空间 $\mathcal{X}$ 是 n 维实数向量空间 R^n , $x_i, x_j \in \mathcal{X}$, $x_i = (x_i^1, x_i^2, \dots, x_i^n)^T$, $x_j = (x_j^1, x_j^2, \dots, x_j^n)^T$, x_i, x_j 的 L_p 距离是:

$$L_p(x_i, x_j) = \left(\sum_{l=1}^n |x_i^l - x_j^l|^p \right)^{\frac{1}{p}}$$

Example

当 $p = 2$ 时, 称为欧氏距离, 即

$$L_2(x_i, x_j) = \left(\sum_{l=1}^n |x_i^l - x_j^l|^2 \right)^{\frac{1}{2}}$$

当 $p = 1$ 时, 称为曼哈顿距离; 当 $p = \infty$ 时, 是各个坐标距离的最大值.

7.7.2.3 k 值的选择

如果选择较小的 k 值, 就相当于用较小的邻域中的训练实例进行预测, 学习的近似误差 (approximation error) 会减小, 只有与输入实例较近 (相似的) 训练实例会起作用. 缺点是估计误差 (estimation error) 会增大, 预测结果会对近邻的实例点非常敏感. **k 值的减小就意味着整个模型变得更复杂, 容易出现过拟合.**

Remark

如果 $k = N$, 那么无论输入实例是什么, 都将简单地预测它属于在训练实例中最多的类, 这时模型过于简单, 完全忽略了训练实例中的大量有用信息, 是不可取的. 在应用中, k 值一般取一个较小的值, 通常采用**交叉验证法**来选取最优的 k 值.

7.7.2.4 分类决策规则

多数表决规则有如下解释: 如果分类的损失函数为 0-1 损失函数, 分类函数为:

$$f: R^n \rightarrow \{c_1, c_2, \dots, c_K\}$$

对给定的实例 $x \in \mathcal{X}$, 其最近邻的 k 个训练实例点构成集合 $N_k(x)$. 如果涵盖 $N_k(x)$ 的区域的类别是 c_j , 那么误分类率是

$$\frac{1}{k} \sum_{x_i \in N_k(x)} I(y_i \neq c_j) = 1 - \frac{1}{k} \sum_{x_i \in N_k(x)} I(y_i = c_j)$$

[cg] 多数表决规则等价于经验风险最小化.

7.7.3 k 近邻法的实现: kd 树

[ch] 额外注意: kd 树是存储 k 维空间数据的树结构,这里的 k 与 k 近邻法的 k 意义不同。(其实 k 近邻法实现方法有很多,这里介绍的 kd 树是其中一种)

Algorithm 7.7.2

kd 树生成算法

输入: k 维空间数据集 $T = \{x_1, x_2, \dots, x_N\}$,其中 $x_i = (x_i^{(1)}, x_i^{(2)}, \dots, x_i^{(k)})^T, i = 1, 2, \dots, N$;

输出: kd 树.

1. **开始:** 构造根结点,根结点对应于包含 T 的 k 维空间的超矩形区域.选择 $x^{(1)}$ 为坐标轴,以 T 中所有实例的 $x^{(1)}$ 坐标的中位数为切分点,将根结点对应的超矩形区域切分为两个子区域.切分由通过切分点并与坐标轴 $x^{(1)}$ 垂直的超平面实现.由根结点生成深度为1的左、右子结点:左子结点对应坐标 $x^{(1)}$ 小于切分点的子区域,右子结点对应于坐标 $x^{(1)}$ 大于切分点的子区域.将落在切分超平面上的实例点保存在根结点.将落在切分超平面上的实例点保存在该结点.
2. **重复:** 对深度为 j 的结点,选择 $x^{(l)}$ 为切分的坐标轴, $l = j(\bmod k) + 1$,以该结点的区域中所有实例的 $x^{(l)}$ 坐标的中位数为切分点,将该结点对应的超矩形区域切分为两个子区域.切分由通过切分点并与坐标轴 $x^{(l)}$ 垂直的超平面实现.由该结点生成深度为 $j + 1$ 的左、右子结点:左子结点对应坐标 $x^{(l)}$ 小于切分点的子区域,右子结点对应坐标 $x^{(l)}$ 大于切分点的子区域.
3. 直到两个子区域没有实例存在时停止.从而形成 kd 树的区域划分.

KDTree 的结构是 BST,即二叉搜索树,每个子树维护的是一个 k 维的长方体,而每个点维护的为其所在子树长方体内的一个点.可以发现,这样要求儿子的长方体包含在父亲的内,且一个点的子树内含有这个 k 维长方体内的所有点.这样 KDTree 的结构已经呼之欲出了,我们只需要从当前的长方体内选一个维度和一个点,按照这个点的这个维度把长方体分成两个部分,把这两个部分作为这个点的左右子树,之后对这两个子树递归建树,直到长方体退化为单个点.

对于静态的点集,我们只需要在 KDTree 结构的描述基础上,稍微修改一些部分就可以得到建树的方法.以长方体的切割为基本思路,每次选出一个点集内的点作为子树的根,然后选出一个维度切割,将切割出的点作为左右子树,如果一个长方体内没有点,则这个子树置为空即可.

Remark

我们可以感性理解一下,肯定是要尽可能把点分得均匀,才能保证 BST 的层数尽可能低,所以维度应该选择方差最大的那个,点应该选择当前维度上的中位数.这样,我们可以保证建出的 BST 树高是 $O(\log(n))$ 级别的.

其实这个的整体思路,有点偏向于是先把点划分完之后,建立起 kd 树,后面用这棵树的时候,先去定位目标点的归属,然后不断定位与他归属相同的点之间的距离,与目标点到划分界限之间的距离大小关系.因为这决定了是否需要去关心划分界限之外的其他的点,即:

Remark

如果实例点是随机分布的, kd 树搜索的平均计算复杂度是 $O(\log(n))$,这里 N 是训练实例数. kd 树更适用于训练实例数远大于空间维数时的 k 近邻搜索.当空间维数接近训练实例数时,它的效率会迅速下降,几乎接近线性扫描.

Algorithm 7.7.3

k 近邻算法

输入: 已构造的 kd 树,目标点 x .

输出: x 的最近邻.

1. 在 kd 树中找出包含目标点 x 的叶结点: 从根结点出发,递归地向下访问 kd 树.若目标点 x 当前维的坐标小于切分点的坐标,则移动到左子结点,否则移动到右子结点,直到子结点为叶结点为止,并以此叶结点为“当前最近点”.
2. 递归地向上回退,在每个结点进行以下操作:
 - 如果该结点保存的实例点比当前最近点距离目标点更近,则以该实例点为“当前最近点”.
 - 当前最近点一定存在于该结点一个子结点对应的区域.检查该子结点的父结点的另一子结点对应的区域是否有更近的点.具体地,检查另一子结点对应的区域是否与以目标点为球心、以目标点与“当前最近点”间的距离为半径的超球体相交.如果相交,可能在另一个子结点对应的区域内存在距目标点更近的点,移动到另一个子结点.接着,递归地进行最近邻搜索.如果不相交,向上回退.
3. 当回退到根结点时,搜索结束.最后的“当前最近点”即为 x 的最近邻点.

Example

给定一个二维空间数据集 $\{(2,3), (5,4), (9,6), (4,7), (8,1), (7,2)\}$,构造一个平衡 kd 树.查找目标点 $(3,4.5)$ 的最近邻点.

进行二叉查找,先从 $(7,2)$ 查找到 $(5,4)$ 节点,在进行查找时是由 $y=4$ 为分割超平面的,接着进入右子空间查找到 $(4,7)$,形成搜索路径: $(7,2) \rightarrow (5,4) \rightarrow (4,7)$,取 $(4,7)$ 为当前最近邻点.以目标查找点为圆心,目标查找点到当前最近点的距离 2.69 为半径确定一个红色圆.然后回溯到 $(5,4)$,用同样的方法再次确定一个绿色的圆,可见该圆和 $y=4$ 超平面相交,所以进入 $(5,4)$ 结点的另一个子空间进行查找.进一步最近邻点更新为 $(2,3)$,最近距离更新为 1.8,确定一个蓝色圆.接着回退到根结点 $(7,2)$,蓝色圆与 $x=7$ 的超平面不相交,因此不用进入 $(7,2)$ 的右子空间进行查找.至此,搜索路径回溯完,返回最近邻点 $(2,3)$,最近距离 1.8.

8 聚类

8.1 聚类方法

形式化地：

[ci] 聚类 (Clustering) 是按照某个特定标准 (如距离) 把一个数据集分割成不同的类或簇, 使得同一个簇内的数据对象的相似性尽可能大, 同时不在同一个簇中的数据对象的差异性也尽可能地大. 聚类后同一类的数据尽可能聚集到一起, 不同类数据尽量分离.

硬聚类 (hardclustering) 方法: 一个聚类方法假定一个样本只能属于一个类或类的交集为空集.

软聚类 (softclustering) 方法: 一个样本可以属于多个类或类的交集不为空集.

假定**样本集** $D = \{x_1, x_2, \dots, x_m\}$ 包含 **m 个无标记样本**, 每个样本 $x_i = \{x_{i1}, x_{i2}, \dots, x_{in}\}$ 是一个 **n 维的特征向量**, 聚类算法将样本集 D 划分成 **k 个不相交的簇** $\{C_l | l = 1, 2, \dots, k\}$, 其中 $C_{l'} \cap C_l = \emptyset$, 且 $D = \bigcup_{l=1}^k C_l$;

相应地, 用 $\lambda_j \in \{1, 2, \dots, k\}$ 表示**样本 x_j 的簇标记 (cluster label)**, 即 $x_j \in C_{\lambda_j}$. 于是, 聚类的结果可用**包含 m 个元素的簇标记向量** $\lambda = \{\lambda_1, \lambda_2, \dots, \lambda_m\}$ 来表示.

Remark

相似性测度、聚类准则和聚类算法称为聚类分析的三要素.

8.1.1 聚类的基本概念

聚类的对象是观测数据或样本集合. 假设有 **n 个样本**, 每个样本由 **m 个属性**的特征向量组成. 矩阵的第 **j 列**表示第 **j 个样本**:

$$X = [x_{ij}]_{m \times n} = \begin{pmatrix} x_{11} & x_{12} & \dots & x_{1n} \\ x_{21} & x_{22} & \dots & x_{2n} \\ \vdots & \vdots & & \vdots \\ x_{m1} & x_{m2} & \dots & x_{mn} \end{pmatrix}$$

在聚类中, 可以将样本集合看作是向量空间中点的集合, 以该空间的距离表示样本之间的相似度.

聚类试图将数据集中的样本划分为若干个通常是不相交的子集, 每个子集称为一个簇.

用 **G** 表示类或簇(cluster), 用 x_i, x_j 表示类中的样本, 用 **n_G** 表示 **G** 中样本的个数, 用 **d_{ij}** 表示样本 x_i 与 x_j 之间的距离.

Definition 8.1.1

类或簇

设 **T** 为给定的正数. 若对于集合 **G** 中任意两个样本 x_i, x_j , 有

$$d_{ij} \leq T$$

则称 **G** 为一个类或簇.

类的特征可以通过不同角度来刻画, 常用的特征有下面 3 种:

1. 类的均值 $\bar{x}_G$, 又称为类的中心

$$\bar{x}_G = \frac{1}{n_G} \sum_{i=1}^{n_G} x_i$$

式中 **n_G** 是类的样本个数.

2. 类的直径(diameter) D_G , 类的直径是类中任意两个样本之间的最大距离, 即:

$$D_G = \max_{x_i, x_j \in G} d_{ij}$$

3. 类的样本散布矩阵(scatter matrix) A_G 与样本协方差矩阵(covariance matrix) S_G 类的样本散布矩阵 **A_G** 为

$$A_G = \sum_{i=1}^{n_G} (x_i - \bar{x}_G)(x_i - \bar{x}_G)^T$$

样本协方差矩阵 **S_G** 为

$$S_G = \frac{1}{n_G - 1} A_G$$

$$= \frac{1}{n_G - 1} \sum_{i=1}^{n_G} (x_i - \bar{x}_G)(x_i - \bar{x}_G)^T$$

考虑类 G_p 与类 G_q 之间的距离 $D(p, q)$, 也称为连接. 类与类之间的距离也有多种定义.
 设类 G_p 包含 n_p 个样本, G_q 包含 n_q 个样本, 分别用 $\bar{x}_p$ 和 $\bar{x}_q$ 表示 G_p 和 G_q 的均值, 即类的中心.

1. 最短距离或单连接(single linkage): 定义类 G_p 的样本与类 G_q 的样本之间的最短距离为两类之间的距离:

$$D_{pq} = \min\{d_{ij} \mid x_i \in G_p, x_j \in G_q\}$$

2. 最长距离或完全连接(complete linkage): 定义类 G_p 的样本与类 G_q 的样本之间的最长距离为两类之间的距离:

$$D_{pq} = \max\{d_{ij} \mid x_i \in G_p, x_j \in G_q\}$$

3. 中心距离: 定义类 G_p 的样本与类 G_q 的中心之间的距离为两类之间的距离:

$$D_{pq} = d_{\bar{x}_p, \bar{x}_q}$$

4. 平均距离: 定义类 G_p 的样本与类 G_q 任意两个样本之间距离的平均值为两类之间的距离:

$$D_{pq} = \frac{1}{n_p n_q} \sum_{x_i \in G_p} \sum_{x_j \in G_q} d_{ij}$$

8.2 性能指标

[cj] 评价标准分成了两派:

- 外部指标: 事先知道数据的真实分类 (参考模型).
- 内部指标: 完全不知道真实分类, 只能通过测量数据点之间的实际物理/几何距离来评价.

8.2.1 外部指标

通过考察所有样本对 (x_i, x_j) (其中 $i < j$), 根据它们在 C 和 C^* 中是否属于同一个簇, 将其分为四类:

- $a = |\text{SS}|$: Same-Same. 在聚类结果中属于同簇, 在参考模型中也属于同簇. (预测正确)
- $b = |\text{SD}|$: Same-Different. 在聚类结果中属于同簇, 但在参考模型中属于不同簇. (预测错误)
- $c = |\text{DS}|$: Different-Same. 在聚类结果中属于不同簇, 但在参考模型中属于同簇. (预测错误)
- $d = |\text{DD}|$: Different-Different. 在聚类结果中属于不同簇, 在参考模型中也属于不同簇. (预测正确)

$a + b + c + d = \frac{m(m-1)}{2} = C_m^2$ [ck] 即从 m 个样本中任取两个样本的组合数.

有了 a, b, c, d 我们就可以拼装出不同的评价指标:

[cl]

1. Jaccard 系数 (JC) 完全忽略了 d (因为大部分毫不相干的点被分在不同组是很正常的, 数值往往极大, 会掩盖有效信息). 它只关心: 在所有“被算法凑在一起”或“原本就该在一起”的样本对中, 真正凑对成功的比例是多少.
2. Rand 指数 (RI) 是最直观的“准确率”分子 $a + d$ 是算法所有做对的决定 (不管是成功凑对还是成功拆散), 分母是总共有多少对.
3. Fowlkes and Mallows 指数 (FMI) 是两个比例的几何平均数. $\frac{a}{a+b}$ 衡量了“算法聚在一起的对中, 有多少是真的” (查准率); $\frac{a}{a+c}$ 衡量了“原本该在一起的对中, 算法找出了多少” (查全率).

- Jaccard 系数 (JC): $\frac{a}{a+b+c}$
- FM 指数 (FMI): $\sqrt{\frac{a}{a+b} \cdot \frac{a}{a+c}}$
- Rand 指数 (RI): $\frac{a+d}{a+b+c+d}$ (即预测正确的样本对比例)

上述指标的结果均在 $[0, 1]$ 区间, 值越大越好.

8.2.2 内部指标

Definition 8.2.1

1. 簇内样本平均距离 $\text{avg}(C)$

$$\text{avg}(C) = \frac{2}{|C|(|C| - 1)} \sum_{x_i, x_j \in C} \text{dist}(x_i, x_j)$$

2. 簇内样本最远距离 (直径) $\text{diam}(C)$

$$\text{diam}(C) = \max_{x_i, x_j \in C} \text{dist}(x_i, x_j)$$

3. 簇间最近样本距离 $d_{\min}(C_i, C_j)$

$$d_{\min}(C_i, C_j) = \min_{x_i \in C_i, x_j \in C_j} \text{dist}(x_i, x_j)$$

[cm]

1. 把同一个簇里的人两两之间量一下距离,算个平均数.这个值越小越好,说明簇内足够紧凑.
 2. 同一个簇里,相距最远的两个人有多远.这就好比一个团队的“直径”.这个值同样越小越好,说明圈子没有拉得太长.
 3. A 簇边缘的人和 B 簇边缘的人,最容易发生混淆的两个点的距离.这个值越大越好,说明两个圈子界限分明,井水不犯河水.
4. 定义 $\mu_i = \frac{1}{|C_i|} \sum_{x \in C_i} x$ 为簇 C_i 内样本的均值向量,则两个聚类簇 C_i 和 C_j 的**中心距离**为:

$$d_{\text{cen}}(C_i, C_j) = \text{dist}(\mu_i, \mu_j)$$

聚类分析常用的内部指标,簇内越紧密越好(高内聚),簇间越疏远越好(低耦合):

$$\text{DBI} = \frac{1}{k} \sum_{i=1}^k \max_{j \neq i} \left(\frac{\text{avg}(C_i) + \text{avg}(C_j)}{d_{\text{cen}}(\mu_i, \mu_j)} \right)$$

$$\text{DI} = \min_{1 \leq i \leq k} \left\{ \min_{j \neq i} \left(\frac{d_{\min}(C_i, C_j)}{\max_{1 \leq l \leq k} \text{diam}(C_l)} \right) \right\}$$

[cn] **DB 指数 (Davies-Bouldin Index, DBI)**, 衡量“最坏情况下的簇间重叠程度”, 用“簇内的平均分散程度”(簇有多松散)除以“簇中心之间的距离”(簇隔得有多远)。

Dunn 指数 (Dunn Index, DI), 衡量“最小的簇间间距”相对于“最大的簇内直径”的比率, 找出所有簇之间最近的距离,除以所有簇中最大的那个直径。

DBI 指数越小(或 DI 指数越大)就越意味着簇内距离越小同时簇间距离越大,说明聚类的性能越好。

8.3 距离计算

对**函数** $\text{dist}(\cdot, \cdot)$, 若它是一个“距离度量”(distance measure), 则应满足一些基本性质:

- **非负性:** $\text{dist}(x_i, x_j) \geq 0$;
- **同一性:** $\text{dist}(x_i, x_j) = 0$ 当且仅当 $x_i = x_j$
- **对称性:** $\text{dist}(x_i, x_j) = \text{dist}(x_j, x_i)$
- **直通性:** $\text{dist}(x_i, x_j) \leq \text{dist}(x_i, x_k) + \text{dist}(x_k, x_j)$

8.3.1 有序属性

Definition 8.3.1

闵可夫斯基距离

给定样本集合 X , X 是 m 维实数向量空间 R^m 中点的集合, 其中 $x_i, x_j \in X, x_i = (x_{1i}, x_{2i}, \dots, x_{mi})^T, x_j = (x_{1j}, x_{2j}, \dots, x_{mj})^T$, 样本 x_i 与 x_j 的闵可夫斯基距离(Minkowski distance)定义为

$$d_{ij} = \left(\sum_{k=1}^m |x_{ki} - x_{kj}|^p \right)^{\frac{1}{p}}$$

$p \geq 1$, 当 $p = 2$ 时称为欧氏距离, $p = 1$ 时称为曼哈顿距离, $p = \infty$ 时称为切比雪夫距离:

$$d_{ij} = \max_k |x_{ki} - x_{kj}|$$

Definition 8.3.2

马哈拉诺比斯距离(马氏距离)

[co] 马氏距离本质上解决两个直觉问题: “单位不统一”和“变量之间有裙带关系(相关性)”。考虑各个分量(特征)之间的相关性,且与各个分量的尺度无关.马哈拉诺比斯距离越大,相似度越小.因此计算距离时要用方差归一化.

给定一个样本集合 X , $X = [x_{(ij)}]_{m \times n}$, 其协方差矩阵记作 S . 样本 x_i 与 x_j 之间的马氏距离 d_{ij} 定义为

$$d_{ij} = \left[(x_i - x_j)^T S^{-1} (x_i - x_j) \right]^{\frac{1}{2}}$$

其中,

$$(x_i = (x_{1i}, x_{2i}, \dots, x_{mi})^T, \quad x_j = (x_{1j}, x_{2j}, \dots, x_{mj})^T$$

$$S = \frac{1}{m} \sum_{i=1}^m (x_i - M)(x_i - M)^T$$

1. 先减: 每个样本都减去均值(去中心化)。
2. 再乘: 每个样本生成一个 $n \times n$ 的小矩阵。
3. 最后加: 把这 m 个小矩阵全部叠在一起,再除以 m (或者 $m-1$)。

Definition 8.3.3

相关系数

样本 x_i 与 x_j 之间的相关系数定义为

$$r_{ij} = \frac{\sum_{k=1}^m (x_{ki} - \bar{x}_i)(x_{kj} - \bar{x}_j)}{\left[\sum_{k=1}^m (x_{ki} - \bar{x}_i)^2 \sum_{k=1}^m (x_{kj} - \bar{x}_j)^2 \right]^{\frac{1}{2}}}$$

其中,

$$\bar{x}_i = \frac{1}{m} \sum_{k=1}^m x_{ki}, \quad \bar{x}_j = \frac{1}{m} \sum_{k=1}^m x_{kj}$$

[cp] 相关系数的绝对值越接近 1,表示样本越相似; 越接近 0,表示样本越不相似。

Definition 8.3.4

夹角余弦

样本 x_i 与 x_j 之间的夹角余弦定义为

$$s_{ij} = \frac{\sum_{k=1}^m x_{ki} x_{kj}}{\left(\sum_{k=1}^m x_{ki}^2 \sum_{k=1}^m x_{kj}^2 \right)^{\frac{1}{2}}}$$

8.3.2 无序属性

1. 连续属性 (continuous attribute): 在定义域上有无穷多个可能的取值
2. 离散属性 (categorical attribute): 在定义域上是有限个可能的取值
3. 有序属性 (ordinal attribute): 例如定义域为{1,2,3}的离散属性,“1”与“2”比较接近、与“3”比较远,称为有序属性
4. 无序属性 (non-ordinal attribute): 例如定义域为{飞机,火车,轮船}这样的离散属性,不能直接在属性值上进行计算,称为无序属性

Definition 8.3.5

VDM

$$VDM_p(a, b) = \sum_{i=1}^k \left| \frac{m_{u,a,i}}{m_{u,a}} - \frac{m_{u,b,i}}{m_{u,b}} \right|^p$$

1. 分母 $m_{u,a}$ 和 $m_{u,b}$: 这就是在整个数据集里,属性 u (比如“职业”) 取值为 a (比如“医生”) 的总人数。
2. 分子 $m_{u,a,i}$: 这是在第 i 个簇里,属性 u 取值为 a 的人数。
3. 分数 $\frac{m_{u,a,i}}{m_{u,a}}$: 代表了取值为 a 的样本掉进第 i 个簇的概率. 比如: 所有的“医生”里,有 80% 都在 1 号簇(常出现在医院的组)。
4. 相减并求和 $\sum | \dots - \dots |^p$: 比较“医生”和“护士”在所有 k 个簇里的分布概率差. 如果他们在每个簇里的占比都差不多,减出来的差值就很小,距离也就很近。

8.3.3 MinkovDM

$$\text{MinkovDM}_p(x_i, x_j) = \left(\sum_{u=1}^{n_c} |x_{iu} - x_{ju}|^p + \sum_{u=n_c+1}^n \text{VDM}_p(x_{iu}, x_{ju}) \right)^{\frac{1}{p}}$$

[cq] 在现实世界里,一个样本往往既有数字(身高、年龄),又有文字(性别、职业)。前半部分 $\sum |x_{iu} - x_{ju}|^p$: 这是传统的闵可夫斯基距离,用来算数字属性(有序属性)。

后半部分 $\sum \text{VDM}_p(x_{iu}, x_{ju})$: 这是刚才学的 VDM,用来算文字属性(无序属性)。

逻辑: 把能减的数字直接减,不能减的文字用 VDM 算,最后全部加起来开个方。

8.4 原型聚类

原型是指样本空间中具有代表性的“点”。通常情况下,算法先对原型进行初始化,再对原型进行迭代更新求解。

8.4.1 k 均值聚类

Remark

随机选一些样本为中心->计算所有样本到各个中心的距离->归到最近的那一类->每一类更新自己的中心。重复。

8.4.1.1 k 均值聚类的算法理念与流程

[cr] 每个样本只能属于一个类,所以 k 均值聚类是硬聚类。下面分别介绍 k 均值聚类的模型、策略、算法,讨论算法的特性及相关问题。

给定 n 个样本的集合 $X = \{x_1, x_2, \dots, x_n\}$, 每个样本由一个特征向量表示,特征向量的维数是 m 。 k 均值聚类的目标是将 n 个样本分到 k 个不同的类或簇中,这里假设 $k < n$ 。 k 个类 $G_1, G_2, \dots, G_k$ 满足

$$G_i \cap G_j = \emptyset, \bigcup_{i=1}^k G_i = X$$

· 用 C 表示划分,一个划分对应着一个聚类结果。

策略是通过损失函数的最小化选取最优的划分或函数 C^* 。

首先采用欧氏距离平方作为样本之间的距离 $d(x_i, x_j)$, 然后定义样本与其所属类的中心之间的距离之和为损失函数,即

$$E_C = \sum_{l=1}^k \sum_{C(i)=l} \|x_i - \bar{x}_l\|^2$$

式中 $\bar{x}_l = (\bar{x}_{1l}, \bar{x}_{2l}, \dots, \bar{x}_{ml})^T$ 是第 l 个类的均值或中心, $n_l = \sum_{i=1}^n \mathbb{I}(C(i) = l)$, k 均值聚类就是求解最优化问题:

$$C^* = \arg \min_C E_C$$

相似的样本被聚到同类时,损失函数值最小,这个目标函数的最优化能达到聚类的目的。

[cs] k 均值聚类的最优解求解是 NP 困难问题,现实中采用迭代的方法求解。

Algorithm 8.4.1

K means 聚类

输入: 样本集 $D = (x_1, x_2, \dots, x_m)$; 聚类簇数 k 。

输出: 簇划分 $C = (C_1, C_2, \dots, C_k)$ 。

过程:

1. 从 D 中随机选择 k 个样本作为初始均值向量 $(\mu_1, \mu_2, \dots, \mu_k)$ 。
2. repeat
 1. 令 $C_i = \emptyset$ ($1 \leq i \leq k$)。
 2. for $j = 1, 2, \dots, m$ do
 1. 计算样本 x_j 与各均值向量 μ_i ($1 \leq i \leq k$) 的距离: $d_{ji} = \|x_j - \mu_i\|^2$ 。
 2. 根据距离最近的均值向量确定 x_j 的簇标记:

$$\lambda_j = \arg \min_{i \in \{1, 2, \dots, k\}} d_{ji}$$
 3. 将样本 x_j 划入相应的簇: $C_{\lambda_j} = C_{\lambda_j} \cup (x_j)$ 。
 3. end for
 4. for $i = 1, 2, \dots, k$ do
 1. 计算新均值向量: $\mu_{i'} = \frac{1}{|C_i|} \sum_{x \in C_i} x$ 。
 2. if $\mu_{i'} \neq \mu_i$ then 将当前均值向量 μ_i 更新为 $\mu_{i'}$ 。
 3. else 保持当前均值向量不变。
 4. end if
 5. end for
3. until 当前均值向量均未更新

k 均值聚类算法的复杂度是 $O(mnk)$, 其中 m 是样本维数, n 是样本个数, k 是类别个数.

8.4.1.2 k 均值聚类的算法特点与局限性

[ct]

类中心在聚类的过程中会发生移动,但是往往不会移动太大

聚类结果的质量可以用类的平均直径来衡量.一般地,类别数变小时,平均直径会增加;类别数变大超过某个值以后,平均直径会不变,而这个值正是最优的值.下图说明类别数与平均直径的关系.实验时,可以采用二分查找,快速找到最优的值.

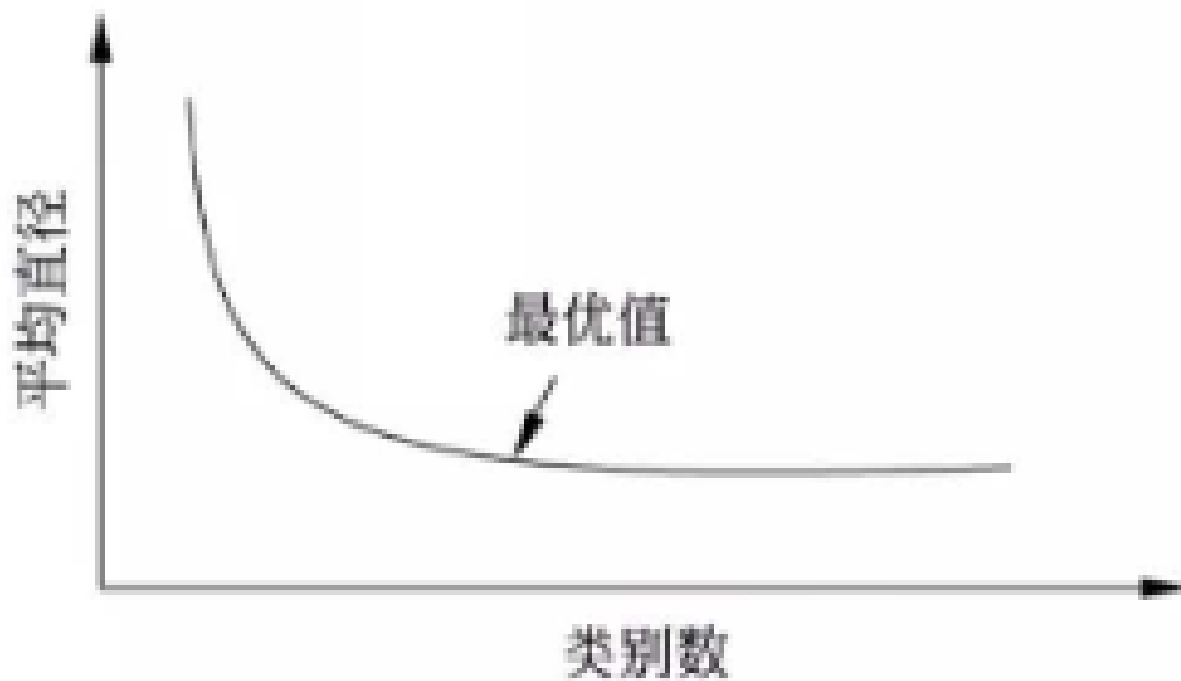


图 15 类别数与平均直径的关系

K 均值适合的聚类数据类型是自然族：球形、等尺寸、等密度. 待聚类数据中各个自然类的样本数据量不平衡时；待聚类数据中各个自然类的样本密度不一致时；待聚类数据中各个自然类的样本分布不是球形时； k 均值的效果不太理想.

Remark

k 均值聚类有以下特点：

- 基于划分的聚类方法；
- 类别数 k 事先指定；
- 以欧氏距离平方表示样本之间的距离,以中心或样本的均值表示类别；
- 以样本和其所属类的中心之间的距离的总和为最优化的目标函数；
- 得到的类别是**平坦的、非层次化的**.算法是迭代算法,不能保证得到全局最优.
 - “平坦”意味着所有生成的簇（Clusters）处于同一层级,地位平等.
 - “非层次化”意味着簇之间不存在“嵌套”或“包含”关系（即没有父类和子类的概念）.

[cu] 本章介绍两种原型聚类方法：**学习向量量化（LVQ）**和**高斯混合聚类（GMM）**.二者的共同思路是用少量“原型”代表整个数据集,每个原型对应一个簇,样本按与各原型的距离或隶属概率完成划分.区别在于原型的形式:LVQ 用一个向量,GMM 用一个完整的高斯分布.

8.4.2 学习向量量化（LVQ）

8.4.2.1 定位与动机

LVQ（Learning Vector Quantization）**假设数据集带有类别标记**,在学习过程中借助这些监督信息,不断**更新原型向量**,从而得到更符合类别边界的划分结果.

8.4.2.2 问题设定

给定带标记训练集：

$$D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_m, y_m)\}$$

其中 $\mathbf{x}_j \in \mathbb{R}^n$ 为 n 维特征向量, $y_j \in \mathcal{Y}$ 为类别标记.

目标是学得 q 个 n 维原型向量 $\{p_1, p_2, \dots, p_q\}$, 每个 p_i 附带一个预设的类别标记 $t_i \in \mathcal{Y}$.

[cv] 这是说, 事先决定要用 q 个代表点来划分空间, 并且每个代表点预先指定它代表哪个类别. 比如用 3 个代表点, 2 个代表猫、1 个代表狗.

训练完成后, 每个原型向量定义其对应的 **Voronoi 区域**——样本空间中与该原型距离最近的所有点都归入对应的簇.

8.4.2.3 算法流程

Algorithm 8.4.2

输入: 样本集 $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$;
原型向量个数 q 及各原型对应的类别标记 $\{t_1, t_2, \dots, t_q\}$;
学习率 $\eta \in (0, 1)$.

过程:

1. 随机初始化一组原型向量 $\{p_1, p_2, \dots, p_q\}$
2. **repeat**
 - 从 D 中随机取一个样本 (x_j, y_j)
 - 计算 x_j 与每个 p_i 的距离: $d_i = \|x_j - p_i\|$
 - 找到最近原型: $p^* = \arg \min_{i \in \{1, \dots, q\}} d_i$, 记其类别标记为 t^*
 - **if** $y_j = t^*$ **then** # 类别相同, 令 p^* 靠近 x_j
 $p' = p^* + \eta \cdot (x_j - p^*)$
 - **else** # 类别不同, 令 p^* 远离 x_j
 $p' = p^* - \eta \cdot (x_j - p^*)$

[cw] 更新规则的几何解释

两种更新方式的本质是调整 p^* 的位置.

情形一: 标记相同, p^* 靠近

更新后两者距离为:

$$\|p' - x_j\| = (1 - \eta) \|p^* - x_j\|$$

即距离缩短为原来的 $(1 - \eta)$ 倍, p^* 向 x_j 移动.

情形二: 标记不同, p^* 远离

更新后两者距离为:

$$\|p' - x_j\| = (1 + \eta) \|p^* - x_j\|$$

即距离扩大为原来的 $(1 + \eta)$ 倍, p^* 远离 x_j .

[cx] **学习率 η 的作用:** η 越大, 原型向量对单个样本的响应越激烈; η 越小, 收敛越平稳但速度较慢. 实践中通常随迭代轮次衰减.

3. 将 p^* 更新为 p'

until 满足停止条件

输出: 原型向量 $\{p_1, p_2, \dots, p_q\}$

8.4.3 高斯混合聚类 (GMM)

LVQ 以一个点代表一个簇, 表达能力有限. 高斯混合模型 (Gaussian Mixture Model, GMM) 改用一个**高斯分布**来刻画每个簇, 能够表达更丰富的形状 (椭圆形、旋转簇等), 同时通过软分配给出每个样本属于各簇的概率.

8.4.3.1 基础: 多元高斯分布

对 n 维随机向量 x , 若其服从均值为 μ 、协方差矩阵为 Σ 的高斯分布, 概率密度函数为:

$$p(x) = \frac{1}{(2\pi)^{n/2} |\Sigma|^{1/2}} \exp\left(-\frac{1}{2}(x - \mu)^\top \Sigma^{-1}(x - \mu)\right)$$

简记为 $p(x | \mu, \Sigma)$, 其中:

- μ : n 维均值向量, 描述分布的中心位置;
- Σ : $n \times n$ 正定协方差矩阵, 描述分布的形状与方向;
- $|\Sigma|$: Σ 的行列式, 出现在归一化常数中.

8.4.3.2 高斯混合分布的定义

高斯混合模型将 k 个高斯分量加权叠加:

$$p_{\mathcal{M}}(\mathbf{x}) = \sum_{i=1}^k \alpha_i \cdot p(\mathbf{x} | \boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i)$$

每个分量由三元组 $(\alpha_i, \boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i)$ 描述:

参数	含义	约束
α_i	混合系数 (先验权重)	$\alpha_i > 0$, 且 $\sum_{i=1}^k \alpha_i = 1$
$\boldsymbol{\mu}_i$	第 i 个分量的均值向量	无约束
$\boldsymbol{\Sigma}_i$	第 i 个分量的协方差矩阵	正定矩阵

8.4.3.3 样本生成过程与隐变量

高斯混合模型的所有样本来源可以理解为一个两阶段生成过程:

1. 以先验概率 α_i 选择第 i 个混合成分. 令隐变量 $z_j \in \{1, \dots, k\}$ 表示生成的样本 $\mathbf{x}_j$ 所来自的混合成分, 即 $P(z_j = i) = \alpha_i$;
2. 从选定的高斯分量 $\mathcal{N}(\boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)$ 中采样, 得到观测样本 $\mathbf{x}_j$.

实际上 z_j 未知, 利用贝叶斯定理求 z_j 的**后验概率** (已观测到的 $\mathbf{x}_j$ 来自第 i 个分量的概率):

$$\gamma_{ji} = p_{\mathcal{M}}(z_j = i | \mathbf{x}_j) = \frac{P(z_j = i) \cdot p(\mathbf{x}_j | z_j = i)}{p_{\mathcal{M}}(\mathbf{x}_j)} = \frac{\alpha_i \cdot p(\mathbf{x}_j | \boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i)}{\sum_{l=1}^k \alpha_l \cdot p(\mathbf{x}_j | \boldsymbol{\mu}_l, \boldsymbol{\Sigma}_l)}$$

[cy] γ_{ji} 是软聚类的核心量: 它不是 0/1 的硬分配, 而是样本 $\mathbf{x}_j$ 对各成分连续的隶属概率, 满足 $\sum_{i=1}^k \gamma_{ji} = 1$.

8.4.3.4 聚类规则

求得所有 γ_{ji} 后, 将样本 $\mathbf{x}_j$ 划入使后验概率最大的那个成分所对应的簇:

$$\lambda_j = \arg \max_{1 \leq i \leq k} \gamma_{ji}$$

从原型聚类角度理解: 高斯混合聚类以高斯分布作为原型, 簇划分由后验概率 (样本属于该成分的平均后验概率) 确定, 即每个样本被划入后验概率最大的簇标记所对应的簇.

8.4.3.5 参数估计: 最大对数似然

给定样本集 D , 模型参数 $\{(\alpha_i, \boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i)\}$ 通过最大化对数似然来确定:

$$LL(D) = \ln \prod_{j=1}^m p_{\mathcal{M}}(\mathbf{x}_j) = \sum_{j=1}^m \ln \left(\sum_{i=1}^k \alpha_i \cdot p(\mathbf{x}_j | \boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i) \right)$$

对 $\boldsymbol{\mu}_i$ 求偏导并令其为零:

$$\frac{\partial LL(D)}{\partial \boldsymbol{\mu}_i} = \sum_{j=1}^m \frac{\alpha_i \cdot p(\mathbf{x}_j | \boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i)}{\sum_{l=1}^k \alpha_l \cdot p(\mathbf{x}_j | \boldsymbol{\mu}_l, \boldsymbol{\Sigma}_l)} \cdot (\mathbf{x}_j - \boldsymbol{\mu}_i) = \sum_{j=1}^m \gamma_{ji} (\mathbf{x}_j - \boldsymbol{\mu}_i) = 0$$

解出 $\boldsymbol{\mu}_i$, 并类似地对 $\boldsymbol{\Sigma}_i$ 和 α_i (利用拉格朗日乘子法处理 $\sum \alpha_i = 1$ 的约束) 求导, 得到参数的闭式更新公式:

$$\begin{aligned} \boldsymbol{\mu}_{(i)'} &= \frac{\sum_{j=1}^m \gamma_{ji} \mathbf{x}_j}{\sum_{j=1}^m \gamma_{ji}} \\ \boldsymbol{\Sigma}_{(i)'} &= \frac{\sum_{j=1}^m \gamma_{ji} (\mathbf{x}_j - \boldsymbol{\mu}_i)(\mathbf{x}_j - \boldsymbol{\mu}_i)^{\top}}{\sum_{j=1}^m \gamma_{ji}} \\ \alpha_{(i)'} &= \frac{\sum_{j=1}^m \gamma_{ji}}{m} \end{aligned}$$

直观理解: $\boldsymbol{\mu}_{(i)'}$ 是所有样本以后验概率 γ_{ji} 为权重的加权均值; $\boldsymbol{\Sigma}_{(i)'}$ 是对应的加权协方差; $\alpha_{(i)'}$ 等于各样本归属第 i 个成分的平均后验概率.

8.4.3.6 EM 算法框架

由于 γ_{ji} 依赖参数、参数又依赖 γ_{ji} , 形成循环依赖, 无法直接求解, 因此采用 **EM (期望最大化) 算法** 迭代求解:

- **E 步 (Expectation)** : 固定当前参数 $\{(\alpha_i, \mu_i, \Sigma_i)\}$, 按 2.3 节的公式计算每个样本对每个成分的后验概率 γ_{ji} ;
- **M 步 (Maximization)** : 固定 γ_{ji} , 按 2.5 节的公式更新所有参数;
- 重复 E/M 两步直至收敛 (参数变化量低于阈值)。

每次迭代都保证对数似然不减, 算法收敛至局部最优。

8.4.3.7 完整算法伪代码

Algorithm 8.4.3

高斯混合聚类 (GMM) 算法

输入: 样本集 $D = \{x_1, x_2, \dots, x_m\}$, 其中 $x_j \in \mathcal{X} \subseteq \mathbb{R}^n$; 高斯混合成分个数 k 。

输出: 簇划分结果 $C = \{C_1, C_2, \dots, C_k\}$ 。

1. 初始化高斯混合分布的模型参数

对 $i = 1, 2, \dots, k$, 设定初始的三元组参数: $(\alpha_i, \mu_i, \Sigma_i)$

[cz] 初始化时, 混合系数需满足 $\alpha_i > 0$ 且 $\sum_{i=1}^k \alpha_i = 1$ 。每一组参数代表一个初始假设的“高斯概率气团”。在工程实践中, 通常会先运行一遍 K-means, 用其聚类中心和各个簇的协方差来为这些气团进行“垫底”初始化。

2. 反复迭代更新, 直到满足停止条件 (EM 算法) :

[da] 通过“左右互搏”的方式解决参数与成分归属之间的循环依赖:

- E 步 (期望步)** : 固定当前的气团参数, 去计算每个数据点属于各个气团的后验概率 γ_{ji} 。这是一种“软分配”, 每个点都以不同的概率隶属于所有簇。
- M 步 (最大化步: 均值)** : 利用算好的概率作为权重, 重新计算气团的中心。对该气团归属感越强的样本, 其坐标对新中心 $\mu_{i'}$ 的拉动作用就越大。
- M 步 (最大化步: 协方差)** : 基于新的权重分布, 更新高斯气团的胖瘦和旋转方向 $\Sigma_{i'}$ 。
- M 步 (最大化步: 权重)** : 看看全场有多少样本实质上归属于该气团, 重新对它在整体中所占的资源比例 $\alpha_{i'}$ 进行洗牌。

- [E 步]** 对 $j = 1, 2, \dots, m$ 和 $i = 1, 2, \dots, k$, 计算当前模型下成分 i 生成样本 x_j 的后验概率:

$$\gamma_{ji} = \frac{\alpha_i p(x_j | \mu_i, \Sigma_i)}{\sum_{l=1}^k \alpha_l p(x_j | \mu_l, \Sigma_l)}$$

- [M 步]** 计算各高斯分量的新均值向量:

$$\mu_{i'} = \frac{\sum_{j=1}^m \gamma_{ji} x_j}{\sum_{j=1}^m \gamma_{ji}}$$

- [M 步]** 计算各高斯分量的新协方差矩阵:

$$\Sigma_{i'} = \frac{\sum_{j=1}^m \gamma_{ji} (x_j - \mu_i)(x_j - \mu_i)^T}{\sum_{j=1}^m \gamma_{ji}}$$

- [M 步]** 计算各高斯分量的新混合系数:

$$\alpha_{i'} = \frac{\sum_{j=1}^m \gamma_{ji}}{m}$$

- [参数更新]** 将模型参数替换为最新值, 若未达到收敛阈值 (如对数似然函数的增量极小), 则返回步骤 (a) 继续循环:

$$\{(\alpha_i, \mu_i, \Sigma_i)\} \rightarrow \{(\alpha_{i'}, \mu_{i'}, \Sigma_{i'})\}$$

3. 根据最终概率划分确定的簇 (一锤定音)

[db] 当高斯气团的位置和形状完全稳定、算法收敛后, 我们由“软分配”正式切换为“硬分配”。遍历每个样本, 看哪家机器认领它的后验概率最大, 它就死心塌地地被归入哪一个簇。

初始化各个簇集合 $C_i = \emptyset$ ($1 \leq i \leq k$)。对每一个样本 x_j , 计算其最可能来自的成分标记 λ_j 并分类:

$$\lambda_j = \arg \max_{1 \leq i \leq k} \gamma_{ji}$$

$$C_{\lambda_j} = C_{\lambda_j} \cup \{x_j\}$$

8.4.4 两种方法横向对比

维度	LVQ	高斯混合聚类
原型形式	向量 (空间中一个点)	高斯分布 (均值 + 协方差)
是否需要标记	是 (利用类别标记)	否 (纯无监督)
簇的表达能力	凸、各向同性区域	可表达椭圆形、旋转簇

样本归属方式	硬分配（最近原型）	软分配（后验概率最大）
求解方式	在线迭代（逐样本更新）	批量迭代（EM 算法）
参数数量	$q \times n$ （原型坐标）	$k \times (1 + n + n^2)$ （混合系数、均值、协方差）

8.4.5 关键概念速查

Voronoi 区域: 每个原型向量“管辖”的样本空间子集——所有与该原型欧氏距离最近的点.LVQ 的聚类边界即由 Voronoi 图给出.

混合系数 α_i : 先验概率,反映第 i 个高斯分量在整体分布中的比重,可理解为“随机抽取一个样本,它来自成分 i 的先验概率”.

后验概率 γ_{ji} : 在观测到 $\mathbf{x}_j$ 之后,该样本来自第 i 个成分的概率.它综合了先验权重 α_i 和该分量对 $\mathbf{x}_j$ 的拟合程度 $p(\mathbf{x}_j | \mu_i, \Sigma_i)$,是软聚类决策的依据.

EM 算法: 专门处理含隐变量的最大似然估计问题的迭代框架.E 步计算隐变量的期望（即后验概率）,M 步在给定期望下最大化似然函数,两步交替直至收敛.每次迭代保证对数似然单调不减.

软分配 vs. 硬分配: GMM 的 γ_{ji} 是连续概率值（软分配）,每个样本同时“部分属于”多个簇; LVQ 和 k-均值则将样本唯一地分配给一个簇（硬分配）.软分配在簇边界附近更具鲁棒性,也更自然地表达了不确定性.

8.5 密度聚类

8.5.1 DBSCAN 聚类密度

密度聚类也称为**基于密度的聚类**（density-based clustering）,此类算法假设聚类结构能通过**样本分布的紧密程度**来确定.通常情况下,密度聚类算法从样本密度的角度来考察样本之间的可连接性,并基于可连接样本不断扩展聚类簇来获得最终的聚类结果.

DBSCAN 是一种基于密度的聚类算法（density-based clustering）,其基于一组“邻域”参数（ ϵ 、 $MinPts$ ）来刻画样本分布的紧密程度.

给定样本集 $D = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_m\}$,定义如下几个概念:

Definition 8.5.1 ε-邻域

对 $\mathbf{x}_j \in D$,其 ϵ -邻域包含样本集 D 中与 $\mathbf{x}_j$ 的距离不大于 ϵ 的样本（假设为欧氏距离）:

$$N_\epsilon(\mathbf{x}_j) = \{\mathbf{x}_i \in D \mid \text{dist}(\mathbf{x}_i, \mathbf{x}_j) \leq \epsilon\}$$

Definition 8.5.2 核心对象

若 $\mathbf{x}_j$ 的 ϵ -邻域至少包含 $MinPts$ 个样本,则 $\mathbf{x}_j$ 是一个核心对象:

$$|N_\epsilon(\mathbf{x}_j)| \geq MinPts$$

Definition 8.5.3 密度直达

若 $\mathbf{x}_j$ 位于 $\mathbf{x}_i$ 的 ϵ -邻域中,且 $\mathbf{x}_i$ 是核心对象,则称 $\mathbf{x}_j$ 由 $\mathbf{x}_i$ 密度直达.

Definition 8.5.4 密度可达

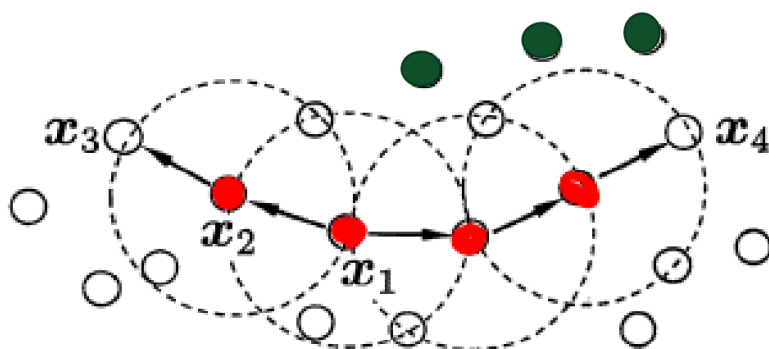
对 $\mathbf{x}_i$ 与 $\mathbf{x}_j$,若存在样本序列 $p_1, p_2, \dots, p_n$,其中 $p_1 = \mathbf{x}_i, p_n = \mathbf{x}_j$ 且 p_{i+1} 由 p_i 密度直达,则称 $\mathbf{x}_j$ 由 $\mathbf{x}_i$ 密度可达.

Definition 8.5.5 密度相连

对 $\mathbf{x}_i$ 与 $\mathbf{x}_j$,若存在 $\mathbf{x}_k$ 使得 $\mathbf{x}_i$ 与 $\mathbf{x}_j$ 均由 $\mathbf{x}_k$ 密度可达,则称 $\mathbf{x}_i$ 与 $\mathbf{x}_j$ 密度相连.

[dc] 形象理解:

- ϵ 描述了某一样本的邻域距离阈值
- $MinPts$ 描述了某一样本的距离为 ϵ 的邻域中样本个数的阈值
- 密度直达: 就像“密切接触者”——直接在一个邻域内
- 密度可达: 就像“次密接”——通过中间人间接连接
- 密度相连: 就像“次次密接”——有一个共同的源头



- x_1 到 x_2 是密度直达
- x_1 到 x_3 、 x_4 是密度可达
- x_3 与 x_4 是密度相连

图 16 密度直达/可达/相连的概念图示 (红色点为核心对象, $MinPts = 3$)

- 绿色点均为异常点,既不是核心点也不是边界点

8.5.2 DBSCAN 聚类簇的定义

基于上述概念,DBSCAN 将“簇”定义为: 由密度可达关系导出的最大的密度相连样本集合.给定邻域参数 (ϵ 、 $MinPts$),簇 $C \subseteq D$ 是满足以下性质的非空样本子集:

1. 连续性: $x_i \in C, x_j \in C \Rightarrow x_i$ 与 x_j 密度相连
2. 最大性: $x_i \in C$, 若 x_j 由 x_i 密度可达 $\Rightarrow x_j \in C$

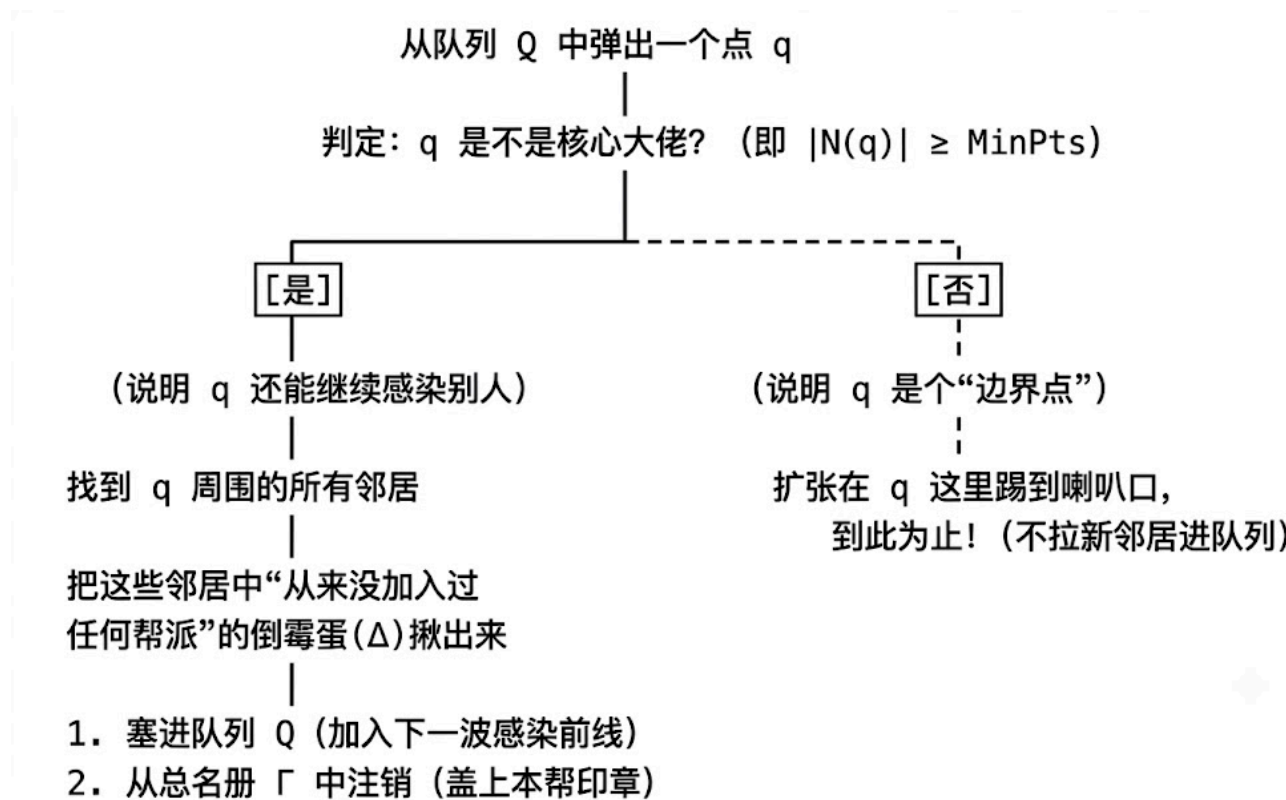
Remark

一个 DBSCAN 的簇里面可以有一个或者多个核心对象:

- 如果只有一个核心对象,则簇里其他的非核心对象样本都在这个核心对象的 ϵ -邻域里
- 如果有多个核心对象,则簇里的任意一个核心对象的 ϵ -邻域中一定有一个其他的核心对象,否则这两个核心对象无法密度可达.这些核心对象的 ϵ -邻域里所有的样本的集合组成的一个 DBSCAN 聚类簇.

8.5.3 DBSCAN 聚类步骤

DBSCAN 算法初始任意选择一个核心对象,然后找到所有这个核心对象能够密度可达的样本集合,即为一个聚类簇.接着继续选择另一个没有类别的核心对象去寻找密度可达的样本集合,这样就得到另一个聚类簇.一直运行到所有核心对象都有类别为止.

**Algorithm 8.5.6**

输入: 样本集 $D = \{x_1, x_2, \dots, x_m\}$; 邻域参数 $(\epsilon, MinPts)$.

输出: 簇划分 $\mathcal{C} = \{C_1, C_2, \dots, C_k\}$.

过程:

1. 初始化核心对象集合: $\Omega = \emptyset$
2. for $j = 1, 2, \dots, m$ do
 1. 确定样本 x_j 的 ϵ -邻域 $N_\epsilon(x_j)$
 2. if $|N_\epsilon(x_j)| \geq MinPts$ then
 1. 将样本 x_j 加入核心对象集合: $\Omega = \Omega \cup \{x_j\}$
 3. end if
3. end for
4. 初始化聚类簇数: $k = 0$
5. 初始化未访问样本集合: $\Gamma = D$
6. while $\Omega \neq \emptyset$ do
 1. 记录当前未访问样本集合: $\Gamma_{old} = \Gamma$
 2. 随机选取一个核心对象 $o \in \Omega$, 初始化队列 $Q = \langle o \rangle$
 3. $\Gamma = \Gamma \setminus \{o\}$
 4. while $Q \neq \emptyset$ do
 1. 取出队列 Q 中的首个样本 q
 2. if $|N_\epsilon(q)| \geq MinPts$ then
 1. 令 $\Delta = N_\epsilon(q) \cap \Gamma$
 2. 将 Δ 中的样本加入队列 Q
 3. $\Gamma = \Gamma \setminus \Delta$
 3. end if
 5. end while
 6. $k = k + 1$, 生成聚类簇 $C_k = \Gamma_{old} \setminus \Gamma$
 7. $\Omega = \Omega \setminus C_k$
7. end while

算法执行过程示意:

8.5.4 DBSCAN 聚类分析

[dd] 参数选择经验:

- ϵ 过小 $\rightarrow$ 簇数过多,大部分点被识别为噪声
- ϵ 过大 $\rightarrow$ 簇数过少,多个簇被合并
- $MinPts$ 过小 $\rightarrow$ 容易将噪声误判为核心对象
- $MinPts$ 过大 $\rightarrow$ 核心对象减少,簇被割裂

实用技巧: 通常先固定 $MinPts$ (经验值 4 10),然后通过绘制 K-距离图 (K-distance graph) 来选择合适的 ϵ 值.

8.5.4.1 DBSCAN 算法特点

一般来说,DBSCAN 算法有以下几个特点:

特点	说明
需要确定参数	需要提前确定 ϵ 和 $MinPts$ 值
无需预设簇数	不需要提前设置聚类类别的个数 (与 K-means 的关键区别)
对初值不敏感	对初始核心对象选取不敏感
对噪声不敏感	能有效识别并处理异常点/噪声
密度不均效果差	对密度不均的数据聚合效果不好

DBSCAN 与 K-means 的对比:

| 特性 | K-means | DBSCAN | |——|——|——| | 簇形状 | 球形/凸形 | 任意形状 | | 簇数 | 需预先指定 | 自动确定 | | 噪声处理 | 无法处理 | 能识别噪声 | | 参数 | 簇数 k | ϵ , $MinPts$ | | 密度不均 | 效果好 | 效果差 | | 复杂度 | $O(kmn)$ | $O(mn)$ 或使用空间索引 $O(n \log n)$ |

8.5.5 算法复杂度分析

DBSCAN 算法的时间复杂度主要取决于邻域查询的效率:

- 朴素实现: 对每个样本计算与所有其他样本的距离,复杂度为 $O(m^2)$,其中 m 为样本数
- 使用空间索引 (如 k-d tree、R-tree): 可将复杂度降至 $O(m \log m)$
- 最坏情况: 当所有点都是核心对象且密度相连时,仍需遍历所有点对,复杂度退化为 $O(m^2)$

空间复杂度为 $O(m)$,主要用于存储样本集、核心对象集合和聚类结果.

8.5.6 应用场景与局限性

8.5.6.1 适用场景

1. 簇形状不规则: 数据呈现非球形、非凸形的簇结构
2. 含噪声数据: 数据集中存在异常点或噪声,需要鲁棒的聚类方法
3. 簇数未知: 无法预先确定聚类簇的个数
4. 密度可分: 不同簇之间存在密度差异,簇内密度相对均匀

8.5.6.2 局限性

1. 密度不均: 当数据集中不同簇的密度差异较大时,难以用一组参数同时处理好所有簇
2. 高维数据: 在高维空间中,“距离”的概念变得模糊 (维度灾难), ϵ 的选择变得困难
3. 参数敏感: ϵ 和 $MinPts$ 的选择对结果影响显著,需要经验或交叉验证
4. 边界点归属: 边界点可能被多个簇共享,DBSCAN 通常将其随机分配给最先发现的簇

Remark

改进算法: 针对 DBSCAN 的局限性,研究者提出了多种改进算法:

- OPTICS: 通过可达距离图展示不同密度下的聚类结构,避免了单一参数选择的困难
- HDBSCAN: 层次 DBSCAN,自动确定参数,能处理密度不均的数据
- DENCLUE: 基于密度分布函数的聚类方法

8.6 层次聚类

8.6.1 层次聚类思想

层次聚类 (Hierarchical Clustering) 假设类别之间存在层次结构,将样本聚到层次化的类中.因为每个样本只属于一个类,所以层次聚类属于硬聚类.
层次聚类算法可以自上而下或自下而上实现:

方法	方向	核心思想
凝聚式 (agglomerative)	自下而上	将每个对象作为单独的一组,然后根据同类相近、异类相异的原则,合并对象,直到所有的组合并成一个,或达到一个终止条件为止
分裂式 (divisive)	自上而下	将所有的对象置于一类,在迭代的每一步中,一个类不断地分为更小的类,直到每个对象在单独的一个类中,或达到一个终止条件

层次聚类其实是一组有**依赖关系**的聚类,后一聚类类依赖前一聚类结果,是在前一聚类结果的基础上,按照一定规则进行新的聚类.因为**有依赖关系**,因此这一组聚类就形成一个**树状结构**.

Remark

凝聚式层次聚类需要预先确定下面 3 个要素:

1. **距离或相似度**: 如何度量样本或类之间的距离
2. **合并规则**: 选择哪两个类进行合并
3. **停止条件**: 何时终止聚类过程

8.6.2 凝聚式层次聚类算法

以下讨论的是**凝聚式层次聚类法**.

8.6.2.1 算法描述

凝聚式层次聚类的核心思想是: 从每个样本自成一类开始,逐步合并距离最近的两个类,直到满足停止条件.

Algorithm 8.6.1

输入: 样本集合 $D = \{x_1, x_2, \dots, x_N\}$; 停止条件.

输出: 对样本集合的一个层次化聚类 (树状结构) .

过程:

1. N 个初始模式样本自成一类,即建立 N 类: $G_1(0), G_2(0), \dots, G_N(0)$
2. 计算各类之间 (即各样本间) 的距离,得一 $N \times N$ 维距离矩阵 $D(0)$
3. 假设已求得距离矩阵 $D(n)$ (n 为逐次聚类合并的次数),找出 $D(n)$ 中的最小元素,将其对应的两类合并为一类.由此建立新的分类: $G_1(n+1), G_2(n+1), \dots$
4. 再次计算: 经过合并后,各个类别之间的距离,得 $D(n+1)$. 显然, $D(n+1)$ 的维数是 $D(n)$ 的维数减一.
5. 跳至上一步,重复计算及合并.
6. 结束条件 (满足其一即可)
 1. **取距离阈值 T** : 当 $D(n)$ 的最小分量**超过给定值 T** 时 (即: 当前各聚类类间的距离,都足够大),算法停止.所得即为聚类结果.
 2. **不设阈值**: 一直将全部样本聚成一类为止,输出聚类的分级树 (得到全部可能的聚类结果——谱系聚类的名称由来) .

这里我们举一个欧氏距离的例子,可以看出聚合层次聚类算法的复杂度是 $O(n^3m)$,其中 m 是样本的维数, n 是样本个数.通过使用**优先队列 (堆)** 或 $k-d$ tree 等数据结构优化,可以将复杂度降低到 $O(n^2 \log n)$ 甚至更低,但最原始的朴素实现是 $O(n^3m)$.

关键点理解:

- 层次聚类的每一次合并都**依赖前一次的结果**,因此形成一个不可逆的树状结构
- 一旦两个类被合并,就无法再分开 (这是与 K-means 的本质区别)
- 最终得到的树状图 (Dendrogram) 可以在任意高度“切割”,得到不同粒度的聚类结果

8.6.3 类间距离计算方法

层次聚类最关键的一步就是**计算两个类簇的相似度**.定义类间距离的方法不同,分类结果会不太一致.

8.6.3.1 最短距离法 (单链距离法)

如 H 、 K 是两个聚类,则两类间的最短距离定义为:

$$D_{HK} = \min\{D(X_H, X_K)\} \quad X_H \in H, X_K \in K$$

其中 $D(X_H, X_K)$ 为 H 类中的某个样本 X_H 和 K 类中的某个样本 X_K 之间的欧氏距离.

D_{HK} : H 类中所有样本与 K 类中所有样本之间的最小距离.

若 K 类由 I 和 J 两类合并而成,且 H 与 I 、 H 与 J 的距离,已先期算出,则得到**递推公式**:

$$D_{HK} = \min\{D_{HI}, D_{HJ}\}$$

特点:最短距离法的**包容性极强**——只要两个簇中有一对样本距离足够近,就会将两个簇合并,这可能导致“链式效应”(chaining effect),即簇被拉得很长。

8.6.3.2 最长距离法（全链距离法）

$$D_{HK} = \max\{D(\mathbf{X}_H, \mathbf{X}_K)\} \quad \mathbf{X}_H \in H, \mathbf{X}_K \in K$$

若 K 类由 I 、 J 两类合并而成,则

$$\begin{aligned} D_{HI} &= \max\{D(\mathbf{X}_H, \mathbf{X}_I)\} \quad \mathbf{X}_H \in H, \mathbf{X}_I \in I \\ D_{HJ} &= \max\{D(\mathbf{X}_H, \mathbf{X}_J)\} \quad \mathbf{X}_H \in H, \mathbf{X}_J \in J \end{aligned}$$

有递推公式:

$$D_{HK} = \max\{D_{HI}, D_{HJ}\}$$

特点:最长距离法**只要存在缺点就坚决不合并**——只有当两个簇中所有样本对之间的距离都较小时,才会合并,这倾向于产生紧凑的、球形的簇。

8.6.3.3 类平均距离法（组平均距离法）

$$D_{HK} = \sqrt{\frac{1}{n_H n_K} \sum_{i \in H} \sum_{j \in K} d_{ij}^2}$$

其中 d_{ij}^2 : H 类任一样本 $\mathbf{X}_i$ 和 K 类任一样本 $\mathbf{X}_j$ 之间的欧氏距离平方。
若 K 类由 I 和 J 类合并产生,则递推式为:

$$D_{HK} = \sqrt{\frac{n_I}{n_I + n_J} D_{HI}^2 + \frac{n_J}{n_I + n_J} D_{HJ}^2}$$

特点:均连接法是从全局出发顾全大局的一种度量——考虑了两个簇中所有样本对的平均距离,兼顾了单链接和全链接的特点。

[de] 实际问题中常用几种不同的方法,比较分类结果,从而选择一个比较切合实际的分类。
三种方法的直观对比:

- **单链接:** 看两个簇“最近”的两个点——容易合并,但可能产生链式效应
- **全链接:** 看两个簇“最远”的两个点——谨慎合并,簇更紧凑
- **均链接:** 看两个簇所有点的“平均”距离——折中方案,兼顾全局

8.6.4 AGNES 聚类方法

AGNES (AGglomerative NESting) 聚类是一种采用**自底向上聚合策略**的层次聚类算法,该算法通过不断合并距离最近的聚类簇,直到达到预设的聚类簇个数或满足其他停止条件为止。

8.6.4.1 AGNES 流程

假设有 m 个待聚类的样本:

1. **初始化:** 把每个样本归为一簇,计算每两个簇之间的距离,也就是样本与样本之间的相似度
2. **合并:** 寻找各个簇之间最近的两个簇,把他们归为一类 (簇的总数减少一个)
3. **更新:** 重新计算新生成的这个簇与各个旧簇之间的**相似度**
4. **重复:** 重复步骤 2 和 3 直到所有样本点都归为一簇,结束

8.6.4.2 类簇相似度度量的三种方式

方法	定义与公式
单链接 (single-linkage)	取类间最小距离 $d_{\min}(C_i, C_j) = \min_{\mathbf{x} \in C_i, \mathbf{z} \in C_j} \text{dist}(\mathbf{x}, \mathbf{z})$
全链接 (complete-linkage)	取类间最大距离 $d_{\max}(C_i, C_j) = \max_{\mathbf{x} \in C_i, \mathbf{z} \in C_j} \text{dist}(\mathbf{x}, \mathbf{z})$
均链接 (average-linkage)	取类间两两的平均距离

$$d_{\text{avg}}(C_i, C_j) = \frac{1}{|C_i| |C_j|} \sum_{x \in C_i} \sum_{z \in C_j} \text{dist}(x, z)$$

Remark

三种链接方法的特点总结：

- 单链接：包容性极强——容易合并,对噪声敏感
- 全链接：只要存在缺点就坚决不合并——对异常值敏感,簇更紧凑
- 均链接：从全局出发顾全大局——折中方案,鲁棒性较好

8.6.5 层次聚类方法的特点

特点类型	说明
无需指定簇数	层次聚类算法不要求我们指定聚类的数量,我们甚至可以选择哪个聚类看起来最好
对距离度量的选择	该算法对距离度量的选择不敏感
可恢复层次结构	当底层数据具有层次结构时,可以恢复层次结构
时间复杂度高	层次聚类的优点是以低效率为代价的,通常具有 $O(n^3)$ 的时间复杂度,与 k-means 和高斯混合模型的线性时间复杂度不同

层次聚类 vs. 原型聚类：

- 层次聚类：产生树状结构,可以灵活选择聚类粒度；但计算开销大,不适合大规模数据
- 原型聚类（K-means、LVQ、GMM）：产生“平坦的”划分；计算效率高,但需要预设簇数

8.6.6 延伸阅读：复杂网络的社区发现

近 20 年来,复杂网络 (complex network) 研究广泛,是复杂系统的抽象.人们发现复杂网络具有一定的社区结构,即复杂网络并不是一大批性质完全相同的节点随机连接在一起,也不是各种类型的节点之间不相关的随意链接,而是“乱中有序”——相同类型节点之间连接较多,构成一个一个的小社区 (Community) .

层次聚类的树状结构天然适合社区发现任务：通过在不同高度切割树状图,可以得到不同粒度的社区划分. (111)

9 特征选择

9.1 子集搜索与评价

9.1.1 特征的基本概念

[df]

Example

西瓜的特征分类

当前任务：判断西瓜是否是好瓜。

- 无关特征：颜色、纹理、触感
- 相关特征：根蒂、声音

特征选择的任务就是从这五个特征中选出与当前任务相关的特征（根蒂和声音）。

特征 (Feature)：描述物体的各种属性,例如形状、颜色、大小、材质等。

根据特征与当前学习任务的关系,可将特征分为三类：

1. 相关特征：对当前学习任务有用的属性。
2. 无关特征：与当前学习任务无关的属性。
3. 冗余特征：其所包含信息能由其他特征推演出来.若某个冗余特征对应学习任务的中间概念,则该冗余特征是有益的。

9.1.2 特征选择的定义与动机

[dg] 就是我们熟知的在训练模型之前的预处理步骤.通常,从给定的特征集合中选出任务相关特征子集,确保不丢失重要特征. 一般来说,对候选特征子集 $\Lambda \triangleq \{X_1, X_2, ..., X_n\}$,直接遍历所有子集是不可行的.一般分为子集搜索/子集评价两个子问题. 特征

选择 (Feature Selection)：从给定的特征集合中选出任务相关特征子集的过程. 它是重要的数据预处理过程,一般先进行特征选择,再训练学习器。

进行特征选择的主要原因有两个：

1. 减轻维数灾难：在少量属性上构建模型,避免高维空间中数据稀疏导致的困难。
2. 降低学习难度：去除不相关特征,留下关键因素,更易看清真相。

[dh] 特征选择必须确保不丢失重要特征,否则会导致模型性能下降.

9.1.3 特征选择的一般框架

遍历所有可能的子集（无先验知识）在计算上遭遇组合爆炸,不可行.特征选择的一般方法:

- 1. 产生初始候选子集: 从原始特征集合出发,生成一个候选特征子集.
- 2. 评价候选子集的好坏: 使用某种评价准则衡量该子集的优劣.
- 3. 基于评价结果产生下一个候选子集: 根据评价结果调整子集,进入下一轮迭代.

两个关键环节: 子集搜索和子集评价.

9.1.4 子集搜索策略

- 1. 每个人自成一派: 给定特征集合 $\{a_1, a_2, \dots, a_d\}$,可将每个特征看作一个候选子集,对这 d 个候选单特征子集进行评价,假定 $\{a_2\}$ 最优,于是将 $\{a_2\}$ 作为第一轮选定集.
- 2. 在上一步的选定集中加入一个特征,构成包含两个特征的候选子集,假定在这 $d - 1$ 个候选两特征子集中 $\{a_2, a_4\}$ 最优,且优于 $\{a_2\}$,于是将 $\{a_2, a_4\}$ 作为本轮的选定集.

常见的搜索策略有三种:

策略	方向	描述
前向搜索 (forward)	自下而上	从零开始,逐渐增加相关特征.若第 $k + 1$ 轮最优候选不如上一轮,则停止,将上一轮选定集作为结果.
后向搜索 (backward)	自上而下	从完整特征集合开始,每次尝试去掉一个无关特征,逐渐减少特征.
双向搜索 (bidirectional)	双向结合	前向与后向结合,每一轮逐渐增加选定相关特征,同时减少无关特征.

Remark

上述策略都是贪心的,因为它们仅考虑了使本轮选定集最优.例如在第三轮假定选择 a_5 优于 a_6 ,于是选定集为 $\{a_2, a_4, a_5\}$,然而在第四轮却可能是 $\{a_2, a_4, a_6, a_8\}$ 比所有的 $\{a_2, a_4, a_5, a_i\}$ 都更优.若不进行穷举搜索,则这样的问题无法避免.

9.1.5 子集评价方法

Definition 9.1.1

子集评价

假定样本属性均为离散型.给定数据集 D ,假定 D 中第 i 类样本所占的比例为 p_i ($i = 1, 2, \dots, |Y|$). 对属性子集 A ,根据其取值将 D 分成了 V 个子集 $\{D^1, D^2, \dots, D^V\}$,每个子集中的样本在 A 上取值相同,属性子集 A 的信息增益定义为:

[di] 信息增益越大,意味着特征子集 A 包含的有助于分类的信息越多.于是对每个候选特征子集,可基于训练数据集 D 来计算其信息增益,以此作为评价准则.

Example

想象你面前有一大筐山楂,里面混着好山楂和烂山楂.现在挑一个特征:“果皮是否有黑点”,把这一大筐山楂分成了两个小筐:有黑点筐、没黑点筐.

- 1. 如果分完之后,第一个筐里几乎全是烂的,第二个筐里几乎全是好的——特征太牛了!(混乱程度大幅下降,信息增益高).
- 2. 如果分完之后,两个筐里依然是好烂参半——这特征没卵用.(混乱程度没变,信息增益低).

$$\text{Gain}(A) = \text{Ent}(D) - \sum_{v=1}^V \left(\frac{|D^v|}{|D|} \right) \text{Ent}(D^v)$$

信息熵定义为:

$$\text{Ent}(D) = - \sum_{k=1}^{|Y|} p_k \log_2 p_k$$

Remark

特征子集 A 实际上确定了对数据集 D 的一个划分,每个划分区域对应着 A 上的一个取值,而样本标记信息 Y 则对应着对 D 的真实划分.通过估算这两个划分的差异,就能对 A 进行评价.与 Y 对应的划分的差异越小,则说明 A 越好.信息熵仅是判断这个差异的一种途径,其他能判断两个划分差异的机制都能用于特征子集评价.

9.1.6 子集搜索与评价的结合

将特征子集搜索机制与子集评价机制相结合,即可得到**特征选择方法**.
例如将前向搜索与信息熵相结合,这与**决策树算法**非常相似.事实上决策树可用于特征选择,树结点的划分属性所组成的集合就是选择出的特征子集.
其他的特征选择方法未必像决策树特征选择这么明显,但它们在本质上都是显式或隐式地**结合了某种(或多种)子集搜索机制和子集评价机制**.

9.2 过滤式选择

Definition 9.2.1

过滤式选择

先对数据集进行特征选择,然后再训练学习器,特征选择过程与**后续学习器无关**.这相当于先用特征选择过程对初始特征进行过滤,再用过滤后的特征来训练模型.

Relief (Relevant Features) 是一种著名的过滤式特征选择方法,该方法设计了一个**相关统计量**来度量特征的重要性.

9.2.1 Relief 方法

Relief 方法的统计量是一个**向量**,其每个分量分别对应于一个初始特征,而特征子集的重要性则是由于子集中每个特征所对应的相关统计量**分量之和**来决定.最终只需指定一个阈值 τ ,然后选择比 τ 大的相关统计量分量所对应的特征即可;也可指定欲选取的特征个数 k ,然后选择相关统计量**分量最大的 k 个特征**.

Algorithm 9.2.2

Relief 算法

输入: 训练集 $\{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_m, y_m)\}$.
输出: 各属性的相关统计量分量 δ^j .
过程:

- 对每个示例 $\mathbf{x}_i$:
 - 在 $\mathbf{x}_i$ 的同类样本中寻找其最近邻 $\mathbf{x}_{i,nh}$,称为**猜中近邻 (near-hit)**.
 - 从 $\mathbf{x}_i$ 的异类样本中寻找其最近邻 $\mathbf{x}_{i,nm}$,称为**猜错近邻 (near-miss)**.
- 相关统计量对应于属性 j 的分量为:

$$\delta^j = \sum_i \left[-\text{diff}(\mathbf{x}_i^j, \mathbf{x}_{i,nh}^j)^2 + \text{diff}(\mathbf{x}_i^j, \mathbf{x}_{i,nm}^j)^2 \right]$$

[dj]

- 若 $\mathbf{x}_i$ 与其猜中近邻 $\mathbf{x}_{i,nh}$ 在属性 j 上的距离**小于** $\mathbf{x}_i$ 与其猜错近邻 $\mathbf{x}_{i,nm}$ 的距离,则说明属性 j 对区分同类与异类样本是有益的,于是**增大**属性 j 所对应的统计量分量.
- 反之,若 $\mathbf{x}_i$ 与其猜中近邻 $\mathbf{x}_{i,nh}$ 在属性 j 上的距离**大于** $\mathbf{x}_i$ 与其猜错近邻 $\mathbf{x}_{i,nm}$ 的距离,则说明属性 j 起负面作用,于是**减小**属性 j 所对应的统计量分量.

其中 $\mathbf{x}_a^j$ 表示样本 $\mathbf{x}_a$ 在属性 j 上的取值. $\text{diff}(\mathbf{x}_a^j, \mathbf{x}_b^j)$ 取决于属性 j 的类型: 离散型取值 0 或 1,连续型取为差的绝对值(属性值已规范到 $[0, 1]$).

最后,对基于不同样本得到的估计结果进行平均,就得到各属性的相关统计量分量,**分量值越大,则对应属性的分类能力就越强**.

Example

Relief 算法手工计算示例

假设有 3 个水果样本,每个水果有两个特征:

- $j = 1$: 甜度(连续型,规范化到 $[0, 1]$)
- $j = 2$: 硬度(离散型,1 代表硬,0 代表软)

数据矩阵与标记如下:

样本	特征 1 (甜度)	特征 2 (硬度)	标签 y (种类)
$\mathbf{x}_1$	0.8	1	苹果
$\mathbf{x}_2$	0.7	1	苹果
$\mathbf{x}_3$	0.2	0	梨

初始化统计量分数: $\delta^1 = 0, \delta^2 = 0$. 之后轮流将每个样本当作“主角”:

1. 以 x_1 (苹果) 为主角:
 - 寻找近邻: 同类最近邻 $x_{1,nh} = x_2$, 异类最近邻 $x_{1,nm} = x_3$.
 - 特征 1 (甜度): 同类距离为 $|0.8 - 0.7| = 0.1$, 异类距离为 $|0.8 - 0.2| = 0.6$. 单项贡献为 $-(0.1)^2 + (0.6)^2 = +0.35$. 累加得 $\delta^1 = 0.35$.
 - 特征 2 (硬度): 同类相同 (距离为 0), 异类不同 (距离为 1). 单项贡献为 $-(0)^2 + (1)^2 = +1.0$. 累加得 $\delta^2 = 1.0$.
2. 以 x_2 (苹果) 为主角:
 - 寻找近邻: 同类最近邻 $x_{2,nh} = x_1$, 异类最近邻 $x_{2,nm} = x_3$.
 - 特征 1 (甜度): 同类距离为 $|0.7 - 0.8| = 0.1$, 异类距离为 $|0.7 - 0.2| = 0.5$. 单项贡献为 $-(0.1)^2 + (0.5)^2 = +0.24$. 累加得 $\delta^1 = 0.35 + 0.24 = 0.59$.
 - 特征 2 (硬度): 同类距离为 0, 异类距离为 1. 单项贡献为 $-(0)^2 + (1)^2 = +1.0$. 累加得 $\delta^2 = 1.0 + 1.0 = 2.0$.
3. 以 x_3 (梨) 为主角:
 - 寻找近邻: 设同类距离为 0. 对比 x_3 到异类样本的距离: 到 x_2 的甜度差为 0.5、硬度差为 1; 到 x_1 的甜度差为 0.6、硬度差为 1. 因此, 离 x_3 最近的异类样本是 x_2 , 即猜错近邻 $x_{3,nm} = x_2$.
 - 特征 1 (甜度): 同类距离为 0, 异类距离为 $|0.2 - 0.7| = 0.5$. 单项贡献为 $-(0)^2 + (0.5)^2 = +0.25$. 累加得 $\delta^1 = 0.59 + 0.25 = 0.84$.
 - 特征 2 (硬度): 同类距离为 0, 异类距离为 $|0 - 1| = 1$. 单项贡献为 $-(0)^2 + (1)^2 = +1.0$. 累加得 $\delta^2 = 2.0 + 1.0 = 3.0$.

最终平均发榜: 最后, 将累加的分数除以样本总数 $m = 3$ (求平均):

- 特征 1 (甜度): $\delta^1 = \frac{0.84}{3} = 0.28$
- 特征 2 (硬度): $\delta^2 = \frac{3.0}{3} = 1.00$

结论分析: 定量结果表明 $\delta^2 > \delta^1$, 说明在当前小数据集里, “硬度”的分类分辨能力高于“甜度”. 若只保留 1 个特征, 算法将保留“硬度”, 扔掉“甜度”.

9.2.2 Relief-F 扩展

Relief 是为二分类问题设计的, 其扩展变体 **Relief-F** 能处理多分类问题.

假定数据集 D 中的样本来自 $|Y|$ 个类别, 对示例 x_i , 若它属于第 k 类 ($k \in \{1, 2, \dots, |Y|\}$), 则 Relief-F 先在第 k 类的样本中寻找 x_i 的最近邻示例 $x_{i,nh}$ 并将其作为猜中近邻, 然后在第 k 类之外的每个类中找到一个 x_i 的最近邻示例作为猜错近邻, 记为 $x_{i,l,nm}$ ($l = 1, 2, \dots, |Y|; l \neq k$). 于是, 相关统计量对应于属性 j 的分量为:

$$\delta^j = \sum_i \left[-\text{diff}(x_i^j, x_{i,nh}^j)^2 + \sum_{l \neq k} (p_l \times \text{diff}(x_i^j, x_{i,l,nm}^j)^2) \right]$$

其中 p_l 为第 l 类的先验概率 (指的就是第 l 类样本在整个数据集中所占的“比例”).

9.3 包裹式选择

Definition 9.3.1

包裹式特征选择

直接把最终将要使用的学习器的性能作为特征子集的评价准则. 目的就是为给定学习器选择最有利于其性能、“量身定做”的特征子集.

一般而言, 由于包裹式特征选择方法直接针对给定学习器进行优化, 因此从最终学习器性能来看, 包裹式特征选择比过滤式特征选择更好; 另一方面, 由于在特征选择过程中需多次训练学习器, 因此计算开销通常大得多.

9.3.1 LVW 方法

LVW (Las Vegas Wrapper) 是一个典型的包裹式特征选择方法, 它在拉斯维加斯方法 (Las Vegas method) 框架下使用随机策略来进行子集搜索, 并以最终分类器的误差为特征子集评价准则.

Algorithm 9.3.2

LVW (Las Vegas Wrapper) 算法

输入: 数据集 D ; 特征集 A ; 学习算法 $\mathcal{L}$; 停止条件控制参数 T .

输出: 特征子集 A^* .

过程:

1. 初始化: 设置当前最小误差 $E = \infty$, 当前最小特征数 $d = |A|$, 最优特征子集 $A^* = A$, 循环控制计数器 $t = 0$.
2. 循环搜索 (当 $t < T$ 时重复执行以下步骤):
 - 随机产生候选子集: 随机产生一个特征子集 A' , 令 $d' = |A'|$.
 - 评估特征子集: 在数据集 D 上, 利用交叉验证法评估在特征子集 A' 下学习器 $\mathcal{L}$ 的分类误差 E' .

[dk] 评估特征子集时, 使用交叉验证法来估计学习器在所选特征子集上的误差. 只有当误差降低, 或误差相同但特征数减少时, 才会更新记录.

• **更新决策:**

- 若 $E' < E$ (即误差更小) ,或 $(E' = E) \wedge (d' < d)$ (即误差相当但特征数更少) ,则更新最优记录: 令 $A^* = A', E = E', d = d'$, 并将计数器 t 重置为 0.
- 否则, 保持当前最优记录不变, 将计数器自增: $t = t + 1$.

3. **输出结果:** 当计数器 t 达到停止参数 T 时停止循环, 输出最终特征子集 A^* .

Remark

由于 LVW 算法中特征子集搜索采用了**随机策略**, 而每次特征子集评价都需训练学习器, 计算开销很大, 因此算法设置了停止条件控制参数 T . 然而整个算法是基于拉斯维加斯方法框架, 若**初始特征数很多**, 则**算法可能运行很长时间都达不到停止条件**.

9.4 嵌入式选择与 L1 正则化

Definition 9.4.1

嵌入式特征选择

在过滤式和包裹式特征选择方法中, 特征选择过程与学习器训练过程有明显的分别. 与此不同, 嵌入式特征选择是将特征选择过程与学习器训练过程**融为一体**, 两者在同一个优化过程中完成, 即在学习器训练过程中自动地进行了特征选择.

给定数据集 $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$, 其中 $x \in \mathbb{R}^d$ 并且 $y \in \mathbb{R}$. 考虑最简单的**线性回归模型**, 以平方误差为损失函数, 则优化目标为:

$$\min_w \sum_{i=1}^m (y_i - w^T x_i)^2$$

当样本特征很多, 而样本数相对较少时, 式 (11.5) 很容易陷入过拟合. 为了**缓解过拟合问题**, 可对式 (11.5) 引入正则化项.

9.4.1 岭回归 (L2 正则化)

若使用 L_2 范数正则化, 则有:

$$\min_w \sum_{i=1}^m (y_i - w^T x_i)^2 + \lambda \|w\|_2^2$$

其中正则化参数 $\lambda > 0$, 式 (11.6) 称为**岭回归 (ridge regression)**. 通过引入 L_2 范数正则化, 确能显著降低过拟合的风险.

9.4.2 LASSO (L1 正则化)

将正则化项中的 L_2 范数替换为 L_p 范数, 若令 $p = 1$, 即采用 L_1 范数, 则有:

$$\min_w \sum_{i=1}^m (y_i - w^T x_i)^2 + \lambda \|w\|_1$$

其中正则化参数 $\lambda > 0$, 上式称为 **LASSO**.

L_1 范数和 L_2 范数正则化都有助于降低过拟合风险, 但前者还会带来一个额外的好处: 它比后者**更易于获得稀疏 (sparse) 解**, 即它求得的 w 会有**更少的非零分量**.

9.4.3 L1 正则化的几何解释

假定 x 仅有两个属性, 于是无论式 (11.6) 还是 (11.7) 解出的 w 都只有两个分量, 即 w_1, w_2 . 将其作为两个坐标轴, 然后在图中绘制出式 (11.6) 与 (11.7) 的第一项的等值线.

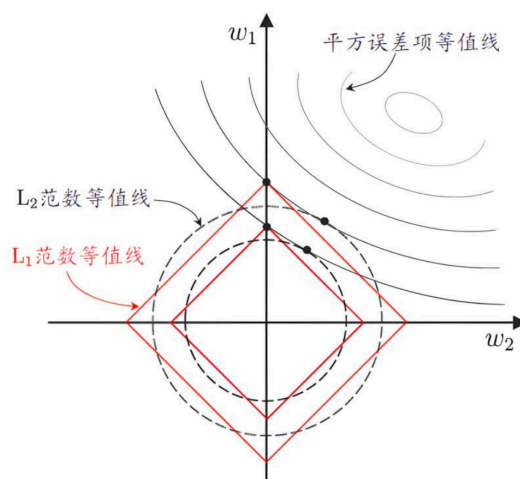


图 17 L1 与 L2 正则化的等值线对比

由图可看出,采用 L_1 范数比 L_2 范数更易于得到稀疏解.

注意到 w 取得稀疏解意味着初始的 d 个特征中仅有对应着 w 的非零分量的特征才会出现在最终模型中,于是求解 L_1 范数正则化的结果是得到了仅采用一部分初始特征的模型.换言之,基于 L_1 正则化的学习方法就是一种嵌入式特征选择方法,其特征选择过程与学习器训练过程融为一体,同时完成.

9.4.4 L1 正则化问题的求解: 近端梯度下降

[d1] 本节解决了一个矛盾: 我们想用梯度下降来求解含有 L_1 正则化的模型 (比如 LASSO), 但 L_1 范数 (绝对值之和) 在原点处是尖的, 根本没法求导.

L_1 正则化问题的求解可使用近端梯度下降 (Proximal Gradient Descent, 简称 PGD). 具体来说, 令 ∇ 表示微分算子, 对优化目标:

$$\min_{\mathbf{x}} f(\mathbf{x}) + \lambda \|\mathbf{x}\|_1$$

若 $f(\mathbf{x})$ 可导, 且 ∇f 满足 L -Lipschitz 条件, 即存在常数 $L > 0$ 使得:

$$\|\nabla f(\mathbf{x}') - \nabla f(\mathbf{x})\|_2^2 \leq L \|\mathbf{x}' - \mathbf{x}\|_2^2 \quad (\forall \mathbf{x}, \mathbf{x}')$$

则在 $\mathbf{x}_k$ 附近可将 $f(\mathbf{x})$ 通过二阶泰勒展开式近似为:

$$\begin{aligned} \hat{f}(\mathbf{x}) &\approx f(\mathbf{x}_k) + \langle \nabla f(\mathbf{x}_k), \mathbf{x} - \mathbf{x}_k \rangle + \frac{L}{2} \|\mathbf{x} - \mathbf{x}_k\|_2^2 \\ &= \frac{L}{2} \left\| \mathbf{x} - \left(\mathbf{x}_k - \frac{1}{L} \nabla f(\mathbf{x}_k) \right) \right\|_2^2 + \text{const} \end{aligned}$$

其中 “const” 是与 $\mathbf{x}$ 无关的常数, $\langle \cdot, \cdot \rangle$ 表示内积. 显然上式的最小值在如下 $\mathbf{x}_{k+1}$ 获得:

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \frac{1}{L} \nabla f(\mathbf{x}_k)$$

若通过梯度下降法对 $f(\mathbf{x})$ 进行最小化, 则每一步梯度下降迭代实际上等价于最小化二次函数 $\hat{f}(\mathbf{x})$. 将这个思想推广到式 (11.8), 则能类似地得到其每一步迭代应为:

$$\mathbf{x}_{k+1} = \arg \min_{\mathbf{x}} \frac{L}{2} \left\| \mathbf{x} - \left(\mathbf{x}_k - \frac{1}{L} \nabla f(\mathbf{x}_k) \right) \right\|_2^2 + \lambda \|\mathbf{x}\|_1$$

即在每一步对 $f(\mathbf{x})$ 进行梯度下降迭代的同时考虑 L_1 范数最小化.

对于上式, 可先计算 $\mathbf{z} = \mathbf{x}_k - \frac{1}{L} \nabla f(\mathbf{x}_k)$, 然后求解:

$$\mathbf{x}_{k+1} = \arg \min_{\mathbf{x}} \frac{L}{2} \|\mathbf{x} - \mathbf{z}\|_2^2 + \lambda \|\mathbf{x}\|_1$$

令 x_i 表示 $\mathbf{x}_i$ 的第 i 个分量, 将式 (11.13) 按分量展开可看出, 其中不存在 $x^i x^j$ ($i \neq j$) 这样的项, 即 $\mathbf{x}$ 的各分量互不影响, 于是式 (11.13) 有闭式解:

$$x_{k+1}^i = \begin{cases} z^i - \frac{\lambda}{L} & \text{if } \frac{\lambda}{L} < z^i \\ 0 & \text{if } |z^i| \leq \frac{\lambda}{L} \\ z^i + \frac{\lambda}{L} & \text{if } z^i < -\frac{\lambda}{L} \end{cases}$$

其中 x_{k+1}^i 与 z^i 分别是 $\mathbf{x}_{k+1}$ 与 $\mathbf{z}$ 的第 i 个分量. 因此, 通过 PGD 能使 LASSO 和其他基于 L_1 范数最小化的方法得以快速求解.

9.5 稀疏表示与字典学习

不妨把数据集 D 考虑成一个矩阵,其每行对应于一个样本,每列对应于一个特征.

1. 特征稀疏(列稀疏):特征选择所考虑的问题.矩阵中的许多列与当前学习任务无关,通过特征选择去除这些列,则学习器训练过程仅需在较小的矩阵上进行,学习任务的难度可能有所降低,涉及的计算和存储开销会减少,学得模型的可解释性也会提高.
2. 元素稀疏(点状稀疏): D 所对应的矩阵中存在很多零元素,但这些零元素并不是以整列、整行形式存在的,在不少现实应用中我们会遇到这样的情形(文档分类).

当样本具有稀疏表达形式时,对学习任务来说会有不少好处.例如线性支持向量机之所以能在文本数据上有很好的性能,恰是由于文本数据在使用上述的**字频表示**后具有高度的稀疏性,使大多数问题变得线性可分.

同时,稀疏样本并不会造成存储上的巨大负担,因为稀疏矩阵已有很多**高效的存储方法**.

若给定数据集 D 是**稠密的**,即普通非稀疏数据,能否将其**转化为稀疏表示**(sparse representation)形式,从而享有稀疏性所带来的好处呢?

Definition 9.5.1

字典学习与稀疏编码

学习出一个字典,为普通稠密表达的样本找到合适的字典,将样本转化为合适的稀疏表示形式,从而使学习任务得以简化,模型复杂度得以降低,通常称为**字典学习**(dictionary learning),亦称**稀疏编码**(sparse coding).

[dm] 这两个称谓稍有差别:字典学习更侧重于学得字典的过程,而稀疏编码则更侧重于对样本进行稀疏表达的过程.由于两者通常是在同一个优化求解过程中完成的,因此下面笼统地称为字典学习.

9.5.1 字典学习的基本形式

给定数据集 $\{x_1, x_2, \dots, x_m\}$,字典学习最简单的形式为:

$$\min_{B, \alpha_i} \sum_{i=1}^m \|x_i - B\alpha_i\|_2^2 + \lambda \sum_{i=1}^m \|\alpha_i\|_1$$

其中 $B \in \mathbb{R}^{d \times k}$ 为字典矩阵, k 称为字典的**词汇量**,通常由用户指定. $\alpha_i \in \mathbb{R}^k$ 则是样本 $x_i \in \mathbb{R}^d$ 的**稀疏表示**.第一项是重构误差(做 B 的列向量的线性组合),希望由 α_i 能很好地重构 x_i ,第二项则是希望 α_i **尽量稀疏**.

与 LASSO 相比,此式显然麻烦得多,但可以采用类似的**变量交替优化策略**求解.

9.5.2 变量交替优化

1. **固定住字典 B** .若将式按分量展开,可看出其中不涉及 $\alpha_i^u \alpha_i^v$ ($u \neq v$) 这样的交叉项,式子就变成了完全独立的 m 个 LASSO 问题.用 PGD (近端梯度下降) 求解下式,从而为每个样本 x_i 找到相应的 α_i :

$$\min_{\alpha_i} \|x_i - B\alpha_i\|_2^2 + \lambda \|\alpha_i\|_1$$

2. **固定住 α_i 来更新字典 B** : 式子变成了关于 B 的最小二乘法问题.

$$\min_B \|X - BA\|_F^2$$

其中 $X = (x_1, x_2, \dots, x_m) \in \mathbb{R}^{d \times m}$, $A = (\alpha_1, \alpha_2, \dots, \alpha_m) \in \mathbb{R}^{k \times m}$. $\|\cdot\|_F$ 是矩阵的 **Frobenius 范数**.

[dn] 把矩阵里所有元素的平方加起来.

有多种求解方法,常用的有基于**逐列更新策略**的 KSVD,每次我只更新字典里的第 i 个词(第 i 列 b_i).令 b_i 表示字典矩阵 B 的第 i 列, α^i 表示稀疏矩阵 A 的第 i 行,式(11.17)可重写为:

[do] E_i 就是公式里的 $X - \sum_{j \neq i} b_j \alpha_j^j$. 如果我们把字典里的第 i 个词扣掉,只用剩下的 $k-1$ 个词去拼这个世界,拼出来的效果和真实世界还差多少?

$$\begin{aligned} \min_B \|X - BA\|_F^2 &= \min_{b_i} \|X - \sum_{j=1}^k b_j \alpha_j^j\|_F^2 \\ &= \min_{b_i} \left\| X - \sum_{j \neq i} b_j \alpha_j^j \right\|_F^2 - b_i \alpha^i \|_F^2 \\ &= \min_{b_i} \|E_i - b_i \alpha^i\|_F^2 \end{aligned}$$

在更新字典的第 i 列时,其他各列都是固定的,因此 $E_i = X - \sum_{j \neq i} b_j \alpha_j^j$ 是固定的.于是原则上只需对 E_i 进行奇异值分解以取得最大奇异值所对应的正交向量.然而,直接对 E_i 进行奇异值分解会同时修改 b_i 和 α^i ,从而可能破坏 A 的稀疏性.

为避免发生这种情况,KSVD 对 E_i 和 α^i 进行专门处理: α^i 仅保留非零元素, E_i 则仅保留 b_i 与 α^i 的非零元素的乘积项,然后再进行奇异值分解,这样就保持了第一步所得到的稀疏性。

[dp] 假设第 i 个词是“硬度”。在第一步算完后,10,000 个水果里,其实只有 5 个水果用到了“硬度”这个词(对应的权重不是 0),剩下的 9,995 个水果根本不需要这个词(权重是 0)。

1. 瘦身残差矩阵: 把误差矩阵 E_i 砍一刀,只留下这 5 个水果对应的列,记为 E_i^R 。
2. 瘦身权重行: 把权重行 α^i 里原本就是 0 的元素扔掉,只留下那 5 个非零的权重,记为 α_R^i 。
3. 精准 SVD: 现在只对瘦身后的 E_i^R 做奇异值分解。

反复迭代上述两步,即可求得字典和样本的稀疏表示。

9.6 压缩感知

在现实任务中,我们常希望根据部分信息来恢复全部信息。例如在数据通讯中要将模拟信号转换为数字信号,根据奈奎斯特(Nyquist)采样定理,令采样频率达到模拟信号最高频率的两倍,则采样后的数字信号就保留了模拟信号的全部信息;换言之,由此获得的数字信号能精确重构原模拟信号。

然而,为了便于传输、存储,在实践中人们通常对采样的数字信号进行压缩,这有可能损失一些信息。而在信号传输过程中,由于信道出现丢包等问题,又可能损失部分信息。

那么,接收方基于收到的信号,能否精确地重构出原信号呢? 压缩感知(compressed sensing)为解决此类问题提供了新的思路。

9.6.1 压缩感知的基本模型

[dq] 压缩感知关注利用稀疏性,从部分观测样本恢复原始信号。

假定有长度为 m 的离散信号,不妨假定我们以远小于奈奎斯特采样定理要求的采样率进行采样,得到长度为 n 的采样后信号 $y, n \ll m$ (例如 $n=200, m=1000$),即:

$$y = \Phi x$$

其中 $\Phi \in \mathbb{R}^{n \times m}$ 是对信号的测量矩阵(采样规则),它确定了以什么频率进行采样以及如何将采样样本组成采样后的信号。

在已知离散信号 x 和测量矩阵 Φ 时要得到测量值 y 很容易。然而,若将测量值和测量矩阵传输出去,接收方一般不能还原出原始信号 x 。这是由于 $n \ll m$,因此 y 、 x 以及 Φ 组成的 $y = \Phi x$ 是一个欠定方程,无法轻易求出数值解,也就是说还原不出原始信号了。

既然直接求 x 无解,提出假设: 现实世界中的绝大多数自然信号,如果你对它做一个变换(比如离散余弦变换/小波变换 Ψ),它其实是非常稀疏的。

不妨假设存在某个线性变换 $\Psi \in \mathbb{R}^{m \times m}$ 使得

$$x = \Psi s$$

Ψ 是稀疏基(比如小波变换矩阵); s 是信号在稀疏基底下的系数向量,极其稀疏(包含大量的 0)。

于是 y 可表示为:

$$y = \Phi \Psi s = A s$$

其中 $A = \Phi \Psi \in \mathbb{R}^{n \times m}$ 。于是,若能根据 y 恢复出 s ,则可通过 $x = \Psi s$ 来恢复出信号 x 。

方程看起来依然是欠定的(200 个方程,1000 个未知的 s),但是这 1000 个未知的 s 里面,真正的非零项可能只有 10 个,其余 990 个项全部都是 0。既然绝大多数项都是 0,那么真正起作用的未知数实际上远远小于 200 个。这样一来,本来拥有“无数个解”的欠定方程,在“强迫解必须极度稀疏(L_1 范数最小化)”下,能在数学上收敛到唯一确定的、唯一的那个正确解。一旦我们通过这个方法把稀疏系数 s 算出来了,由于 $x = \Psi s$ 是已知的,我们就可以完美重构出高维的原始信号 x 。

A 类似于字典。

9.6.2 限定等距性(RIP)

前面我们学到,压缩感知可以用很少的方程解出稀疏的信号。但数学家必须回答一个严谨的问题: 是不是任意一个测量矩阵 A 都能办到这件事?

答案显然不是。这个矩阵必须满足一个硬性指标,就是限定等距性(RIP)。

Definition 9.6.1

限定等距性(Restricted Isometry Property, RIP)

[dr] 如果它满足 RIP,意思就是: 当一个稀疏信号 s 通过矩阵 A 投影到低维空间时,它的长度(或两个不同信号之间的距离)几乎不会发生缩水或扭曲。

RIP 描述一个矩阵在低维子空间上保留信号距离的能力,是判断稀疏信号能否通过线性测量矩阵准确恢复的重要性质。

对大小为 $n \times m$ ($n \ll m$) 的矩阵 A ,若存在常数 $\delta_k \in (0, 1)$ 使得对于任意向量 s 和 A 的所有子矩阵 $A_k \in \mathbb{R}^{n \times k}$ 有:

$$(1 - \delta_k) \|s\|_2^2 \leq \|A_k s\|_2^2 \leq (1 + \delta_k) \|s\|_2^2$$

其中 δ_k 是一个很小的正数: 高维的距离在压缩后,最多只在极其微小的范围内波动。

则称 A 满足 k 限定等距性(k -RIP)。

此时可通过下面的优化问题近乎完美地从 y 中恢复出稀疏信号 s ,进而恢复出 x :

$$\min_s \|s\|_0, \text{ s.t. } y = As$$

然而,上式涉及 L_0 范数最小化,这是个 NP 难问题.不过 L_1 范数最小化在一定条件下与 L_0 范数最小化问题共解,于是实际上只需关注:

$$\min_s \|s\|_1, \text{ s.t. } y = As$$

这样,压缩感知问题就可通过 L_1 范数最小化问题求解,将其转化为 LASSO 的等价形式通过近端梯度下降法求解.

9.6.3 矩阵补全

[ds] 工业界用它完成协同过滤推荐系统 (比如电影推荐) .

Definition 9.6.2

矩阵补全 (matrix completion)

通过观察到的矩阵部分已知值来推测未知值的一种技术,这种方法在大规模数据分析中被广泛应用,尤其是在数据稀疏性较高的场景下.

[dt] 有 1 万名用户 ($m = 10000$),平台上有 5000 部电影 ($n = 5000$) .

画一个巨大的用户对电影的评分矩阵 A ,大小是 10000×5000 .

任何一个用户撑死也就看过几百部电影,剩下的格子全是空白 (未知的评分).已有的已知评分集合就是公式里的 Ω .

任务: 把这些空白的格子填满 (补全矩阵 X),从而预测用户会给没看过的电影打几分,把高分电影推荐给他.

约束条件:

1. 补全过后的矩阵 X ,在那些原本就有评分的格子里必须和原观测矩阵 A 一模一样.
2. 优化目标 (min): 在满足上面条件的前提下,希望补全后的矩阵 X 的秩 (Rank) 尽可能小.也就是强迫矩阵去寻找背后最精简、最核心的“大盘规律” (低秩性),而不是胡乱瞎填.

矩阵补全的形式为:

$$\min_X \text{rank}(X), \text{ s.t. } (X)_{ij} = (A)_{ij}, \quad (i, j) \in \Omega$$

其中 X 表示需恢复的稀疏信号,rank(X) 表示矩阵 X 的秩, A 是已观测信号, Ω 是 A 中确定元素的下标集合.

直接最小化矩阵的“秩”仍然是个 NP 难问题;用核范数 $\|X\|_*$ 去替代 rank(X).

注意到 rank(X) 在集合 $\{X \in \mathbb{R}^{m \times n} : \|X\|_F^2 \leq 1\}$ 上的凸包是 X 的核范数 (nuclear norm) :

$$\|X\|_* = \sum_{j=1}^{\min\{m,n\}} \sigma_j(X)$$

其中 $\sigma_j(X)$ 表示 X 的奇异值,即矩阵的核范数为矩阵的奇异值之和

[du] 对角矩阵 Σ 里面排满了奇异值: $[\sigma_1, \sigma_2, \sigma_3, \dots]$. 把他们都加起来. 于是可通过最小化矩阵核范数来近似求解式 (11.24). 以下问题选择凸优化方法求解:

$$\min_X \|X\|_* \text{ s.t. } (X)_{ij} = (A)_{ij}, \quad (i, j) \in \Omega$$

10 半监督学习

10.1 未标记样本与半监督学习

10.1.1 背景与现实需求

在传统的监督学习中,我们依赖大量的有标记样本 (Labeled Samples) 来训练分类器.然而,在许多现实任务中,获取标记是一个需要消耗大量人力、物力和时间的过程.相比之下,未标记样本 (Unlabeled Samples) 的收集则显得极其容易且成本低廉.

例如,在医学影像分析中,医院可以轻易获得数以万计的医学影像,但是如果希望医学专家对这些影像中的病灶进行逐一精确标识,则是不现实的.

Definition 10.1.1

半监督学习 (Semi-Supervised Learning)

让学习器不依赖外界交互,自动地利用未标记样本来提升学习性能的学习方法,即为半监督学习.

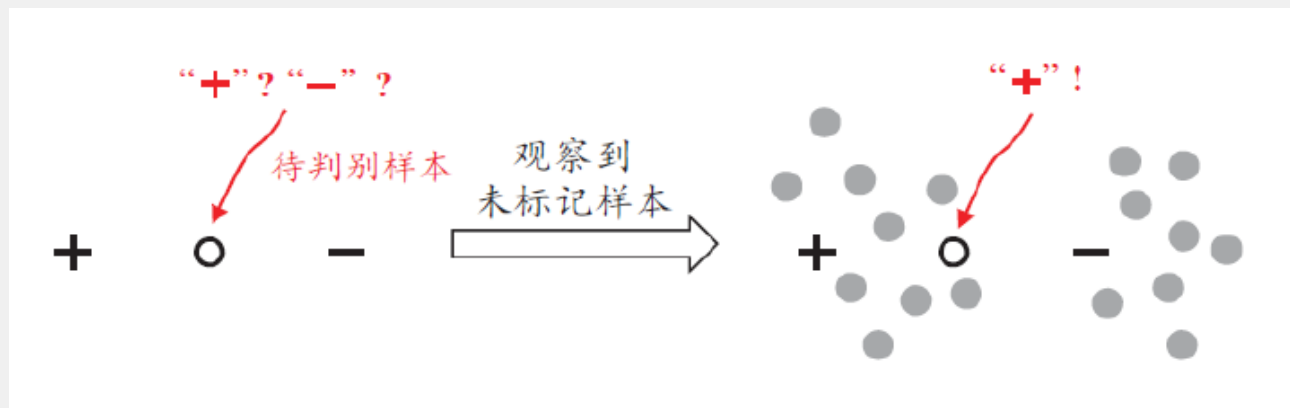
10.1.2 未标记样本的效用与直观理解

[dv]

Example

未标记样本的撮合效用

假设在二维空间中只有一个正样本 $+$ 和一个负样本 $-$ 。如果此时输入一个待预测样本 x 正好位于两者的中点,仅基于监督学习只能随机猜测。

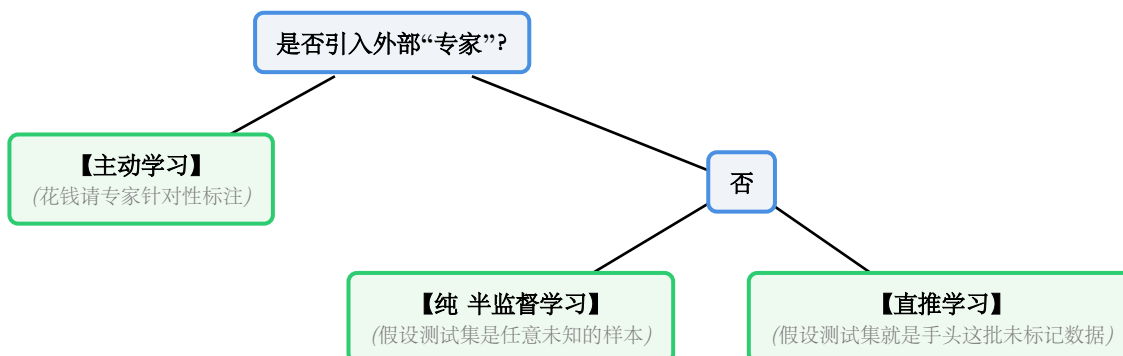


如果我们能观察到大量灰色的未标记样本,根据它们揭示的密度结构,我们可以非常笃定地将中点处的样本判别为正类。

未标记样本虽然没有直接的标记信息,但如果它们与有标记样本是从相同的数据源独立同分布采样而来,则它们所包含的关于数据分布的信息对建立模型将大有裨益。

为了更好地理解未标记样本的利用方式,我们将相关的方法分类并与主动学习进行对比:

1. **主动学习 (Active Learning)**: 引入外部“标注者”(如专家)进行交互.它首先用少量标记数据训练模型,然后利用某种挑选策略在未标记样本中筛选出“最不确定”或“信息量最大”的样本,向标注者查询标记,最后用新标记的样本更新模型.其主动权在算法,旨在用尽量少的标注代价获取尽可能好的性能.
2. **(纯) 半监督学习 (Pure Semi-Supervised Learning)**: 自动利用未标记样本,不需要与外界交互.它是基于**开放世界假设 (Open World Assumption)**,即假定训练集中的未标记样本并非最终的待预测样本,目标是获得对未知新样本的最优泛化性能.
3. **直推学习 (Transductive Learning)**: 同样不与外界交互.但它是基于**封闭世界假设 (Closed World Assumption)**,假定学习过程中观察到的未标记样本恰好就是待预测的测试数据,目标是仅在这些特定的未标记样本上取得最优预测效果.



10.1.3 未标记样本利用的核心假设

要利用未标记样本,必然要做一些将数据分布信息与类别标记相联系的假设.最基础的假设为“相似的样本拥有相似的输出”.具体细分为两种:

- **聚类假设 (Cluster Assumption)**: 假设数据存在簇结构,同一个簇的样本属于同一个类别.这意味着决策边界应当尽量穿过数据稀疏的区域,而不是穿过高密度的簇内部.
- **流形假设 (Manifold Assumption)**: 假设数据分布在一个流形结构上,邻近的样本拥有相似的输出值.邻近程度常用相似程度来刻画.流形假设可看作聚类假设的推广,但它对输出值没有限制(可以是连续值),适用范围更广,可用于回归等更多学习任务.

10.2 生成式方法 (Generative Methods)

10.2.1 基本假设

Definition 10.2.1

生成式半监督学习

直接基于生成式模型的方法.此类方法假设所有数据(有标记和无标记)都是由同一个潜在的生成式模型生成的.

通过潜在模型的参数,我们可以将未标记数据与学习目标联系起来.未标记数据的标记可以看作是模型的缺失参数,通常可基于 **EM 算法 (Expectation-Maximization)** 进行极大似然估计求解.

10.2.2 高斯混合模型 (GMM) 的引入

假定样本 $\mathbf{x}$ 的真实类别标记 $y \in \mathcal{Y} = \{1, 2, \dots, N\}$. 数据样本是基于如下高斯混合模型生成:

$$p(\mathbf{x}) = \sum_{i=1}^N \alpha_i \cdot p(\mathbf{x} \mid \mu_i, \Sigma_i)$$

其中,混合系数 $\alpha_i \geq 0$ 且 $\sum_{i=1}^N \alpha_i = 1$; μ_i 和 Σ_i 分别为第 i 个高斯混合成分的均值向量和协方差矩阵.

令 $f(\mathbf{x}) \in \mathcal{Y}$ 表示模型对 $\mathbf{x}$ 的预测标记, $\Theta \in \{1, 2, \dots, N\}$ 表示样本隶属的高斯混合成分. 根据最大化后验概率,分类规则定义为:

$$\begin{aligned} f(\mathbf{x}) &= \arg \max_{j \in \mathcal{Y}} p(y = j \mid \mathbf{x}) \\ &= \arg \max_{j \in \mathcal{Y}} \sum_{i=1}^N p(y = j \mid \Theta = i, \mathbf{x}) \cdot p(\Theta = i \mid \mathbf{x}) \end{aligned}$$

不失一般性,若假定第 i 个类别对应于第 i 个高斯混合成分,即当且仅当 $i = j$ 时 $p(y = j \mid \Theta = i, \mathbf{x}) = 1$, 否则为 0. 此时上式简化为:

$$f(\mathbf{x}) = \arg \max_{j \in \mathcal{Y}} p(\Theta = j \mid \mathbf{x})$$

其中,样本由第 i 个高斯混合成分生成的后验概率为:

$$p(\Theta = i \mid \mathbf{x}) = \frac{\alpha_i \cdot p(\mathbf{x} \mid \mu_i, \Sigma_i)}{\sum_{k=1}^N \alpha_k \cdot p(\mathbf{x} \mid \mu_k, \Sigma_k)}$$

[dw] 在式 (13.2) 中:

- 估计类别与混合成分的对应该率 $p(y = j \mid \Theta = i, \mathbf{x})$ 时必须知道标记,故仅能使用有标记数据.
- 估计混合成分后验概率 $p(\Theta = i \mid \mathbf{x})$ 时不涉及标记,因此可以同时利用有标记和未标记数据.引入海量未标记样本将极大地提高对 $p(\Theta = i \mid \mathbf{x})$ 估计的准确性.

10.2.3 联合对数似然估计

给定有标记样本集 $D_l = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_l, y_l)\}$ 和未标记样本集 $D_u = \{\mathbf{x}_{l+1}, \dots, \mathbf{x}_{l+u}\}$, $l \ll u$. 假设样本独立同分布,则高斯混合模型参数的联合对数似然为:

$$\begin{aligned} \mathbb{L}(D_l \cup D_u) &= \sum_{(\mathbf{x}_j, y_j) \in D_l} \ln \left(\sum_{i=1}^N \alpha_i \cdot p(\mathbf{x}_j \mid \mu_i, \Sigma_i) \cdot p(y_j \mid \Theta = i, \mathbf{x}_j) \right) \\ &\quad + \sum_{\mathbf{x}_j \in D_u} \ln \left(\sum_{i=1}^N \alpha_i \cdot p(\mathbf{x}_j \mid \mu_i, \Sigma_i) \right) \end{aligned}$$

[dx] 上式第一项为基于有标记数据的**有监督项**: 模型不仅要解释为什么这里有个点,还要解释为什么这个点是个猫.要算的是联合概率 $p(\mathbf{x}_j, y_j)$.

- $\alpha_i \cdot p(\mathbf{x}_j \mid \mu_i, \Sigma_i)$: 这和前面一样,是第 i 个高斯成分产生出这个点 $\mathbf{x}_j$ 的概率.
- $p(y_j \mid \Theta = i, \mathbf{x}_j)$: 这是一个修正因子:“如果这个点确实是由第 i 个高斯成分产生的,那么这个成分恰好是 y_j (猫) 的概率是多少?”

第二项为基于未标记数据的**无监督项**.

- 由第 i 个高斯生成的概率是 $p(\mathbf{x}_j \mid \mu_i, \Sigma_i)$.
- 第 i 个高斯的先验概率是 α_i .
- 两者相乘,再把所有 N 个高斯成分的结果加起来 ($\sum_{i=1}^N$),就得到了这个点在整个高斯混合模型里的总出场概率.

Remark

最普遍的数学推导中,我们还没做“一个高斯成分完美对应一个类别”的强假设.

10.2.4 EM 算法求解步骤

高斯混合模型的参数估计可以通过 EM 算法迭代更新求解:

- E 步 (Expectation)**: 根据当前模型参数计算未标记样本 $\mathbf{x}_j \in D_u$ 属于各高斯混合成分的后验概率:

$$\gamma_{ji} = \frac{\alpha_i \cdot p(\mathbf{x}_j \mid \mu_i, \Sigma_i)}{\sum_{k=1}^N \alpha_k \cdot p(\mathbf{x}_j \mid \mu_k, \Sigma_k)}$$

- M 步 (Maximization)**: 更新模型参数.令 l_i 表示第 i 类的有标记样本数目 (即 $l_i = \sum_{(\mathbf{x}_j, y_j) \in D_l} \mathbb{I}(y_j = i)$), 更新公式为:

$$\mu_i = \frac{1}{\sum_{\mathbf{x}_j \in D_u} \gamma_{ji} + l_i} \left(\sum_{\mathbf{x}_j \in D_u} \gamma_{ji} \mathbf{x}_j + \sum_{(\mathbf{x}_j, y_j) \in D_l \wedge y_j = i} \mathbf{x}_j \right)$$

[dy] 本质上是一个加权平均:

1. 右边项 (硬证据): $\sum \mathbf{x}_j$. 这是正牌的有标记数据 (D_l). 如果标签写着它是猫 ($y_j = i$), 那它的权重就是整整的 1.
2. 左边项 (软证据): $\sum \gamma_{ji} \mathbf{x}_j$. 这是未标记数据 (D_u). 若只有 0.8 的概率是猫, 那就只贡献 0.8 个点的力量.
3. 分母 (总贡献点数): $\sum \gamma_{ji} + l_i$. 把有标记的猫的数量 (l_i) 和未标记数据里凑出来的“虚拟猫”的数量 ($\sum \gamma_{ji}$) 加在一起, 作为总分母.

方差 Σ_i 和混合系数 α_i 也是用这种“真猫”与“虚拟猫”合力的方式更新出来的.

$$\begin{aligned} \Sigma_i &= \frac{1}{\sum_{\mathbf{x}_j \in D_u} \gamma_{ji} + l_i} \left(\sum_{\mathbf{x}_j \in D_u} \gamma_{ji} (\mathbf{x}_j - \mu_i)(\mathbf{x}_j - \mu_i)^T \right. \\ &\quad \left. + \sum_{(\mathbf{x}_j, y_j) \in D_l \wedge y_j = i} (\mathbf{x}_j - \mu_i)(\mathbf{x}_j - \mu_i)^T \right) \\ \alpha_i &= \frac{1}{m} \left(\sum_{\mathbf{x}_j \in D_u} \gamma_{ji} + l_i \right) \end{aligned}$$

其中 $m = l + u$ 为总样本数.

Algorithm 10.2.2

生成式半监督高斯混合模型 EM 算法

输入: 有标记样本集 $D_l = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_l, y_l)\}$;

未标记样本集 $D_u = \{\mathbf{x}_{l+1}, \mathbf{x}_{l+2}, \dots, \mathbf{x}_{l+u}\}$;

高斯混合成分个数 N .

输出: 高斯混合模型参数 $\alpha_i, \mu_i, \Sigma_i$ ($i = 1, 2, \dots, N$).

过程:

1. 初始化高斯混合模型参数 $\alpha_i, \mu_i, \Sigma_i$ ($i = 1, 2, \dots, N$).
2. 循环迭代直至收敛 (如对数似然函数值不再显著增加):
 - E 步 (Expectation): 根据当前模型参数计算每个未标记样本 $\mathbf{x}_j \in D_u$ 属于各高斯混合成分的后验概率 (软标签):

$$\gamma_{ji} = \frac{\alpha_i \cdot p(\mathbf{x}_j | \mu_i, \Sigma_i)}{\sum_{k=1}^N \alpha_k \cdot p(\mathbf{x}_j | \mu_k, \Sigma_k)}$$

- M 步 (Maximization): 利用有标记数据与未标记数据的组合信息更新模型参数. 令 l_i 表示第 i 类的有标记样本数目 (即 $l_i = \sum_{(\mathbf{x}_j, y_j) \in D_l} \mathbb{I}(y_j = i)$), 对每个成分 $i = 1, 2, \dots, N$ 更新:
 - ▶ 更新均值向量 μ_i 、协方差矩阵 Σ_i ;
 - ▶ 更新混合系数 α_i (其中总样本数 $m = l + u$).

10.2.5 生成式方法的特点

- 推广: 若将高斯混合模型换成混合专家模型、朴素贝叶斯模型等, 即可推导出其他的生成式半监督学习方法.
- 优点: 算法简单易实现, 在标记样本极少时由于先验结构强, 往往表现优异.
- 局限性: 此类方法极其依赖模型假设的准确性. 若假设的生成式模型与真实数据分布不吻合, 利用未标记数据反而会降低模型的泛化性能. 因此, 除非拥有充分可靠的领域先验知识, 否则在现实任务中很难直接保证假设的精确度.

10.3 半监督支持向量机 (S3VM)

10.3.1 TSVM 基本原理

半监督支持向量机中最著名的是 TSVM (Transductive Support Vector Machine, 直推支持向量机). 与标准 SVM 一样, TSVM 也是针对二分类问题的学习方法.

Remark

S3VM 与标准 SVM 决策边界的差异 (低密度间隔划分)

- 标准 SVM 仅寻找使有标记样本间隔最大化的超平面, 这可能会把超平面直接切在未标记样本密集的区域.
- TSVM 则同时考虑有标记和未标记样本, 它试图穿过未标记样本最稀疏的地方 (低密度区域), 并让所有样本 (包括带有伪标记的未标记样本) 的整体间隔最大化.

TSVM 试图考虑对未标记样本进行各种可能的标记指派 (Label Assignment), 在所有可能的结果中, 寻求一个在所有样本上间隔最大化的划分超平面. 确定划分超平面后, 未标记样本的指派标记即为最终预测结果.

10.3.2 TSVM 形式化优化目标

给定 $D_l = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_l, y_l)\}$, 其中 $y_i \in \{-1, +1\}$, 以及未标记样本集 $D_u = \{\mathbf{x}_{l+1}, \dots, \mathbf{x}_{l+u}\}$, $l \ll u$. TSVM 的目标是尝试为 D_u 给出预测标记 $\hat{\mathbf{y}} = (\hat{y}_{l+1}, \dots, \hat{y}_{l+u})$ ($\hat{y}_j \in \{-1, +1\}$) 以最小化如下目标函数:

$$\begin{aligned} \min_{\mathbf{w}, b, \hat{\mathbf{y}}, \boldsymbol{\xi}} \quad & \frac{1}{2} \|\mathbf{w}\|_2^2 + C_l \sum_{i=1}^l \xi_i + C_u \sum_{i=l+1}^{l+u} \xi_i \\ \text{s.t.} \quad & y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i, \quad i = 1, 2, \dots, l \\ & \hat{y}_j(\mathbf{w}^T \mathbf{x}_j + b) \geq 1 - \xi_j, \quad j = l+1, l+2, \dots, l+u \\ & \xi_i \geq 0, \quad i = 1, 2, \dots, l+u \end{aligned} \quad (13.9)$$

其中, $(\mathbf{w}, b)$ 确定了划分超平面, $\boldsymbol{\xi}$ 为松弛向量; C_l 和 C_u 是用户指定的折中参数, 分别用于控制有标记与未标记样本在模型惩罚项中的重要程度.

由于尝试所有可能的 $\hat{\mathbf{y}}$ 是一个穷举的过程 (复杂度达 2^u), 仅当 u 很小时才可能直接求解. 在一般情形下, 必须考虑高效的局部搜索近似策略.

10.3.3 类别不平衡问题及其改进

在对未标记样本进行指派及调整的过程中, 极易出现指派的类别严重失衡的问题, 进而干扰 SVM 的正常训练.

为了减轻类别不平衡所造成的不利影响, 可对算法进行改进: 将优化目标中的 $C_u \sum_{i=l+1}^{l+u} \xi_i$ 项拆分为 C_u^+ 和 C_u^- 两项, 分别对应指派为正例和反例的未标记样本:

$$C_u \sum_{i=l+1}^{l+u} \xi_i \rightarrow C_u^+ \sum_{i: \hat{y}_i=1} \xi_i + C_u^- \sum_{j: \hat{y}_j=-1} \xi_j$$

并在初始化时令:

$$C_u^+ = \frac{u_-}{u_+} C_u^-$$

其中, u_+ 与 u_- 分别为基于伪标记指派为正、反例的未标记样本数.

10.3.4 TSVM 算法流程与标记置换机制

TSVM 通过局部搜索迭代寻找近似解.

Algorithm 10.3.1

TSVM 算法

输入: 有标记样本集 D_l ; 未标记样本集 D_u ; 折中参数 C_l, C_u .

输出: 未标记样本的预测结果 $\hat{\mathbf{y}}$.

过程:

1. 用 D_l 训练一个标准 SVM _{l} ;
2. 用 SVM _{l} 对 D_u 中的样本进行预测, 得到初始伪标记 $\hat{\mathbf{y}} = (\hat{y}_{l+1}, \dots, \hat{y}_{l+u})$;
3. 初始化参数 $C_u \ll C_l$;
4. **while** $C_u < C_l$ **do**
5. 基于 $D_l, D_u, \hat{\mathbf{y}}, C_l, C_u$ 求解式 (13.9), 得到 $(\mathbf{w}, b), \boldsymbol{\xi}$;
6. **while** $\exists i, j$ 满足 $(\hat{y}_i \hat{y}_j < 0) \wedge (\xi_i > 0) \wedge (\xi_j > 0) \wedge (\xi_i + \xi_j > 2)$ **do**

[dz] 如果在某轮求解中, 存在一对未标记样本 $\text{bold}(\mathbf{x})_i$ 和 $\text{bold}(\mathbf{x})_j$:

1. 它们被指派了不同的伪标记 ($\text{hat}(\mathbf{y})_i \text{hat}(\mathbf{y})_j < 0$);
2. 且它们的松弛变量满足 $\xi_i > 0$ 和 $\xi_j > 0$;
3. 并且它们的松弛变量之和 $\xi_i + \xi_j > 2$.

这说明它们在当前划分面下都位于错误的一侧, 且错误非常严重. 如果将它们的标记对调重新训练, 可以严格保证式目标函数值单调下降.

7. $\hat{y}_i = -\hat{y}_i$;
8. $\hat{y}_j = -\hat{y}_j$;
9. 基于 $D_l, D_u, \hat{\mathbf{y}}, C_l, C_u$ 重新求解式 (13.9), 得到新参数 $(\mathbf{w}, b), \boldsymbol{\xi}$;
10. **end while**
11. $C_u = \min(2C_u, C_l)$;
12. **end while**

尽管如此, 搜索每一对可能出错的伪标记样本并进行调整, 计算开销仍然极大. 因此, 半监督支持向量机的研究重点在于如何设计更加高效的大规模优化求解策略.

10.4 图半监督学习 (Graph-Based Semi-Supervised Learning)

10.4.1 基本思想与图映射关系

图半监督学习方法将数据集映射为一个图 $G = (V, E)$:

- 数据集中每个样本对应于图中的一个结点 $v \in V$.
- 若两个样本之间的相似度很高,则对应的结点之间存在一条边,边的强度(权值)正比于样本之间的相似度.
- 有标记样本对应的结点已被染色(有初始标签),未标记样本对应的结点尚未染色.半监督学习的过程就是颜色在图上顺着边不断扩散、传播的动态过程.

10.4.2 亲和矩阵与图拉普拉斯矩阵

给定数据集,我们首先构建亲和矩阵 W .通常基于高斯函数定义为:

$$W_{ij} = \begin{cases} \exp\left(-\frac{\|x_i - x_j\|_2^2}{2\sigma^2}\right) & \text{if } i \neq j \\ 0 & \text{otherwise} \end{cases}$$

其中 $\sigma > 0$ 是用户指定的高斯函数带宽参数.度矩阵 $D = \text{diag}(d_1, \dots, d_{l+u})$ 是一个对角矩阵,其对角线元素为 $d_i = \sum_{j=1}^{l+u} W_{ij}$.

10.4.3 二分类标记传播闭式解

假定我们希望学得一个实值函数 $f: V \rightarrow \mathbb{R}$,对应的分类规则为 $y_i = \text{sign}(f(x_i))$.直观上,相似的样本应具有相似的标记(为了满足流形假设(或平滑性约束)),故可定义关于 f 的能量函数:

[ea] 可以把这个能量理解为“社会舆论压力”或“朋友圈的不和谐度”.

如果我和你是死党 (W_{ij} 很大),但是我的观点和你的观点差了十万八千里 ($(f_i - f_j)^2$ 很大),那我们俩在一起就会天天吵架,整个网络的“不和谐能量”就会飙升.

算法的目标是:最小化这个能量.也就是强迫关系铁的人,必须拥有相似的观点.所谓的图拉普拉斯矩阵 $L = D - W$,在数学上没有魔法,它纯粹是一个优雅的包装盒.有了它,我们就能把全网所有人两两吵架的能量,用一个极简的矩阵乘法 $f^T L f$ 表示出来,方便后续求导.

$$\begin{aligned} E(f) &= \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m (W)_{ij} (f(x_i) - f(x_j))^2 \\ &= \frac{1}{2} \left(\sum_{i=1}^m d_i f^2(x_i) + \sum_{j=1}^m d_j f^2(x_j) - 2 \sum_{i=1}^m \sum_{j=1}^m (W)_{ij} f(x_i) f(x_j) \right) \\ &= \sum_{i=1}^m d_i f^2(x_i) - \sum_{i=1}^m \sum_{j=1}^m (W)_{ij} f(x_i) f(x_j) \\ &= f^T (D - W) f \end{aligned}$$

其中 $L = D - W$ 为拉普拉斯矩阵.注意,这里的 $D \triangleq \text{diag}\{d_i\}$, $d_i = \sum_{j=1}^n (W)_{ij}$.将函数 f 划分为有标记部分 f_l 和未标记部分 f_u .以第 l 行与第 l 列为界,采用分块矩阵表示方式:

$$W = \begin{pmatrix} W_{ll} & W_{lu} \\ W_{ul} & W_{uu} \end{pmatrix}, \quad D = \begin{pmatrix} D_{ll} & 0_{lu} \\ 0_{ul} & D_{uu} \end{pmatrix}$$

能量函数 $E(f)$ 可重写为:

$$E(f) = \begin{pmatrix} f_l^T & f_u^T \end{pmatrix} \left(\begin{pmatrix} D_{ll} & 0_{lu} \\ 0_{ul} & D_{uu} \end{pmatrix} - \begin{pmatrix} W_{ll} & W_{lu} \\ W_{ul} & W_{uu} \end{pmatrix} \right) \begin{pmatrix} f_l \\ f_u \end{pmatrix}$$

最小化能量的函数在有标记样本上必须满足 $f_l = y_l$,为了求在未标记样本上的输出,我们对 f_u 求偏导并令其为 0:

$$\begin{aligned} \frac{\partial E(f)}{\partial f_u} &= 0 \\ \Rightarrow (D_{uu} - W_{uu})f_u - W_{ul}f_l &= 0 \\ \Rightarrow f_u &= (D_{uu} - W_{uu})^{-1} W_{ul}f_l \end{aligned}$$

令标记传播矩阵 $P = D^{-1}W$,其对应分块为 $P_{uu} = D_{uu}^{-1}W_{uu}$, $P_{ul} = D_{uu}^{-1}W_{ul}$.上式可进一步重写为:

$$f_u = (I - P_{uu})^{-1} P_{ul} f_l$$

[eb] 把人分成两拨:

- 铁杆派 (f_l): 这帮人有初始标签,立场坚定,打死也不改颜色.
- 墙头草 (f_u): 这帮人没标签,他们的颜色完全取决于周围的朋友.
 - 为了让全网的吵架能量最小,这群墙头草最终会达到一个平衡: 每个墙头草的颜色,正好等于他所有邻居颜色的加权平均.

10.4.4 多分类标记传播与规范拉普拉斯矩阵

在多分类任务中,定义非负标记矩阵 $F \in \mathbb{R}^{(l+u) \times |Y|}$, 其第 i 行向量对应于示例 x_i 的类别后验概率. 首先将 F 初始化为 Y , 其中:

$$Y_{ij} = \begin{cases} 1 & \text{if } (1 \leq i \leq l) \wedge (y_i = j) \\ 0 & \text{otherwise} \end{cases}$$

其前 l 行为已知有标记样本的 One-hot 编码, 剩余行为 0. 基于度矩阵构建标记传播矩阵 $S = D^{-\frac{1}{2}} W D^{-\frac{1}{2}}$. 算法采用如下迭代计算式:

$$F(t+1) = \alpha S F(t) + (1-\alpha) Y$$

其中, $\alpha \in (0, 1)$ 用于控制标记传播项与初始化项的折中比重.

[ec]

- α 趋近于 1 时: 模型极其信任网络中“朋友之间的小道消息传话”. 颜色在图上会扩散得非常剧烈、非常远, 但缺点是传着传着, 最初那几个有标记样本的真实标签 (正确答案) 就会被稀释、甚至被洗脑洗掉.
- α 趋近于 0 时: 模型极其保守, 只认死理. 大家几乎不怎么传话, 未标记样本很难从周围邻居那里学到有效的分布信息.

由于 S 的特征值绝对值不大于 1, 当 $t \rightarrow \infty$ 时, 迭代必收敛. 收敛解为:

$$F^* = (1-\alpha)(I - \alpha S)^{-1} Y$$

Remark

正则化框架等价性

上述迭代算法的收敛解, 在数学上等价于如下正则化框架的最优解:

$$\min_F \frac{1}{2} \left(\sum_{i,j=1}^{l+u} W_{ij} \left\| \frac{1}{\sqrt{d_i}} F_i - \frac{1}{\sqrt{d_j}} F_j \right\|_2^2 + \mu \sum_{i=1}^l \|F_i - Y_i\|_2^2 \right)$$

其中第一项为流形约束 (强迫近邻样本具有相似的归一化表示), 第二项为拟合约束 (强迫有标记样本的预测值逼近真实标签), $\mu = \frac{1-\alpha}{\alpha}$ 为正则化参数.

10.4.5 图半监督学习的局限性

1. **存储和计算开销巨大**: 对于拥有 m 个样本的数据集, 亲和矩阵的规模达 $O(m^2)$. 当数据规模较大时, 矩阵逆运算和存储都难以直接承受.
2. **外部样本预测困难 (Out-of-Sample Issue)**: 构图时无法预知未出现的测试样本. 当接收到新样本时, 要么必须将其加入原数据集重新构建整个图并重新运行标记传播, 要么必须额外引入一个预测机制 (如归纳学习器) 来逼近传播结果.

10.5 基于分歧的方法 (Disagreement-Based Methods)

10.5.1 协同训练与多视图数据

基于分歧的方法使用多个学习器, 而学习器之间的分歧 (Disagreement) 对未标记数据的利用至关重要.

其代表是 协同训练 (Co-training) 算法. 协同训练最初是针对多视图 (Multi-view) 数据设计的, 是多视图学习 (Multi-view learning) 的里程碑.

[ed]

Example

电影数据的多视图性质

在现实生活中, 一个数据对象往往拥有多个属性集, 每个属性集构成一个视图 (View):

- 图像画面特征集 (视图 1)
- 声音频域特征集 (视图 2)
- 字幕文本特征集 (视图 3)

这些视图从不同侧面描述了同一部电影, 它们是相通且互补的.

在多视图数据中, 每个样本可以表示为 $((x^1, x^2), y)$, 其中 x^1 和 x^2 分别是样本在两个不同视图下的属性向量.

10.5.2 协同训练算法流程

Remark

协同训练的基本思路是: 在每个视图上训练一个独立的分类器, 然后让各自的分类器去 **挑选置信度最高的未标记样本**, 将其预测结果赋予 **伪标记**, 并 **提供给另一个视图下的分类器** 作为新增的有标记样本进行训练.

Algorithm 10.5.1

协同训练 (Co-training) 算法

输入: 有视图标记样本集 D_l ; 未标记样本集 D_u ; 缓冲池大小 s ; 每轮挑选正、反例数 p, n ; 基学习算法 $\mathcal{L}$; 训练轮数 T .

输出: 视图分类器 h_1, h_2 .

过程:

1. 从 D_u 中随机抽取 s 个样本构成缓冲池 $D_s, D_u = D_u \setminus D_s$;

[ee] 为什么不直接从海量的未标记集 D_u 里挑样本,非要搞一个大小为 s 的“缓冲池 D_s ”?

为了防止标签漂移 (Label Drift) 和类别不平衡.如果直接从几百万数据的 D_u 里挑,分类器可能每轮挑出来的全都是长得极其相似的、极端的“超级大正例”.这样不仅无法给另一个分类器带来新知识,还会让数据集瞬间失去平衡.

搞一个小的缓冲池,强迫分类器只能在这一小撮电影里挑选最自信的 p 个正例和 n 个反例.这样能保证挑出来的样本具有随机性和多样性,让两边的训练能稳步、均衡地推进.

2. 根据 D_l 分离出各视图对应的标记数据集 D_l^1, D_l^2 ;

3. **for** $t = 1, 2, \dots, T$ **do**

4. $h_1 = \mathcal{L}(D_l^1)$;

5. $h_2 = \mathcal{L}(D_l^2)$;

6. 计算 h_1 在 D_s 上的预测置信度,选出 p 个正例和 n 个反例,组成带伪标记的数据: $\tilde{D}_p^2 = \{(x_i^2, +1) \mid x_i^1 \in D_p\}$,
 $\tilde{D}_n^2 = \{(x_i^2, -1) \mid x_i^1 \in D_n\}$;

7. 计算 h_2 在 D_s 上的预测置信度,选出 p 个正例和 n 个反例,组成带伪标记的数据: $\tilde{D}_p^1 = \{(x_i^1, +1) \mid x_i^2 \in D_p\}$,
 $\tilde{D}_n^1 = \{(x_i^1, -1) \mid x_i^2 \in D_n\}$;

8. 从缓冲池中移除这些被挑选的样本: $D_s = D_s \setminus (D_p \cup D_n)$;

9. **if** h_1 和 h_2 的参数在本轮迭代中均未发生改变 **then** break;

10. **else**

11. $D_l^1 = D_l^1 \cup (\tilde{D}_p^1 \cup \tilde{D}_n^1)$;

12. $D_l^2 = D_l^2 \cup (\tilde{D}_p^2 \cup \tilde{D}_n^2)$;

13. 从 D_u 中随机抽取 $2p + 2n$ 个样本补充到缓冲池 D_s 中;

14. **end if**

15. **end for**

理论证明,如果两个视图满足:

- 充分性 (即每个视图单独就足以让分类器收敛): 任何一个单独的视图,都包含了足够完整的信息,多给点数据它自己就能把题解出来.
- 条件独立性 (给定类别标记时,两个视图的属性条件独立)

[ef] 充分性保证了两个分类器能“互相提供有用的新知识”; 条件独立性保证了它们“不会互相传染对方的偏见和错误”. 则利用未标记样本通过协同训练可以将弱分类器的泛化性能提升到任意高.

[eg] 条件独立性要求“给定类别时,两个视图完全不相干”,但这在现实中是不可能的.

比如一部动作片 (给定类别),它的画面往往有“追逐爆炸” (视图 1),而它的音效也必然会在暗中是有强烈的因果和物理关联的,绝对做不到数学上的完全独立.所以现实中协同训练高精度.

虽然在现实任务中“条件独立性”几乎无法完美满足,但协同训练在实际应用中依然能带来显著的性能提升.

10.5.3 单视图协同训练与分歧特征

对于只有单个视图的数据,依然可以通过基于分歧的方法进行半监督学习.此时我们必须通过人为引入差异来构建“分歧”:

- 使用不同的基学习器算法 (如 SVM 与 决策树 结合);
- 采用不同的样本子集采样 (如 Bagging 的自助法采样);
- 使用差异明显的模型参数设置.

只要多个弱学习器之间保持显著的分歧,它们就能通过相互提供伪标记样本的形式来提高整体的泛化性能.该方法较少受到强模型假设的制约,简单有效,因而得到了广泛的应用.

10.6 半监督聚类 (Semi-Supervised Clustering)

10.6.1 监督信息的形式

聚类通常是无监督学习,但在很多现实任务中,我们可以获得一些额外的监督信息.通过引入监督信息,我们可以显著约束和指导聚类方向.

监督信息通常有两种基本形式:

- **必连 (Must-link) 与勿连 (Cannot-link) 约束:** 必连约束 $\mathcal{M}$ 规定两个样本必须被划分到同一个簇,勿连约束 $\mathcal{C}$ 规定两个样本不能被划分到同一个簇.
- **少量有标记样本:** 直接提供少量已知类别 (或目标簇标签) 的样本.

10.6.2 约束 k 均值 (Constrained k-means) 算法

约束 k 均值算法利用第一类监督信息 (即约束集合 $\mathcal{M}$ 和 $\mathcal{C}$)。它在常规 k -means 指派样本时,强迫每一次划分都符合已知的必连和勿连约束。

Algorithm 10.6.1

约束 k -means 算法

输入: 样本集 $D = \{x_1, x_2, \dots, x_m\}$;

必连约束集 $\mathcal{M}$, 规定必须划入同一簇的样本对;

勿连约束集 $\mathcal{C}$, 规定绝不能划入同一簇的样本对;

聚类簇数 k .

输出: 簇划分 $\{C_1, C_2, \dots, C_k\}$.

过程:

1. 选择初始中心: 从样本集 D 中随机选取 k 个样本作为各个簇的初始均值向量 $\{\mu_1, \mu_2, \dots, \mu_k\}$.

2. 策略迭代 (重复执行以下步骤,直至均值向量不再发生更新):

- 清空当前的簇分配,令每个簇 $C_j = \emptyset (1 \leq j \leq k)$.
- 对每个示例 $x_i (i = 1, 2, \dots, m)$,在其合规的有效候选簇集合 $\mathcal{K}_{\text{valid}}$ 中,寻找距离其最近的簇索引 r :

$$r = \arg \min_{j \in \mathcal{K}_{\text{valid}}} \|x_i - \mu_j\|_2$$

- 安全审查与合法候选集 $\mathcal{K}_{\text{valid}}$: 集合 $\mathcal{K}_{\text{valid}}$ 初始为 $\{1, 2, \dots, k\}$.在考核样本 x_i 拟塞入簇 j 时,必须通过以下两条硬性红线的政审:
- 必连遵从: 若存在约束 $(x_i, x_a) \in \mathcal{M}$,且已指派样本 $x_a \in C_A$,则必须满足 $j = A$.
- 勿连回避: 若存在约束 $(x_i, x_b) \in \mathcal{C}$,且已指派样本 $x_b \in C_B$,则必须满足 $j \neq B$.

随后将该样本并入对应簇中: $C_r = C_r \cup \{x_i\}$.

- 在全体样本指派完成后,基于各簇吸纳的新成员,重新计算并动态更新其均值中心:

$$\mu_j = \frac{1}{|C_j|} \sum_{x \in C_j} x, \quad (j = 1, 2, \dots, k)$$

[eh] 规则死锁与中止: 若某个样本身上背负的必连与勿连约束过于繁重或自相矛盾,导致遍历所有簇均告违规 (即有效候选集完全归空 $\mathcal{K}_{\text{valid}} = \emptyset$),意味着在当前中心布局下无解,算法将立即中断并抛出异常退出.

[ei] 以西瓜数据集 4.0 (30 个样本) 为例,在限定必连集 $\mathcal{M}$ 和勿连集 $\mathcal{C}$ 条件下,设置 $k = 3$,经历 5 轮迭代收敛,给出三个簇的最终划分结果.

10.6.3 约束种子 k 均值 (Constrained Seed k-means) 算法

当我们拥有少量有标记样本时,可以直接将它们作为种子 (Seeds). 约束种子 k -means 将这些种子用于初始化聚类中心,并在随后的迭代中,固定种子样本的簇隶属关系,仅允许未标记样本改变归属.

Algorithm 10.6.2

约束种子 k -means 算法

输入: 样本集 $D = \{x_1, x_2, \dots, x_m\}$;

少量有标记样本集合 $S = \bigcup_{j=1}^k S_j$, 其中 S_j 为已经明确归属于第 j 簇的种子样本集;

聚类簇数 k .

输出: 簇划分 $\{C_1, C_2, \dots, C_k\}$.

过程:

1. 锚定初始中心: 对每个簇 $j = 1, 2, \dots, k$,严格基于其专属的种子样本集 S_j 计算初始均值向量:

$$\mu_j = \frac{1}{|S_j|} \sum_{x \in S_j} x$$

2. 重复执行以下步骤,直至均值向量不再发生更新:

- 种子复位: 强行让所有带标签的种子选手回到各自的大本营,作为各簇的固定底座,令 $C_j = S_j (1 \leq j \leq k)$.
- 普通兵就近归队: 对每一个未标记样本 $x_i \in D \setminus S$,计算其到各个簇中心的欧氏距离,直接将其划分至距离最近的那个簇 C_r 中:

$$r = \arg \min_{j \in \{1, 2, \dots, k\}} \|x_i - \mu_j\|_2$$

$$C_r = C_r \cup \{x_i\}$$

- 中心动态修正: 在全体未标记样本吸纳完毕后,融合种子样本与新并入的样本,重新计算并动态调整各簇的均值中心:

$$\mu_j = \frac{1}{|C_j|} \sum_{x \in C_j} x, \quad (j = 1, 2, \dots, k)$$

Remark

- 种子样本在此算法中扮演了不可动摇的约束角色.与常规 k -means 相比,种子样本的簇归属关系在前期的“种子复位”阶段被强制固定,在后续的迭代中**不允许改变归属**.它们的存在就像磁铁,牢牢吸引特征相似的未标记样本向其靠拢.
- 算法的鲁棒性**: 由于不引入离散的“必连/勿连”链条约束,样本指派时不需要进行条件排除,因而算法运行平滑高效,**彻底免疫**了因约束冲突而导致的死锁和崩盘退出风险.

有标记样本在这个算法中扮演了“引路人”的硬性锚点角色,它强制均值中心贴近物理类别分布,使无监督的聚类结构能够自底而上地与我们希望的真实类别语义保持高度契合.

11 强化学习

11.1 强化学习基本设置

11.1.1 强化学习的直观理解与应用场景

强化学习 (Reinforcement Learning, RL) 致力于解决智能体 (Agent) 在与环境 (Environment) 的不断交互中学习最优决策序列的问题.

典型应用场景:

- 机器人控制 (如 Boston Dynamics 的机器狗、双足机器人行走)
- 游戏 AI (如吃豆人、Backgammon 双陆棋、机器人足球赛)

“试错 (Trial-and-error)”和“延迟奖赏 (Delayed Reward)”是强化学习面临的核心挑战.

11.1.2 马尔可夫决策过程 (Markov Decision Process, MDP)

强化学习的基础数学框架是马尔可夫决策过程.

Definition 11.1.1

马尔可夫决策过程

MDP 是对强化学习环境交互过程的严格数学抽象.它假设环境的下一个状态仅依赖于当前状态和当前动作,而与更早的历史状态无关 (即马尔可夫性).

- 智能体与环境闭环**: 机器 (智能体) 观察到当前的环境状态 x , 执行动作 a , 环境根据该动作转移到新状态, 并反馈给机器一个标量奖赏 r .
- 四元组定义**: 一个 MDP 可以用一个四元组来完整描述:

$$E = \langle X, A, P, R \rangle$$

- 状态空间 X** : 机器能够感知到的所有可能状态的集合, $x \in X$.
 - 动作空间 A** : 机器能够执行的所有可能动作的集合, $a \in A$.
 - 转移概率 P** : $P: X \times A \times X \rightarrow \mathbb{R}$. 状态 x 执行动作 a 后, 转移到下一个状态 x' 的概率.
 - 奖赏函数 R** : $R: X \times A \times X \rightarrow \mathbb{R}$ 或简写为 $R: X \times X \rightarrow \mathbb{R}$. 表示发生状态转移时环境给予的反馈信号.
- 策略的数学表达**: 在给定状态下执行的行动.
 - 确定性策略**: 直接将状态映射到唯一动作, $\pi: X \rightarrow A$, 此时动作可以直接写为 $a = \pi(x)$.
 - 随机性策略**: 将状态映射到动作的概率分布,

$$\pi: X \times A \rightarrow \mathbb{R}$$

这意味着在状态 x 下采取某动作 a 依据的是一个概率值.

- 交互轨迹**: 从初始状态 x_0 开始, 智能体根据策略 π 选择动作 a_0 , 环境反馈奖赏 r_1 并转移到 x_1 . 以此类推, 形成一条长度为 T 的交互序列:

$$H_T = \langle x_0, a_0, r_1, x_1, a_1, r_2, \dots, x_{T-1}, a_{T-1}, r_T, x_T \rangle$$

11.1.3 优化目标: 累积奖赏 (Cumulative Reward)

Remark

强化学习的根本目标不是追求某一步的即时奖赏最大化, 而是寻找一个最优策略 π^* , 使得长期累积奖赏的期望值达到最大.

1. T 步累积奖赏 (Finite-horizon):

适用于有限步数结束的任务. 目标是最大化这 T 步内奖赏的平均期望:

$$\mathbb{E} \left[\frac{1}{T} \sum_{t=1}^T r_t \right]$$

2. **折扣累积奖赏 (Infinite-horizon Discounted)**: 适用于可能无限进行下去的任务. 引入折扣因子 $\gamma \in (0, 1)$, 使得越远未来的奖赏权重越低, 既体现了近期奖赏的确定性, 也保证了总和级数的收敛:

$$\mathbb{E} \left[\sum_{t=0}^{+\infty} \gamma^t r_{t+1} \right]$$

11.1.4 强化学习与监督学习的本质对比

在基础设置的最后, 我们需要厘清强化学习与传统机器学习的边界:

- **监督学习 (分类器/回归器)**: 模型 f 的输入是示例 (Features), 输出是标记 (Labels). 训练数据直接提供了明确的“正确答案” (Ground Truth), 模型通过最小化误差进行学习.
- **强化学习**: 策略 π 的输入是状态, 输出是动作. 环境中 **不存在** 明确的“正确动作”标记, 只有等到最终结果揭晓才能反思之前动作 (通过**试错**尝试不同的动作), 观察环境给出的延迟标量反馈 (即奖赏 r) 来间接评估和优化策略.

[ej] 在某种意义上而言, 强化学习可被认为是具有“延迟标记能力”的监督学习.

11.2 K-摇臂赌博机 (K-armed Bandit)

在本节中, 我们通过一个简化的单状态模型 (丢弃了 MDP 最核心的“状态转移结构”和“未来价值预测”能力) 来深入探讨强化学习中最核心的权衡问题.

11.2.1 问题定义与背景

Definition 11.2.1

K-摇臂赌博机 (K-Armed Bandit)

K-摇臂赌博机是一个只有**单个状态**、**K 个动作** (即摇臂) 的强化学习模型, 每个摇臂的奖赏服从某个期望未知的分布.

在每个时刻从 **K** 个摇臂中选择一个并按下, 根据该摇臂背后的未知概率分布立即产生一个奖赏 r , **不涉及状态转移**. 该问题的目标是通过多次尝试, 最大化累积奖赏.

在 K-摇臂赌博机中, 智能体面临着经典的 **探索-利用窘境 (Exploration-Exploitation dilemma)**:

- **利用 (Exploitation)**: 按下目前已知平均奖赏最高的摇臂, 以获取即时高收益.
- **探索 (Exploration)**: 尝试那些目前表现一般甚至较差的摇臂, 以发现潜在的更高收益摇臂, 防止陷入局部最优.

11.2.2 ϵ -贪心算法

这是解决探索-利用问题最简单直观的方法.

Algorithm 11.2.2

ϵ -贪心算法

在每一次选择动作时:

- 以 $1 - \epsilon$ 的概率进行“利用”: 选择当前平均奖赏最大的摇臂, 即 $a = \arg \max_k Q(k)$.
- 以 ϵ 的概率进行“探索”: 从所有 **K** 个摇臂中以均匀概率随机选择一个动作.

[ek] 算法中的 $Q(k)$ 记录了当前摇臂 k 的平均奖赏.

随时间减小 ϵ 的值 (如 $\epsilon = \frac{1}{\sqrt{t}}$) 可以使系统在初期充分探索, 在后期趋于稳定利用.

11.2.3 Softmax 策略 (Boltzmann 探索)

ϵ -贪心算法在探索时对所有摇臂一视同仁, 而 Softmax 策略则根据摇臂的估计价值来分配探索概率.

Definition 11.2.3

Softmax 选择机制

基于每个摇臂的当前平均奖赏估计 $Q(k)$, 按下第 k 个摇臂的概率 $P(k)$ 定义为:

$$P(k) = \frac{e^{\frac{Q(k)}{\tau}}}{\sum_{i=1}^K e^{\frac{Q(i)}{\tau}}}$$

- **高温度 ($\tau \rightarrow +\infty$)**: 各摇臂选中的概率趋于相等, 系统表现为完全随机探索.
- **低温度 ($\tau \rightarrow 0$)**: 高奖赏摇臂被选中的概率极大增加, 系统表现为趋向于完全利用.

Remark

用累计奖赏代替 K-摇臂赌博机算法中的奖赏函数, 离散状态、离散动作上的多步强化学习可以将每个状态上动作选择看作一个 K-摇臂赌博机问题, 基于赌博机算法选择动作, 但没有考虑马尔可夫决策过程的特性. 因为该模型不存在状态的概念, 算法不会考虑这个动作会把智能体带到一个“好”的状态还是一个“坏”的状态; 也忽略了延迟奖赏.

11.3 有模型学习 (Model-based Learning)

理想情况——即所谓的“白盒”环境：强化学习的环境是完全已知的。

具体而言,MDP的四元组 $E = \langle X, A, P, R \rangle$ 对智能体来说是透明的,智能体清楚地知道在状态 x 下执行动作 a 后,转移到下一个状态 x' 的精确概率 $P_{x \rightarrow x'}^a$,以及能够获得的准确奖赏 $R_{x \rightarrow x'}^a$ 。

[e1] 既然环境的运转规律已经完全被掌握,强化学习就不再需要像赌博机那样去“试错”。此时的问题完全转化为一个通过规划和计算来寻找最优策略 π^* 的动态规划问题。

11.3.1 值函数的定义 (Value Functions)

Remark

如果说“策略 π ”是决定智能体在特定状态下**该怎么做**,那么“值函数”就是用来评估智能体处于某个特定状态(或执行某个特定动作)**到底有多好**.它衡量的是在未来长期交互中所能获得累积奖赏的期望值,是评价不同策略优劣的“量尺”。

为了全面评估,我们需要定义两种不同的值函数:

1. **状态值函数 $V^\pi(x)$** : **评估这个状态有多好**. 表示从状态 x 开始,一直按照策略 π 行动,所能获得的累积奖赏期望. 根据累积奖赏类型的不同,可分为:

- **T 步累积状态值函数:**

$$V_T^\pi(x) = \mathbb{E}_\pi \left[\frac{1}{T} \sum_{t=1}^T r_t \mid x_0 = x \right]$$

- **折扣累积状态值函数:**

$$V_\gamma^\pi(x) = \mathbb{E}_\pi \left[\sum_{t=0}^{+\infty} \gamma^t r_{t+1} \mid x_0 = x \right]$$

2. **状态-动作值函数 $Q^\pi(x, a)$** : **评估在给定状态下的动作有多好**. 表示从状态 x 开始,第一步先强制执行动作 a ,然后再继续按照策略 π 行动,所能获得的累积奖赏期望.

- **T 步累积动作值函数:**

$$Q_T^\pi(x, a) = \mathbb{E}_\pi \left[\frac{1}{T} \sum_{t=1}^T r_t \mid x_0 = x, a_0 = a \right]$$

- **折扣累积动作值函数:**

$$Q_\gamma^\pi(x, a) = \mathbb{E}_\pi \left[\sum_{t=0}^{+\infty} \gamma^t r_{t+1} \mid x_0 = x, a_0 = a \right]$$

11.3.2 贝尔曼等式

Remark

直接计算长期的累积期望非常困难.贝尔曼等式揭示了强化学习中最深刻的动态规划思想:当前状态的价值,等于“即时获得的奖赏”加上“下一个状态的后续价值”.它巧妙地将一个长远未来的累积过程,转化为一个只关心当前步与下一步的简单递归方程。

Bellman Equation:

$$V_T^\pi(x) = \sum_{a \in A} \pi(x, a) \sum_{x' \in X} P_{x \rightarrow x'}^a \left(\frac{1}{T} R_{x \rightarrow x'}^a + \frac{T-1}{T} V_{T-1}^\pi(x') \right)$$

将长度为 T 的累积奖赏拆分:

$$\begin{aligned} V_T^\pi(x) &= \mathbb{E}_\pi \left[\frac{1}{T} \sum_{t=1}^T r_t \mid x_0 = x \right] \\ &= \mathbb{E}_\pi \left[\frac{1}{T} r_1 + \frac{T-1}{T} \cdot \frac{1}{T-1} \sum_{t=2}^T r_t \mid x_0 = x \right] \end{aligned}$$

接着,利用全概率公式,将第一步所有可能采取的动作 a (根据策略 $\pi(x, a)$ 采样) 以及随后可能转移到的下一个状态 x' (根据转移概率 $P_{x \rightarrow x'}^a$) 展开:

$$V_T^\pi(x) = \sum_{a \in A} \pi(x, a) \sum_{x' \in X} P_{x \rightarrow x'}^a \left(\frac{1}{T} R_{x \rightarrow x'}^a + \frac{T-1}{T} \underbrace{\mathbb{E}_\pi \left[\frac{1}{T-1} \sum_{t=1}^{T-1} r_t \mid x_0 = x' \right]}_{V_{T-1}^\pi(x')} \right)$$

于是得到原式.

同理,折扣累积奖赏的贝尔曼等式可以推导为:

$$V_\gamma^\pi(x) = \sum_{a \in A} \pi(x, a) \sum_{x' \in X} P_{x \rightarrow x'}^a (R_{x \rightarrow x'}^a + \gamma V_\gamma^\pi(x'))$$

而状态-动作值函数 Q 可以直接写为给定动作后的期望:

$$Q_T^\pi(x, a) = \sum_{x' \in X} P_{x \rightarrow x'}^a \left(\frac{1}{T} R_{x \rightarrow x'}^a + \frac{T-1}{T} V_{T-1}^\pi(x') \right);$$

$$Q_\gamma^\pi(x, a) = \sum_{x' \in X} P_{x \rightarrow x'}^a (R_{x \rightarrow x'}^a + \gamma V_\gamma^\pi(x')).$$

11.3.3 最优策略与最优值函数

最优策略是在所有状态下都能获得最大累积奖励的策略:

$$\pi^* = \arg \max_{\pi} \sum_{x \in X} V^\pi(x)$$

在最优策略下,对于任意状态 x ,最优状态值函数 $V^*(x)$ 必然等于在当前状态下采取能带来最高期望价值的动作对应的 Q 值.这被称为贝尔曼最优等式 (Bellman Optimality Equation):

$$V_\gamma^*(x) = \max_{a \in A} \sum_{x' \in X} P_{x \rightarrow x'}^a (R_{x \rightarrow x'}^a + \gamma V_\gamma^*(x'))$$

$$V^*(x) = \max_{a \in A} Q^{\pi^*}(x, a)$$

基于上述推导,如果我们已经知道了最优的值函数,那么最优策略就只需在每一步贪心地选择使得 $Q(x, a)$ 最大的动作:

[em] 这不是废话吗,每一步选最好的动作.

$$\pi'(x) = \arg \max_{a \in A} Q^\pi(x, a)$$

11.3.4 策略迭代算法 (Policy Iteration)

结合对值函数的评估和对策略的贪心改进,我们便可以得到有模型学习中的核心算法框架.

Algorithm 11.3.1

策略迭代 (Policy Iteration)

策略迭代通过“策略评估”和“策略改进”两个阶段的交替,不断优化策略,直至收敛.

输入: MDP 四元组 $E = \langle X, A, P, R \rangle$, 累积奖赏参数 T .

过程:

1. **初始化**: 对所有的状态 $x \in X$, 初始化值函数 $V(x) = 0$, 并设定一个均匀分布的随机初始策略 $\pi(x, a) = \frac{1}{|A(x)|}$.
2. **策略评估 (Policy Evaluation)**: 在当前策略 π 下, 根据贝尔曼等式迭代计算值函数. 对于所有的状态 $x \in X$ 、有限步数 T , 循环更新 $V'(x)$ 直至 $t = T$;

$$V'(x) = \sum_{a \in A} \pi(x, a) \sum_{x' \in X} P_{x \rightarrow x'}^a \left(\frac{1}{t} R_{x \rightarrow x'}^a + \frac{t-1}{t} V(x') \right), \text{ then } V = V'$$

3. **策略改进 (Policy Improvement)**: 使用最新计算得到的值函数, 提取出对每个动作的 $Q(x, a)$ 估计, 并贪心地选出最好的动作, 更新策略:

$$\pi'(x) = \arg \max_{a \in A} Q(x, a)$$

4. **终止判定**: 检查新策略 π' 是否与旧策略 π 完全相同. 若没有任何状态的动作发生改变, 说明策略已经是最优的, 跳出循环; 否则令 $\pi = \pi'$, 回到第 2 步重新评估.

输出: 最优策略 π^* .

11.4 免模型学习 (Model-free Learning)

在有模型学习中,我们依赖于对环境动态特性 (转移概率 P 和奖赏函数 R) 的完全掌握. 然而, 现实世界中的大多数问题都过于复杂, 我们往往面对的是一个“黑盒”环境.

Definition 11.4.1

免模型学习 (Model-free Learning)

当环境的转移概率 P 和奖赏函数 R 未知时, 智能体无法通过内部的马尔可夫推演来计算值函数. 此时, 智能体必须通过在环境中实际执行动作、收集真实交互轨迹 (Trajectory) 来积累经验, 并直接根据经验数据来估算值函数并优化策略.

11.4.1 蒙特卡洛方法 (Monte Carlo Methods)

通过大数定律, 用样本的经验平均值来逼近真实的数学期望.

Remark

蒙特卡洛方法

既然无法直接算出某个状态-动作对 (x, a) 的期望累积奖赏, 那就在环境中多跑几次完整的游戏 (Episode). 把每次经过状态 x 并执行动作 a 后得到的实际总奖赏记录下来, 最后取平均值, 作为对 $Q(x, a)$ 的无偏估计.

1. **轨迹采样**: 在环境 E 中执行策略 π 产生一条完整轨迹:

$$\langle x_0, a_0, r_1, x_1, a_1, r_2, \dots, x_{T-1}, a_{T-1}, r_T, x_T \rangle$$

其中 x_i 是状态, a_i 是策略, r_i 是奖赏.

2. **计算回报 (Return)**: 对于轨迹中的时刻 t , 从当前时刻到任务结束, 其向后的平均累积奖赏可以表示为:

$$R = \frac{1}{T-t} \sum_{i=t+1}^T r_i$$

3. **增量式更新**: 为了避免保存所有历史数据, 我们采用增量更新的方法. 设 $\text{count}(x_t, a_t)$ 为状态-动作对被访问的次数, 则新的估计值为:

$$Q(x_t, a_t) = \frac{Q(x_t, a_t) \times \text{count}(x_t, a_t) + R}{\text{count}(x_t, a_t) + 1}$$

在蒙特卡洛控制中, 为了保证持续的探索, 我们通常将更新后的值函数结合 ϵ -贪心法来提取新策略. 根据产生样本的策略与待优化的策略是否相同, 分为两类:

- **同策略 (On-policy)**: 评估和改进的是同一个策略 (即使用 ϵ -贪心策略生成轨迹, 同时也优化这个 ϵ -贪心策略).

Algorithm 11.4.2

同策略蒙特卡洛方法

1. $Q(x, a) = 0, \text{count}(x, a) = 0, \pi(x, a) = \frac{1}{|A(x)|}$;
2. **for** $s = 1, 2, \dots$ **do**
3. 在 E 中执行策略 π 产生轨迹:
 $\langle x_0, a_0, r_1, x_1, a_1, r_2, \dots, x_{T-1}, a_{T-1}, r_T, x_T \rangle$;
4. **for** $t = 0, 1, \dots, T-1$ **do**
5. $R = \frac{1}{T-t} \sum_{i=t+1}^T r_i$;
6. $Q(x_t, a_t) = \frac{Q(x_t, a_t) \times \text{count}(x_t, a_t) + R}{\text{count}(x_t, a_t) + 1}$;
7. $\text{count}(x_t, a_t) = \text{count}(x_t, a_t) + 1$;
8. **end for**
9. 对所有已见状态 x : $\pi(x, a) = \begin{cases} \arg \max_{a'} Q(x, a'), & \text{以概率 } 1-\epsilon \\ \text{以均匀概率从 } A \text{ 中选取动作, 以概率 } \epsilon \end{cases}$
10. **end for**

输出: 策略 π

- **异策略 (Off-policy)**: 评估和改进的是目标策略 π (通常是确定性的贪心策略), 但生成数据的行为策略 (Behavior Policy) 是另一个具有探索性的策略 (如 ϵ -贪心: $\pi^*(x, a) = \begin{cases} \arg \max_a Q(x, a), & 1-\epsilon \\ \text{random}, & \epsilon \end{cases}$).
- 为了修正分布差异, 异策略蒙特卡洛必须引入**重要性采样 (Importance Sampling)**. 如果行为策略在第 j 步选择动作的概率为 p_j , 那么目标策略的回报需要通过累积概率比率进行加权:

$$R = \frac{1}{T-t} \sum_{i=t+1}^T \left(r_i \times \prod_{j=i}^{T-1} \frac{1}{p_j} \right)$$

Algorithm 11.4.3

异策略蒙特卡罗强化学习算法

1. $Q(x, a) = 0, \text{count}(x, a) = 0, \pi(x, a) = \frac{1}{|A(x)|}$;
2. **for** $s = 1, 2, \dots$ **do**
3. 在 E 中执行 π 的 ϵ -贪心策略产生轨迹:
 $\langle x_0, a_0, r_1, x_1, a_1, r_2, \dots, x_{T-1}, a_{T-1}, r_T, x_T \rangle$;
4. $p_i = \begin{cases} 1-\epsilon + \frac{\epsilon}{|A|}, & a_i = \pi(x) \\ \frac{\epsilon}{|A|}, & a_i \neq \pi(x) \end{cases}$;
5. **for** $t = 0, 1, \dots, T-1$ **do**
6. $R = \frac{1}{T-t} \sum_{i=t+1}^T \left(r_i \times \prod_{j=i}^{T-1} \frac{1}{p_j} \right)$;
7. $Q(x_t, a_t) = \frac{Q(x_t, a_t) \times \text{count}(x_t, a_t) + R}{\text{count}(x_t, a_t) + 1}$;
8. $\text{count}(x_t, a_t) = \text{count}(x_t, a_t) + 1$;
9. **end for**
10. $\pi(x) = \arg \max_a Q(x, a')$
11. **end for**

输出: 策略 π

Remark

同策略的核心是“边做边学”, 评估当前正在执行的带有探索性的策略; 异策略的核心是“看别人做来学自己”, 将探索过程与目标策略的优化解耦, 能学习到更为纯粹的最优策略.

11.4.2 时序差分学习 (Temporal Difference, TD)

蒙特卡洛方法的一个致命弱点是：必须等待一个完整的轨迹结束（即走到终点状态 x_T ）才能计算回报并更新价值。在连续任务或步数极长的任务中，这种延迟是不可接受的。

Remark

时序差分学习

核心概念：时序差分结合了蒙特卡洛的“免模型采样”思想和动态规划的“自举 (Bootstrapping)”思想。它不必等待回合结束，而是在每走一步后，利用下一步的价值估计来更新当前步的价值估计。

我们已经有经典的更新公式

$$Q_{t+1}(x, a) = Q_t(x, a) + \frac{1}{t+1}(R - Q_t(x, a))$$

有时候我们不能模拟完整的轨迹，只能通过单步模拟来更新值函数。记 x' 是下一步的状态， a' 是下一步的动作。

$$Q(x, a) = Q(x, a) + \alpha(r + \gamma Q(x', a') - Q(x, a))$$

- α ：学习率（更新步长）。
- $r + \gamma Q(x', a')$ ：被称为 **TD 目标 (TD Target)**，它是根据下一步的即时奖赏和下一步的估计价值拼凑出的当前真实价值的近似。
- $r + \gamma Q(x', a') - Q(x, a)$ ：被称为 **TD 误差 (TD Error)**，代表了我们估计的偏差。

[en] 时序差分方法由于每步都能更新，收敛速度通常远快于蒙特卡洛方法，是现代强化学习最基石的算法族。

11.4.3 Sarsa 算法与 Q-learning 算法

基于 TD 思想的动作价值控制算法，同样分为同策略和异策略两个经典分支：Sarsa 和 Q-learning。

Algorithm 11.4.4

Sarsa 算法 (同策略时序差分控制)

更新时需要用到五元组：当前状态 S 、当前动作 A 、即时奖赏 R 、下一状态 S' 、下一动作 A' 。

1. 智能体在状态 x 依据 ϵ -贪心策略选择动作 a 。
2. 执行动作 a ，获得奖赏 r ，并转移到新状态 x' 。
3. 在状态 x' 处，再次依据相同的 ϵ -贪心策略选择下一步动作 a' 。
4. 利用实际选定的 a' 进行价值更新：

$$Q(x, a) = Q(x, a) + \alpha(r + \gamma Q(x', a') - Q(x, a))$$

5. 贪心化当前策略 $\pi(x) = \arg \max_{a''} Q(x, a'')$ ，步进到下一状态，重复循环。

Algorithm 11.4.5

Q-learning 算法 (异策略时序差分控制)

Q-learning 是强化学习中最著名的算法之一，它打破了必须用实际执行动作的价值来更新当前价值的限制。

核心过程：

1. 智能体在状态 x 依据 ϵ -贪心策略（行为策略）选择执行动作 a 。
2. 执行动作 a ，获得奖赏 r ，并转移到新状态 x' 。
3. 在更新 $Q(x, a)$ 时，它不去管智能体下一步实际会走哪步，而是直接假设智能体会采取最优策略，即直接取状态 x' 下所有可能动作的 Q 值最大者：

$$Q(x, a) = Q(x, a) + \alpha \left(r + \gamma \max_{a''} Q(x', a'') - Q(x, a) \right)$$

4. 贪心化目标策略 $\pi(x) = \arg \max_{a''} Q(x, a'')$ ，步进到下一状态，重复循环。

Remark

Sarsa 是保守的，它在更新时会考虑策略本身的探索性带来的潜在风险；而 Q-learning 是激进的，无论当前如何探索，它始终在逼近那个绝对最优的确定性策略 π^* 。

11.5 值函数近似 (Value Function Approximation)

探讨强化学习面对复杂现实问题时如何解决由于状态空间过大或连续而导致的“维度灾难”问题。

11.5.1 表格型值函数的局限性

无论是动态规划、蒙特卡洛方法还是时序差分学习，我们都在一个前提下进行：假定状态空间和动作空间都是离散且有限的。

在这些场景下，值函数可以通过一个表格来精确记录。这意味着我们在内存中维护着一个查找表，表中的每一个条目精确对应一个状态 x 或状态-动作对 (x, a) 的价值。

[eo] 然而，现实世界中的状态往往是连续的（例如机器人的关节角度、自动驾驶的坐标与速度）。即使进行离散化，状态数量也会随维度呈指数级爆炸，导致无法有效泛化。

11.5.2 值函数近似的概念

改变值函数的表达方式,引入值函数近似 (Value Function Approximation) .

Definition 11.5.1

值函数近似 (Value Function Approximation)

不再试图精确记录每一个状态的独立价值,而是引入一个参数化的数学模型 (如线性模型、神经网络等) 来拟合价值空间的整体分布.通过学习模型的参数 θ ,使得模型对于任意输入的状态 x ,都能直接计算并输出近似的价值估计.

这种方法不仅极大地压缩了存储空间 (从存储无限个状态值变为仅存储参数 θ),更重要的是赋予了模型**泛化能力**: 算法可以通过学习经历过的状态,推断出那些从未见过的相似状态的价值.

11.5.3 值函数近似

对于状态值函数,我们将其表达为状态特征向量的线性函数: $V_\theta(x) = \theta^\top x$, 其中 x 是表示环境状态的特征向量,而 θ 则是模型需要学习的参数向量.

优化目标: 为了找到最优的参数 θ ,采用最小二乘误差 MSE 来度量我们学到的近似值函数 $V_\theta(x)$ 与环境中真实的值函数 $V^\pi(x)$ 之间的差异. 其目标函数定义为:

$$\varepsilon_\theta = \mathbb{E}_{x \sim \pi} [(V^\pi(x) - V_\theta(x))^2]$$

最经典的做法是使用梯度下降法 (Gradient Descent) 来迭代更新参数向量 θ .

在实际的免模型学习中,真实的 $V^\pi(x)$ 是未知的.因此,算法通常会使用蒙特卡罗方法的回报 R 或时序差分方法的 TD 目标 $(r + \gamma V_\theta(x'))$ 来作为真实价值的替代标签,从而计算梯度并更新 θ .

11.5.4 状态-动作值函数近似

$Q(x, a)$ 的线性近似可以表示为:

$$Q_{\theta(x,a)} = \theta^\top(x,a)$$

输入变成了状态和动作的联合特征向量.在假定动作空间是有限的离散空间时,可以使用 One-hot encoding.动作特征可以表示为一个除第 k 位为 1,其余 $K-1$ 位全为 0 的向量:

$$a = (0, \dots, 1, \dots, 0)$$

将该动作特征与状态特征进行拼接或张量积计算,即可作为线性模型的输入.

Remark

虽然线性近似简单高效,但其表达能力有限.在面对极其复杂的非线性环境时,通常需要引入更强大的非线性值函数近似技术,例如核方法 (Kernel Methods) 或者是现代深度强化学习 (Deep Reinforcement Learning) 中广泛使用的深度神经网络 (如 DQN 中的神经网络) .

11.6 模仿学习 (Imitation Learning)

在经典的强化学习任务中,我们通常假设环境能够提供明确的奖赏函数 $R(x)$ (例如游戏得分).但在许多现实场景中 (如自动驾驶、机器人抓取),人工设计一个完美反映任务目标的奖赏函数极其困难.

Definition 11.6.1

模仿学习 (Imitation Learning)

当环境奖赏函数难以定义时,模仿学习提供了一种替代方案: 直接利用人类专家在环境中的决策记录 (即专家范例) 来进行学习,让机器模仿人类专家的行为模式.

11.6.1 专家轨迹数据集 (Expert Demonstrations)

在模仿学习中,智能体不依赖环境奖赏,而是依赖一组由专家展示的优质轨迹数据集.假设我们收集了 m 条专家轨迹,构成数据集 D : $D = \{\tau_1, \tau_2, \dots, \tau_m\}$, 其中,每一条轨迹 τ_i 记录了专家在完整回合中经历的“状态-动作”序列:

$$\tau_i = \langle s_1^i, a_1^i, s_2^i, a_2^i, \dots, s_{n_i+1}^i \rangle$$

这里 n_i 表示第 i 条轨迹的步数.

11.6.2 行为克隆 (Behavioral Cloning)

Remark

行为克隆直接将模仿学习问题退化为一个传统的监督学习问题.它不再考虑状态转移的长期影响,而是简单粗暴地学习一个从“状态”到“动作”的直接映射.

- 数据展平与重组:** 首先,将包含时序关系的轨迹数据集 D 拆解为一个个独立的“状态-动作”对.此时,状态 s 视为特征输入,专家采取的动作 a 视为标签 (Label): $D = \left\{ (s_1, a_1), (s_2, a_2), \dots, (s_{\sum_{i=1}^m n_i}, a_{\sum_{i=1}^m n_i}) \right\}$

2. 监督学习寻优:

- 如果动作空间是**离散**的（如走迷宫的上下左右），则训练一个**分类器** $\pi(s)$ ，最小化交叉熵等分类损失。
- 如果动作空间是**连续**的（如方向盘转角），则训练一个**回归器** $\pi(s)$ ，最小化均方误差等回归损失。

[ep] 行为克隆实现简单且收敛快，但它的致命缺点是缺乏对长远状态转移的规划。一旦智能体在执行中因微小误差偏离了专家轨迹分布，就会进入从未见过的状态，导致误差随时间步不断累积甚至崩溃。

11.6.3 逆强化学习 (Inverse Reinforcement Learning, IRL)

在 IRL 的标准设定中，我们通常假设真实的奖赏函数 $R(x)$ 可以被建模为状态特征向量 x 的线性组合： $R(x) = w^T x$ ，其中 w 是我们从专家数据中学习的未知权重向量。

我们的目标是寻找一个权重 w^* ，使得在此奖赏函数下，专家策略的期望累积奖赏（即状态特征的期望加权和）显著优于任何其他随机采样的策略。

Algorithm 11.6.2

迭代式逆强化学习算法 (Iterative IRL)

该算法通过交替更新“奖赏函数权重”与“最优策略”来逼近专家的真实意图。

输入：环境 E ，状态空间 X ，动作空间 A ，专家轨迹数据集 $D = \{\tau_1, \tau_2, \dots, \tau_m\}$ 。

过程：

1. **计算专家特征期望**：从范例轨迹 D 中计算出专家的状态加权和的均值向量 $\bar{x}^*$ （这代表了专家的平均表现水平）。
2. **初始化**：随机初始化一个策略 π 。
3. **for** $t = 1, 2, \dots$ **do** :
 - 在环境中执行当前策略 π ，采样生成轨迹，并计算出该策略下的状态加权和均值向量 $\bar{x}_t^{\pi}$ 。
 - **更新奖赏权重** w ：

[eq] 寻找一个新的权重 w^* ，使得在此权重下，专家的表现尽可能大地拉开与当前已知所有策略 π_i （其中 $i = 1 \dots t$ ）表现的差距。这是一个最大最小化优化问题。

$w^* = \arg \max_w \min_{i=1}^t w^T (\bar{x}^* - \bar{x}_i^{\pi_i})$ s.t. $\|w\| \leq 1$

- **更新策略** π ：在更新了奖赏函数的环境 $\langle X, A, \tilde{R}(x) = (w^*)^T x \rangle$ 中，利用任何常规强化学习算法（如有模型的值迭代、免模型的 Q-learning）重新求解出当前环境的最优策略，赋给 π 。

7. **end for**

输出：反推得到的奖赏函数 $R(x) = (w^*)^T x$ 以及最终收敛的最优策略 π 。

Remark

迭代式 IRL 算法本质上是一个生成对抗过程：奖赏函数试图区分专家轨迹和机器生成的轨迹（找出能最大化表现差距的 w ），而策略优化的过程则试图生成更高质量的轨迹来最大化当前奖赏，直到机器的表现逼近专家的特征期望 $\bar{x}^*$ ，无法被进一步区分。常规强化学习是“已知奖赏函数，求最优策略”；逆强化学习则是“已知最优策略（专家轨迹），反推潜在的奖赏函数”。

11.7 基于人类反馈的强化学习 (RLHF)

11.7.1 智能体分类

根据智能体内部优化的核心目标，可分为三类：

1. **基于值的智能体 (Value-based Agent)**：显式地学习价值函数，隐式地学习策略。其策略是从所学到的价值函数推算得到的。
2. **基于策略的智能体 (Policy-based Agent)**：直接学习策略函数。策略函数的输入为一个状态，输出为对应动作的概率。不学习价值函数，价值函数隐式地表达在策略函数中。
3. **Actor-Critic 智能体**：结合上述两者。Actor（演员）负责输出策略，Critic（评论家）负责评估价值函数，Critic 辅助 Actor 更好、更稳地更新。

11.7.2 大模型对齐挑战与 3H 原则

大语言模型（如基于 Transformer 架构的模型）本质上是“文字接龙”的概率模型，通过海量语料进行预训练。但这会导致模型可能输出有害、偏见或不符合人类价值观的内容。

AI 对齐 (AI Alignment) 是指确保人工智能系统的目标和行为与人类的价值观、意图相一致。为了规范 LLM 的行为，OpenAI 等机构提出了模型回复必须遵循的 3H 原则：Helpfulness (有用性)、Honesty (诚实性)、Harmlessness (无害性)。

传统监督学习很难兼顾这种多维度的、模糊的偏好，这就促使了 RLHF 技术的诞生。

11.7.3 RLHF 核心流水线 (Pipeline)

RLHF 并非单一算法，而是一套将人类主观偏好转化为机器可优化目标的工程流水线。整个过程通常分为三个核心步骤。

1. **监督微调 (Supervised Fine-Tuning, SFT)**：预训练模型具备了丰富的世界知识，但还不擅长“对话”。在这一步，我们收集高质量的人类问答对 $(x, y_1, y_2, \dots, y_T)$ ，利用传统的交叉熵损失函数对模型进行监督微调。目的：让模型学会遵循指令和基础的对话格式，此时得到的基础模型称为 **SFT 模型**。
2. **训练奖励模型 (Reward Model, RM)**：既然无法用代码写出完美的奖赏函数 $R(x)$ ，我们就训练一个神经网络来充当“人类裁判”。
 - **数据收集**：给定一个输入 x ，让 SFT 模型生成多个不同的回答。

- **人类标注**：人类标注员根据 3H 原则,对这些回答进行好坏排序（人类偏好）。
 - **模型训练**：训练一个奖励模型,输入文本,输出一个标量奖励值 r ,使得 RM 的打分能够完美拟合人类的排序偏好。
3. 近端策略优化 (Proximal Policy Optimization, PPO) 闭环迭代

这是 RLHF 中数学和工程最复杂的环节,通常采用 Actor-Critic 架构的变体算法 PPO.

四个模型 协作:

1. **策略模型 (Policy Model / Actor)**: 初始化为 SFT 模型,生成回复.
 2. **参考模型 (Reference Model)**: 被冻结的原始 SFT 模型,用于计算 KL 散度惩罚,防止策略模型在追求高奖励时“走火入魔”,偏离正常人类语言分布.
 3. **奖励模型 (Reward Model)**: 被冻结的裁判,对策略模型生成的回答给出环境奖赏 r .
 4. **评论模型 (Critic Model)**: 预测当前状态（对话上下文）的价值,辅助 Actor 更新.
1. **采样阶段**: 用户输入提示词 x ,策略模型逐步生成回答 y_t .生成完毕后,奖励模型给出整句话的得分.同时,参考模型给出相同序列的概率分布,用于计算相对于初始分布的 KL 散度偏移.
2. **广义优势估计 (Generalized Advantage Estimation, GAE)**:
- 评论模型输出每个时刻的价值预测.
 - 利用真实获得的回报 (Return) 与评论模型的预测值作差,计算出 **TD 误差 (TD Error)**.
 - GAE 结合多步 TD 误差,精确估算出当前动作的 **优势函数 (Advantage Function)** (即当前动作比平均表现好多少)。
3. **参数更新阶段**: 基于计算出的优势函数,更新网络参数.同时利用参考模型确保更新的策略不会有太大的变化,从而维持模型的稳定性.

11.8 强化学习与有监督学习的区别

11.8.1 相较于有监督学习更有可能考虑整体影响

有监督学习针对单个词元进行反馈,其目标是要求模型针对给定的输入给出确切的答案.而强化学习是针对整个输出文本进行反馈,并不针对特定的词元.反馈粒度的不同,使强化学习更适合大语言模型,既可以兼顾表达多样性,又可以增强对微小变化的敏感性.自然语言十分灵活,可以用多种不同的方式表达相同的语义.而有监督学习很难支持上述学习方式.强化学习则可以允许模型给出不同的多样性表达.另外,有监督微调通常采用交叉熵损失作为损失函数,由于总和规则,造成这种损失对个别词元变化不敏感.如果改变个别的词元,只会对整体损失产生小的影响.但是,一个否定词可以完全改变文本的整体含义.强化学习则可以通过奖励函数同时兼顾多样性和微小变化敏感性两个方面.

11.8.2 强化学习更容易解决幻觉问题

用户在大语言模型上主要有三类输入: (a) 文本型 (Text-Grounded), 用户输入相关文本和问题,让模型基于所提供的文本生成答案 (例如,“本文中提到的人名和地名有哪些”); (b) 求知型 (Knowledge-Seeking), 用户仅提出问题,模型根据内在知识提供真实回答 (例如,“流感的常见原因是什么”); (c) 创造型 (Creative), 用户提供问题或说明,让模型进行创造性输出 (例如,“写一个关于.....的故事”). 有监督学习算法非常容易使得求知型查询产生幻觉.在模型并不包含或者知道答案的情况下,有监督训练仍然会促使模型给出答案.而使用强化学习方法,则可以通过定制奖励函数,将正确答案赋予非常高的分数,将放弃回答的答案赋予中低分数,将不正确的答案赋予非常高的负分,使得模型学会依赖内部知识选择放弃回答,从而在一定程度上缓解模型的幻觉问题.

11.8.3 强化学习可以更好地解决多轮对话奖励累积问题

多轮对话能力是大语言模型重要的基础能力之一.多轮对话是否达成最终目标,需要考虑多次交互过程的整体情况,因此很难使用有监督学习的方法构建.而使用强化学习方法,可以通过构建奖励函数,根据整个对话的背景及连贯性对当前模型输出的优劣进行判断.

12 名词索引

- A**
- AdaBoost 2, 70
 - AdaBoost 算法 71
- B**
- Bagging 79
 - Boosting 69
 - 半监督学习 125
 - 半朴素贝叶斯 2, 34
 - 包裹式选择 3, 120
 - 贝叶斯决策论 2, 32
 - 贝叶斯网络 36
- C**
- C4.5 2, 63
 - CART 64
 - CART 剪枝算法 69
 - CART 生成算法 67
 - 策略迭代 (Policy Iteration) 137
- D**
- DBSCAN 3, 111
 - 袋外样本 78
 - 迭代式逆强化学习算法 (Iterative IRL) 141
 - 度量学习 96
 - 多维缩放 84
- E**
- Epsilon-贪心算法 135
- F**
- 泛化误差 9
 - 非线性支持向量机学习算法 54
- G**
- 高斯混合聚类 3, 108, 110
 - 广义瑞利商 18
 - 广义线性回归 23
 - 过滤式选择 3, 119
 - 过拟合 11
- H**
- 函数间隔 40
 - 核函数 52
 - 合页损失 50
 - 回归问题的提升树算法 74
- I**
- ID3 61
 - ID3 算法生成决策树 61
 - ISOMAP 96
- J**
- 监督学习 4
 - 间隔最大化 38
 - 剪枝 63
 - 交叉熵 27
 - 假设空间 6
- 吉布斯采样 2, 38
- 集成学习 69
- 极大似然估计 27
- 结构风险最小化 6
- 几何间隔 40
- 经验风险最小化 6
- 基尼指数 66
- 近邻成分分析 2, 98
- 近邻成分分析 (NCA) 98
- 机器学习 4
- 计算信息增益 61
- 局部线性嵌入 2, 96
- 决策树 2, 59
- 聚类 102
- 矩阵补全 3, 125
- K**
- K d 树生成算法 101
 - K means 聚类 106
 - K 近邻法 2, 99
 - K 近邻算法 99, 101
- L**
- L1 正则化 22
 - L2 正则化 21
 - LASSO 121
 - LLE 96
 - LVW (Las Vegas Wrapper) 算法 120
 - 拉普拉斯修正 34
 - 拉索回归 21
 - 类间散度矩阵 18
 - 类内散度矩阵 18
 - 岭回归 21
 - 流形学习 2, 95
 - 逻辑斯蒂回归 2, 26
 - 逻辑斯蒂回归模型 28
- M**
- MDS 85
 - 马尔可夫决策过程 134
 - 马氏距离 97
- O**
- OOB 特征重要性计算 (MDA) 81
- P**
- 判别函数 16
 - 朴素贝叶斯 32
 - 朴素贝叶斯法 34
- Q**
- Q-learning 算法 (异策略时序差分控制) 139
 - 强化学习 5, 134
 - 嵌入式选择 3, 121
- R**
- Relief 119
 - Relief 算法 119
- 弱分类器 70
- S**
- Sarsa 算法 (同策略时序差分控制) 139
 - Sigmoid 函数 27
 - SMO 算法 54, 56
 - Softmax 28
 - 熵 59
 - 生成式半监督高斯混合模型 EM 算法 128
 - 随机森林 2, 79
 - 随机森林算法 79
 - 损失函数 6
- T**
- TSVM 算法 129
 - 特征选择 117
 - 条件独立性 33
 - 梯度提升算法 76
 - 同策略蒙特卡洛方法 138
- V**
- VC 维度 8
- W**
- 维数灾难 84
 - 无监督学习 4
- X**
- 线性回归 20
 - 线性可分支持向量机 44
 - 线性判别分析 18
 - 线性支持向量机 48
 - 协同训练 (Co-training) 算法 132
 - 信息增益 60
 - 信息增益率 61
 - 稀疏表示 3, 123
 - 学习向量量化 3, 107
- Y**
- 样本主成分分析: 基于相关矩阵的特征值分解 91
 - 样本主成分分析: 基于样本的奇异值分解 94
 - 样本主成分分析: 基于 (样本) 协方差矩阵的特征值分解 86
 - 异策略蒙特卡罗强化学习算法 138
 - 约束 k-means 算法 133
 - 约束种子 k-means 算法 133
- Z**
- 支持向量 42
 - 支持向量机 38
 - 直推学习 126
 - 主成分分析 85
 - 字典学习 123
 - 自助采样法 78
- 最大熵模型 2, 29, 31
- 最小二乘法 21
- 同**
- 同质集成/异质集成 2, 69
- 梯**
- 梯度提升树 2, 76